



Продвинутая разработка нейросетей от ядра до продакшена

Модуль 1 - Перегрузка глубокого обучения 2.0

Урок 1 - Основы кастомных моделей

Шаг 01:

Миссия 1

Во время настройки пользовательской модели Алиса заметила, что обучающая и валидационная loss расходятся: валидационная растёт. Что скорее всего происходит?

Подсказка: См. классическую проблему, когда модель «слишком хорошо» подгоняется под данные.

Ответ:

- Переобучение (overfitting)
- Недообучение (underfitting)
- Ошибка вычисления градиента

Шаг 02:

Миссия 2

 Разработчик Борис хочет изменить архитектуру нейросети и использовать остаточные блоки (residual). Зачем добавляют skip-соединения?

Подсказка: Подумайте о том, как передавать «исходный сигнал» через множество слоёв.

Ответ:

- Чтобы облегчить обратный проход градиентов и избежать исчезновения градиента
- Чтобы уменьшить число параметров сети
- Чтобы ускорить работу на GPU

💡 **Шаг 03:**

Миссия 3

Катя экспериментирует: использует нестандартную функцию активации, но сеть «взрывается» (градиенты становятся бесконечными). Какой подход часто помогает стабилизировать обучение?

Подсказка: Посмотри на методы нормализации.

Ответ:

- Добавление слоёв **BatchNorm / LayerNorm**
- Увеличение скорости обучения
- Удаление регуляризации

💡 **Шаг 04:**

Миссия 4

🧠 При тренировке кастомной модели Денис хочет делать контроль над весами, чтобы не позволить им стать слишком большими. Что он может применить?

Подсказка: Регуляризация часто добавляет штрафы к весам.

Ответ:

- **L2-регуляризация (weight decay)**
- Пропуск обучения (drop learning)
- Увеличение batch size

💡 **Шаг 05:**

Миссия 5

🛠 Команда разрабатывает модель для мобильного приложения, и нагрузка на память критична. Какой трюк поможет уменьшить модель без сильной потери качества?

Подсказка: Подумайте о снижении точности представления весов.

Ответ:

- **Квантование параметров (quantization)**
- Добавление случайного шума к весам
- Уменьшение числа слоёв до одного
- Увеличение dropout до 0.9

💡 **Шаг 06:**

Миссия 6

Сопоставьте термин и его описание в контексте кастомных моделей.

Термины:

Ответ:

- Transfer learning
- Fine-tuning

- Feature extractor
- Head (голова модели)

Описания:

1. Использование предобученной модели как базовых слоёв, замораживая их
2. Настройка верхних слоёв модели под конкретную задачу
3. Слои, которые отвечают за классификацию/регрессию после базового тела
4. Модель без головы, которая извлекает эмбединги

Подсказка: Обрати внимание: некоторые термины перекрываются, но каждый имеет своё уникальное назначение.

- **Transfer learning** — Использование предобученной модели как базовых слоёв, замораживая их
- **Fine-tuning** — Настройка верхних слоёв модели под конкретную задачу
- **Feature extractor** — Модель без головы, которая извлекает эмбединги
- **Head (голова модели)** — Слои, которые отвечают за классификацию/регрессию после базового тела

Шаг 07:

Миссия 7

 Сопоставьте прием оптимизации модели с его назначением или эффектом.

Приёмы:

Ответ:

- Dropout
- Early stopping
- Gradient clipping
- Learning rate scheduler

Эффекты:

1. Остановка обучения, когда валидационная метрика перестаёт улучшаться
2. Ограничение максимальной нормы градиента
3. Снижение скорости обучения по расписанию
4. Случайное обнуление части нейронов во время тренировки

Подсказка: Подумайте, какие методы защищают от переобучения, а какие регулируют градиенты.

- **Dropout** — Случайное обнуление части нейронов во время тренировки
- **Early stopping** — Остановка обучения, когда валидационная метрика перестаёт улучшаться
- **Gradient clipping** — Ограничение максимальной нормы градиента
- **Learning rate scheduler** — Снижение скорости обучения по расписанию

Шаг 08:

Миссия 8

 Сопоставьте понятие и пример из жизни разработчика нейросети.

Понятия:

Ответ:

- Batch normalization

- Layer normalization
- Group normalization
- Instance normalization

Примеры:

1. Нормализация по каналам внутри одного изображения (GAN, стилизация)
2. Нормализация по мини-батчам, часто в сверточных сетях
3. Нормализация по слоям во временных или трансформерных моделях
4. Нормализация в группах каналов, чтобы работать независимо от размера батча

Подсказка: Обрати внимание на отличие по размеру групп и по тому, над какими осями нормализуют.

- **Batch normalization** — Нормализация по мини-батчам, часто в сверточных сетях
- **Layer normalization** — Нормализация по слоям во временных или трансформерных моделях
- **Group normalization** — Нормализация в группах каналов, чтобы работать независимо от размера батча
- **Instance normalization** — Нормализация по каналам внутри одного изображения (GAN, стилизация)

🔮 Шаг 09:

Миссия 9

🔗 Сопоставьте тип обучающей ошибки и её причина.

Тип ошибки:

Ответ:

- Underfitting
- Overfitting
- High bias
- High variance

Причины:

1. Модель слишком простая, недостаточная сложность
2. Модель слишком сложная, чрезмерная гибкость на шум
3. Смещение в обучающих данных или слишком строгая регуляризация
4. Чувствительность к шуму данных, модель «перепрыгивает» на выбросы

Подсказка: Bias и variance — классическая дилемма моделей обучения.

- **Underfitting** — Модель слишком простая, недостаточная сложность
- **Overfitting** — Модель слишком сложная, чрезмерная гибкость на шум
- **High bias** — Смещение в обучающих данных или слишком строгая регуляризация
- **High variance** — Чувствительность к шуму данных, модель «перепрыгивает» на выбросы

🔮 Шаг 10:

Миссия 10

⚙️ Сопоставьте компонент кастомной модели с его ролью:

Компоненты:

Ответ:

- Embedding layer
- Positional encoding
- Attention mechanism
- Feed-forward block

Роли:

1. Добавляет информации о порядке элементов в последовательности
2. Прямые полносвязные слои после внимания
3. Превращает входные токены в плотные векторы
4. Селективно взвешивает связи между элементами последовательности

Подсказка: В трансформерных архитектурах части известны из базовых теорий внимания.

- **Embedding layer** — Превращает входные токены в плотные векторы
- **Positional encoding** — Добавляет информации о порядке элементов в последовательности
- **Attention mechanism** — Селективно взвешивает связи между элементами последовательности
- **Feed-forward block** — Прямые полносвязные слои после внимания

 Шаг 11:

Миссия 11

У программиста Павла задача — упорядочить этапы подготовки данных для кастомной модели. Расположите шаги в правильном порядке:

1. Нормализация / стандартизация признаков
2. Сбор и чистка данных
3. Разделение на train / validation / test
4. Обработка категориальных признаков (one-hot, embedding и др.)
5. Обработка пропущенных значений

Подсказка: Подумайте, какие этапы нельзя делать раньше разделения, а какие — после базовой очистки.

Ответ:

- Сбор и чистка данных
- Обработка пропущенных значений
- Обработка категориальных признаков (one-hot, embedding и др.)
- Нормализация / стандартизация признаков
- Разделение на train / validation / test

 Шаг 12:

Миссия 12

 Когда программист Олег разрабатывает цикл обучения, он хочет упорядочить ключевые этапы эпохи (epoch). Расположите шаги:

1. Обратный проход (backpropagation) и обновление весов
2. Прямой проход (forward pass), вычисление предсказаний
3. Обнуление градиентов
4. Подсчёт функции потерь (loss) и метрик

Подсказка: Каждая эпоха начинается заново с чистых градиентов.

Ответ:

- Обнуление градиентов
- Прямой проход (forward pass), вычисление предсказаний
- Подсчёт функции потерь (loss) и метрик
- Обратный проход (backpropagation) и обновление весов

Шаг 13:

Миссия 13

 В команде Марии была задача корректно проводить валидацию и тестирование после обучения. Расположите этапы:

1. Оценка модели на тестовом наборе
2. Обучение (train) на тренировочных данных
3. Выбор гиперпараметров по validation-набору
4. Финальное дообучение на объединённом train+validation (по необходимости)

Подсказка: Подумай: зачем нужен validation, и когда мы используем все данные.

Ответ:

- Обучение (train) на тренировочных данных
- Выбор гиперпараметров по validation-набору
- Финальное дообучение на объединённом train+validation (по необходимости)
- Оценка модели на тестовом наборе

Шаг 14:

Миссия 14

 Ивану необходимо правильно запустить пайплайн инференса кастомной модели. Что должно быть первым, а что — последним? Расположите этапы:

1. Получение входных данных (например, картинка или текст)
2. Постобработка предсказаний (декодирование, фильтрация и т.д.)
3. Приведение входа к нужному формату / нормализация
4. Запуск прямого прохода модели (forward inference)

Подсказка: Подумай: сначала подготовка, потом предсказание, затем интерпретация результата.

Ответ:

- Получение входных данных (например, картинка или текст)
- Приведение входа к нужному формату / нормализация
- Запуск прямого прохода модели (forward inference)
- Постобработка предсказаний (декодирование, фильтрация и т.д.)

Шаг 15:

Миссия 15

 В процессе CI/CD для модели разработчик Стас упорядочил этапы развёртывания кастомной модели. Расположите их:

1. Мониторинг метрик модели после запуска в продакшене
2. Интеграция API модели в сервис

3. Тестирование API и функциональные тесты
4. Деплой (развёртывание) модели на сервере/кластере

Подсказка: По логике сначала подключаем, затем проверяем, затем запускаем и наблюдаем.

Ответ:

- Интеграция API модели в сервис
- Тестирование API и функциональные тесты
- Деплой (развёртывание) модели на сервере/кластере
- Мониторинг метрик модели после запуска в продакшене

Шаг 16:

Миссия 16

✳ В процессе A/B-тестирования решений с кастомной моделью, команда Лены делает эксперимент. Расположите этапы:

1. Сбор статистики и сравнение метрик
2. Определение тестовых групп / распределение трафика
3. Запуск обеих версий (контрольной и экспериментальной)
4. Формулировка гипотезы и критериев успеха

Подсказка: Подумай, что должно быть определено до запуска теста, и как анализ проводится после.

Ответ:

- Формулировка гипотезы и критериев успеха
- Определение тестовых групп / распределение трафика
- Запуск обеих версий (контрольной и экспериментальной)
- Сбор статистики и сравнение метрик

Шаг 17:

Миссия 17

🔄 В команде Ани модель периодически обновляют: от подготовки данных до переобучения. Расположите этапы полного цикла обновления модели:

1. Сбор новых данных / расширение датасета
2. Тонкая донастройка (fine-tuning) предобученной версии
3. Оценка новой версии на тестовых данных и сравнение с текущей в продакшене
4. Развёртывание обновлённой модели

Подсказка: Полный цикл начинается с данных, затем обучение и затем внедрение.

Ответ:

- Сбор новых данных / расширение датасета
- Тонкая донастройка (fine-tuning) предобученной версии
- Оценка новой версии на тестовых данных и сравнение с текущей в продакшене
- Развёртывание обновлённой модели

Урок 2 - Архитектуры 2020-2025 что изменилось

Шаг 01:

Миссия 1

Инженер Катя читает статью о архитектуре Vision Transformer (ViT). Что является основным отличием ViT от классической сверточной сети?

Подсказка: Думайте о том, как сеть разбивает изображение на части и обрабатывает их.

Ответ:

- **Использование токенов-патчей и внимания вместо свёрток**
- Непрерывные свёртки с ядром 7×7
- Отсутствие слоёв позиционного кодирования

🔮 Шаг 02:

Миссия 2

🔧 Разработчик Илья хочет применить архивацию сжатия параметров (parameter pruning) в модели 2023 года, чтобы ускорить инференс. Какой риск наиболее вероятен?

Подсказка: Сжатие может убрать «важные» параметры.

Ответ:

- **Понижение точности на некоторых классах данных**
- Увеличение объёма модели
- Снижение сложности модели до линейной

🔮 Шаг 03:

Миссия 3

👤 В команде Алисы внедряют архитектуру GPT-4-style. Что характерно для таких крупных моделей последних лет?

Подсказка: Обратите внимание на размер параметров и механизм обучения.

Ответ:

- **Многомодальность и обучение с учителем и без учителя гибридно**
- Использование только сверточных блоков
- Минимальное количество параметров (< 10 млн)

🔮 Шаг 04:

Миссия 4

🏢 При внедрении LLaMA-подобной модели в облаке, Дмитрий замечает, что память GPU быстро заполняется. Что из нижеперечисленного поможет уменьшить пиковую нагрузку?

Подсказка: Это приём, используемый при обучении длинных последовательностей.

Ответ:

- **Комплексный контроль градиентов** — gradient checkpointing
- Увеличение batch size
- Удаление слоёв внимания

🔮 Шаг 05:

Миссия 5

📖 Разработчица Маша читает исследование про цепочки внимания (sparse attention) в моделях 2022–2024 годов. В чём преимущество разреженного внимания?

Подсказка: Думайте о снижении сложности по длине последовательности.

Ответ:

- **Снижение квадратичной сложности до почти линейной**
- Увеличение числа параметров
- Отсутствие внимания между всеми парами токенов
- Полная блокировка границ последовательности

🔮 Шаг 06:

Миссия 6

🔍 Сопоставьте архитектуры 2020–2025 и ключевые нововведения, которые они предлагают.

Архитектуры:

Ответ:

- Swin Transformer
- Perceiver
- MLP-Mixer
- Reformer

Нововведения / особенности:

1. Использует локальные окна внимания с переходом между уровнями
2. Обработывает разные виды данных (аудио, текст, изображение) в единой модели
3. Замена внимания слоями MLP, разделёнными по токенам и каналам
4. Разреженное внимание + рекурсивные локальные внимательные блоки для длинных последовательностей

Подсказка: Подумайте, какая архитектура была задумана для мультимодальности, какая — для уменьшения сложности внимания.

- **Swin Transformer** — Использует локальные окна внимания с переходом между уровнями
- **Perceiver** — Обработывает разные виды данных (аудио, текст, изображение) в единой модели
- **MLP-Mixer** — Замена внимания слоями MLP, разделёнными по токенам и каналам
- **Reformer** — Разреженное внимание + рекурсивные локальные внимательные блоки для длинных последовательностей

🔮 Шаг 07:

Миссия 7

Сопоставьте концепции новой архитектуры с их эффектом или применением.

Концепции:

Ответ:

- FlashAttention
- Mixture of Experts (MoE)
- Fused attention + MLP (например, FuseFormer)
- Adapters / LoRA (Low-Rank Adaptation)

Эффекты / применения:

1. Адаптация больших моделей с минимальным количеством параметров для fine-tuning
2. Экономия памяти и ускорение через оптимальный доступ к GPU памяти во внимание
3. Динамическое переключение экспертных ветвей сети для разных примеров
4. Интеграция внимания и MLP-блоков для более компактного дизайна

Подсказка: FlashAttention — об оптимизации самого вычисления внимания, MoE — о ветвлении, Adapters — о тонкой адаптации.

- **FlashAttention** — Экономия памяти и ускорение через оптимальный доступ к GPU памяти во внимание
- **Mixture of Experts (MoE)** — Динамическое переключение экспертных ветвей сети для разных примеров
- **Fused attention + MLP (например, FuseFormer)** — Интеграция внимания и MLP-блоков для более компактного дизайна
- **Adapters / LoRA (Low-Rank Adaptation)** — Адаптация больших моделей с минимальным количеством параметров для fine-tuning

🔧 Шаг 08:

Миссия 8

🔗 Сопоставьте термин с архитектурным приёмом, появившимся после 2020 года.

Термины:

Ответ:

- Memory-efficient attention
- Cross-attention fusion
- Residual MLP streams
- Dynamic token pruning

Приёмы / описание:

1. Уменьшение числа токенов во время обработки (отсечение нерелевантных)
2. Использование внимания между слоями для слияния информации
3. Схемы внимания, работающие с ограничением памяти при длинных последовательностях
4. Параллельные потоки MLP с остаточными соединениями для разных аспектов модели

Подсказка: Думайте о современных методах ускорения и оптимизации архитектур.

- **Memory-efficient attention** — Схемы внимания, работающие с ограничением памяти при длинных последовательностях
- **Cross-attention fusion** — Использование внимания между слоями для слияния информации
- **Residual MLP streams** — Параллельные потоки MLP с остаточными соединениями для разных аспектов модели
- **Dynamic token pruning** — Уменьшение числа токенов во время обработки (отсечение нерелевантных)

🔧 Шаг 09:

Миссия 9

🧠 Сопоставьте названия архитектур или подходов 2020+ с их ключевой слабостью или ограничением.

Архитектуры / подходы:

Ответ:

- Mixture of Experts (MoE)
- Vision Transformer (ViT)
- MLP-Mixer
- Dynamic token pruning

Ограничения / слабости:

1. Чувствительность к шуму и необходимость большого количества данных
2. Сложность экспертов и неравномерное распределение нагрузки
3. Отсутствие локальной инвариантности, как в свёртках
4. Потеря информации при агрессивном отсечении токенов

Подсказка: У каждой современной методики есть компромисс — подумайте, что теряется при выигрыше в скорости или параметрах.

- **Mixture of Experts (MoE)** — Сложность экспертов и неравномерное распределение нагрузки
- **Vision Transformer (ViT)** — Чувствительность к шуму и необходимость большого количества данных
- **MLP-Mixer** — Отсутствие локальной инвариантности, как в свёртках
- **Dynamic token pruning** — Потеря информации при агрессивном отсечении токенов

🔮 Шаг 10:

Миссия 10

🔧 Сопоставьте методы оптимизации архитектур после 2020 года с целью, которую они решают.

Методы:

Ответ:

- Quantization + mixed precision
- Distillation (Knowledge Distillation)
- Pruning / sparse weights
- Efficient attention (Linformer, Nyström, BigBird и др.)

Цели / эффекты:

1. Уменьшение размера модели и ускорение инференса
2. Перенос знаний из большой модели в малую
3. Уменьшение числа ненужных параметров в модели
4. Снижение вычислительной сложности внимания для длинных последовательностей

Подсказка: Часто современные архитектуры сочетают несколько методов оптимизации.

- **Quantization + mixed precision** — Уменьшение размера модели и ускорение инференса
- **Distillation (Knowledge Distillation)** — Перенос знаний из большой модели в малую
- **Pruning / sparse weights** — Уменьшение числа ненужных параметров в модели
- **Efficient attention (Linformer, Nyström, BigBird и др.)** — Снижение вычислительной сложности внимания для длинных последовательностей

🔮 Шаг 11:

Миссия 11

🔧 Разработчик Сергей хочет воспроизвести жизненный цикл архитектуры с 2020 по 2025 год. Упорядочите ключевые этапы развития архитектурных тенденций:

1. Эксперименты с MLP-базируемыми архитектурами
2. Появление моделей с sparse / efficient attention
3. Переход к мультимодальным архитектурам и унифицированным моделям
4. Интеграция техник оптимизации (quantization, pruning, distillation и др.)

Подсказка: Подумайте о том, как эволюционировали модели: сначала простые конструкции, потом оптимизации, затем мультизадачность.

Ответ:

- Эксперименты с MLP-базируемыми архитектурами
- Появление моделей с sparse / efficient attention
- Переход к мультимодальным архитектурам и унифицированным моделям
- Интеграция техник оптимизации (quantization, pruning, distillation и др.)

Шаг 12:

Миссия 12

При проектировании инференс-пайплайна для архитектуры 2024 года, инженер Ольга упорядочивает стадии:

1. Прогон входа через ускоренные attention-модули (sparse / efficient attention)
2. Приведение входных токенов / embeddings
3. Постобработка вывода модели (декодирование, фильтрация, softmax и т.д.)
4. Подготовка батчей / паддинг токенов

Подсказка: Сначала подготавливаем данные, затем прогон, затем интерпретацию.

Ответ:

- Приведение входных токенов / embeddings
- Подготовка батчей / паддинг токенов
- Прогон входа через ускоренные attention-модули (sparse / efficient attention)
- Постобработка вывода модели (декодирование, фильтрация, softmax и т.д.)

Шаг 13:

Миссия 13

 В команде Павла разрабатывается модель с MoE и distillation. Какой логический порядок этапов?

1. Обучение большой экспертной модели (teacher)
2. Перенос знаний в компактную модель (distillation)
3. Интеграция компактной модели в продакшен
4. Мониторинг и адаптация производительности

Подсказка: Сначала строят сильную модель, затем упрощают и внедряют.

Ответ:

- Обучение большой экспертной модели (teacher)
- Перенос знаний в компактную модель (distillation)
- Интеграция компактной модели в продакшен
- Мониторинг и адаптация производительности

Шаг 14:

Миссия 14

 Команда Насти строит CI/CD для архитектурных инноваций. Упорядочите шаги:

1. Автоматическое тестирование новых слоёв и модулей
2. Построение и проверка модели с новыми архитектурными блоками
3. Развёртывание новой версии модели

4. Мониторинг производительности и отклонений

Подсказка: Сначала проверяем изменения, затем запускаем, затем оцениваем.

Ответ:

- Автоматическое тестирование новых слоёв и модулей
- Построение и проверка модели с новыми архитектурными блоками
- Развёртывание новой версии модели
- Мониторинг производительности и отклонений

Шаг 15:

Миссия 15

🔗 Для архитектуры с динамическим отсечением токенов (dynamic token pruning) команда Иры выстраивает цикл обработки:

1. Анализ активаций токенов и решение, какие отбросить
2. Принятие решения о сокращении токенов
3. Прогон оставшихся токенов через основной блок внимания / трансформера
4. Объединение результатов с контекстом

Подсказка: Отсечение токенов — это динамический фильтр перед основной обработкой.

Ответ:

- Анализ активаций токенов и решение, какие отбросить
- Принятие решения о сокращении токенов
- Прогон оставшихся токенов через основной блок внимания / трансформера
- Объединение результатов с контекстом

Шаг 16:

Миссия 16

📊 При сравнении версий архитектур 2022 и 2025 годов команда строит процесс оценки:

1. Запуск обеих моделей на одном тестовом наборе данных
2. Сбор и сравнение метрик (точность, латенты, потребление ресурсов)
3. Решение, стоит ли мигрировать на новую архитектуру
4. Развёртывание новой модели, если решено перейти

Подсказка: Первым идёт контрольный запуск, затем анализ, потом выбор, потом внедрение.

Ответ:

- Запуск обеих моделей на одном тестовом наборе данных
- Сбор и сравнение метрик (точность, латенты, потребление ресурсов)
- Решение, стоит ли мигрировать на новую архитектуру
- Развёртывание новая модель, если решено перейти

Шаг 17:

Миссия 17

При имплементации полной архитектурной миграции инженеры следуют этому порядку:

1. Разработка и тестирование нового архитектурного блока (offline)
2. Интеграция нового блока с текущей моделью (canary / shadow mode)
3. Оценка производительности и стабильности

4. Полный переход (switchover) на новую архитектуру

Подсказка: Сначала тестируешь офлайн, потом внедряешь осторожно, потом переключаешь.

Ответ:

- Разработка и тестирование нового архитектурного блока (offline)
- Интеграция нового блока с текущей моделью (canary / shadow mode)
- Оценка производительности и стабильности
- Полный переход (switchover) на новую архитектуру

Урок 3 - Тренировочные циклы от простого к функциональному

Шаг 01:

Миссия 1

Разработчик Пётр реализует тренировочный цикл с ручным `forward` и `backward`. Сейчас он обновляет веса сразу после каждого примера. Что из ниже поможет сгладить шум в градиентах?

Подсказка: Усреднение градиентов по группе примеров часто помогает.

Ответ:

- **Групповое обновление** — копить градиенты и обновлять после mini-batch
- Уменьшить momentum
- Применить сильный L2-регуляризатор
- Прекратить использовать функцию потерь

Шаг 02:

Миссия 2

Анна хочет внедрить возможность валидации после каждой эпохи. Когда лучше всего выполнять валидацию?

Подсказка: После завершения всех батчей эпохи модель стабилизирована на текущих весах.

Ответ:

- **После завершения всех батчей текущей эпохи**
- После каждого батча
- Перед каждой эпохой
- Каждый шаг градиента

Шаг 03:

Миссия 3

Миша хочет реализовать early stopping: прекращать обучение, если валидационная потеря не улучшаются. Что нужно хранить при этом?

Подсказка: Нужно помнить наилучшие веса и лучший результат.

Ответ:

- **Лучший показатель валидационной потери + соответствующие веса**
- Только финальные веса модели
- Последнюю пару вход-выход
- Гиперпараметры обучения

🔮 Шаг 04:

Миссия 4

Разработчица Катя хочет логировать метрики обучения: loss, accuracy, lr. Что удобнее всего логировать в рамках цикла?

Подсказка: Логгинг внутри цикла часто делают после каждого батча или эпохи.

Ответ:

- **Логировать после каждого батча или в конце эпохи**
- Логировать только перед обучением
- Логировать раз в месяц
- Не логировать вовсе

🔮 Шаг 05:

Миссия 5

Герой Иван использует циклы обучения и сталкивается с терминами. Сопоставьте термин и его корректное описание:

Термины:

Ответ:

- Epoch
- Batch
- Iteration
- Overfitting

Определения:

1. Обучение слишком точно «запоминает» тренировочные данные и теряет обобщающую способность.
2. **Один проход по мини-бату** — шаг обновления весов.
3. Полный проход по всему тренировочному набору.
4. Небольшая часть выборки, обрабатываемая одновременно.

Подсказка: Epoch — полный проход всех данных, iteration связан с batch, overfitting — про обобщение.

- **Epoch** — Полный проход по всему тренировочному набору.
- **Batch** — Небольшая часть выборки, обрабатываемая одновременно.
- **Iteration** — Один проход по мини-бату — шаг обновления весов.
- **Overfitting** — Обучение слишком точно «запоминает» тренировочные данные и теряет обобщающую способность.

🔮 Шаг 06:

Миссия 6

Алиса реализует scheduler learning rate. Сопоставьте тип scheduler и его поведение:

Типы:

Ответ:

- Step decay
- Exponential decay
- Cyclical LR
- Cosine annealing

Поведение:

1. LR уменьшается по косинусной кривой к нулю.
2. LR меняется в пределах минимума и максимума циклично.
3. LR уменьшается экспоненциально с каждой итерацией.
4. **LR резко падает с шагом** — через фиксированное число эпох.

Подсказка: Подумайте о классических стратегиях изменения LR.

- **Step decay** — LR резко падает с шагом — через фиксированное число эпох.
- **Exponential decay** — LR уменьшается экспоненциально с каждой итерацией.
- **Cyclical LR** — LR меняется в пределах минимума и максимума циклично.
- **Cosine annealing** — LR уменьшается по косинусной кривой к нулю.

Шаг 07:

Миссия 7

Борис хочет комфортно переключаться между обучением и валидацией. Сопоставьте часть кода и её роль:

Части кода:

Ответ:

- `model.train()`
- `model.eval()`
- `with torch.no_grad()`
- `optimizer.zero_grad()`

Роли:

1. Переключает модель в режим оценки (отключая dropout, batchnorm поведение).
2. Обнуляет накопленные градиенты перед шагом backward.
3. Отключает градиенты, чтобы не считать их на валидации.
4. Переключает модель в режим обучения (включает dropout, batchnorm).

Подсказка: Валидация не требует градиентов, training/ eval переключают поведение модуля.

- **`model.train()`** — Переключает модель в режим обучения (включает dropout, batchnorm).
- **`model.eval()`** — Переключает модель в режим оценки (отключая dropout, batchnorm поведение).
- **`with torch.no_grad()`** — Отключает градиенты, чтобы не считать их на валидации.
- **`optimizer.zero_grad()`** — Обнуляет накопленные градиенты перед шагом backward.

Шаг 08:

Миссия 8

Светлана сталкивается с понятием градиентного клиппинга. Сопоставьте метод и его эффект:

Методы:

Ответ:

- Clipping by value
- Clipping by norm
- Gradient scaling
- Gradient accumulation

Эффекты:

1. Если норма градиента превышает порог, масштабирование всего вектора.
2. Обрезание каждого компонента по фиксированному диапазону.
3. Умножать градиенты на коэффициент для стабилизации.
4. Собирать градиенты из нескольких батчей перед обновлением.

Подсказка: Clipping by norm — про норму, by value — про компонентные пределы, accumulation — про накопление.

- **Clipping by value** — Обрезание каждого компонента по фиксированному диапазону.
- **Clipping by norm** — Если норма градиента превышает порог, масштабирование всего вектора.
- **Gradient scaling** — Умножать градиенты на коэффициент для стабилизации.
- **Gradient accumulation** — Собирать градиенты из нескольких батчей перед обновлением.

Шаг 09:

Миссия 9

Разработчик Игорь хочет контролировать метрики для разных частей модели, например, `loss_feature` и `loss_class`. Сопоставьте концепцию и её описание:

Концепции:

Ответ:

- Multi-loss
- Weighted sum
- Loss balancing
- Auxiliary loss

Описание:

1. Дополнительная функция потерь, помогающая основному обучению.
2. Взвешенное суммирование нескольких потерь.
3. Корректировка весов потерь, чтобы они были сопоставимы по масштабу.
4. Использование сразу нескольких функций потерь.

Подсказка: Multi-loss — общая категория, weighted sum — как комбинируем, balancing — как подгоняем вес, auxiliary — вспомогательная.

- **Multi-loss** — Использование сразу нескольких функций потерь.
- **Weighted sum** — Взвешенное суммирование нескольких потерь.
- **Loss balancing** — Корректировка весов потерь, чтобы они были сопоставимы по масштабу.
- **Auxiliary loss** — Дополнительная функция потерь, помогающая основному обучению.

Шаг 10:

Миссия 10

Кадровый нейроинженер Коля планирует порядок шагов в тренировочном цикле (с использованием mini-batch, валидации и сохранения чекпоинта). Расположите шаги в правильной последовательности:

1. Сохранить checkpoint при улучшении метрики валидации
2. Обнулить градиенты `optimizer.zero_grad()`
3. Пропустить forward + backward + update по каждому батчу
4. Выполнить валидацию модели на валид наборе

5. Инициализация обучения (задание гиперпараметров, модели и optimizer)

Подсказка: Сначала нужно инициализировать, потом проход батчей, затем валидация и возможен чекпоинт.

Ответ:

- Инициализация обучения (задание гиперпараметров, модели и optimizer)
- Обнулить градиенты `optimizer.zero_grad()`
- Пропустить `forward + backward + update` по каждому батчу
- Выполнить валидацию модели на валид наборе
- Сохранить checkpoint при улучшении метрики валидации

Шаг 11:

Миссия 11

Марина работает над циклом, в котором хочет встроить scheduler и логгирование. Расположите этапы в рамках одной эпохи:

1. Обновление lr через `scheduler.step()`
2. Пропуск batch: `forward, loss, backward, optimizer.step()`
3. Логгирование метрик после батча или эпохи
4. Обнулить градиенты `optimizer.zero_grad()`

Подсказка: Помните, что lr меняют после шага optimizer часто либо в конце батча/эпохи.

Ответ:

- Обнулить градиенты `optimizer.zero_grad()`
- Пропуск batch: `forward, loss, backward, optimizer.step()`
- Обновление lr через `scheduler.step()`
- Логгирование метрик после батча или эпохи

Шаг 12:

Миссия 12

Павел интегрирует early stopping, чекпоинты и обучение с накоплением градиентов. Расположите шаги логики внутри контроля цикла:

1. **Если на валид метрике улучшение** — сохранить checkpoint и сбросить счётчик терпения
2. Иначе увеличить счётчик терпения
3. **Если счётчик терпения превысил порог** — остановка обучения
4. Накопление градиентов по батчам и обновление после N батчей
5. Валидация после каждой эпохи

Подсказка: Контроль остановки и чекпоинт — после валидации, accumulation — внутри обучения.

Ответ:

- Валидация после каждой эпохи
- **Если на валид метрике улучшение** — сохранить checkpoint и сбросить счётчик терпения
- Иначе увеличить счётчик терпения
- **Если счётчик терпения превысил порог** — остановка обучения
- Накопление градиентов по батчам и обновление после N батчей

Шаг 13:

Миссия 13

Таня строит пайплайн: загрузка данных, обучение, валидация, тест. Какой правильный порядок операций?

1. Обучение модели (несколько эпох)
2. Тестирование на тестовом наборе
3. Подготовка данных (split, препроцессинг)
4. Валидация после каждой эпохи

Подсказка: Подумайте, что должно готовиться перед обучением, а что после него.

Ответ:

- Подготовка данных (split, препроцессинг)
- Обучение модели (несколько эпох)
- Валидация после каждой эпохи
- Тестирование на тестовом наборе

Шаг 14:

Миссия 14

В сценарии продакшена Лев хочет гибко менять стратегию обучения: сначала фиксированный lr, потом decay, потом остановка. Строим обработку этапов:

1. Применение decay scheduler после каждой эпохи
2. Режим обучения с постоянным lr некоторое время
3. Early stopping по валидации
4. Чекпоинты при улучшении на валидации
5. Полный тренировочный цикл (эпохи и батчи)

Подсказка: Спланируйте логически: сначала стабильный режим, потом decay, затем контроль остановки.

Ответ:

- Режим обучения с постоянным lr некоторое время
- Полный тренировочный цикл (эпохи и батчи)
- Применение decay scheduler после каждой эпохи
- Чекпоинты при улучшении на валидации
- Early stopping по валидации

Шаг 15:

Миссия 15

София планирует интегрировать logging, checkpointing, scheduler и early stopping. Расположите этапы внутри основной обучающей функции:

1. Для каждой эпохи: обучение по батчам
2. Запуск scheduler.step()
3. Валидация и проверка улучшения метрики
4. Сохранение checkpoint, если улучшилась метрика
5. Проверка условия ранней остановки
6. Логирование метрик за эпоху

Подсказка: scheduler часто срабатывает после эпохи, checkpoint и early stopping — после валидации, лог — под конец.

Ответ:

- Для каждой эпохи: обучение по батчам
- Запуск scheduler.step()
- Валидация и проверка улучшения метрики
- Сохранение checkpoint, если улучшилась метрика
- Проверка условия ранней остановки
- Логирование метрик за эпоху

Шаг 16:

Миссия 16

Финальный квест: Влада собирает все элементы: инициализация, обучение, accumulation, scheduler, validation, early stopping, checkpoint, тест. Расположите все шаги в правильном порядке:

1. Подготовка данных и сплит (train / valid / test)
2. Инициализация модели, optimizer и scheduler
3. **Для каждой эпохи:** — обнулить градиенты — накопление градиентов по батчам и update — scheduler.step()
4. Валидация после эпохи
5. Если улучшение: сохранить checkpoint и сбросить терпение
6. **Иначе увеличить счётчик терпения и, если превышен** — остановка
7. После окончания обучения: загрузить лучший checkpoint и протестировать модель

Подсказка: В конце вы должны получить готовую модель, валидирование и тестирование после лучшего состояния.

Ответ:

- Подготовка данных и сплит (train / valid / test)
- Инициализация модели, optimizer и scheduler
- **Для каждой эпохи:** — обнулить градиенты — накопление градиентов по батчам и update — scheduler.step()
- Валидация после эпохи
- Если улучшение: сохранить checkpoint и сбросить терпение
- **Иначе увеличить счётчик терпения и, если превышен** — остановка
- После окончания обучения: загрузить лучший checkpoint и протестировать модель

Урок 4 - Чек-лист инженера

Шаг 01:

Миссия 1

Выстроить приоритет из чек-листа: что первым делом проверит инженер при приёме нейросети?

Подсказка: Начинаем с самого базового — проверить данные и метрики.

Ответ:

- **Качество данных: распределение, пропуски, выбросы**
- Оптимизация GPU-памяти
- Мониторинг latency в продакшене

🔮 Шаг 02:

Миссия 2

✂ Выбор метрики: что чаще всего принимают за «главную» метрику классификации при проверке модели?

Подсказка: Часто выбирают метрику, понятную бизнесу и чувствительную к ошибкам.

Ответ:

- Среднее квадратичное отклонение
- **ROC AUC / ROC-AUC**
- PSNR
- BLEU

🔮 Шаг 03:

Миссия 3

📦 Выбор версии модели: что гарантирует воспроизводимость при деплое?

Подсказка: Важно, чтобы можно было откатиться и пересобрать модель.

Ответ:

- **Фиксация версии модели + контроль зависимостей**
- Непосредственный деплой из локальной ветки без тегов
- Использование latest-тега без версионирования

🔮 Шаг 04:

Миссия 4

🔒 Безопасность: что обязательно должен проверить инженер при работе с API модели?

Подсказка: Не забываем про авторизацию и ограничения доступа.

Ответ:

- Всегда использовать HTTP (не HTTPS)
- **Проверка токенов доступа и прав пользователей**
- Открытый доступ без проверки токенов

🔮 Шаг 05:

Миссия 5

📊 Мониторинг: что из списка наиболее важно отслеживать в продакшн-модели?

Подсказка: Ошибки, drift и производительность — ключевые сигналы.

Ответ:

- **Сдвиг распределения (data drift)**
- **Ошибки и исключения (error rates)**
- Цвет фона веб-интерфейса
- **Latency / время ответа**

🔮 Шаг 06:

Миссия 6

Сопоставьте чек-элементы инженера и их значение:

Чек-элемент:

Ответ:

- Data drift detection
- Логирование предсказаний
- Версионирование модели
- Автотесты на качество

Значение:

1. Уменьшить риск деградации на продакшене
2. Отслеживать ввод и вывод модели
3. Обеспечить контроль изменений модели
4. Гарантировать метрики качества и регрессионный контроль

Подсказка: Подумайте, зачем нужен каждый элемент чек-листа — что он защищает или контролирует.

- **Data drift detection** — Уменьшить риск деградации на продакшене
- **Логирование предсказаний** — Отслеживать ввод и вывод модели
- **Версионирование модели** — Обеспечить контроль изменений модели
- **Автотесты на качество** — Гарантировать метрики качества и регрессионный контроль

 Шаг 07:

Миссия 7

🔍 Сопоставьте понятие с типичной проблемой в продакшене:

Понятия:

Ответ:

- Concept drift
- Model staleness
- Overfitting на тренировочном наборе
- Непредсказуемые входы

Проблемы:

1. Результаты становятся некорректны из-за смещения в целевой функции
2. Модель устаревает по времени
3. Слишком хорошее качество на train, слабое на продакшене
4. Модель не умеет обрабатывать неожиданное распределение входов

Подсказка: Каждый термин — отражение реальной слабости модели в живой системе.

- **Concept drift** — Результаты становятся некорректны из-за смещения в целевой функции
- **Model staleness** — Модель устаревает по времени
- **Overfitting на тренировочном наборе** — Слишком хорошее качество на train, слабое на продакшене
- **Непредсказуемые входы** — Модель не умеет обрабатывать неожиданное распределение входов

 Шаг 08:

Миссия 8

⚙ Сопоставьте компонент системы и его функция в жизненном цикле нейросети:

Компоненты:

Ответ:

- Feature store
- Рабочее окружение (runtime)
- Pipeline преобразования данных
- Мониторинг и алертинг

Функции:

1. Хранение и доставка функций признаков
2. Обеспечение среды для выполнения модели
3. Очистка и подготовка данных к инференсу
4. Обнаружение аномалий, уведомления о сбоях

Подсказка: Каждый компонент имеет свою конкретную роль — функция должна соответствовать компоненту.

- **Feature store** — Хранение и доставка функций признаков
- **Рабочее окружение (runtime)** — Обеспечение среды для выполнения модели
- **Pipeline преобразования данных** — Очистка и подготовка данных к инференсу
- **Мониторинг и алертинг** — Обнаружение аномалий, уведомления о сбоях

 Шаг 09:

Миссия 9

 Расположите этапы перед развёртыванием модели в продакшен:

1. Тестирование на интеграции с API
2. Валидация метрик на отложенной выборке
3. Код ревью и проверка безопасности
4. Сбор контейнера и упаковка модели
5. Запуск тестового деплоя (staging)

Подсказка: Подумайте, что идёт после проверки качества, а что — прямо перед деплоем.

Ответ:

- Валидация метрик на отложенной выборке
- Код ревью и проверка безопасности
- Сбор контейнера и упаковка модели
- Тестирование на интеграции с API
- Запуск тестового деплоя (staging)

 Шаг 10:

Миссия 10

 Расположите контрольные точки при эксплуатации модели:

1. Анализ ошибок и подозрительных случаев
2. Автообучение / дообучение
3. Сбор логов и метрик
4. Обнаружение drift
5. Алерты и уведомления по метрикам

Подсказка: Сначала собираем данные, затем анализируем, дальше — детектируем и реагируем.

Ответ:

- Сбор логов и метрик
- Обнаружение drift
- Алерты и уведомления по метрикам
- Анализ ошибок и подозрительных случаев
- Автообучение / дообучение

 Шаг 11:

Миссия 11

Расположите шаги при откате модели (rollback):

1. Восстановление предыдущей версии модели
2. Переключение трафика на стабильный сервис
3. Уведомление команды об откате
4. Проверка стабильности после отката

Подсказка: Думайте как инженер SRE: сначала откат, потом переключение, затем проверка и уведомление.

Ответ:

- Восстановление предыдущей версии модели
- Переключение трафика на стабильный сервис
- Проверка стабильности после отката
- Уведомление команды об откате

 Шаг 12:

Миссия 12

🕒 Расставьте по порядку шаги реагирования на спад качества модели:

1. Сигнал тревоги по метрикам
2. Диагностика причин спада
3. Решение: дообучение, смена данных, откат и т.д.
4. Мониторинг после изменения

Подсказка: Что делать после тревоги? Сначала разобраться, затем действовать, затем проверить.

Ответ:

- Сигнал тревоги по метрикам
- Диагностика причин спада
- Решение: дообучение, смена данных, откат и т.д.
- Мониторинг после изменения

 Шаг 13:

Миссия 13

🔧 Расположите этапы отладки продакшен-модели:

1. Воспроизведение ошибки локально
2. Сбор контекстных логов на продакшене
3. Корректировка кода/параметров модели
4. Развёртывание исправления

5. Наблюдение за эффектом

Подсказка: Ошибка в продакшене — сначала логируй, потом воспроизводи, правь, деплой и проверяй.

Ответ:

- Сбор контекстных логов на продакшене
- Воспроизведение ошибки локально
- Корректировка кода/параметров модели
- Развёртывание исправления
- Наблюдение за эффектом

Шаг 14:

Миссия 14

 Расположите шаги запуска A/B теста модели:

1. Определение контрольной и экспериментальной групп
2. Выбор метрик и порогов успеха
3. Запуск модели-эксперимента параллельно со старой моделью
4. Сбор статистики и анализ различий
5. Решение: принять, отклонить или смешать модели

Подсказка: A/B тест — это эксперимент, сначала группы, метрики, запуск, анализ, решение.

Ответ:

- Определение контрольной и экспериментальной групп
- Выбор метрик и порогов успеха
- Запуск модели-эксперимента параллельно со старой моделью
- Сбор статистики и анализ различий
- Решение: принять, отклонить или смешать модели

Шаг 15:

Миссия 15

 Расположите действия при масштабировании системы:

1. Профилирование узких мест (CPU, память, I/O)
2. Горизонтальное разделение модели / шардирование
3. Кеширование результатов инференса
4. Балансировка нагрузки
5. Мониторинг масштабируемости

Подсказка: Сначала найти узкое место, затем масштабировать, ретушировать и мониторить.

Ответ:

- Профилирование узких мест (CPU, память, I/O)
- Горизонтальное разделение модели / шардирование
- Кеширование результатов инференса
- Балансировка нагрузки
- Мониторинг масштабируемости

Шаг 16:

Миссия 16

Расположите уровни логирования в порядке от самого детального до минимального:

1. DEBUG
2. INFO
3. WARNING
4. ERROR
5. CRITICAL

Подсказка: DEBUG содержит максимум информации, CRITICAL — только серьёзные сбои.

Ответ:

- DEBUG
- INFO
- WARNING
- ERROR
- CRITICAL

Шаг 17:

Миссия 17

 Расположите слои архитектуры микросервиса нейросети (от пользователя до модели):

1. Клиент / фронтенд
2. API Gateway / маршрутизация
3. Сервис управления версиями модели
4. Сервис инференса / модель
5. Хранилище логов / мониторинг

Подсказка: Сначала пользователь, потом запрос идёт через API, затем выбор версии, инференс и запись логов.

Ответ:

- Клиент / фронтенд
- API Gateway / маршрутизация
- Сервис управления версиями модели
- Сервис инференса / модель
- Хранилище логов / мониторинг

Урок 5 - Настройка среды Docker, Conda, Jupyter

Шаг 01:

Миссия 1

 Анна запускает новый ноутбук и хочет установить окружение для экспериментов. Что первым делом ей стоит сделать?

Подсказка: Без управления пакетами начинается хаос.

Ответ:

- **Создать и активировать Conda-окружение**
- Сразу установить все пакеты в системный Python
- Запустить Docker без настроек

🔧 Шаг 02:

Миссия 2

Борис хочет запустить Jupyter Notebook в Docker для проекта с зависимостями. Какой базовый образ он выберет?

Подсказка: Ищите готовый образ с поддержкой Jupyter и Python.

Ответ:

- **jupyter/base-notebook**
- ubuntu:latest
- tensorflow/tensorflow:latest-gpu

🔧 Шаг 03:

Миссия 3

🔗 В контейнере модель выдает ошибку из-за отсутствия библиотеки. Что нужно добавить в Dockerfile?

Подсказка: Убедитесь, что все зависимости указаны.

Ответ:

- **RUN pip install -r requirements.txt**
- RUN rm -rf /var/lib/apt/lists/*
- EXPOSE 80

🔧 Шаг 04:

Миссия 4

📁 Катя хочет использовать локальное Jupyter внутри Docker и маппить папку проекта. Какой флаг Docker ей понадобится?

Подсказка: Флаг, позволяющий связывать локальную папку с контейнером.

Ответ:

- **-v /path/on/host:/app**
- --network host
- --rm

🔧 Шаг 05:

Миссия 5

🔍 Сопоставьте команды Conda и их назначение:

Команды:

Ответ:

- conda create
- conda activate
- conda list
- conda env export

Назначения:

1. Просмотреть установленные пакеты
2. Создать новое окружение
3. Активировать окружение

4. Сохранить файл зависимостей (environment.yml)

Подсказка: Подумайте, какие команды чаще всего применяют для обмена окружениями.

- **conda create** — Создать новое окружение
- **conda activate** — Активировать окружение
- **conda list** — Просмотреть установленные пакеты
- **conda env export** — Сохранить файл зависимостей (environment.yml)

Шаг 06:

Миссия 6

Сопоставьте инструкции Dockerfile с их ролью:

Инструкции:

Ответ:

- FROM
- COPY
- RUN
- CMD

Роли:

1. Указывает базовый образ
2. Копирует файлы из локальной файловой системы в контейнер
3. Выполняет команды при сборке образа
4. Задаёт команду по умолчанию при запуске контейнера

Подсказка: FROM — начало, RUN — сборка, CMD — запуск.

- **FROM** — Указывает базовый образ
- **COPY** — Копирует файлы из локальной файловой системы в контейнер
- **RUN** — Выполняет команды при сборке образа
- **CMD** — Задаёт команду по умолчанию при запуске контейнера

Шаг 07:

Миссия 7

 Сопоставьте виды портов и их описания при запуске Jupyter в контейнере:

Тип порта:

Ответ:

- 8888:8888
- 0.0.0.0:8888
- 127.0.0.1:8888
- --network host

Описание:

1. Привязка к localhost на хосте
2. Доступ со всех интерфейсов на хосте
3. Проброс порта контейнера к хосту
4. Использовать сеть хоста без проброса портов

Подсказка: 8888:8888 — классический проброс, host — без проброса.

- **8888:8888** — Проброс порта контейнера к хосту
- **0.0.0.0:8888** — Доступ со всех интерфейсов на хосте
- **127.0.0.1:8888** — Привязка к localhost на хосте
- **--network host** — Использовать сеть хоста без проброса портов

🔮 Шаг 08:

Миссия 8

🔧 Сопоставьте команды Docker и их описание:

Команды:

Ответ:

- docker build
- docker run
- docker ps
- docker exec

Описание:

1. Запустить контейнер из образа
2. Построить Docker-образ по Dockerfile
3. Список запущенных контейнеров
4. Выполнить команду внутри запущенного контейнера

Подсказка: build сначала, run запускает, ps показывает, exec — внутри контейнера.

- **docker build** — Построить Docker-образ по Dockerfile
- **docker run** — Запустить контейнер из образа
- **docker ps** — Список запущенных контейнеров
- **docker exec** — Выполнить команду внутри запущенного контейнера

🔮 Шаг 09:

Миссия 9

🔧 Расположите шаги создания Docker-контейнера с Jupyter и зависимостями:

1. Написать Dockerfile
2. Собрать образ
3. Запустить контейнер с маппингом тома
4. Проверить, что Jupyter доступен в браузере
5. Установить kernel для Jupyter (внутри контейнера)

Подсказка: От пишем → строим → запускаем → проверяем → настраиваем kernel.

Ответ:

- Написать Dockerfile
- Собрать образ
- Запустить контейнер с маппингом тома
- Установить kernel для Jupyter (внутри контейнера)
- Проверить, что Jupyter доступен в браузере

🔮 Шаг 10:

Миссия 10

🔗 Расположите шаги отладки контейнера, если Jupyter не запускается:

1. Посмотреть логи контейнера (docker logs)
2. Проверить Dockerfile на ошибки синтаксиса
3. Запустить контейнер с shell и вручную выполнить jupyter
4. Проверить, что порты проброшены правильно
5. Проверить, что зависимости установлены

Подсказка: Логги → порт → зависимости → ручной запуск → исправление Dockerfile.

Ответ:

- Посмотреть логи контейнера (docker logs)
- Проверить Dockerfile на ошибки синтаксиса
- Запустить контейнер с shell и вручную выполнить jupyter
- Проверить, что порты проброшены правильно
- Проверить, что зависимости установлены

💡 Шаг 11:

Миссия 11

🔗 Расположите шаги обновления образа при изменении зависимостей:

1. Изменить environment.yml или requirements.txt
2. Обновить Dockerfile (если нужно)
3. Пересобрать образ
4. Запустить новый контейнер для тестирования
5. Замена старого контейнера новым (rolling update)

Подсказка: Изменили зависимости → пересобрали → протестировали → обновили сервис.

Ответ:

- Изменить environment.yml или requirements.txt
- Обновить Dockerfile (если нужно)
- Пересобрать образ
- Запустить новый контейнер для тестирования
- Замена старого контейнера новым (rolling update)

💡 Шаг 12:

Миссия 12

🌐 Расположите уровни взаимодействия между Jupyter, Docker и хостом:

1. Пользователь в браузере
2. Jupyter внутри контейнера
3. Port binding Docker (хост ↔ контейнер)
4. Файлы проекта на хосте (мапинг тома)
5. Среди них kernel, зависимости и расширения

Подсказка: Пользователь → порт → контейнер → файлы → зависимости.

Ответ:

- Пользователь в браузере
- Jupyter внутри контейнера
- Port binding Docker (хост ↔ контейнер)

- Файлы проекта на хосте (мапинг тома)
- Среди них kernel, зависимости и расширения

🔮 Шаг 13:

Миссия 13

🔒 Расположите меры безопасности при запуске Jupyter в Docker:

1. Установка пароля / токена доступа
2. Ограничение интерфейсов (127.0.0.1 vs 0.0.0.0)
3. Ограничение прав пользователя внутри контейнера
4. Необходимость HTTPS / шифрование
5. Ограничение доступа по сети или VPN

Подсказка: Защита всегда многоуровневая.

Ответ:

- Установка пароля / токена доступа
- Ограничение интерфейсов (127.0.0.1 vs 0.0.0.0)
- Ограничение прав пользователя внутри контейнера
- Необходимость HTTPS / шифрование
- Ограничение доступа по сети или VPN

🔮 Шаг 14:

Миссия 14

Расположите шаги настройки kernel Jupyter в Docker для проекта:

1. Установить ipykernel в окружении контейнера
2. Запустить `python -m ipykernel install`
3. Назначить имя ядра (`-name`)
4. Перезапустить Jupyter / ядро распознано
5. Выбрать ядро в интерфейсе Notebook

Подсказка: Сначала установить библиотеку, затем зарегистрировать kernel, потом перезапуск и выбор.

Ответ:

- Установить ipykernel в окружении контейнера
- Запустить `python -m ipykernel install`
- Назначить имя ядра (`-name`)
- Перезапустить Jupyter / ядро распознано
- Выбрать ядро в интерфейсе Notebook

🔮 Шаг 15:

Миссия 15

⚙️ Финал: расположите полный рабочий цикл от локальной разработки до запуска Jupyter в Docker:

1. Создать и настроить Conda-окружение локально
2. Экспортировать зависимости (`environment.yml` или `requirements.txt`)
3. Подготовить Dockerfile с зависимостями и Jupyter
4. Собрать и протестировать Docker-образ локально

5. Запустить контейнер с пробросом томов и портов
6. Подключиться к Jupyter из браузера и проверить ядро

Подсказка: Последовательность должна быть логичной: среда → зависимости → контейнер → запуск → проверка.

Ответ:

- Создать и настроить Conda-окружение локально
- Экспортировать зависимости (environment.yml или requirements.txt)
- Подготовить Dockerfile с зависимостями и Jupyter
- Собрать и протестировать Docker-образ локально
- Запустить контейнер с пробросом томов и портов
- Подключиться к Jupyter из браузера и проверить ядро

Модуль 2 - Строим своё ядро

Урок 1 - ⚙ Низкоуровневое API PyTorch и TensorFlow

Шаг 01:

Миссия 1

Алексей сидит перед ноутбуком и хочет создать тензор из списка Python для дальнейших вычислений в PyTorch. Что он должен сделать?

Подсказка: В PyTorch тензоры создаются через класс torch.Tensor или фабричные функции.

Ответ:

- **torch.tensor([1.0, 2.0, 3.0])**
- [1.0, 2.0, 3.0] + 5
- np.array([1.0, 2.0]) + torch

Шаг 02:

Миссия 2

Марина работает с TensorFlow и хочет включить автоматическое дифференцирование градиентов для обучения модели. Что нужно включить?

Подсказка: В TensorFlow вы используете GradientTape для записи операций.

Ответ:

- **tf.GradientTape()** как контекстный менеджер
- tf.no_grad()
- disable_gradients()

Шаг 03:

Миссия 3

Иван готовит данные и хочет переместить тензоры на GPU в PyTorch. Какой метод использовать?

Подсказка: У тензоров есть метод to(device).

Ответ:

- **tensor.to('cuda')**
- tensor.gpu_move()
- tensor.cuda_move()

🔮 Шаг 04:

Миссия 4

Ольга хочет вычислить среднее значение элементов тензора в TensorFlow. Что применить?

Подсказка: `tf.reduce_mean` подходит для агрегаций по тензорам.

Ответ:

- **`tf.reduce_mean(tensor)`**
- `tensor.mean_py()`
- `tensor.aggregate()`

🔮 Шаг 05:

Миссия 5

Павел реализует ручной обратный проход в PyTorch для простого скаляра. Какой тензор нужно пометить для градиентов?

Подсказка: В PyTorch используется параметр `requires_grad`.

Ответ:

- **`torch.tensor(3.0, requires_grad=True)`**
- `torch.tensor(3.0).requires_grad()`
- `torch.tensor(3.0).grad=True`

🔮 Шаг 06:

Миссия 6

☞ Сопоставьте низкоуровневые операции и их эквиваленты в PyTorch / TensorFlow:

Операции:

Ответ:

- Матрица умножения (PyTorch)
- Матрица умножения (TensorFlow)
- Выбор максимума по оси
- Сумма по всем элементам

Эквиваленты:

1. `tf.reduce_sum(x, axis=...)`
2. `torch.matmul(a, b)`
3. `tf.matmul(a, b)`
4. `tf.reduce_max(x, axis=...)`
5. `torch.sum(x)`
6. `torch.max(x, dim=...)`

Подсказка: Обратите внимание на именования функций и аргументы в библиотеках.

- **Матрица умножения (PyTorch)** — `torch.matmul(a, b)`
- **Матрица умножения (TensorFlow)** — `tf.matmul(a, b)`
- **Сумма по всем элементам** — `torch.sum(x)`
- **Выбор максимума по оси** — `tf.reduce_max(x, axis=...)`

🔮 Шаг 07:

Миссия 7

🔍 Сопоставьте режимы с поведением в обучении и инференсе:

Режимы:

Ответ:

- PyTorch режим `train()`
- PyTorch режим `eval()`
- TensorFlow в режиме тренировки (`training=True`)
- TensorFlow в режиме оценки (`training=False`)

Поведение:

1. Отключение Dropout и BatchNorm неактивны
2. Включение Dropout, BatchNorm обучает статистику
3. Используется поведение как при обучении
4. Используется поведение как при инференсе

Подсказка: Подумайте, как себя ведут слои Dropout и BatchNorm в разных режимах.

- **PyTorch режим `train()`** — Включение Dropout, BatchNorm обучает статистику
- **PyTorch режим `eval()`** — Отключение Dropout и BatchNorm неактивны
- **TensorFlow `training=True`** — Используется поведение как при обучении
- **TensorFlow `training=False`** — Используется поведение как при инференсе

🔧 Шаг 08:

Миссия 8

Сопоставьте функции оптимизации с их поведением:

Функции:

Ответ:

- SGD
- Adam
- RMSProp
- AdaGrad

Характеристики:

1. Адаптивный learning rate с накоплением градиентов
2. Простой стохастический градиентный спуск
3. Использует моментум и адаптивный шаг
4. Учитывает квадраты градиентов для адаптации шага

Подсказка: Подумайте о ключевых характеристиках Adam: моментум + адаптивный шаг.

- **SGD** — Простой стохастический градиентный спуск
- **Adam** — Использует моментум и адаптивный шаг
- **RMSProp** — Учитывает квадраты градиентов для адаптации шага
- **AdaGrad** — Адаптивный learning rate с накоплением градиентов

🔧 Шаг 09:

Миссия 9

🧠 В проектах часто нужно сопоставлять операции автограда. Сопоставьте:

Операция:

Ответ:

- `backward()`
- `zero_grad()`
- `step()`
- `clip_grad_norm_()`

Назначение:

1. Обнуляет накопленные градиенты
2. Ограничивает норму градиентов
3. Выполняет шаг оптимизации
4. Запускает обратное распространение градиентов

Подсказка: Помните порядок: `backward` → `clip` → `step` → `zero`.

- **`backward()`** — Запускает обратное распространение градиентов
- **`zero_grad()`** — Обнуляет накопленные градиенты
- **`step()`** — Выполняет шаг оптимизации
- **`clip_grad_norm_()`** — Ограничивает норму градиентов

 Шаг 10:

Миссия 10

Ленка планирует цикл обучения модели. Упорядочьте шаги в правильной последовательности:

1. `optimizer.step()`
2. `loss.backward()`
3. `optimizer.zero_grad()`
4. подать батч данных, вычислить предсказание и `loss`

Подсказка: Сначала очистка градиентов, затем `forward` + `loss`, затем `backward` и шаг оптимизации.

Ответ:

- `optimizer.zero_grad()`
- подать батч данных, вычислить предсказание и `loss`
- `loss.backward()`
- `optimizer.step()`

 Шаг 11:

Миссия 11

✂ Василий отлаживает смешанный код PyTorch + NumPy. Нужно правильно располагать операции:

1. преобразовать NumPy в тензор
2. выполнить вычисления в PyTorch с градиентами
3. преобразовать результат в NumPy для визуализации
4. включить `requires_grad`, если нужно градиенты

Подсказка: Сначала настройка тензора, затем вычисления, а уже потом преобразование обратно.

Ответ:

- преобразовать NumPy в тензор

- включить `requires_grad`, если нужно градиенты
- выполнить вычисления в PyTorch с градиентами
- преобразовать результат в NumPy для визуализации

🔧 Шаг 12:

Миссия 12

🔧 В команде нужно реализовать ручное вычисление градиентов: упорядочите шаги:

1. обновить параметры: `w = w - lr * grad_w`
2. пройти forward и вычислить loss
3. обнулить градиенты
4. вызвать `backward()` на loss

Подсказка: Убедитесь, что вы сначала очищаете старые градиенты, затем вычисляете, затем обновляете.

Ответ:

- обнулить градиенты
- пройти forward и вычислить loss
- вызвать `backward()` на loss
- обновить параметры: `w = w - lr * grad_w`

🔧 Шаг 13:

Миссия 13

🧠 Инна готовит pipeline вывода модели: упорядочите этапы:

1. вызвать `model.eval()`
2. перевести вход на устройство GPU/CPU
3. получить предсказания через `model(input)`
4. превратить предсказания в NumPy / список

Подсказка: Для инференса сначала переключитесь в режим оценки, затем подайте вход.

Ответ:

- вызвать `model.eval()`
- перевести вход на устройство GPU/CPU
- получить предсказания через `model(input)`
- превратить предсказания в NumPy / список

🔧 Шаг 14:

Миссия 14

♥ Алекс решает реализовать обучение с градиентным клиппингом. Упорядочите шаги:

1. `loss.backward()`
2. `clip_grad_norm_(model.parameters(), max_norm)`
3. `optimizer.step()`
4. `optimizer.zero_grad()`

Подсказка: Ни шаг оптимизации, ни очистка градиентов не должны происходить раньше, чем вы ограничите градиенты.

Ответ:

- `optimizer.zero_grad()`
- `loss.backward()`
- `clip_grad_norm_(model.parameters(), max_norm)`
- `optimizer.step()`

🔮 **Шаг 15:**

Миссия 15

🔮 В продакшен системе Алиса хочет использовать JIT-компиляцию в PyTorch. Какой способ выбрать?

Подсказка: TorchScript через `@torch.jit.script` или `torch.jit.trace`.

Ответ:

- **`torch.jit.script(model)` или `torch.jit.trace(model, example_inputs)`**
- `tf.function(model)`
- `use_gpu_jit()`

🔮 **Шаг 16:**

Миссия 16

🌐 Команда готовит модель TensorFlow к экспорту и хочет преобразовать её в формат, подходящий для обслуживания (serving). Что выбрать?

Подсказка: SavedModel — стандартный формат для сервиса TensorFlow.

Ответ:

- **`tf.saved_model.save(model, path)`**
- `model.to_serving_binary()`
- `model.export_serving()`

📖 Урок 2 - 🔁 Forward-pass вручную

🔮 **Шаг 01:**

Миссия 1

Алексей только начал писать собственный forward-pass для нейросети. Что является первым шагом в реализации прямого прохода?

Подсказка: Начинайте с самой базовой операции — как данные проходят через слой.

Ответ:

- **Умножение входных данных на веса слоя**
- Инициализация весов случайными числами
- Подключение функции потерь
- Оптимизация гиперпараметров

🔮 **Шаг 02:**

Миссия 2

🔮 Оля добавляет активацию в forward-pass. Какой тип функции активации чаще всего используется в современных нейросетях?

Подсказка: Подумайте о функции, которая обнуляет отрицательные значения и пропускает положительные.

Ответ:

- **ReLU (Rectified Linear Unit)**
- Sigmoid
- Tanh
- Softmax

Шаг 03:

Миссия 3

🔑 Иван реализует forward-pass для полносвязного слоя. Что нужно сделать после умножения входа на веса и добавления смещения?

Подсказка: Обычно на этом этапе применяется нелинейная функция.

Ответ:

- **Применить функцию активации**
- Вычислить градиенты
- Обновить веса
- Сохранить выходы для backpropagation

Шаг 04:

Миссия 4

🔑 Михаил сопоставляет части кода и их назначение в forward-pass. Соотнесите элементы и их функции.

Подсказка: Подумайте, что делает каждая часть кода в рамках прямого прохода.

Элементы:

Ответ:

- Матрица весов
- Входной вектор
- Функция активации
- Смещение (bias)

Назначения:

1. Добавляет сдвиг для увеличения выразительности модели
2. Передаёт сигнал следующему слою
3. Весовые коэффициенты для связи входов и выходов
4. Нелинейная трансформация результата

- **Матрица весов** — Весовые коэффициенты для связи входов и выходов
- **Входной вектор** — Передаёт сигнал следующему слою
- **Функция активации** — Нелинейная трансформация результата
- **Смещение (bias)** — Добавляет сдвиг для увеличения выразительности модели

Шаг 05:

Миссия 5

✂ Катя работает с forward-pass и хочет сопоставить этапы вычислений и их описания.

Подсказка: Каждый этап важен для корректной работы слоя.

Этапы:

Ответ:

- 1. Умножение входа на веса
- 2. Добавление смещения
- 3. Применение активации
- 4. Передача результата дальше

Описание:

1. Обеспечивает линейную трансформацию входных данных
 2. Обеспечивает сдвиг для более гибкой модели
 3. Вводит нелинейность для обучения сложных зависимостей
 4. Результат слоя становится входом для следующего слоя
- **1. Умножение входа на веса** — Обеспечивает линейную трансформацию входных данных
 - **2. Добавление смещения** — Обеспечивает сдвиг для более гибкой модели
 - **3. Применение активации** — Вводит нелинейность для обучения сложных зависимостей
 - **4. Передача результата дальше** — Результат слоя становится входом для следующего слоя

 Шаг 06:

Миссия 6

✳ Денис сопоставляет функции активации и их основные свойства.

Подсказка: Вспомните, какие функции дают выход в диапазоне $[0,1]$, какие — $[-1,1]$, а какие — обрезают отрицательные значения.

Функции активации:

Ответ:

- Sigmoid
- Tanh
- ReLU
- Leaky ReLU

Свойства:

1. Выход в диапазоне $[0,1]$
 2. Выход в диапазоне $[-1,1]$
 3. Обнуляет отрицательные значения
 4. Слабый положительный наклон для отрицательных значений
- **Sigmoid** — Выход в диапазоне $[0,1]$
 - **Tanh** — Выход в диапазоне $[-1,1]$
 - **ReLU** — Обнуляет отрицательные значения
 - **Leaky ReLU** — Слабый положительный наклон для отрицательных значений

 Шаг 07:

Миссия 7

🔍 Лена сопоставляет термины, связанные с forward-pass, и их определения.

Подсказка: Вспомните базовые понятия нейросетей и как они влияют на процесс обучения.

Термины:

Ответ:

- Forward-pass
- Весовые коэффициенты
- Активация
- Смещение (bias)

Определения:

1. Процесс прохождения данных через сеть от входа к выходу
 2. Параметры, которые сеть обучается изменять для лучшего результата
 3. Функция, добавляющая нелинейность к выходу слоя
 4. Дополнительное значение, позволяющее сдвигать активацию
- **Forward-pass** — Процесс прохождения данных через сеть от входа к выходу
 - **Весовые коэффициенты** — Параметры, которые сеть обучается изменять для лучшего результата
 - **Активация** — Функция, добавляющая нелинейность к выходу слоя
 - **Смещение (bias)** — Дополнительное значение, позволяющее сдвигать активацию

 Шаг 08:

Миссия 8

 Сергей сопоставляет виды слоев и операции в forward-pass.

Подсказка: Подумайте, как меняются операции для разных типов слоев.

Слои:

Ответ:

- Полносвязный слой (Dense)
- Сверточный слой (Conv)
- Слой нормализации (BatchNorm)
- Слой активации

Операции:

1. Выполняет свертку входного тензора с фильтрами
 2. Умножение входов на веса и суммирование
 3. Применение нелинейной функции
 4. Нормализация выходов слоя для ускорения обучения
- **Полносвязный слой (Dense)** — Умножение входов на веса и суммирование
 - **Сверточный слой (Conv)** — Выполняет свертку входного тензора с фильтрами
 - **Слой нормализации (BatchNorm)** — Нормализация выходов слоя для ускорения обучения
 - **Слой активации** — Применение нелинейной функции

 Шаг 09:

Миссия 9

 Максим работает над собственным forward-pass. Расположите шаги вычисления выхода полносвязного слоя в правильном порядке:

1. Добавление смещения (bias)
2. Применение функции активации
3. Умножение входных данных на веса

4. Передача результата следующему слою

Подсказка:

Обратите внимание, что сначала происходит линейная комбинация, затем добавляется смещение, потом функция активации, и только после этого результат уходит дальше.

Ответ:

- Умножение входных данных на веса
- Добавление смещения (bias)
- Применение функции активации
- Передача результата следующему слою

Шаг 10:

Миссия 10

✂ Вера пишет forward-pass для нейросети с несколькими слоями. Расположите действия по порядку при вычислении выхода на слое с активацией:

1. Передача результата активации дальше
2. Суммирование произведений входа и веса с bias
3. Умножение входных данных на матрицу весов
4. Применение функции активации

Подсказка:

Линейная часть (умножение и суммирование с bias) всегда происходит до активации.

Ответ:

- Умножение входных данных на матрицу весов
- Суммирование произведений входа и веса с bias
- Применение функции активации
- Передача результата активации дальше

Шаг 11:

Миссия 11

✂ Андрей разрабатывает forward-pass с несколькими слоями. Расположите шаги вычисления результата для сверточного слоя:

1. Добавление смещения к результату свертки
2. Передача результата сверточного слоя дальше
3. Выполнение свертки входных данных с ядрами фильтров
4. Применение функции активации

Подсказка:

Сначала идет свертка, потом bias, активация и дальше передача.

Ответ:

- Выполнение свертки входных данных с ядрами фильтров
- Добавление смещения к результату свертки
- Применение функции активации
- Передача результата сверточного слоя дальше

Шаг 12:

Миссия 12

Ксения реализует forward-pass с batch normalization. Расположите действия по порядку:

1. Вычисление среднего и дисперсии по батчу
2. Нормализация входов слоя
3. Применение функции активации
4. Передача нормализованных и активированных данных дальше

 Подсказка:

BatchNorm сначала вычисляет статистики, затем нормализует, после чего следует активация и передача.

Ответ:

- Вычисление среднего и дисперсии по батчу
- Нормализация входов слоя
- Применение функции активации
- Передача нормализованных и активированных данных дальше

 Шаг 13:

Миссия 13

 Дмитрий пишет собственный forward-pass. Расположите этапы обработки данных в сверточном слое с последующим применением ReLU:

1. Передача результата дальше
2. Добавление bias
3. Выполнение свертки
4. Применение ReLU

 Подсказка:

Логика та же, что и в Dense-слое: сначала линейное преобразование, затем активация, потом выход.

Ответ:

- Выполнение свертки
- Добавление bias
- Применение ReLU
- Передача результата дальше

 Шаг 14:

Миссия 14

★ Олег проверяет порядок действий в forward-pass с функцией Softmax на выходном слое. Расположите этапы:

1. Вычисление экспоненты для каждого элемента
2. Суммирование экспонент
3. Деление экспоненты каждого элемента на сумму
4. Передача результатов дальше

 Подсказка:

Softmax сначала преобразует выходы в положительные значения, затем нормирует их, чтобы получить вероятности.

Ответ:

- Вычисление экспоненты для каждого элемента

- Суммирование экспонент
- Деление экспоненты каждого элемента на сумму
- Передача результатов дальше

🔮 Шаг 15:

Миссия 15

Вы помогаете программисту Алексею понять, какой слой нейросети отвечает за активацию нейронов после линейной комбинации входов. Что это за слой?

Подсказка: Подумайте, какой слой добавляет нелинейность в модель.

Ответ:

- **Активационный слой (activation layer)**
- Слой нормализации (normalization layer)
- Слой свёртки (convolutional layer)
- Входной слой (input layer)

🔮 Шаг 16:

Миссия 16

Инженер Марина строит forward-pass нейросети. Помогите ей сопоставить шаги с их описаниями:

Шаги:

Ответ:

- Линейное преобразование
- Применение активации
- Обработка входных данных
- Передача результата на следующий слой

Описания:

1. Складываем входы с весами и прибавляем смещение.
2. Преобразуем данные с помощью нелинейной функции.
3. Получение данных из предыдущего слоя или изначальных данных.
4. Передаем вычисленный вектор дальше по сети.

Подсказка: Запомните порядок операций в прямом проходе.

- **Линейное преобразование** — Складываем входы с весами и прибавляем смещение.
- **Применение активации** — Преобразуем данные с помощью нелинейной функции.
- **Обработка входных данных** — Получение данных из предыдущего слоя или изначальных данных.
- **Передача результата на следующий слой** — Передаем вычисленный вектор дальше по сети.

🔮 Шаг 17:

Миссия 17

Алексей проектирует слой нейросети. Сопоставьте элементы вычисления forward-pass и их роль:

Элементы:

Ответ:

- Вес (weight)
- Смещение (bias)
- Активация (activation)
- Вход (input)

Роли:

1. Позволяет сдвинуть функцию и повысить гибкость модели.
2. Множитель входного сигнала для влияния на результат.
3. Нелинейная функция, которая помогает модели решать сложные задачи.
4. Начальные данные для вычислений слоя.

Подсказка: Вспомните, как каждый элемент влияет на выход слоя.

- **Вес (weight)** — Множитель входного сигнала для влияния на результат.
- **Смещение (bias)** — Позволяет сдвинуть функцию и повысить гибкость модели.
- **Активация (activation)** — Нелинейная функция, которая помогает модели решать сложные задачи.
- **Вход (input)** — Начальные данные для вычислений слоя.

 Шаг 18:

Миссия 18

Разработчик Катя хочет понять, в каком порядке выполняются шаги forward-pass в нейросети. Расположите их правильно:

1. Применение активационной функции
2. Получение входных данных
3. Вычисление линейной комбинации (умножение на веса и добавление смещения)
4. Передача результата следующему слою

Подсказка: Подумайте, что происходит сначала, а что — в конце прямого прохода.

Ответ:

- Получение входных данных
- Вычисление линейной комбинации (умножение на веса и добавление смещения)
- Применение активационной функции
- Передача результата следующему слою

 Урок 3 - Расчёт градиентов руками

 Шаг 01:

Миссия 1

Вы как инженер-нейросетей решили вручную просчитать градиент функции потерь $L = (\hat{y} - y)^2$ для одного примера. Что будет выражением $dL/d\hat{y}$?

Подсказка: Используйте цепное правило: $L = (\hat{y} - y)^2 = u^2$, где $u = \hat{y} - y$.

Ответ:

- $2 \cdot (\hat{y} - y)$
- $(\hat{y} - y)^2$
- $-2 \cdot (\hat{y} - y)$
- $\hat{y} - y$

🔮 Шаг 02:

Миссия 2

У персонажа Алисы есть нейрон с активацией $\sigma(z) = 1/(1 + e^{-z})$. В ручном градиенте, какую форму должен принять $d\sigma/dz$?

Подсказка: Помните известную формулу для сигмоиды: $\sigma' = \sigma \cdot (1 - \sigma)$.

Ответ:

- $\sigma(z) \cdot (1 - \sigma(z))$
- $\sigma(z)^2$
- $1/(1 + e^{-z})^2$
- $1 - \sigma(z)$

🔮 Шаг 03:

Миссия 3

Разработчик Борис вручную считает градиент веса w в линейном слое: $z = w \cdot x + b$, $L = (\sigma(z) - y)^2$. Самый первый шаг — что нужно найти в цепочке?

Подсказка: Разбейте $dL/dw = (dL/d\sigma) \cdot (d\sigma/dz) \cdot (dz/dw)$.

Ответ:

- $dL/d\sigma$
- $d\sigma/dz$
- dz/dw
- dL/dz

🔮 Шаг 04:

Миссия 4

Вы посчитали $dL/d\sigma = 2 \cdot (\sigma(z) - y)$. Что это выражение означает в контексте ошибки?

Подсказка: Это производная квадратичной ошибки по предсказанию σ .

Ответ:

- **Отклонение сигмоиды от истинного y , умноженное на 2**
- Сигмоида сама по себе
- Квадрат ошибки
- Нулевая величина

🔮 Шаг 05:

Миссия 5

🔗 Сопоставьте элементы цепного правила с их значениями в выражении $dL/dw = (dL/d\sigma) \cdot (d\sigma/dz) \cdot (dz/dw)$.

Элементы цепочки:

Ответ:

- $dL/d\sigma$
- $d\sigma/dz$
- dz/dw

Выражения:

1. $2 \cdot (\sigma(z) - y)$
2. $\sigma(z) \cdot (1 - \sigma(z))$

3. x

Подсказка: dz/dw — это просто x , т.к. $z = w \cdot x + b$.

- $dL/d\sigma = 2 \cdot (\sigma(z) - y)$
- $d\sigma/dz = \sigma(z) \cdot (1 - \sigma(z))$
- $dz/dw = x$

🔮 Шаг 06:

Миссия 6

🔑 Теперь сопоставьте компоненты полного градиента с частичным выражением:

Ответ:

- $dL/d\sigma \cdot d\sigma/dz \cdot dz/dw$
- $dL/d\sigma \cdot d\sigma/dz$
- $d\sigma/dz \cdot dz/dw$

Части формулы:

1. $2 \cdot (\sigma(z) - y) \cdot \sigma(z) \cdot (1 - \sigma(z)) \cdot x$
2. $2 \cdot (\sigma(z) - y) \cdot \sigma(z) \cdot (1 - \sigma(z))$
3. $\sigma(z) \cdot (1 - \sigma(z)) \cdot x$

Подсказка: Полный градиент — это произведение всех трёх частей.

- $dL/d\sigma \cdot d\sigma/dz \cdot dz/dw = 2 \cdot (\sigma(z) - y) \cdot \sigma(z) \cdot (1 - \sigma(z)) \cdot x$
- $dL/d\sigma \cdot d\sigma/dz = 2 \cdot (\sigma(z) - y) \cdot \sigma(z) \cdot (1 - \sigma(z))$
- $d\sigma/dz \cdot dz/dw = \sigma(z) \cdot (1 - \sigma(z)) \cdot x$

🔮 Шаг 07:

Миссия 7

🔑 Сопоставьте, для градиента по смещению b (bias), какая часть аналогична чему:

Ответ:

- $dL/d\sigma$
- $d\sigma/dz$
- dz/db

Выражения:

1. $2 \cdot (\sigma(z) - y)$
2. $\sigma(z) \cdot (1 - \sigma(z))$
3. 1

Подсказка: $z = w \cdot x + b$, какова производная по b ?

- $dL/d\sigma = 2 \cdot (\sigma(z) - y)$
- $d\sigma/dz = \sigma(z) \cdot (1 - \sigma(z))$
- $dz/db = 1$

🔮 Шаг 08:

Миссия 8

🧠 Расположите шаги ручного вычисления градиентов для одного параметра w в верном порядке:

1. Подставить значения $dL/d\sigma$, $d\sigma/dz$, dz/dw и умножить
2. Найти $dL/d\sigma = \partial L/\partial \sigma$
3. Найти $d\sigma/dz$
4. Найти dz/dw
5. Вычислить итоговый dL/dw

Подсказка: Сначала отделите части цепочки, затем объедините.

Ответ:

- Подставить значения $dL/d\sigma$, $d\sigma/dz$, dz/dw и умножить
- Найти $dL/d\sigma = \partial L/\partial \sigma$
- Найти $d\sigma/dz$
- Найти dz/dw
- Вычислить итоговый dL/dw

🔗 Шаг 09:

Миссия 9

📐 Теперь усложним: у нас две точки (x_1, y_1) и (x_2, y_2) . Расположите шаги усреднённого градиента в порядке:

1. Сложить индивидуальные dL/dw от каждой точки
2. Вычислить градиент для каждого примера
3. Разделить сумму на число примеров
4. Вернуть усреднённый dL/dw

Подсказка: Сложите, затем усредните.

Ответ:

- Сложить индивидуальные dL/dw от каждой точки
- Вычислить градиент для каждого примера
- Разделить сумму на число примеров
- Вернуть усреднённый dL/dw

🔗 Шаг 10:

Миссия 10

🔄 Многослойная сеть: у вас два слоя (вход → скрытый → выход). Расположите шаги обратного распространения градиента:

1. Градиент на выходном слое → передать назад
2. Вычислить локальный градиент скрытого слоя
3. Получить dL/dW для первого слоя (input→hidden)
4. Получить dL/dW для второго слоя (hidden→output)
5. Суммировать градиенты, если мини-батч

Подсказка: Градиенты идут от выхода назад к входу.

Ответ:

- Градиент на выходном слое → передать назад
- Вычислить локальный градиент скрытого слоя
- Получить dL/dW для первого слоя (input→hidden)

- Получить dL/dW для второго слоя (hidden→output)
- Суммировать градиенты, если мини-батч

🔗 Шаг 11:

Миссия 11

🔗 Сопоставьте ситуации ручного градиентного спуска с правильным описанием:

Ответ:

- Изменение веса w на основании градиента
- Накопление градиента в минибатче
- Применение регуляризации L2 к градиенту

Описание:

1. $w_{\text{new}} = w_{\text{old}} - \eta \cdot (\partial L / \partial w)$
2. $\text{gradients_sum} += \partial L / \partial w$ для каждого примера
3. $\partial L_{\text{total}} / \partial w += \lambda \cdot w_{\text{old}}$

Подсказка: Регуляризация — это дополнительный вклад к градиенту.

- **Изменение веса w на основании градиента** — $w_{\text{new}} = w_{\text{old}} - \eta \cdot (\partial L / \partial w)$
- **Накопление градиента в минибатче** — $\text{gradients_sum} += \partial L / \partial w$ для каждого примера
- **Применение регуляризации L2 к градиенту** — $\partial L_{\text{total}} / \partial w += \lambda \cdot w_{\text{old}}$

🔗 Шаг 12:

Миссия 12

🔗 Сопоставьте виды ошибок при ручном градиенте и их причины:

Ответ:

- Градиент = 0, хотя сеть ещё не обучилась
- Слишком большие скачки при обновлении
- Медленная сходимость

Причины:

1. взят нулевой участок ReLU или saturating sigmoid
2. слишком большой learning rate η
3. learning rate слишком мал / маленький шаг

Подсказка: Подумайте, когда градиент может «всплыть» нулём или переэкстремиться.

- **Градиент = 0, хотя сеть ещё не обучилась** — взят нулевой участок ReLU или saturating sigmoid
- **Слишком большие скачки при обновлении** — слишком большой learning rate η
- **Медленная сходимость** — learning rate слишком мал / маленький шаг

🔗 Шаг 13:

Миссия 15

⚙ Представьте три слоя подряд (вход → скрытый → скрытый2 → выход). Вам нужно вручную проложить градиенты, расположите шаги:

1. Вычислить dL/dW последнего слоя
2. Применить цепное правило от выхода к скрытому2

3. Вычислить внутренние градиенты скрытого слоя
4. Получить dL/dW для каждого слоя от конца к началу
5. Суммировать градиенты при батче

Подсказка: Начинайте с выхода, двигайтесь назад через слои.

Ответ:

- Вычислить dL/dW последнего слоя
- Применить цепное правило от выхода к скрытому2
- Вычислить внутренние градиенты скрытого слоя
- Получить dL/dW для каждого слоя от конца к началу
- Суммировать градиенты при батче

Шаг 14:

Миссия 14

У вас сеть с residual connection: выход слоя = $F(x) + x$. Расположите операторы в градиенте:

1. градиент через $F(x)$
2. градиент через прямую связь x
3. сумма двух градиентов для следующего слоя

Подсказка: В residual соединении градиенты «делятся» через оба пути.

Ответ:

- градиент через $F(x)$
- градиент через прямую связь x
- сумма двух градиентов для следующего слоя

Шаг 15:

Миссия 15

🔑 Финальное испытание: у вас сеть с ветвями: два параллельных пути, потом сумма. Расположите шаги обратного распространения:

1. Разделить градиент на оба пути
2. Передать градиент к каждому пути отдельно
3. Обратное распространение внутри каждого пути
4. Суммировать обновления параметров

Подсказка: Сначала «разветвление», затем «обратка», потом объединение.

Ответ:

- Разделить градиент на оба пути
- Передать градиент к каждому пути отдельно
- Обратное распространение внутри каждого пути
- Суммировать обновления параметров

Урок 4 - Квест реализуй MLP без .Sequential

Шаг 01:

Миссия 1

👤 Иван, младший исследователь ИИ, впервые приступает к написанию простого

****перцептрона**** вручную. Какой из вариантов — самый базовый шаг при реализации линейного

слоя (в контексте MLP)? *Подсказка:* Начните с самого простого — как выглядит матричное умножение в нейронной сети?

Ответ:

- $y = W \cdot x + b$
- $y = x \div W + b$
- $y = W + x \cdot b$

🔧 Шаг 02:

Миссия 2

☞ Мария уже реализовала $y = W \cdot x + b$. Что ей нужно реализовать ****далее****, чтобы получить выход после нелинейности (активации) в нейроне? *Подсказка:* Подумайте, какая функция добавляет «сигмоиду» или «relu» после линейной части.

Ответ:

- суммировать все выходы
- **применить нелинейную функцию $\sigma(y)$ к линейному выходу**
- умножить результат снова на W

🔧 Шаг 03:

Миссия 3

☞ Алекс писал MLP вручную и хочет обрабатывать мини-батчи (batch). Как лучше всего организовать вычисления линейного слоя при входе размером $(batch_size, in_features)$? *Подсказка:* Используйте тензоры и правила линейной алгебры, а не цикл по элементам.

Ответ:

- $Y = X @ W.T + b$
- $Y = \text{sum over batch}$ потом умножить
- цикл: for каждый x, $y = W \cdot x + b$

🔧 Шаг 04:

Миссия 4

☞ Виталий реализует компоненты MLP вручную. Сопоставьте названия части с их назначением (функция ядра): **Названия:** - Linear Layer - Activation Function - Loss Function - Backpropagation **Назначения:** 1. Обратный проход ошибок и обновление градиентов 2. Линейное преобразование входа 3. Вычисление ошибки между предсказанием и меткой 4. Применение нелинейности к линейному слою *Подсказка:* Подумайте, какая цепочка: линейное преобразование → активация → ошибка → обратный ход.

Ответ:

- **Linear Layer** — Линейное преобразование входа
- **Activation Function** — Применение нелинейности к линейному слою
- **Loss Function** — Вычисление ошибки между предсказанием и меткой
- **Backpropagation** — Обратный проход ошибок и обновление градиентов

🔧 Шаг 05:

Миссия 5

Елена пишет свой класс MLP и внутри сохраняет градиенты вручную. Сопоставьте название тензора с его ролью в обучении: **Тензоры:** - dW - db - dX - output **Роли:** 1. Градиент по входу, передача дальше в предыдущий слой 2. Выход нейрона после активации 3. Градиент по весам

слоя 4. Градиент по смещениям слоя *Подсказка*: dX — градиент по входу, dW — градиент по весам и т.д.

Ответ:

- dW — Градиент по весам слоя
- db — Градиент по смещениям слоя
- dX — Градиент по входу, передача дальше в предыдущий слой
- **output** — Выход нейрона после активации

🔗 Шаг 06:

Миссия 6

🔗 Дмитрий хочет, чтобы его реализация MLP вручную обрабатывала проход вперёд (forward) и обратный (backward). Расположите этапы ****прямого + обратного**** прохода в правильном порядке:

1. Применение функции потерь (loss)
2. Вычисление градиентов (backpropagation)
3. Линейное преобразование ($X @ W + b$)
4. Обратная передача dX к предыдущему слою
5. Применение активации

Подсказка: Сначала идёт фронтальный проход от входа к выходу, затем назад — от ошибки к обновлению параметров.

Ответ:

- Линейное преобразование ($X @ W + b$)
- Применение активации
- Применение функции потерь (loss)
- Вычисление градиентов (backpropagation)
- Обратная передача dX к предыдущему слою

🔗 Шаг 07:

Миссия 7

🌐 София реализовала сеть из двух скрытых слоёв вручную. Она хочет добавить ****регуляризацию L2****. Что ей нужно ****добавить**** к функции потерь? *Подсказка*: L2-регуляризация — это сумма квадратов весов с коэффициентом λ .

Ответ:

- $\lambda \cdot \text{sum } |W|$
- $\lambda \cdot \text{sum } W^2$
- $\lambda \cdot \text{sum } \text{sqrt}(W^2)$

🔗 Шаг 08:

Миссия 8

Максим реализует ручной градиент, хочет применить ****шаг обучения η ****. Как правильно обновить веса W и bias b в слое? *Подсказка*: Напоминаем: $W_{\text{new}} = W_{\text{old}} - \eta \cdot dW$.

Ответ:

- $W \leftarrow W - \eta \cdot dW, b \leftarrow b - \eta \cdot db$
- $W \leftarrow W + \eta \cdot dW$
- $W \leftarrow W - dW / \eta$

🔧 Шаг 09:

Миссия 9

📖 Ксения собирает последовательность операций в своём цикле обучения. Расположите шаги одной итерации (forward + loss + backward + update) в правильном порядке:

1. Compute loss
2. Forward pass through all слои
3. Backward pass (градиенты)
4. Update parameters (W, b)

Подсказка: Всегда: сначала forward, потом loss, затем backward, и обновление в конце.

Ответ:

- Forward pass through all слои
- Compute loss
- Backward pass (градиенты)
- Update parameters (W, b)

🔧 Шаг 10:

Миссия 10

Иван уже сделал MLP вручную и хочет добавить **dropout** и **batch normalization**. Сопоставьте технику с её эффектом: **Техники:** - Dropout - Batch Normalization **Эффекты:** 1. Снижение колебаний активаций, ускорение сходимости 2. Случайное обнуление части нейронов на обучении *Подсказка:* Dropout — случайное исключение нейронов, BN — стандартизация активаций внутри мини-батча.

Ответ:

- **Dropout** — Случайное обнуление части нейронов на обучении
- **Batch Normalization** — Снижение колебаний активаций, ускорение сходимости

🔧 Шаг 11:

Миссия 11

🔗 Ольга внедряет **He initialization** вручную в свой MLP. В чём смысл этой инициализации при активации ReLU? *Подсказка:* Инициализация подбирается так, чтобы сохранять дисперсию входов и выходов при прямом прохождении.

Ответ:

- Весам задаётся распределение с дисперсией $\sim 2 / fan_in$
- Весам задаётся распределение с дисперсией $\sim 1 / fan_out$
- Весам назначается нулевое смещение

🔧 Шаг 12:

Миссия 12

Константин хочет вручную реализовать **mini-batch SGD с моментумом**. Как будет выглядеть обновление веса $\hat{v}$ (с учётом предыдущего импульса) и $\hat{W}$? *Подсказка:* $v \leftarrow \mu \cdot v - \eta \cdot dW$, $W \leftarrow W + v$.

Ответ:

- $v \leftarrow \mu \cdot v - \eta \cdot dW$, $W \leftarrow W + v$
- $v \leftarrow v - \eta \cdot dW$, $W \leftarrow W + \mu \cdot v$
- $W \leftarrow W - \eta \cdot dW + \mu \cdot v$

🔔 Шаг 13:

Миссия 13

🔍 Дарья хочет выбрать ****стратегию изменения lr**** (learning rate schedule). Сопоставьте стратегию с её описанием: **Стратегии:** - Step decay - Exponential decay - Cosine annealing - Constant LR **Описания:** 1. Уменьшение LR на фиксированный множитель через равные эпохи 2. $LR = LR_0 \cdot \exp(-k \cdot t)$ 3. LR изменяется по косинусной кривой от LR_0 до минимума 4. LR остаётся постоянным всю тренировку *Подсказка:* подумайте, как постепенно или резко меняется LR в этих стратегиях.

Ответ:

- **Step decay** — Уменьшение LR на фиксированный множитель через равные эпохи
- **Exponential decay** — $LR = LR_0 \cdot \exp(-k \cdot t)$
- **Cosine annealing** — LR изменяется по косинусной кривой от LR_0 до минимума
- **Constant LR** — LR остаётся постоянным всю тренировку

🔔 Шаг 14:

Миссия 14

Артём хочет задать ****процесс обучения с разбиением на эпохи и батчи****. Какие шаги входят в одну эпоху и в каком порядке? Расположите:

1. Обнулять градиенты перед новым батчем
2. Итерировать по батчам: forward + loss + backward + обновление
3. Оценивать метрики (accuracy, loss) на валидации
4. Шейкать данные (перемешивать) перед каждой эпохой

Подсказка: в начале эпохи — перемешивание, потом батчи, обнуление градиентов, обновление, и в конце — валидация.

Ответ:

- Шейкать данные (перемешивать) перед каждой эпохой
- Итерировать по батчам: forward + loss + backward + обновление
- Обнулять градиенты перед новым батчем
- Оценивать метрики (accuracy, loss) на валидации

🔔 Шаг 15:

Миссия 15

✂ Полина реализовала весь MLP вручную и хочет добавить ****градиентный клиппинг**** (gradient clipping), чтобы защититься от взрывных градиентов. Как она может ограничить норму градиента $\|dW\|$, если она превысит порог max_norm ? *Подсказка:* обрезать (scale) градиенты, если их норма больше пороговой.

Ответ:

- если $\|dW\| > \text{max_norm}$, $dW \leftarrow dW * (\text{max_norm} / \|dW\|)$
- $dW \leftarrow dW - \text{max_norm}$
- $dW \leftarrow dW / \text{max_norm}$

🔔 Шаг 16:

Миссия 16

🔍 Зоя анализирует поведение сети и хочет сопоставить ****сигналы переобучения / недообучения**** с их характеристиками: **Сигналы:** - Переобучение (overfitting) - Недообучение (underfitting) **Характеристики:** 1. Тренировочная ошибка низкая, валидационная — высокая 2. И

тренировочная, и валидационная ошибки большие и близкие друг к другу *Подсказка:* переобучение — когда модель слишком подгоняется под тренировочные данные.

Ответ:

- **Переобучение (overfitting)** — Тренировочная ошибка низкая, валидационная — высокая
- **Недообучение (underfitting)** — И тренировочная, и валидационная ошибки большие и близкие друг к другу

Шаг 17:

Миссия 17

🏁 Финальная миссия: команда готовит ручной MLP к ****продакшен-внедрению**** в простом веб-сервисе. Расположите этапы от кода ядра до деплоя:

1. Обёртывание ядра (MLP) в API (например, Flask или FastAPI)
2. Написание тестов (unit / интеграционные) для ядра и API
3. Контейнеризация (Docker)
4. Настройка CI/CD пайплайна запуска тестов и деплоя
5. Мониторинг модели на продакшене (логирование, алерты)

Подсказка: сначала тесты, затем упаковка, затем деплой и мониторинг.

Ответ:

- Написание тестов (unit / интеграционные) для ядра и API
- Обёртывание ядра (MLP) в API (например, Flask или FastAPI)
- Контейнеризация (Docker)
- Настройка CI/CD пайплайна запуска тестов и деплоя
- Мониторинг модели на продакшене (логирование, алерты)

Урок 5 - Реализация активации и нормализации с нуля

Шаг 01:

Миссия 1

Вы только что приступили к реализации собственного ядра нейросети, и перед вами выбор активации первого слоя: что вы выберете?

Подсказка: подумайте, какая активация часто используется как базовая точка старта.

Ответ:

- **ReLU**
- Step function
- Sign activation
- Absolute value

Шаг 02:

Миссия 2

Ваша коллега предлагает для второго слоя использовать сигмоид в ядре. Вы сомневаетесь: какой из пунктов — главный недостаток сигмоидальной активации?

Подсказка: что происходит с градиентом, когда вход очень большой по модулю?

Ответ:

- **Затухающие градиенты**
- Нелинейность слишком большая

- Выходы становятся отрицательными
- Недостаток выразительности

🔮 Шаг 03:

Миссия 3

Вы проектируете этап нормализации перед активацией. Какой метод будете реализовывать первым?

Подсказка: подумайте, какой метод проще по вычислениям и часто служит базой.

Ответ:

- **Batch Normalization**
- Layer Normalization
- Instance Normalization
- Group Normalization

🔮 Шаг 04:

Миссия 4

При реализации BatchNorm вам нужно выбирать параметры обучения: что важно задать для сглаживания оценок дисперсии?

Подсказка: вспомните, как сохранить устойчивость вычислений при малых батчах.

Ответ:

- **eps (ϵ) маленькая положительная константа**
- momentum = 0
- learning rate для gamma
- max_norm constraint

🔮 Шаг 05:

Миссия 5

Вы решаете, будет ли ваша активация с параметром (например, PReLU). Какой выбор правильный по умолчанию?

Подсказка: параметр обычно начинается небольшим положительным числом или нулём.

Ответ:

- **alpha = 0.25**
- alpha = 1
- alpha = -0.5
- alpha = 10

🔮 Шаг 06:

Миссия 6

Вы обсуждаете с коллегами: почему иногда слой BatchNorm отключают на время инференса? Что меняется?

Подсказка: подумайте, что вы используете вместо статистики текущего батча.

Ответ:

- **Использование накопленных средних и дисперсий**
- Отключение γ и β
- Установка learning rate = 0

- Приведение к среднему = 0

🔮 Шаг 07:

Миссия 7

Настал момент сопоставить термин и его роль в активации/нормализации.

Подсказка: вспомните, что делает gamma и beta и зачем добавляют ϵ .

Понятия:

Ответ:

- γ (gamma)
- β (beta)
- ϵ (epsilon)
- running mean
- running variance
- централизовать (центровка)

Определения:

1. Смещение после нормализации
 2. Накопленная средняя активаций по эпохам
 3. Множитель при масштабировании после нормализации
 4. Добавка для численной устойчивости
 5. Накопленная дисперсия по эпохам
 6. Выравнивание так, чтобы среднее стало нулём
- **γ (gamma)** — Множитель при масштабировании после нормализации
 - **β (beta)** — Смещение после нормализации
 - **ϵ (epsilon)** — Добавка для численной устойчивости
 - **running mean** — Накопленная средняя активаций по эпохам
 - **running variance** — Накопленная дисперсия по эпохам
 - **централизовать (центровка)** — Выравнивание так, чтобы среднее стало нулём

🔮 Шаг 08:

Миссия 8

У вас есть разные варианты нормализации: сопоставьте каждому его преимущество.

Подсказка: подумайте о независимости по элементам, стабилизации обучения и затратам.

Методы:

Ответ:

- BatchNorm
- LayerNorm
- InstanceNorm
- GroupNorm

Преимущества:

1. Независимость от размера батча
 2. Хорошо для стилевых задач (образы)
 3. Стабилизирует входы к активациям внутри слоя
 4. Гибкая группировка каналов для баланса
- **BatchNorm** — Стабилизирует входы к активациям внутри слоя

- **LayerNorm** — Независимость от размера батча
- **InstanceNorm** — Хорошо для стилизованных задач (образы)
- **GroupNorm** — Гибкая группировка каналов для баланса

🔮 Шаг 09:

Миссия 9

В ходе дебага вашего ядра сопоставьте ошибку/симптом и вероятную причину.

Подсказка: градиенты могут исчезать или взрываться, нормализация может быть неправильно применена.

Симптомы:

Ответ:

- Градиенты близки к нулю
- Градиенты огромны и нестабильны
- Незначительное обучение, все выходы почти одинаковы
- Числовые NaN в активациях

Причины:

1. ϵ слишком маленькое или отсутствие защиты от деления на ноль
 2. Плохая инициализация весов / слишком большой learning rate
 3. Применение BatchNorm на размер батча = 1 без адаптации
 4. Сигмоида на больших значениях вызывает затухание градиента
- **Градиенты близки к нулю** — Сигмоида на больших значениях вызывает затухание градиента
 - **Градиенты огромны и нестабильны** — Плохая инициализация весов / слишком большой learning rate
 - **Незначительное обучение, все выходы почти одинаковы** — Применение BatchNorm на размер батча = 1 без адаптации
 - **Числовые NaN в активациях** — ϵ слишком маленькое или отсутствие защиты от деления на ноль

🔮 Шаг 10:

Миссия 10

Вы выбираете порядок операций внутри слоя ядра: сопоставьте, что должно идти до и что после нормализации/активации.

Подсказка: многие современные архитектуры сначала нормализуют, затем применяют активацию, иногда добавляют остаток.

Операции:

Ответ:

- линейное преобразование ($Wx + b$)
- BatchNorm
- активация (ReLU, PReLU и др.)
- добавление residual connection (сумма с входом)

Позиции:

1. Перед активацией
2. После активации
3. После линейного преобразования

4. В конце слоя, после остальных операций

- **линейное преобразование ($Wx + b$)** — После линейного преобразования
- **BatchNorm** — Перед активацией
- **активация (ReLU, PReLU и др.)** — После активации
- **добавление residual connection (сумма с входом)** — В конце слоя, после остальных операций

🔮 Шаг 11:

Миссия 11

В сценарии продакшена ваш модуль нормализации работает в распределённой среде. Сопоставьте понятие и требование.

Подсказка: подумайте, про что говорят sync-batchnorm и глобальные статистики.

Понятия:

Ответ:

- Sync-BatchNorm
- Независимые статистики на каждом устройстве
- Глобальная статистика
- Коммуникация через all-reduce

Определения:

1. Каждое устройство считает свою среднюю и дисперсию
 2. Обмен статистиками между устройствами для усреднения
 3. Использование статистики по всему распределению данных
 4. Нормализация без синхронизации между устройствами
- **Sync-BatchNorm** — Обмен статистиками между устройствами для усреднения
 - **Независимые статистики на каждом устройстве** — Каждое устройство считает свою среднюю и дисперсию
 - **Глобальная статистика** — Использование статистики по всему распределению данных
 - **Коммуникация через all-reduce** — Нормализация без синхронизации между устройствами

🔮 Шаг 12:

Миссия 12

Команда обсуждает, как инициализировать γ и β при старте. Сопоставьте стратегию и её эффект.

Подсказка: $\gamma = 1$ и $\beta = 0$ — часто применяемый старт.

Стратегии:

Ответ:

- $\gamma = 1, \beta = 0$
- $\gamma = 0.1, \beta = 0$
- $\gamma = \text{small random}, \beta = 0$
- $\gamma = 1, \beta = \text{random small}$

Эффекты:

1. начальное сохранение сигнала без масштабирования или сдвига
2. уменьшение эффекта нормализации

3. непредсказуемое поведение с начала обучения

4. слегка смещённое начало обучения

- $\gamma = 1, \beta = 0$ — начальное сохранение сигнала без масштабирования или сдвига
- $\gamma = 0.1, \beta = 0$ — уменьшение эффекта нормализации
- $\gamma = \text{small random}, \beta = 0$ — непредсказуемое поведение с начала обучения
- $\gamma = 1, \beta \text{ random small}$ — слегка смещённое начало обучения

🔮 Шаг 13:

Миссия 13

Расположите этапы прямого прохода через слой с нормализацией и активацией:

1. линейное преобразование
2. применение γ и β
3. центрирование и масштабирование (нормализация)
4. активация

Подсказка: сначала нормализуют (центровка и масштабирование), потом применяют обучаемые параметры, затем активацию.

Ответ:

- линейное преобразование
- центрирование и масштабирование (нормализация)
- применение γ и β
- активация

🔮 Шаг 14:

Миссия 14

При обучении вы хотите задействовать backward-проход. Расположите шаги обратного пропуска через нормализацию + активацию:

1. градиент по γ и β
2. градиент после активации к нормализованному входу
3. градиент по входу к нормализованному слою
4. распространение к линейному ядру

Подсказка: сначала нужно распределить градиенты через активацию, затем нормализацию, потом на вход.

Ответ:

- градиент по γ и β
- градиент после активации к нормализованному входу
- градиент по входу к нормализованному слою
- распространение к линейному ядру

🔮 Шаг 15:

Миссия 15

Вы разрабатываете ядро и хотите встроить residual connection с нормализацией. Расположите правильный порядок операций:

1. вход + остаток
2. линейное преобразование

3. нормализация и γ/β
4. активация
5. добавление резидуала

Подсказка: residual применяется после всех преобразований, активации обычно после нормализации.

Ответ:

- вход + остаток
- линейное преобразование
- нормализация и γ/β
- активация
- добавление резидуала

Шаг 16:

Миссия 16

В продакшен-ядре нужно оптимизировать вычисления: упорядочите следующие оптимизации по степени простоты внедрения (от простого к сложному):

1. инлайн-вычисления активации и нормализации
2. использование предвычисленных γ и β преобразований
3. сведение к fused kernel (смешение операций в одном ядре)
4. оптимизация через SIMD / векторизацию
5. профилирование и удаление избыточных операций

Подсказка: сначала делаются тривиальные упрощения, затем более сложные интеграции.

Ответ:

- инлайн-вычисления активации и нормализации
- использование предвычисленных γ и β преобразований
- сведение к fused kernel (смешение операций в одном ядре)
- оптимизация через SIMD / векторизацию
- профилирование и удаление избыточных операций

Шаг 17:

Миссия 17

Финальный тест: расположите в правильном порядке ваш pipeline при inference с активацией и нормализацией (без обучения):

1. линейное преобразование
2. центрирование и масштабирование (используя накопленные статистики)
3. применение γ и β
4. активация
5. результат (выход слоя)

Подсказка: при inference используйте накопленные статистики, а не статистику батча.

Ответ:

- линейное преобразование
- центрирование и масштабирование (используя накопленные статистики)
- применение γ и β
- активация

- результат (выход слоя)

Модуль 3 - Погружение в сложные архитектуры

Урок 1 - Сравнение архитектур

Шаг 01:

Миссия 1

Анна работает над модулем attention в трансформере и задумывается: какую архитектуру использовать для обработки длинных последовательностей?

Подсказка: Подумайте, какая из моделей уже доказала свою эффективность для длинных входов.

Ответ:

- Классический LSTM
- **Longformer / Performer**
- MLP с большим скрытым слоем

Шаг 02:

Миссия 2

Борис разрабатывает модель детекции изображений и хочет выбрать между свёрточной нейросетью и ViT. Что стоит учитывать в первую очередь?

Подсказка: Подумайте про зависимость от размера обучающего корпуса и структурных локальных свойств изображения.

Ответ:

- Количество слоёв в архитектуре
- **Размер и разнообразие обучающей выборки**
- Количество ядер GPU

Шаг 03:

Миссия 3

Светлана решает, использовать ли ResNet или DenseNet для классификации медицинских снимков. Что преимущественно даст DenseNet?

Подсказка: Подумайте об обмене признаками между слоями.

Ответ:

- **Более плотные связи между слоями и лучшую градиентную передачу**
- Уменьшение числа параметров до минимума
- Автоматическое изменение разрешения входа

Шаг 04:

Миссия 4

Илья хочет выбрать между архитектурами Encoder-Decoder и обычным автокодером для задачи передачи текста в текст (seq2seq). Что ключевое отличие?

Подсказка: Подумайте, нужно ли генерировать выход, длина которого может отличаться от входа.

Ответ:

- **Encoder-Decoder позволяет генерировать выход другой длины**

- Обычный автокодер безопаснее в обучении
- Encoder-Decoder требует меньше параметров

🔮 Шаг 05:

Миссия 5

Марина рассматривает использования U-Net или GAN для сегментации. Почему часто выбирают U-Net?

Подсказка: Подумайте про сохранение низкоуровневой информации и передачу между уровнями.

Ответ:

- **U-Net сохраняет и передаёт детальные признаки между уровнями**
- GAN быстрее обучается
- U-Net имеет меньше слоёв

🔮 Шаг 06:

Миссия 6

Виталий проектирует гибридную архитектуру и хочет понять, как сопоставляются принципы каждой модели. Сопоставьте архитектуру с её сильной стороной:

Архитектуры:

Ответ:

- Transformer
- Convolutional Neural Network (CNN)
- Recurrent Neural Network (RNN)
- MLP / Fully Connected

Сильные стороны:

1. Эффективность при обработке последовательностей и контекстов
2. Локальные инвариантности и распознавание шаблонов
3. Простота структуры и универсальность
4. Гибкость в моделировании зависимостей между всеми токенами

Подсказка: Подумайте, какая модель умеет захватывать глобальные зависимости, а какая — локальные.

- **Transformer** — Гибкость в моделировании зависимостей между всеми токенами
- **Convolutional Neural Network (CNN)** — Локальные инвариантности и распознавание шаблонов
- **Recurrent Neural Network (RNN)** — Эффективность при обработке последовательностей и контекстов
- **MLP / Fully Connected** — Простота структуры и универсальность

🔮 Шаг 07:

Миссия 7

Павел анализирует блоки внимания. Сопоставьте тип внимания с его особенностью:

Типы внимания:

Ответ:

- Scaled Dot-Product Attention
- Multi-Head Attention

- Self-Attention
- Cross-Attention

Особенности:

1. Каждая голова учится своей проекции
2. Взаимодействие внутри одного слоя между токенами
3. Масштабирование через деление на $\sqrt{d_k}$
4. Внимание между кодером и декодером

Подсказка: Подумайте, где применяется cross-attention — между двух частей модели.

- **Scaled Dot-Product Attention** — Масштабирование через деление на $\sqrt{d_k}$
- **Multi-Head Attention** — Каждая голова учится своей проекции
- **Self-Attention** — Взаимодействие внутри одного слоя между токенами
- **Cross-Attention** — Внимание между кодером и декодером

🔮 Шаг 08:

Миссия 8

Ольга комбинирует резидуальные связи и нормализацию в своей архитектуре. Сопоставьте понятие с функцией:

Понятия:

Ответ:

- Residual Connection
- Layer Normalization
- Batch Normalization
- Dropout

Функции:

1. Стабилизирует активации по минипакету
2. Помогает передавать градиенты через слои
3. Нормализация по токенам внутри слоя
4. Случайное обнуление нейронов для регуляризации

Подсказка: Подумайте, где применяется layer norm — по токенам, а batch norm — по батчу.

- **Residual Connection** — Помогает передавать градиенты через слои
- **Layer Normalization** — Нормализация по токенам внутри слоя
- **Batch Normalization** — Стабилизирует активации по минипакету
- **Dropout** — Случайное обнуление нейронов для регуляризации

🔮 Шаг 09:

Миссия 9

Демьян экспериментирует с энкодерами и декодерами. Сопоставьте тип архитектуры с её применением:

Архитектуры:

Ответ:

- Encoder-only
- Decoder-only
- Encoder-Decoder

- Autoencoder

Применения:

1. Генерация текста по подсказке (causal)
2. Машинный перевод (seq2seq)
3. Кодирование признаков (без генерации)
4. Восстановление входа через сжатие и декодирование

Подсказка: Подумайте, когда нужен трансформер с двумя частями — когда вход $\neq$ выход по длине.

- **Encoder-only** — Кодирование признаков (без генерации)
- **Decoder-only** — Генерация текста по подсказке (causal)
- **Encoder-Decoder** — Машинный перевод (seq2seq)
- **Autoencoder** — Восстановление входа через сжатие и декодирование

Шаг 10:

Миссия 10

Софья хочет настроить шаги при сравнении производительности разных архитектур. Расположите шаги в правильном порядке:

1. Запуск измерений (L2, память, время инференса)
2. Выбор метрик для сравнения
3. Интерпретация полученных результатов
4. Выбор наборов архитектур и конфигураций
5. Проверка стабильности результатов (несколько запусков)

Подсказка: Сначала нужно выбрать, что мерить, затем собрать конфигурации, после — запускать и проверять стабильность, и в конце анализировать.

Ответ:

- Выбор метрик для сравнения
- Выбор наборов архитектур и конфигураций
- Запуск измерений (L2, память, время инференса)
- Проверка стабильности результатов (несколько запусков)
- Интерпретация полученных результатов

Шаг 11:

Миссия 11

Денис сравнивает ResNet, DenseNet, и EfficientNet по сложности и скоростным качествам. Расположите архитектуры от самой лёгкой к самой тяжёлой:

1. DenseNet
2. ResNet
3. EfficientNet

Подсказка: Лёгкая — минимальное число операций, тяжёлые — много слоёв и сенситивности.

Ответ:

- DenseNet
- ResNet
- EfficientNet

Шаг 12:

Миссия 12

Катя хочет упорядочить шаги архитектурного эксперимента A/B между двумя моделями:

1. Интеграция модели в тестовую среду
2. Сбор метрик (latency, throughput, качество)
3. Гиперпараметрическая настройка каждой модели
4. Выбор метрик успеха
5. Статистический анализ результатов

Подсказка: Начинайте с определения метрик, потом настройка, интеграция, сбор, анализ.

Ответ:

- Выбор метрик успеха
- Гиперпараметрическая настройка каждой модели
- Интеграция модели в тестовую среду
- Сбор метрик (latency, throughput, качество)
- Статистический анализ результатов

Шаг 13:

Миссия 13

Лена ставит в ряд этапы оценки архитектур на продакшене:

1. Мониторинг ресурсов (CPU, GPU, память)
2. Сравнение через shadow-deployment
3. Измерение качества предсказаний в реальных данных
4. A/B тестирование в реальном трафике
5. Оценка стабильности и латентности

Подсказка: Начинайте с проверок вне трафика или параллельно, затем вживую, затем анализируйте метрики.

Ответ:

- Сравнение через shadow-deployment
- Измерение качества предсказаний в реальных данных
- Мониторинг ресурсов (CPU, GPU, память)
- Оценка стабильности и латентности
- A/B тестирование в реальном трафике

Шаг 14:

Миссия 14

Михаил хочет ранжировать модели по устойчивости к шуму в входных данных:

1. Обычная MLP
2. ResNet с data augmentation
3. Transformer с шумовой регуляризацией
4. Сверточная сеть без регуляризации

Подсказка: Устойчивые модели чаще имеют регуляризацию и архитектурные механизмы защиты.

Ответ:

- Обычная MLP
- Сверточная сеть без регуляризации
- ResNet с data augmentation
- Transformer с шумовой регуляризацией

 Шаг 15:

Миссия 15

Арина ставит в порядок уровни абстракции при сравнении архитектур:

1. Ядро слоя (операции матричных умножений и нелинейности)
2. Блок внимания, сверточный блок и др.
3. Модуль (encoder, decoder, backbone)
4. Система обучения и пайплайн
5. Инференс-движок и продакшен сборка

Подсказка: От низкоуровня к высокоуровню: сначала математика, затем блоки, модули, обучение и продакшен.

Ответ:

- Ядро слоя (операции матричных умножений и нелинейности)
- Блок внимания, сверточный блок и др.
- Модуль (encoder, decoder, backbone)
- Система обучения и пайплайн
- Инференс-движок и продакшен сборка

 Шаг 16:

Миссия 16

Борис планирует этапы исследования новой архитектуры для продакшена. Упорядочите шаги:

1. Прототипирование архитектуры
2. Обучение на контрольных датасетах
3. Профилирование узких мест (memory, time)
4. Оптимизация (прунинг, квантизация и т.д.)
5. Валидация модели на hold-out
6. Интеграция в продакшен среду

Подсказка: Сначала прототип, затем обучение и валидация, после профилирование, оптимизация и интеграция.

Ответ:

- Прототипирование архитектуры
- Обучение на контрольных датасетах
- Валидация модели на hold-out
- Профилирование узких мест (memory, time)
- Оптимизация (прунинг, квантизация и т.д.)
- Интеграция в продакшен среду

 Шаг 17:

Миссия 17

Финальный квест: Упорядочите подходы к сравнению моделей от уровня дизайна до продакшена:

1. Сравнение на контролируемых датасетах (offline)
2. Тестовый прогон в продакшен-окружении (shadow-mode)
3. АВ-тестирование в продакшен режиме
4. Мониторинг и анализ ошибок в живом трафике
5. Обратная связь и итерации архитектуры

Подсказка: Переносите сравнение от офлайна к онлайн, затем итерации.

Ответ:

- Сравнение на контролируемых датасетах (offline)
- Тестовый прогон в продакшен-окружении (shadow-mode)
- АВ-тестирование в продакшен режиме
- Мониторинг и анализ ошибок в живом трафике
- Обратная связь и итерации архитектуры

Урок 2 - Как работает skip connection

Шаг 01:

Миссия 1

Алексей строит нейросеть с residual-блоками. Какую роль выполняет skip connection?

Подсказка: Подумайте, какую проблему он решает при добавлении этого пути.

Ответ:

- **Передать сигнал напрямую, минуя некоторые слои**
- Удалить веса из слоя автоматически
- Уменьшить размер активации
- Добавить шум для регуляризации

Шаг 02:

Миссия 2

 Мария экспериментирует: она добавила skip connection между слоями разной размерности. Что нужно, чтобы это работало?

Подсказка: Обратите внимание на размеры тензоров.

Ответ:

- **Линейное преобразование (1x1 conv) для согласования размерности**
- **Padding или отсечение, чтобы выровнять пространственные размеры**
- Случайный шум для растяжения тензора
- Удвоение числа каналов даже без весов

Шаг 03:

Миссия 3

 Иван настроил skip connections, но обучение стало менее стабильным. Что может быть причиной?

Подсказка: Подумайте про градиенты и особенности потока информации.

Ответ:

- **Слишком сильный прямой путь мешает адаптации весов внутри блока**

- Skip-connection удаляет все нелинейности
- Skip connection вызывает исчезающий градиент
- Skip connection делает модель линейной

🔔 Шаг 04:

Миссия 4

🔍 Светлана рассматривает два варианта skip: «add» или «concat». В каком случае concat предпочтительнее?

Подсказка: Подумайте о том, как вы хотите сохранить информацию из обоих путей.

Ответ:

- **Когда вы хотите сохранить обе части как независимые каналы**
- Когда размерность путей точно совпадает
- Когда необходимо усреднить два сигнала
- Когда хотите уменьшить число параметров

🔔 Шаг 05:

Миссия 5

✂ Николай строит глубокую сеть с множеством skip connections. Какой эффект они дают на обучение?

Подсказка: Подумайте про распространение градиентов сквозь блоки.

Ответ:

- **Улучшение сходимости и уменьшение проблемы исчезающих градиентов**
- Замедление обучения из-за лишних путей
- Полная отмена нелинейностей внутри блока
- Переобучение из-за прямых путей

🔔 Шаг 06:

Миссия 6

🕒 Сопоставьте тип skip connection с его особенностью.

Типы:

Ответ:

- Skip через сложение (add)
- Skip через конкатенацию (concat)
- Skip с линейным преобразованием (1×1 conv)
- Identity skip (просто соединение без трансформации)

Описание особенностей:

1. Требуется совпадения числа каналов
2. Добавляет рост размерности канала, сохраняя оба потока
3. Используется для согласования каналов между путями
4. Самый простой вариант без изменений сигнала

Подсказка: Подумайте, что делает каждое преобразование и когда оно нужно.

- **Skip через сложение (add)** — Требуется совпадения числа каналов
- **Skip через конкатенацию (concat)** — Добавляет рост размерности канала, сохраняя оба потока

- **Skip с линейным преобразованием (1×1 conv)** — Используется для согласования каналов между путями
- **Identity skip (просто соединение без трансформации)** — Самый простой вариант без изменений сигнала

🔗 Шаг 07:

Миссия 7

⚙ Сопоставьте проблему, возникшую при использовании skip, с методом её решения.

Проблемы:

Ответ:

- Несоответствие размерности каналов
- Разные пространственные размеры (высота/ширина)
- Избыточный рост числа параметров
- Преобладающий прямой путь мешает обучению внутренних слоёв

Решения:

1. Использование 1×1 свёртки
2. Подбор padding/upsample/downsample
3. Использование squeeze, bottleneck структуры
4. Добавить weight scaling, регуляризацию или gating

Подсказка: Иногда нужно не просто соединить два пути, а «пропускать с контролем».

- **Несоответствие размерности каналов** — Использование 1×1 свёртки
- **Разные пространственные размеры (высота/ширина)** — Подбор padding/upsample/downsample
- **Избыточный рост числа параметров** — Использование squeeze, bottleneck структуры
- **Преобладающий прямой путь мешает обучению внутренних слоёв** — Добавить weight scaling, регуляризацию или gating

🔗 Шаг 08:

Миссия 8

Сопоставьте архитектурный паттерн с ролью skip connection в нём.

Паттерны:

Ответ:

- ResNet-блок
- DenseNet
- Highway Network
- U-Net

Роли skip connection:

1. Каждый слой подключается ко всем последующим
2. Передача пространственной информации между уровнями энкодера и декодера
3. Прямой путь с learnable gating
4. Упрощённый остаточный путь внутри блока

Подсказка: Подумайте, где именно skip используется: между блоками, уровнями, через гейты и т.д.

- **ResNet-блок** — Упрощённый остаточный путь внутри блока

- **DenseNet** — Каждый слой подключается ко всем последующим
- **Highway Network** — Прямой путь с learnable gating
- **U-Net** — Передача пространственной информации между уровнями энкодера и декодера

🔮 Шаг 09:

Миссия 9

🔍 Сопоставьте проблему обучения с тем, как skip помогает её решать.

Проблемы обучения:

Ответ:

- Исчезающий градиент
- Глубокие сети трудно оптимизировать
- Потеря информации при проходе вперёд
- Плохое распространение ошибок к ранним слоям

Как skip помогает:

1. Создаёт короткие пути для градиента
2. Облегчает оптимизацию, разбивая путь на участки
3. Сохраняет сигналы низкого уровня
4. Уменьшает расстояние, которое должен пройти градиент

Подсказка: Подумайте, какой вклад skip в распространение сигнала и ошибок.

- **Исчезающий градиент** — Создаёт короткие пути для градиента
- **Глубокие сети трудно оптимизировать** — Облегчает оптимизацию, разбивая путь на участки
- **Потеря информации при проходе вперёд** — Сохраняет сигналы низкого уровня
- **Плохое распространение ошибок к ранним слоям** — Уменьшает расстояние, которое должен пройти градиент

🔮 Шаг 10:

Миссия 10

Арина разрабатывает skip блок, расположите этапы внедрения skip connection правильно:

1. Добавление линейного преобразования, если нужно
2. Определение соединяемых слоёв (откуда и куда skip идёт)
3. Тестирование, что обучение сходится лучше
4. Выравнивание размеров тензоров (padding / downsample или upsample)
5. Интеграция skip (add/concat) с основным путём

Подсказка: Сначала нужно выбрать, потом подготовить, потом соединить, потом проверить и скорректировать.

Ответ:

- Определение соединяемых слоёв (откуда и куда skip идёт)
- Выравнивание размеров тензоров (padding / downsample или upsample)
- Добавление линейного преобразования, если нужно
- Интеграция skip (add/concat) с основным путём
- Тестирование, что обучение сходится лучше

🔧 Шаг 11:

Миссия 11

👤 Павел хочет сравнить эффекты разных вариантов skip. Расположите по возрастанию сложности реализации:

1. Identity skip без изменений
2. Skip через сложение (add)
3. Skip через сложение + согласование каналов (1×1 conv)
4. Skip через конкатенацию (concat)
5. Skip с gating (learnable weight пути)

Подсказка: Подумайте, что требует больше параметров и логики.

Ответ:

- Identity skip без изменений
- Skip через сложение (add)
- Skip через сложение + согласование каналов (1×1 conv)
- Skip через конкатенацию (concat)
- Skip с gating (learnable weight пути)

🔧 Шаг 12:

Миссия 12

👤 Максим проектирует U-Net с skip между уровнями. Расположите порядок передачи и объединения сигналов:

1. Передача активаций из энкодера вниз
2. Преобразование (upsample) декодера до нужного размера
3. Объединение (concat или add) с сигналом skip
4. Дополнительная свёртка после объединения
5. Продолжение декодера дальше

Подсказка: Сначала подготовить, потом объединить, потом обработать.

Ответ:

- Передача активаций из энкодера вниз
- Преобразование (upsample) декодера до нужного размера
- Объединение (concat или add) с сигналом skip
- Дополнительная свёртка после объединения
- Продолжение декодера дальше

🔧 Шаг 13:

Миссия 13

👤 Дарья оценивает эффективность skip: расположите виды измерений по мере усилия реализации:

1. Сравнение loss и accuracy с моделью без skip
2. Визуализация градиентов по слоям
3. Измерение скорости сходимости (эпохи до порога)
4. Оценка ресурсоёмкости (память, скорость)
5. Анализ внутренней активации и редундантности сигналов

Подсказка: Простые метрики доступны сразу, глубокий анализ требует больше усилий.

Ответ:

- Сравнение loss и accuracy с моделью без skip
- Визуализация градиентов по слоям
- Измерение скорости сходимости (эпохи до порога)
- Оценка ресурсоёмкости (память, скорость)
- Анализ внутренней активации и редундантности сигналов

 Шаг 14:**Миссия 14**

🔗 Екатерина проводит ablation-исследование skip: расположите к новому эксперименту по усложнению:

1. Модель без skip
2. Skip identity только в часть блоков
3. Skip add во всех блоках
4. Skip add + alignment каналов (1×1 conv)
5. Skip с gating и learnable scale

Подсказка: Усложнение идёт от простого к гибкому.

Ответ:

- Модель без skip
- Skip identity только в часть блоков
- Skip add во всех блоках
- Skip add + alignment каналов (1×1 conv)
- Skip с gating и learnable scale

 Шаг 15:**Миссия 15**

🔗 Финальная проверка: расположите шаги комплексной диагностики skip в модели перед продакшеном:

1. Сравнить поведение модели с/без skip по метрикам
2. Визуализация распределений активаций и сигналов skip
3. Проверка градиентов и их нормы в пути skip и основном пути
4. Нагрузочное тестирование (скорость, память) модели с skip
5. A/B-тестирование в продакшен среде

Подсказка: Диагностика охватывает метрики, внутренние сигналы и поведение в продакшене.

Ответ:

- Сравнить поведение модели с/без skip по метрикам
- Визуализация распределений активаций и сигналов skip
- Проверка градиентов и их нормы в пути skip и основном пути
- Нагрузочное тестирование (скорость, память) модели с skip
- A/B-тестирование в продакшен среде

 Урок 3 - Трюки и паттерны проектирования **Шаг 01:****Миссия 1**

Коллега Борис считает, что паттерн «Фабрика» – это то, что спасёт дизайн вашего ядра. В чём ключевое достоинство этого паттерна?

Подсказка: Подумайте, как фабрика инкапсулирует создание объектов.

Ответ:

- **Скрывает логику создания и позволяет легко менять реализации**
- Гарантирует единственный экземпляр
- Упрощает многопоточность
- Устраняет необходимость тестирования

Шаг 02:

Миссия 2

Вы пишете систему плагинов для нейросети, и хотите, чтобы новые плагины легко подключались. Какой подход обеспечит расширяемость?

Подсказка: Подумайте, как интерфейсы помогают добавлять модули без изменения ядра.

Ответ:

- **Определить интерфейс Plugin и загружать реализации через конфигурацию**
- Жёстко прописывать все плагины в коде ядра
- Передавать весь код плагина как строку
- Использовать глобальные переменные для каждого плагина

Шаг 03:

Миссия 3

Ваша команда предлагает использовать шаблон «Стратегия», чтобы смена поведения обработки модели была динамичной. Что из нижеперечисленного — реальный сценарий применения?

Подсказка: Подумайте, когда может потребоваться менять алгоритм во время выполнения.

Ответ:

- **Переключение функции активации (ReLU / LeakyReLU / Sigmoid) без пересборки**
- Создание единственного синглтона
- Жёсткая привязка модели к интерфейсу
- Передача глобального состояния между стратегиями

Шаг 04:

Миссия 4

Сопоставьте шаблон проектирования с его ключевой функцией в ядре нейросети.

Шаблоны:

Ответ:

- Singleton
- Adapter
- Facade
- Observer

Функции:

1. Предоставляет единый интерфейс для сложной подсистемы
2. Гарантирует один экземпляр и глобальную точку доступа
3. Подписка на события и уведомления при изменениях

4. Преобразует интерфейс одного компонента в нужный другой

Подсказка: Вспомните, когда нужен единственный экземпляр, когда нужен адаптер, и что делает фасад.

- **Singleton** — Гарантирует один экземпляр и глобальную точку доступа
- **Adapter** — Преобразует интерфейс одного компонента в нужный другой
- **Facade** — Предоставляет единый интерфейс для сложной подсистемы
- **Observer** — Подписка на события и уведомления при изменениях

Шаг 05:

Миссия 5

Вы сталкиваетесь с ситуацией: разные реализации слоёв (Dense, Conv, LSTM) должны использовать один интерфейс. Сопоставьте шаблон и описание:

Шаблоны:

Ответ:

- Factory Method
- Bridge
- Composite
- Decorator

Описания:

1. Фабрика создаёт объект нужного класса по параметрам
2. Разделяет абстракцию и её реализацию, чтобы можно было менять их независимо
3. Компонент может содержать другие компоненты (структура дерева)
4. Динамически добавляет поведение объекту без изменения его класса

Подсказка: Подумайте, где вы хотите расширять и где переиспользовать без изменения кода.

- **Factory Method** — Фабрика создаёт объект нужного класса по параметрам
- **Bridge** — Разделяет абстракцию и её реализацию, чтобы можно было менять их независимо
- **Composite** — Компонент может содержать другие компоненты (структура дерева)
- **Decorator** — Динамически добавляет поведение объекту без изменения его класса

Шаг 06:

Миссия 6

В системе логирования при разработке нейросети используется шаблон «Chain of Responsibility». Сопоставьте элементы цепочки и их роли:

Элементы цепочки:

Ответ:

- HandlerA
- HandlerB
- HandlerC
- Client

Роли:

1. Обращается к первому обработчику
2. Обработывает или передаёт далее
3. Обработывает сообщения уровня «информация»

4. Обрабатывает сообщения уровня «ошибки»

Подсказка: Клиент отправляет запрос цепочке, каждая звено решает, обрабатывать ли дальше.

- **Client** — Обращается к первому обработчику
- **HandlerA** — Обрабатывает сообщения уровня «информация»
- **HandlerB** — Обрабатывает или передаёт далее
- **HandlerC** — Обрабатывает сообщения уровня «ошибки»

Шаг 07:

Миссия 7

При оптимизации системы обучения вы хотите предусмотреть точку расширения для новых оптимизаторов (SGD, Adam, RMSProp). Сопоставьте шаблон и роль:

Шаблоны:

Ответ:

- Strategy
- Template Method
- Prototype
- Flyweight

Роли:

1. Определяет семейство алгоритмов (SGD, Adam и др.) и инкапсулирует их
2. Определяет скелет алгоритма, оставляя шаги подклассам
3. Клонирование прототипа вместо создания заново
4. Снижает память за счёт общей части данных

Подсказка: Представьте, что вы хотите менять алгоритм «на лету».

- **Strategy** — Определяет семейство алгоритмов (SGD, Adam и др.) и инкапсулирует их
- **Template Method** — Определяет скелет алгоритма, оставляя шаги подклассам
- **Prototype** — Клонирование прототипа вместо создания заново
- **Flyweight** — Снижает память за счёт общей части данных

Шаг 08:

Миссия 8

Выстройте этапы применения паттерна «Декоратор» к компоненту модели нейросети:

1. Назначение базового компонента
2. Передача базового компонента декоратору
3. Создание цепочки декораторов
4. Применение поведения через декораторы

Подсказка: Сначала есть базовый объект, затем он “оборачивается” декораторами, которые добавляют поведение.

Ответ:

- Назначение базового компонента
- Передача базового компонента декоратору
- Создание цепочки декораторов
- Применение поведения через декораторы

Шаг 09:

Миссия 9

Упорядочите шаги интеграции шаблона «Facade» в сложную систему подсистем:

1. Определение API фасада для клиента
2. Инкапсуляция вызовов подсистем внутри фасада
3. Замена прямых вызовов подсистем клиентским кодом
4. Тестирование, что фасад вызывает всё правильно

Подсказка: Проверьте, что клиент больше не знает детали подсистем.

Ответ:

- Определение API фасада для клиента
- Инкапсуляция вызовов подсистем внутри фасада
- Замена прямых вызовов подсистем клиентским кодом
- Тестирование, что фасад вызывает всё правильно

Шаг 10:

Миссия 10

Расположите этапы внедрения шаблона «Observer» в систему логирования метрик нейросети:

1. Определение интерфейса наблюдателя
2. Регистрация подписчиков (наблюдателей)
3. Генерация события (например, новая метрика)
4. Уведомление всех подписчиков

Подсказка: Сначала вы определяете контракт, потом подписываете слушателей, затем генерируете и рассылаете события.

Ответ:

- Определение интерфейса наблюдателя
- Регистрация подписчиков (наблюдателей)
- Генерация события (например, новая метрика)
- Уведомление всех подписчиков

Шаг 11:

Миссия 11

Выстраиваем порядок при внедрении шаблона «Composite» для слоёв ансамбля моделей:

1. Создать базовый интерфейс компонента модели
2. Создать листовые компоненты (отдельные модели)
3. Создать составной компонент (ансамбль, содержащий несколько моделей)
4. Запуск обучения и предсказания через общий интерфейс

Подсказка: Компоненты и составные объекты должны работать одинаково.

Ответ:

- Создать базовый интерфейс компонента модели
- Создать листовые компоненты (отдельные модели)
- Создать составной компонент (ансамбль, содержащий несколько моделей)
- Запуск обучения и предсказания через общий интерфейс

🔗 Шаг 12:

Миссия 12

При внедрении гибридного ядра с межмодульным кэшем: упорядочите шаги внедрения шаблона «Proxy»:

1. Определение интерфейса доступа к ресурсу
2. Создание прокси-класса, который реализует этот интерфейс
3. Прокси внутри хранит ссылку на реальный объект и добавляет функциональность (например, кеширование)
4. Клиент взаимодействует через прокси вместо прямого объекта

Подсказка: Прокси действует как посредник между клиентом и реальным модулем.

Ответ:

- Определение интерфейса доступа к ресурсу
- Создание прокси-класса, который реализует этот интерфейс
- Прокси внутри хранит ссылку на реальный объект и добавляет функциональность (например, кеширование)
- Клиент взаимодействует через прокси вместо прямого объекта

🔗 Шаг 13:

Миссия 13

Ваша задача — упорядочить шаги применения шаблона «Template Method» для обучения нейросети с вариациями:

1. Определение шаблонного алгоритма (скелет метода обучения)
2. Создание абстрактных шагов (например, forward, loss, backward)
3. Реализация подклассов, которые заполняют абстрактные шаги
4. Запуск «template method», чтобы выполнить весь процесс обучения

Подсказка: Подклассы реализуют лишь части алгоритма, общий скелет остаётся в базовом классе.

Ответ:

- Определение шаблонного алгоритма (скелет метода обучения)
- Создание абстрактных шагов (например, forward, loss, backward)
- Реализация подклассов, которые заполняют абстрактные шаги
- Запуск «template method», чтобы выполнить весь процесс обучения

🔗 Шаг 14:

Миссия 14

При разработке ядра с отложенной инициализацией ресурсов: упорядочите шаги применения шаблона «Lazy Initialization»:

1. Объявление ресурса как lazy (лениво инициализируемого)
2. При первом обращении проверка, инициализирован ли ресурс
3. **Если нет** — создание (инициализация) ресурса
4. Возврат ресурса клиенту

Подсказка: Ленивость даёт ресурс только при необходимости, не раньше.

Ответ:

- Объявление ресурса как lazy (лениво инициализируемого)

- При первом обращении проверка, инициализирован ли ресурс
- **Если нет** — создание (инициализация) ресурса
- Возврат ресурса клиенту

🔮 Шаг 15:

Миссия 15

Вы интегрируете шаблон «Mediator» в систему координации модулей обработки. Упорядочьте шаги:

1. Определение центрального посредника (mediator)
2. Модули взаимодействуют не напрямую друг с другом
3. Модули отправляют сообщения посреднику
4. Посредник перенаправляет сообщения между модулями по логике

Подсказка: Mediator устраняет прямые связи между модулями, управляя коммуникацией.

Ответ:

- Определение центрального посредника (mediator)
- Модули взаимодействуют не напрямую друг с другом
- Модули отправляют сообщения посреднику
- Посредник перенаправляет сообщения между модулями по логике

🔮 Шаг 16:

Миссия 16

Финальный квест: упорядочите комплексный порядок, если вы хотите сочетать множество шаблонов (Factory, Strategy, Observer, Facade) в одном ядре:

1. Определить абстракции и интерфейсы для всех частей
2. Использовать фабрику для создания объектов стратегии и обработчиков
3. Подписать наблюдателей на события изменений состояния
4. Обеспечить фасад как единый интерфейс к подсистемам

Подсказка: Сначала интерфейсы, затем фабрика, затем подписка, затем фасад закрывает детали.

Ответ:

- Определить абстракции и интерфейсы для всех частей
- Использовать фабрику для создания объектов стратегии и обработчиков
- Подписать наблюдателей на события изменений состояния
- Обеспечить фасад как единый интерфейс к подсистемам

📖 Урок 4 - 🎯 Квест модифицируй ResNet под свою задачу

🔮 Шаг 01:

Миссия 1

Вы — инженер ИИ в стартапе, и решили адаптировать ResNet для классификации медицинских снимков (где мало данных). Какое простое изменение архитектуры вы попробуете первым?

Подсказка: Начните с минимальных правок, не ломая базовую структуру.

Ответ:

- **Уменьшить число блоков в каждом слое**
- Полностью убрать остаточные связи

- Замена всех свёрток на полносвязные слои

🔧 Шаг 02:

Миссия 2

🔧 При адаптации ResNet для сегментации изображений вашей команде нужно выбрать тип выхода. Какой выходной слой логично прикрутить к модифицированной ResNet?

Подсказка: Подумайте, потребуется ли плотное предсказание по пикселям.

Ответ:

- **Свертка с 1×1 фильтрами + softmax по классам пикселя**
- Один нейрон с сигмоидой на весь выход
- MaxPooling до скалярного выхода
- Полносвязный слой на весь фичемап

🔧 Шаг 03:

Миссия 3

🔧 Вы хотите ускорить ResNet для мобильного приложения. Какой из вариантов уменьшения вычислений — первый из тех, что вы испытаете?

Подсказка: Сохраняйте совместимость со слоями skip-connections.

Ответ:

- **Замена обычных свёрток на глубинные сепарабельные**
- Уборка всех BatchNorm
- Замена Relu на сигмоиду
- Увеличение числа каналов до очень большого

🔧 Шаг 04:

Миссия 4

🔧 Во время обучения модифицированной ResNet вы замечаете, что градиенты исчезают в глубоких блоках. Что разумно попробовать первым?

Подсказка: Остаточные связи помогают, но можно усилить их эффективность.

Ответ:

- **Добавить масштабирование в skip-соединения (alpha-коэффициент)**
- Убрать все активации
- Использовать сигмоидные блоки вместо Relu
- Судорожно уменьшить learning rate до почти нуля

🔧 Шаг 05:

Миссия 5

🔧 Ваши эксперименты показали переобучение на мелком наборе данных. Что вы попробуете, модифицируя ResNet?

Подсказка: Искусственное расширение данных или регуляризация внутри сети часто помогают.

Ответ:

- **Добавить dropout внутри residual-блоков**
- Увеличить размер сети
- Удалить skip-связи
- Убрать всю регуляризацию

🔮 Шаг 06:

Миссия 6

Вы собираетесь «заморозить» первые слои ResNet при дообучении. Почему это может быть полезно?

Подсказка: Некоторые слои уже выучили базовые фильтры, и менять их — риск.

Ответ:

- Чтобы сохранить базовые признаки и не разрушить их при дообучении
- Чтобы увеличить число обучаемых параметров
- Чтобы ускорить вычисления в переднем блоке

🔮 Шаг 07:

Миссия 7

🔗 Сопоставьте термины архитектурных модификаций ResNet с кратким описанием:

Подсказка: Подумайте, что именно меняют — структуру блоков, количество каналов, или связи.

Термины:

Ответ:

- ResNet-довгий (deep)
- ResNet-широкий (wide)
- ResNet с bottleneck
- ResNet с squeeze-and-excitation

Описания:

1. Увеличение числа каналов внутри блоков при сохранении числа слоёв
 2. Добавление механизмов внимания внутри блока
 3. Использование трёхшаговых bottleneck блоков с уменьшением и расширением каналов
 4. **Увеличение глубины** — больше residual-блоков
- **ResNet-долгий (deep)** — Увеличение глубины — больше residual-блоков
 - **ResNet-широкий (wide)** — Увеличение числа каналов внутри блоков при сохранении числа слоёв
 - **ResNet с bottleneck** — Использование трёхшаговых bottleneck блоков с уменьшением и расширением каналов
 - **ResNet с squeeze-and-excitation** — Добавление механизмов внимания внутри блока

🔮 Шаг 08:

Миссия 8

⚙️ Разработчик вашего проекта предлагает несколько техник регуляризации модифицированной ResNet. Сопоставьте технику и её эффект.

Подсказка: Одни техники борются с переобучением, другие — с шумом.

Регуляризации:

Ответ:

- DropBlock
- ShakeDrop
- Label smoothing
- Stochastic depth

Эффекты:

1. Случайное удаление целых каналов внутри слоя
 2. Умягчение целевых меток для уменьшения переобучения
 3. Случайное «отключение» целых слоёв в ResNet во время обучения
 4. Стохастическое смешивание путей прямого и обратного блока
- **DropBlock** — Случайное удаление целых каналов внутри слоя
 - **ShakeDrop** — Стохастическое смешивание путей прямого и обратного блока
 - **Label smoothing** — Умягчение целевых меток для уменьшения переобучения
 - **Stochastic depth** — Случайное «отключение» целых слоёв в ResNet во время обучения

🔧 Шаг 09:

Миссия 9

🔗 Вы и ваша команда решили ввести мультизадачность (multi-task) на базе вашей модифицированной ResNet. Сопоставьте задачу с изменением архитектуры:

Подсказка: Подумайте, какие ответы требуются от сети одновременно.

Задачи:

Ответ:

- Классификация + сегментация
- Регрессия координат объекта
- Классификация видов + оценка качества (score)
- Сегментация + выделение границ (edge detection)

Модификации:

1. Две головы: softmax + 1×1 conv выводу пиксельно
 2. Выход softmax + линейная регрессия для координат
 3. Объединение обучения сегментации и edge метки как двум каналам
 4. Ветка softmax и параллельная ветка регрессии качества (один neuron)
- **Классификация + сегментация** — Две головы: softmax + 1×1 conv выводу пиксельно
 - **Регрессия координат объекта** — Выход softmax + линейная регрессия для координат
 - **Классификация видов + оценка качества (score)** — Ветка softmax и параллельная ветка регрессии качества (один neuron)
 - **Сегментация + выделение границ (edge detection)** — Объединение обучения сегментации и edge метки как двум каналам

🔧 Шаг 10:

Миссия 10

⌘ При подготовке продакшен-версии вашей модифицированной ResNet нужно учитывать оптимизации. Сопоставьте оптимизацию и её преимущество:

Подсказка: Подумайте о скорости вывода и размерах модели.

Оптимизации:

Ответ:

- Квантование (quantization)
- Прюннинг (pruning)
- Сжатие весов (weight compression)
- Fuse BatchNorm + Conv

Преимущества:

1. Уменьшение точности с плавающей запятой, снижение памяти и ускорение
 2. Удаление наименее значимых весов, снижение параметров и FLOPs
 3. Объединение слоёв во время inference ускоряет прохождение
 4. Кодирование весов компактно сжатым представлением
- **Квантование (quantization)** — Уменьшение точности с плавающей запятой, снижение памяти и ускорение
 - **Прюннинг (pruning)** — Удаление наименее значимых весов, снижение параметров и FLOPs
 - **Сжатие весов (weight compression)** — Кодирование весов компактно сжатым представлением
 - **Fuse BatchNorm + Conv** — Объединение слоёв во время inference ускоряет прохождение

Шаг 11:

Миссия 11

 Упорядочьте этапы, которые вы будете выполнять при подготовке вашей модифицированной ResNet к продакшену:

1. Квантование модели
2. Обучение и валидация
3. Прюннинг и сжатие весов
4. Конвертация модели в формат inference (ONNX / TFLite и др.)
5. Тонкая настройка после оптимизаций (fine-tuning)

Подсказка: Вы сначала тренируете, затем оптимизируете, затем экспортируете.

Ответ:

- Обучение и валидация
- Прюннинг и сжатие весов
- Квантование модели
- Тонкая настройка после оптимизаций (fine-tuning)
- Конвертация модели в формат inference (ONNX / TFLite и др.)

Шаг 12:

Миссия 12

Упорядочите виды тестирования архитектуры или модели при подготовке к продакшену:

1. Тесты на корректность вывода (unit tests для блоков)
2. Тесты на производительность (latency / throughput)
3. Тесты интеграции (встроить модель в пайплайн)
4. A/B тестирование в продакшене
5. Тесты на устойчивость к шуму / adversarial

Подсказка: Сначала локальные тесты, потом нагрузка, потом интеграция, затем запуск в реальном окружении.

Ответ:

- Тесты на корректность вывода (unit tests для блоков)
- Тесты интеграции (встроить модель в пайплайн)
- Тесты на производительность (latency / throughput)
- Тесты на устойчивость к шуму / adversarial

- A/B тестирование в продакшене

Шаг 13:

Миссия 13

🔧 Упорядочите шаги обновления вашей модели в продакшене:

1. Подготовка новой версии модели (обучение + оптимизация)
2. Запуск canary-релиза / деплой части трафика
3. Слежение метрик (latency, accuracy, error)
4. Откат / полный деплой при успехе
5. Мониторинг и логирование в проде

Подсказка: Минимизируйте риск, постепенно выкатывая новую модель.

Ответ:

- Подготовка новой версии модели (обучение + оптимизация)
- Запуск canary-релиза / деплой части трафика
- Слежение метрик (latency, accuracy, error)
- Мониторинг и логирование в проде
- Откат / полный деплой при успехе

Шаг 14:

Миссия 14

🕒 Упорядочите слои в одном residual-блоке (bottleneck) модифицированной ResNet с SE-модулем:

1. 1×1 conv для сжатия каналов
2. 3×3 conv
3. Squeeze-and-Excitation блок
4. 1×1 conv для расширения каналов
5. Складывание с skip-связью и активация

Подсказка: SE обычно применяется после свёрток и перед суммированием.

Ответ:

- 1×1 conv для сжатия каналов
- 3×3 conv
- Squeeze-and-Excitation блок
- 1×1 conv для расширения каналов
- Складывание с skip-связью и активация

Шаг 15:

Миссия 15

🔧 Упорядочите этапы трассировки и оптимизации в inference-движке для модифицированной ResNet:

1. Fusing Conv + BatchNorm
2. Квантование весов и активаций
3. Топологическая оптимизация графа (удаление лишних узлов)
4. Планирование памяти / размещение буферов
5. Запуск inference и верификация на тестовых данных

Подсказка: Граф сначала очищают, потом оптимизируют веса, потом планируют память.

Ответ:

- Топологическая оптимизация графа (удаление лишних узлов)
- Fusing Conv + BatchNorm
- Квантование весов и активаций
- Планирование памяти / размещение буферов
- Запуск inference и верификация на тестовых данных

 Шаг 16:

Миссия 16

 Упорядочите шаги обработки потока данных на вход модели в продакшене (с pre- и post-обработкой):

1. Нормализация / стандартизация входного изображения
2. Аугментации (если применяется онлайн)
3. Пакетная сборка батчей и паддинг
4. Выполнение inference
5. Постобработка: преобразование вывода к формату пользователя

Подсказка: Подумайте, что должно быть до inference, а что — после.

Ответ:

- Нормализация / стандартизация входного изображения
- Аугментации (если применяется онлайн)
- Пакетная сборка батчей и паддинг
- Выполнение inference
- Постобработка: преобразование вывода к формату пользователя

 Шаг 17:

Миссия 17

 Финальный квест: упорядочите весь жизненный цикл модифицированной ResNet от идеи до её поддержки в продакшене:

1. Формулировка задачи и требования бизнес-заказчика
2. Дизайн архитектуры и прототипирование (модификация ResNet)
3. Обучение, валидация и гиперпараметрический поиск
4. Оптимизация (pruning, quantization и др.)
5. Интеграция в inference-движок и тестирование
6. Деплой, мониторинг и итерации после запуска

Подсказка: Начните с постановки задачи, заканчивая реальной поддержкой в продакшене.

Ответ:

- Формулировка задачи и требования бизнес-заказчика
- Дизайн архитектуры и прототипирование (модификация ResNet)
- Обучение, валидация и гиперпараметрический поиск
- Оптимизация (pruning, quantization и др.)
- Интеграция в inference-движок и тестирование
- Деплой, мониторинг и итерации после запуска

📖 Урок 5 - 📐 Архитектуры ViT, DeiT, Swin Transformer

🔮 Шаг 01:

Миссия 1

В команде стартапа Alice оказывается, что Vision Transformer (ViT) разбивает изображение на фрагменты-заплатки (patches). Что из нижеописанного соответствует этому шагу?

Подсказка: подумайте, как делится изображение перед подачей в трансформер.

Ответ:

- **Разбиение изображения на фиксированные неперекрывающиеся патчи**
- Случайное обрезание прямоугольников
- Предварительное сглаживание фильтрами
- Подавление низкочастотных компонент

🔮 Шаг 02:

Миссия 2

🔧 Bob изучает DeiT и хочет понять, чем оно отличается от ViT. Какую ключевую инновацию внедряет DeiT?

Подсказка: обратите внимание на обучение без большого внешнего датасета.

Ответ:

- **Трансфер обучения с помощью distillation token**
- Использование рекуррентных блоков
- Замена внимания на свёртки
- Новая функция потерь на пиксельном уровне

🔮 Шаг 03:

Миссия 3

👩 Carol хочет применить Swin Transformer к медицинским снимкам: она узнаёт, что Swin делает «локальное окно внимания». Что это значит?

Подсказка: подумайте, что означает «окно» в контексте внимания.

Ответ:

- **Внимание считается внутри небольших окон неперекрывающихся блоков**
- Внимание считается глобально сразу на все токены
- **Окно** — это фильтр свёртки перед вниманием
- **Окно** — это подмножество слоёв трансформера

🔮 Шаг 04:

Миссия 4

🔗 Dave собирает гибридную архитектуру: часть сети — CNN, часть — ViT. Он хочет узнать, какое преимущество даёт использование ViT в конце CNN-подсети. Что он получает?

Подсказка: подумайте об общей способности внимания моделировать глобальные зависимости.

Ответ:

- **Возможность глобального взаимодействия между патчами**
- Снижение числа параметров сети
- Автоматическое выравнивание границ свёрток
- Избегание обучения внимания

🔮 Шаг 05:

Миссия 5

👁 Emma замечает, что при обучении DeiT может быть использован «distillation token». Какая его роль?

Подсказка: вспомните, как знания передаются от учителя к студенту.

Ответ:

- Он помогает трансферу информации от модели-учителя к модели-студенту
- Он заменяет CLS-токен
- Он используется как шумовой вектор
- Он выполняет агрегацию предсказаний

🔮 Шаг 06:

Миссия 6

Frank анализирует компоненты архитектур ViT, DeiT и Swin. Сопоставьте компонент с его описанием.

Компоненты:

Ответ:

- Patch Embedding
- Distillation Token
- Shifted Windows
- Self-Attention

Описания:

1. Механизм внимания между всеми токенами одновременно.
2. Токен, используемый для дистилляции знаний от учительской модели.
3. Патчи преобразуются в векторы перед подачей в трансформер.
4. Окна внимания смещаются для связи между блоками.

Подсказка: обратите внимание на детали, что делает каждая архитектура особенной.

- **Patch Embedding** — Патчи преобразуются в векторы перед подачей в трансформер.
- **Distillation Token** — Токен, используемый для дистилляции знаний от учительской модели.
- **Shifted Windows** — Окна внимания смещаются для связи между блоками.
- **Self-Attention** — Механизм внимания между всеми токенами одновременно.

🔮 Шаг 07:

Миссия 7

✂ Gina сравнивает различные типы внимания. Сопоставьте термин и его характерную черту.

Термины:

Ответ:

- Global Attention
- Local Attention
- Window Attention
- Cyclic Shift

Характерные черты:

1. Каждый токен может взаимодействовать со всеми остальными.
2. Взаимодействие ограничено внутри окна фиксированного размера.
3. Окна смещаются циклически для улучшения соединения между окнами.
4. Взаимодействие ограничено соседними токенами вокруг центра.

Подсказка: некоторые термины очень близки — читайте точно.

- **Global Attention** — Каждый токен может взаимодействовать со всеми остальными.
- **Local Attention** — Взаимодействие ограничено соседними токенами вокруг центра.
- **Window Attention** — Взаимодействие ограничено внутри окна фиксированного размера.
- **Cyclic Shift** — Окна смещаются циклически для улучшения соединения между окнами.

Шаг 08:

Миссия 8

 Ivan разрабатывает пайплайн обучения DeiT с дистилляцией. Расположите шаги в порядке:

1. Собрать модель-учитель и обучить её
2. Инициализировать модель-студент
3. Совместное обучение студента с учетом сигнала от учителя и истинных меток
4. Оценить студентскую модель на валидации

Подсказка: сначала нужна «мудрость» учителя, затем студент учится и проверяется.

Ответ:

- Собрать модель-учитель и обучить её
- Инициализировать модель-студент
- Совместное обучение студента с учетом сигнала от учителя и истинных меток
- Оценить студентскую модель на валидации

Шаг 09:

Миссия 9

Julia строит Swin сеть. Расположите следующие модули в порядке прохождения данных:

1. Shifted Window Attention
2. Patches & Patch Embedding
3. Merge & Patch Merging
4. Window Attention

Подсказка: сначала формируются патчи, потом внимание, потом слияние, затем новые окна сдвигаются.

Ответ:

- Patches & Patch Embedding
- Window Attention
- Merge & Patch Merging
- Shifted Window Attention

Шаг 10:

Миссия 10

✳ Kevin оптимизирует inference Swin для мобильного приложения. Расположите оптимизации по возрастанию сложности внедрения:

1. Квантование весов
2. Пулинг перед трансформером
3. Сжатие с помощью дистиллятора
4. Удаление слоёв через прунинг

Подсказка: самые простые изменения — это преобразование представлений, затем веса, затем удаление.

Ответ:

- Пулинг перед трансформером
- Квантование весов
- Удаление слоёв через прунинг
- Сжатие с помощью дистиллятора

🔮 Шаг 11:

Миссия 11

🔄 Laura экспериментирует: сначала ViT, затем DeiT, затем Swin. Расположите архитектуры по уровню локального внимания (от меньшего к большему):

1. ViT
2. DeiT
3. Swin Transformer

Подсказка: ViT и DeiT используют глобальное внимание, Swin — локальное окно внимания с сдвигом.

Ответ:

- ViT
- DeiT
- Swin Transformer

🔮 Шаг 12:

Миссия 12

🔧 Martin развёртывает модель на продакшн. Расположите этапы развёртывания трансформера:

1. Конвертация в оптимальный формат (ONNX, TorchScript и др.)
2. Тестирование производительности (латентность, память)
3. Развёртывание в среде (сервер, облако, edge)
4. Мониторинг и обратная связь

Подсказка: сначала нужно подготовить модель, затем проверить, запустить, потом следить.

Ответ:

- Конвертация в оптимальный формат (ONNX, TorchScript и др.)
- Тестирование производительности (латентность, память)
- Развёртывание в среде (сервер, облако, edge)
- Мониторинг и обратная связь

🔮 Шаг 13:

Миссия 13

📦 Nicole хочет уменьшить размер модели Swin для edge-устройства. Что она может попробовать первой шагом?

Подсказка: начните с наименее разрушительных методов.

Ответ:

- **Квантование до 8-бит**
- Полное замещение слоя внимания
- Удаление всех сдвигов окон
- Сокращение больших окон

🔮 Шаг 14:

Миссия 14

Oliver сталкивается с границей памяти GPU при обучении Swin на большом изображении. Что разумно сделать?

Подсказка: подумайте, как уменьшить нагрузку на память.

Ответ:

- **Уменьшить размер батча**
- Увеличить число слоёв
- Увеличить размер окна внимания
- Добавить дропаут

🔮 Шаг 15:

Миссия 15

🕒 Patricia пытается объяснить менеджеру: «зачем Swin Transformer удобнее для данных высокой разрешающей способности?» Какой аргумент будет наиболее точным?

Подсказка: Think глобальность vs локальность и адаптивность.

Ответ:

- **Он масштабируется на большие изображения, сохраняя локальное внимание через окна и свёртки прихода**
- Он полностью заменяет свёртки
- Он требует меньше данных для обучения
- Он не использует механизм внимания

🔮 Шаг 16:

Миссия 16

🔲 Quentin финализирует курс: он хочет выбрать архитектуру для общего качества и скорости. Что часто называют компромиссом между ними?

Подсказка: подумайте, как архитектура влияет на латентность vs точность.

Ответ:

- **Сложность внимания и размер окна**
- Количество цветовых каналов
- Порядок нормализации
- Использование residual connection

Модуль 4 - Функции потерь — сердце обучения

Урок 1 - Focal, Dice, Triplet, Contrastive

Шаг 01:

Миссия 1

Анна тренирует модель сегментации опухолей и хочет уменьшить влияние легко классифицируемых пикселей. Какую функцию потерь ей стоит попробовать?

Подсказка: Focal loss усиливает внимание к трудно классифицируемым примерам.

Ответ:

- **Focal loss**
- MSE loss
- Binary cross-entropy без взвешивания
- KL divergence

Шаг 02:

Миссия 2

 Борис работает над задачей сегментации язв на коже и хочет оптимизировать пересечение масок. Какую функцию потерь он может рассмотреть?

Подсказка: Dice loss напрямую основана на метрике пересечения / объединения.

Ответ:

- **Dice loss**
- Hinge loss
- Poisson loss
- Huber loss

Шаг 03:

Миссия 3

 Оля работает над системой рекомендательной фильтрации: ей нужно заставить похожие пары сидеть ближе в эмбединг-пространстве. Какую функцию ей стоит применить?

Подсказка: Contrastive loss минимизирует дистанцию между схожими парами и максимизирует — между разными.

Ответ:

- **Contrastive loss**
- Mean Absolute Error
- Categorical cross-entropy
- Cosine similarity loss

Шаг 04:

Миссия 4

 Дан набор эмбедингов лиц, и Никита хочет, чтобы уйти от очень близких негативных примеров: какую loss использовать?

Подсказка: Triplet loss работает с анкером, положительным и негативным примером.

Ответ:

- **Triplet loss**

- Focal loss
- Dice loss
- Poisson loss

🔮 Шаг 05:

Миссия 5

📖 Мария комбинирует сегментацию и классификацию: на выходе есть маска и метка класса. Какую стратегию потерь разумно использовать?

Подсказка: Вы можете комбинировать Dice (маска) + Cross-entropy (класс).

Ответ:

- **Dice loss + cross-entropy**
- **Weighted sum (Dice + CE)**
- Только MSE
- Только KL divergence

🔮 Шаг 06:

Миссия 6

🧠 Павел обучает модель с Triplet loss и хочет пересмотреть терминологию. Сопоставьте термин с определением.

Подсказка: Анкер — ключевой пример, позитив — близкий, негатив — удалённый.

Термины:

Ответ:

- Анкер (anchor)
- Позитивный пример (positive)
- Негативный пример (negative)
- Марджин (margin)

Определения:

1. Пример, который должен располагаться далеко от анкера
2. Минимальное расстояние между позитивом и негативом в loss
3. Образец, к которому позитив и негатив относятся
4. Пара примеров, которую хотят сблизить

- **Анкер (anchor)** — Образец, к которому позитив и негатив относятся
- **Позитивный пример (positive)** — Пара примеров, которую хотят сблизить
- **Негативный пример (negative)** — Пример, который должен располагаться далеко от анкера
- **Марджин (margin)** — Минимальное расстояние между позитивом и негативом в loss

🔮 Шаг 07:

Миссия 7

🔍 Катя экспериментирует с функциями Focal, Dice и Cross-entropy. Сопоставьте loss с её свойством.

Подсказка: Focal уменьшает вес лёгких примеров, Dice оптимизирует пересечение.

Функции:

Ответ:

- Focal loss
- Dice loss
- Cross-entropy loss

Свойства:

1. Работает хорошо при несбалансированных классах, усиливая редкие случаи
 2. Оптимизирует метрику пересечения / объединения в сегментации
 3. Стандартная loss для классификации, логарифмическая ошибка
- **Focal loss** — Работает хорошо при несбалансированных классах, усиливая редкие случаи
 - **Dice loss** — Оптимизирует метрику пересечения / объединения в сегментации
 - **Cross-entropy loss** — Стандартная loss для классификации, логарифмическая ошибка

 Шаг 08:

Миссия 8

✧ Иван интегрирует contrastive loss в графическую модель. Сопоставьте термин с ролью:

Подсказка: Contrastive соединяет пары, понятия «похожие» и «разные» примеры важны.

Термины:

Ответ:

- Пара позитивная
- Пара негативная
- Margin

Роли:

1. Примеры, которые хотят сблизить в эмбединг-пространстве
 2. Примеры, которые хотят разнести в эмбединг-пространстве
 3. Минимальное требуемое расстояние между негативами
- **Пара позитивная** — Примеры, которые хотят сблизить в эмбединг-пространстве
 - **Пара негативная** — Примеры, которые хотят разнести в эмбединг-пространстве
 - **Margin** — Минимальное требуемое расстояние между негативами

 Шаг 09:

Миссия 9

Сергей изучает различия между Dice и IoU. Сопоставьте формулу с названием.

Подсказка: $Dice = 2 \cdot |A \cap B| / (|A| + |B|)$, $IoU = |A \cap B| / |A \cup B|$.

Формулы:

Ответ:

- $2 \cdot |A \cap B| / (|A| + |B|)$
- $|A \cap B| / |A \cup B|$
- $1 - (|A \cap B| / |A \cup B|)$

Названия:

1. Dice coefficient
2. IoU (Jaccard index)
3. Jaccard loss

- $2 \cdot |A \cap B| / (|A| + |B|)$ — Dice coefficient
- $|A \cap B| / |A \cup B|$ — IoU (Jaccard index)
- $1 - (|A \cap B| / |A \cup B|)$ — Jaccard loss

🔮 Шаг 10:

Миссия 10

🔮 Лена комбинирует contrastive и triplet loss. Сопоставьте цель с loss:

Подсказка: Contrastive рассматривает пары, triplet — триплеты.

Цель:

Ответ:

- Минимизировать расстояние для схожих пар, максимизировать для разных
- Минимизировать ссылку анкер-позитив + маргинализацию негативов

Loss:

1. Contrastive loss
2. Triplet loss

- **Минимизировать расстояние для схожих парами, максимизировать для разных** — Contrastive loss
- **Минимизировать ссылку анкер-позитив + маргинализацию негативов** — Triplet loss

🔮 Шаг 11:

Миссия 11

🔮 Расположите стадии работы с Triplet loss от подготовки до обучения:

1. Выбор образцов: якорь, позитив, негатив
2. Вычисление расстояний между парами
3. Применение марджина и составление loss
4. Обратное распространение и оптимизация
5. Предобработка и нормализация эмбедингов

Подсказка: Сначала готовим эмбединги, потом составляем триплеты, затем считаем loss, и только после — оптимизация.

Ответ:

- Предобработка и нормализация эмбедингов
- Выбор образцов: якорь, позитив, негатив
- Вычисление расстояний между парами
- Применение марджина и составление loss
- Обратное распространение и оптимизация

🔮 Шаг 12:

Миссия 12

Расставьте шаги обучения с Contrastive loss:

1. Генерация положительных и негативных пар
2. Слежение за апсэмплингом трудных негативов (hard negatives)
3. Вычисление loss: позитивные близко, негативы далеко
4. Обратное распространение и шаг оптимизации

5. Нормализация эмбеддингов (например, L2-норм)

Подсказка: Контрастивная оптимизация требует заранее сформированных пар и баланса между позитивами и негативами.

Ответ:

- Генерация положительных и негативных пар
- Слежение за апсэмплингом трудных негативов (hard negatives)
- Вычисление loss: позитивные близко, негативы далеко
- Обратное распространение и шаг оптимизации
- Нормализация эмбеддингов (например, L2-норм)

Шаг 13:

Миссия 13

🕒 У Вики комбинированная loss: focal + dice + contrastive для мультизадачи (сегментация + эмбеддинги). Расположите этапы оптимизации:

1. Форвард: вычислить маску и эмбеддинги
2. Составить loss для segmentation (focal + dice)
3. Сформировать пары и вычислить contrastive loss на эмбеддинги
4. Суммировать все части loss с весами
5. Обратное распространение и шаг оптимизации

Подсказка: Вычисление loss по каждому компоненту до объединения — ключ к мультизадачному обучению.

Ответ:

- Форвард: вычислить маску и эмбеддинги
- Составить loss для segmentation (focal + dice)
- Сформировать пары и вычислить contrastive loss на эмбеддинги
- Суммировать все части loss с весами
- Обратное распространение и шаг оптимизации

Шаг 14:

Миссия 14

🕒 Андрей пытается обучить модель с фокусом на редких патологиях и похожих пациентах. Расположите применение функций loss в порядке, как он добавляет уровни сложности:

1. Начать с Dice loss для сегментации
2. Добавить Focal loss для улучшения редких случаев
3. Внедрить Contrastive loss для сопоставления эмбеддингов пациентов
4. Добавить Triplet loss, чтобы расширить негативную разницу
5. Обучить всю модель совместно

Подсказка: Сначала базовая segmentation, затем усиление редких случаев, далее работа с эмбеддингами и более сложные сигналы.

Ответ:

- Начать с Dice loss для сегментации
- Добавить Focal loss для улучшения редких случаев
- Внедрить Contrastive loss для сопоставления эмбеддингов пациентов
- Добавить Triplet loss, чтобы расширить негативную разницу
- Обучить всю модель совместно

🔧 Шаг 15:

Миссия 15

🔧 Расположите уровни диагностики ошибок при комбинированной loss (focal + dice + triplet):

1. Проверка балансировки весов loss-частей
2. Отладка масштаба эмбедингов перед triplet/contrastive
3. Анализ метрик сегментации (Dice, IoU)
4. Проверка распределения дистанций позитивов и негативов
5. Отладка общей производительности и конфигураций обучения

Подсказка: Ошибки могут быть как в части сегментации, так и в части эмбедингов — надо диагностировать отдельно и совместно.

Ответ:

- Проверка балансировки весов loss-частей
- Отладка масштаба эмбедингов перед triplet/contrastive
- Анализ метрик сегментации (Dice, IoU)
- Проверка распределения дистанций позитивов и негативов
- Отладка общей производительности и конфигураций обучения

🔧 Шаг 16:

Миссия 16

🔧 В задачах metric learning часто применяют hard negative mining. Расположите шаги метода hard negative mining:

1. Генерация множества негативных кандидатов для каждого анкера
2. Вычисление расстояний между анкерами и негативами
3. Отбор самых «трудных» негативов (с минимальным расстоянием)
4. Составление triplet-пары с hardest negative
5. Использование этих пар в loss и обучении

Подсказка: Hard negative mining — это способ выбрать негативы, наиболее близкие к анкеру, чтобы «научить» модель лучше.

Ответ:

- Генерация множества негативных кандидатов для каждого анкера
- Вычисление расстояний между анкерами и негативами
- Отбор самых «трудных» негативов (с минимальным расстоянием)
- Составление triplet-пары с hardest negative
- Использование этих пар в loss и обучении

🔧 Шаг 17:

Миссия 17

🔧 Финальная задача: ранжируйте этапы при продакшен-внедрении модели с кастомной комбинированной loss (focal + dice + triplet + contrastive):

1. Провести A/B тестирование модели с loss-комбинацией
2. Собрать эмбединги и метрики на валидации
3. Калибровать веса частей loss и гиперпараметры
4. Интегрировать модель в продакшен пайплайн
5. Мониторить распределения потерь и дистанций на проде

6. Откатить или скорректировать весовые коэффициенты при деградации

Подсказка: В продакшене важны не только точность, но и стабильность, мониторинг и способность к корректировке.

Ответ:

- Собрать эмбединги и метрики на валидации
- Калибровать веса частей loss и гиперпараметры
- Провести A/B тестирование модели с loss-комбинацией
- Интегрировать модель в продакшен пайплайн
- Мониторить распределения потерь и дистанций на проде
- Откатить или скорректировать весовые коэффициенты при деградации

📖 Урок 2 - ✂ Кастомные losses с регуляризацией

💡 Шаг 01:

Миссия 1

Анна разрабатывает нейросеть для классификации изображений и хочет добавить L2-регуляризацию в loss-функцию. Какой из вариантов формулы корректно включает L2-регуляризацию?

Подсказка: Регуляризация обычно добавляется через сумму квадратов весов с коэффициентом.

Ответ:

- **Loss = CrossEntropy + $\lambda * \sum_i w_i^2$**
- Loss = CrossEntropy - $\lambda * \sum_i w_i^2$
- Loss = CrossEntropy + $\lambda * \sum_i |w_i|$
- Loss = CrossEntropy * $\lambda * \sum_i w_i^2$

💡 Шаг 02:

Миссия 2

🐞 Борис внедряет кастомный loss для задачи регрессии и хочет учесть как ошибку MSE, так и регуляризацию L1. Что лучше всего выбрать в качестве общей функции?

Подсказка: Можно просто сложить основную ошибку и регуляризацию с весом.

Ответ:

- **Loss = MSE + $\alpha * \sum_i |w_i|$**
- Loss = MSE * $\alpha * \sum_i |w_i|$
- Loss = $\alpha * \text{MSE} + \sum_i |w_i|$
- Loss = MSE - $\alpha * \sum_i |w_i|$

💡 Шаг 03:

Миссия 3

👤 Светлана работает над нейросетью и хочет, чтобы регуляризация затрагивала только веса, а не смещения (bias). Какая реализация подходит больше всего?

Подсказка: При вычислении регуляризации просто исключите bias-параметры из суммы.

Ответ:

- **$\sum$ over only w, не включая b (bias) в регуляризацию**
- $\sum$ over все параметры, включая bias
- Регуляризация модулей w + bias

- Регуляризация только bias

🔮 Шаг 04:

Миссия 4

🔮 Иван создает loss-функцию для задачи сегментации. Он хочет, чтобы регуляризация была адаптивной: чем большая активация, тем сильнее регуляризация. Что из ниже — разумная идея?

Подсказка: Можно весить регуляризацию функцией от величины параметра.

Ответ:

- **Loss = BaseLoss + $\lambda * \sum_i (w_i^2 * |w_i|)$ (то есть зависимость от величины w_i)**
- Loss = BaseLoss + $\lambda * \sum_i (1 / w_i^2)$
- Loss = BaseLoss $\times \lambda * \sum_i w_i^2$
- Loss = BaseLoss - $\lambda * \sum_i |w_i|$

🔮 Шаг 05:

Миссия 5

🔮 Денис хочет сбалансировать два слагаемых в кастомном loss: основной критерий и регуляризацию. Какой подход помогает настроить их относительный вклад?

Подсказка: Обычно добавляют коэффициент перед регуляризацией.

Ответ:

- **Использовать гиперпараметр λ , который регулирует масштаб регуляризации**
- Всегда ставить коэффициент 1 перед регуляризацией
- Избегать регуляризации и полагаться на данные
- Умножать основной loss на λ

🔮 Шаг 06:

Миссия 6

🔮 Задача: сопоставьте тип регуляризации с его характерным эффектом:

Типы регуляризации:

Ответ:

- L1-регуляризация
- L2-регуляризация
- Dropout как регуляризация
- Elastic Net (смешанная L1 + L2)

Эффекты:

1. Создаёт разреженность (некоторые веса становятся точно ноль)
2. Сглаживает веса, но редко обнуляет их полностью
3. Рандомно отключает нейроны во время обучения
4. Комбинирует плюсы L1 и L2

Подсказка: L1 → нули, L2 → сглаживание. Dropout — про случайное пропадание.

- **L1-регуляризация** — Создаёт разреженность (некоторые веса становятся точно ноль)
- **L2-регуляризация** — Сглаживает веса, но редко обнуляет их полностью
- **Dropout как регуляризация** — Рандомно отключает нейроны во время обучения
- **Elastic Net (смешанная L1 + L2)** — Комбинирует плюсы L1 и L2

🔮 Шаг 07:

Миссия 7

Сопоставьте условие и способ реализации регуляризации в кастомном loss:

Условия:

Ответ:

- Регуляризовать только веса определённого слоя
- Регуляризовать все веса кроме bias
- Регуляризация, взвешенная по величине веса
- Регуляризация, изменяющаяся в течение тренировки (schedule)

Способы:

1. Вычислять Σ только для конкретного слоя
2. Исключать bias-параметры при суммировании регуляризации
3. Умножать регуляризацию на $|w_i|$ или другую функцию от w_i
4. Изменять λ по эпохам (например, линейно или экспоненциально)

Подсказка: Прочитайте каждый подход — что он делает с весами, bias, или как меняется во времени.

- **Регуляризовать только веса определённого слоя** — Вычислять Σ только для конкретного слоя
- **Регуляризовать все веса кроме bias** — Исключать bias-параметры при суммировании регуляризации
- **Регуляризация, взвешенная по величине веса** — Умножать регуляризацию на $|w_i|$ или другую функцию от w_i
- **Регуляризация, изменяющаяся в течение тренировки (schedule)** — Изменять λ по эпохам (например, линейно или экспоненциально)

🔮 Шаг 08:

Миссия 8

🔍 Сопоставьте подход к регуляризации с областью применения:

Подход:

Ответ:

- Регуляризация градиентов (gradient clipping)
- Регуляризация через шум в весах (weight noise)
- Регуляризация активаций (activity regularization)
- Регуляризация через раннюю остановку (early stopping)

Применение / эффект:

1. Ограничение нормы градиентов при апдейте
2. Добавление шума к весам во время обучения
3. Добавление штрафа за большие активации нейронов
4. Остановка тренировки при отсутствии улучшения на валидации

Подсказка: Gradient clipping — контролирует градиенты. Early stopping — прерывание.

- **Регуляризация градиентов (gradient clipping)** — Ограничение нормы градиентов при апдейте
- **Регуляризация через шум в весах (weight noise)** — Добавление шума к весам во время обучения

- **Регуляризация активаций (activity regularization)** — Добавление штрафа за большие активации нейронов
- **Регуляризация через раннюю остановку (early stopping)** — Остановка тренировки при отсутствии улучшения на валидации

🔮 Шаг 09:

Миссия 9

🔍 Сопоставьте вид регуляризации с преимуществом, которое он даёт:

Вид регуляризации:

Ответ:

- L1-регуляризация
- L2-регуляризация
- Dropout
- Early stopping

Преимущества:

1. Снижает переобучение за счёт ограничения величин весов
2. Стимулирует нулевые веса, делая модель разреженной
3. Уменьшает зависимость от отдельных нейронов
4. Прерывает обучение, прежде чем модель начнёт запоминать шум

Подсказка: Если регуляризация «обнуляет», это L1. Dropout — про нейроны.

- **L1-регуляризация** — Стимулирует нулевые веса, делая модель разреженной
- **L2-регуляризация** — Снижает переобучение за счёт ограничения величин весов
- **Dropout** — Уменьшает зависимость от отдельных нейронов
- **Early stopping** — Прерывает обучение, прежде чем модель начнёт запоминать шум

🔮 Шаг 10:

Миссия 10

🔍 Сопоставьте сценарий с подходом регуляризации, который будет наиболее подходящим:

Сценарии:

Ответ:

- **У модели слишком много параметров** — риск переобучения
- **Некоторые слои часто перенастраиваются, а другие** — нет
- Модель начинает «запоминать» шум после определённого числа эпох
- Градиенты внезапно взрываются при обучении RNN

Подходы:

1. Применить L1 / L2 регуляризацию для всех параметров
2. Задать индивидуальные λ для разных слоёв
3. Использовать early stopping
4. Применить градиентное обрезание (gradient clipping)

Подсказка: Подумайте, какая проблема соответствует какому методу.

- **У модели слишком много параметров** — риск переобучения — Применить L1 / L2 регуляризацию для всех параметров

- **Некоторые слои часто перенастраиваются, а другие — нет** — Задать индивидуальные λ для разных слоёв
- **Модель начинает «запоминать» шум после определённого числа эпох** — Использовать early stopping
- **Градиенты внезапно взрываются при обучении RNN** — Применить градиентное обрезание (gradient clipping)

🔮 Шаг 11:

Миссия 11

📖 Расположите этапы внедрения кастомного loss-функциона с регуляризацией:

1. Написание функции потерь с регуляризацией (например, в PyTorch / TensorFlow)
2. Тестирование на небольшом датасете (проверка, что loss спадает)
3. Выбор структуры регуляризации (L1, L2, adaptive, schedule и др.)
4. Подбор коэффициента регуляризации λ через валидацию
5. Интеграция кастомного loss в общий pipeline тренировки

Подсказка: Сначала выбирают структуру, потом пишут, тестируют, интегрируют, а λ подбирают на валидации.

Ответ:

- Выбор структуры регуляризации (L1, L2, adaptive, schedule и др.)
- Написание функции потерь с регуляризацией (например, в PyTorch / TensorFlow)
- Тестирование на небольшом датасете (проверка, что loss спадает)
- Интеграция кастомного loss в общий pipeline тренировки
- Подбор коэффициента регуляризации λ через валидацию

🔮 Шаг 12:

Миссия 12

✂ Устройте последовательность тестов для проверки корректности кастомного loss с регуляризацией:

1. Проверка градиентов на нуле (градиентный тест, имя: grad check)
2. Проверка, что при $w = 0$ регуляризация даёт ноль вклад
3. Проверка на искусственной задаче, что loss уменьшается при корректной оптимизации
4. **Сравнение значений loss на двух одинаковых сетях** — с регуляризацией и без
5. Проверка поведения при больших значениях весов (что регуляризация сильно штрафует)

Подсказка: Убедитесь сначала в тривиальных случаях, затем в сложных, и запустите градиенты.

Ответ:

- Проверка, что при $w = 0$ регуляризация даёт ноль вклад
- Проверка поведения при больших значениях весов (что регуляризация сильно штрафует)
- Проверка на искусственной задаче, что loss уменьшается при корректной оптимизации
- **Сравнение значений loss на двух одинаковых сетях** — с регуляризацией и без
- Проверка градиентов на нуле (градиентный тест, имя: grad check)

🔮 Шаг 13:

Миссия 13

🔍 Упорядочите шаги отладки на продакшен при использовании кастомного loss в рабочей среде:

1. **Логирование компонентов loss** — основного и регуляризации отдельно
2. Внедрение мониторинга метрик качества и переобучения на валидационной выборке
3. Постепенное включение регуляризации (например, $\lambda = 0 \rightarrow \lambda_{\text{target}}$)
4. Сравнение поведения модели с и без регуляризации на смоделированных данных
5. Запуск A/B эксперимента в продакшене с отключённой и включённой регуляризацией

Подсказка: На старте — логировать и сравнивать, затем включать постепенно и тестировать продакшен.

Ответ:

- **Логирование компонентов loss** — основного и регуляризации отдельно
- Сравнение поведения модели с и без регуляризации на смоделированных данных
- Постепенное включение регуляризации (например, $\lambda = 0 \rightarrow \lambda_{\text{target}}$)
- Внедрение мониторинга метрик качества и переобучения на валидационной выборке
- Запуск A/B эксперимента в продакшене с отключённой и включённой регуляризацией

🔮 Шаг 14:

Миссия 14

🧩 Расположите стратегии адаптации λ (коэффициента регуляризации) в порядке возрастания гибкости:

1. Фиксированное λ на всё обучение
2. Линейный рост / убывание λ при эпохах
3. Экспоненциальное изменение λ
4. Обратное изменение λ : $\lambda = \lambda_0 / \sqrt{\text{epoch}}$
5. Автоматическое обновление λ через гиперпараметр-оптимизацию в процессе обучения

Подсказка: Чем сложнее метод — тем большую гибкость он даёт.

Ответ:

- Фиксированное λ на всё обучение
- Линейный рост / убывание λ при эпохах
- Экспоненциальное изменение λ
- Обратное изменение λ : $\lambda = \lambda_0 / \sqrt{\text{epoch}}$
- Автоматическое обновление λ через гиперпараметр-оптимизацию в процессе обучения

🔮 Шаг 15:

Миссия 15

Упорядочите сценарии, в которых регуляризация наиболее критична:

1. Модель с огромным числом параметров и мало данных
2. Режим fine-tuning большой предобученной модели
3. Модель с сильным шумом в данных
4. Модель с очень стабильным и однородным датасетом
5. Ситуация, когда важнее максимальная точность, а переобучение не страшно

Подсказка: Регуляризация особенно важна при малом объёме данных и шуме.

Ответ:

- Модель с огромным числом параметров и мало данных

- Модель с сильным шумом в данных
- Режим fine-tuning большой предобученной модели
- Модель с очень стабильным и однородным датасетом
- Ситуация, когда важнее максимальная точность, а переобучение не страшно

🔮 Шаг 16:

Миссия 16

🔮 Расположите уровни, на которых можно добавить регуляризацию (от низкого к высокому):

1. Регуляризация активаций (penalty на активации нейронов)
2. Регуляризация весов через L1 / L2
3. Шум в параметрах (weight noise)
4. Регуляризация структуры (pruning, sparsity constraint)
5. Глобальные методы: early stopping, контроль переподгонки

Подсказка: Сначала внутри сети, потом структурные, затем глобальные методы.

Ответ:

- Регуляризация активаций (penalty на активации нейронов)
- Регуляризация весов через L1 / L2
- Шум в параметрах (weight noise)
- Регуляризация структуры (pruning, sparsity constraint)
- Глобальные методы: early stopping, контроль переподгонки

🔮 Шаг 17:

Миссия 17

🔮 Финальная миссия: Расположите этапы вывода в продакшен кастомного loss с регуляризациями:

1. Мониторинг статистики loss-компонентов (составной и регуляризации отдельно)
2. Плавное включение регуляризации (λ от нуля к целевому значению)
3. Тестирование в staging-окружении на знакомых данных
4. Контроль метрик на продакшене и сравнение с базовой моделью
5. Обратная коррекция λ или отключение, если переобучение всё ещё растёт

Подсказка: Начинайте с тестов, затем постепенное включение, мониторинг и коррекция.

Ответ:

- Тестирование в staging-окружении на знакомых данных
- Плавное включение регуляризации (λ от нуля к целевому значению)
- Мониторинг статистики loss-компонентов (составной и регуляризации отдельно)
- Контроль метрик на продакшене и сравнение с базовой моделью
- Обратная коррекция λ или отключение, если переобучение всё ещё растёт

📖 Урок 3 - 🎯 Квест реализуй loss для детекции ошибок

🔮 Шаг 01:

Миссия 1

Вы — разработчик ядра обучения, и замечаете, что модель детекции часто предсказывает фон как объект. Что вы проверите в первую очередь?

Подсказка: Ошибочная детекция часто связана с пороговыми значениями или несбалансированностью классов.

Ответ:

- **Порог принятия решения (confidence threshold)**
- Скорость обучения (learning rate)
- Размер батча (batch size)

🔮 Шаг 02:

Миссия 2

🧐 При обучении вы заметили, что одна ошибка (false positive) стоит дороже, чем пропуск объекта (false negative). Что логично сделать с функцией потерь?

Подсказка: Вы можете задать разные веса для типов ошибок.

Ответ:

- **Увеличить вес штрафа за false positive**
- Снизить общий learning rate
- Увеличить momentum

🔮 Шаг 03:

Миссия 3

🕒 Вы добавляете regularization (L_2) к loss для детекции. Как это повлияет на границы принятия решений?

Подсказка: Регуляризация сглаживает веса, уменьшая переобучение.

Ответ:

- **Границы станут менее шумными, более гладкими**
- Границы станут круче, более жёсткими
- Регуляризация не влияет на границы

🔮 Шаг 04:

Миссия 4

🔥 При обучении loss периодически становится NaN. Какое изменение loss-функции может помочь стабилизировать обучение?

Подсказка: В детекции часто используется смягчение логарифма или добавление eps.

Ответ:

- **Добавить ϵ внутрь логарифма (лог-сглаживание)**
- Увеличить количество слоёв
- Увеличить дамп градиентов

🔮 Шаг 05:

Миссия 5

🔧 Вы хотите смешать два компонента loss: classification loss и localisation loss. Какое поведение должно быть при отсутствии объекта (background) в якорном окне?

Подсказка: Если нет объекта, метки координат не заданы.

Ответ:

- **Не использовать localisation loss (обнулить)**

- Добавлять localisation loss как обычно
- Умножать localisation loss на большой коэффициент

🔮 Шаг 06:

Миссия 6

🕒 Вы замечаете, что границы предсказания часто смещены. Какой компонент loss поможет исправить смещение центра объекта?

Подсказка: Используйте отклонение центра x и y как часть локализации.

Ответ:

- **Loss по смещению центра (dx , dy)**
- Loss по ширине/высоте bounding box
- Loss по углу поворота (если есть)

🔮 Шаг 07:

Миссия 7

Назначьте каждому компоненту функции потерь детекции правильное описание:

Компоненты:

Ответ:

- Классификационный loss
- Локализационный loss
- Ограничение на концы рамок (box limits)
- Весовой коэффициент ошибок (α , β)

Описание:

1. Штраф за смещение центра, ширину/высоту рамки
2. Привязка рамок к допустимому диапазону координат
3. Классификация, фон или объект
4. Балансировка важности между компонентами

Подсказка: Классификация отвечает за фон/объект, локализация — координаты; коэффициенты регулируют вклад.

- **Классификационный loss** — Классификация, фон или объект
- **Локализационный loss** — Штраф за смещение центра, ширину/высоту рамки
- **Ограничение на концы рамок (box limits)** — Привязка рамок к допустимому диапазону координат
- **Весовой коэффициент ошибок (α , β)** — Балансировка важности между компонентами

🔮 Шаг 08:

Миссия 8

🕒 Сопоставьте термины с их применением в loss-функции детекции:

Ответ:

- Smooth L1
- Cross-entropy
- Focal Loss
- IoU Loss

1. Снижает влияние легко классифицируемых примеров

2. Традиционный loss для бокс-координат, сглаженный L1
3. Классический логистический loss для classification
4. Учитывает пересечение предсказанной и целевой рамки

Подсказка: Focal — про фокусировку на трудных примерах, IoU — про геометрическое перекрытие.

- **Smooth L1** — Традиционный loss для бокс-координат, сглаженный L1
- **Cross-entropy** — Классический логистический loss для classification
- **Focal Loss** — Снижает влияние легко классифицируемых примеров
- **IoU Loss** — Учитывает пересечение предсказанной и целевой рамки

🔮 Шаг 09:

Миссия 9

🔮 Сопоставьте проблему обучения детекции с её возможным симптомом:

Ответ:

- Класс сильно доминирует (background)
 - Рамки слишком маленькие
 - Неправильные anchor ratios
 - Неустойчивость loss (скачки)
1. Модель не обнаруживает объекты среднего размера
 2. Высокое значение loss на первых эпохах
 3. Большое число false negatives по мелким объектам
 4. Классификационный loss доминирует над локализацией

Подсказка: Подумайте, какая проблема приводит к доминированию фона или к забыванию мелких объектов.

- **Класс сильно доминирует (background)** — Классификационный loss доминирует над локализацией
- **Рамки слишком маленькие** — Большое число false negatives по мелким объектам
- **Неправильные anchor ratios** — Модель не обнаруживает объекты среднего размера
- **Неустойчивость loss (скачки)** — Высокое значение loss на первых эпохах

🔮 Шаг 10:

Миссия 10

Сопоставьте термин и возможное решение для детекционной loss:

Ответ:

- Класс imbalance
 - Нулевые градиенты при отсутствии пересечения
 - Слишком разные масштабы loss-компонентов
 - Модели легко занижают confidence
1. Использовать Focal Loss
 2. Ввести IoU-loss вне пересечения
 3. Нормализовать компоненты и применить веса
 4. Добавить calibration или temperature scaling

Подсказка: Каждая проблема — как слабая сторона loss-функции, подойдёт своё решение.

- **Класс imbalance** — Использовать Focal Loss

- **Нулевые градиенты при отсутствии пересечения** — Ввести IoU-loss вне пересечения
- **Слишком разные масштабы loss-компонентов** — Нормализовать компоненты и применить веса
- **Модели легко занижают confidence** — Добавить calibration или temperature scaling

🔮 Шаг 11:

Миссия 11

🔍 Упорядочьте этапы реализации кастомной loss для детекции:

1. Тестирование на простых примерах
2. Определение формулы для classification и localization
3. Объединение компонентов с весами
4. Добавление сглаживания или ϵ для стабильности
5. Интеграция в training loop и отладка градиентов

Подсказка: Сначала формула, потом стабильность, затем объединение, интеграция и тесты.

Ответ:

- Определение формулы для classification и localization
- Добавление сглаживания или ϵ для стабильности
- Объединение компонентов с весами
- Интеграция в training loop и отладка градиентов
- Тестирование на простых примерах

🔮 Шаг 12:

Миссия 12

🔍 Упорядочьте методы обработки негативных (background) якорей при вычислении loss:

1. Игнорировать hard negative (ощущаются как шум)
2. Выбрать subset негативных примеров (hard negative mining)
3. Вычислить classification loss на выбранных негативных и позитивных примерах
4. Присвоить нулевой localization loss негативам

Подсказка: Сначала фильтрация негативов, потом подсчёт loss.

Ответ:

- Игнорировать hard negative (ощущаются как шум)
- Выбрать subset негативных примеров (hard negative mining)
- Вычислить classification loss на выбранных негативных и позитивных примерах
- Присвоить нулевой localization loss негативам

🔮 Шаг 13:

Миссия 13

🔍 Упорядочите стадии обучения, если loss начал зашкаливать (exploding):

1. Применить градиентный клиппинг
2. Понизить learning rate
3. Добавить ϵ в логарифмы или корни в loss-компонентах
4. Сделать предварительную нормализацию входов или анкерных boxов

Подсказка: Контроль градиентов → снижение шага → стабилизация вычислений → нормализация.

Ответ:

- Применить градиентный клиппинг
- Понизить learning rate
- Добавить ϵ в логарифмы или корни в loss-компонентах
- Сделать предварительную нормализацию входов или анкерных boxов

 Шаг 14:

Миссия 14

 Упорядочите этапы деплоя модели после того, как loss стабилизировался:

1. Проверка поведения loss и метрик на валидации
2. Переобучение модели с включённой loss-функцией на полном датасете
3. Экспорт модели и сохранение весов
4. Тестирование inference с новой loss-компонентой на edge-кейсах
5. Запуск продакшен-инференса

Подсказка: Сначала убедитесь, что всё правильно работает, затем подготовка к вводу в эксплуатацию.

Ответ:

- Проверка поведения loss и метрик на валидации
- Переобучение модели с включённой loss-функцией на полном датасете
- Экспорт модели и сохранение весов
- Тестирование inference с новой loss-компонентой на edge-кейсах
- Запуск продакшен-инференса

 Шаг 15:

Миссия 15

 Упорядочите уровни сложности добавления новых видов ошибок в loss:

1. Добавить кастомный шумовой penalty (например, за нестабильные координаты)
2. Увеличить веса уже существующих ошибок при необходимости
3. Ввести новый компонент (например, аспект ориентации объекта)
4. Интегрировать новый компонент в training pipeline и провести тюнинг

Подсказка: Начинайте с регулировки уже существующего, затем добавляйте новое, и только потом интегрируйте.

Ответ:

- Добавить кастомный шумовой penalty (например, за нестабильные координаты)
- Увеличить веса уже существующих ошибок при необходимости
- Ввести новый компонент (например, аспект ориентации объекта)
- Интегрировать новый компонент в training pipeline и провести тюнинг

 Шаг 16:

Миссия 16

 Упорядочите шаги, если модель ведёт себя устойчиво на train, но на продакшене loss растёт:

1. Собирать реальные данные из продакшена и запускать вычисление loss
2. Проверить распределения входных признаков (drift)
3. Переопределить loss-параметры под новые данные (transfer)

4. Контролировать границы confidence и обновлять thresholds
5. Проводить мониторинг и алерты на рост loss в продакшене

Подсказка: В продакшене важно мониторить drift → обновление loss → контроль порогов → сбор данных.

Ответ:

- Собирать реальные данные из продакшена и запускать вычисление loss
- Проверить распределения входных признаков (drift)
- Переопределить loss-параметры под новые данные (transfer)
- Контролировать границы confidence и обновлять thresholds
- Проводить мониторинг и алерты на рост loss в продакшене

Урок 4 - Метрики Precision, Recall, F1, AUC

Шаг 01:

Миссия 1

Мария обучает сеть классификации спам/не спам. Что из перечисленного — корректная метрика для оценки качества «спам»-класса?

Подсказка: Подумайте: важнее не пропустить спам.

Ответ:

- **Recall (полнота) для класса «спам»**
- Accuracy
- Precision для класса «не спам»

Шаг 02:

Миссия 2

 Иван работает над задачей медицинской диагностики (болен / здоров). Почему AUC важна?

Подсказка: AUC показывает способность модели различать классы на всех порогах.

Ответ:

- **Потому что она учитывает все пороги и баланс между TPR и FPR**
- Потому что она показывает точность предсказания
- Потому что она эквивалентна F1

Шаг 03:

Миссия 3

 Алёна исследует классификатор, у которого много ложных положительных. Что можно улучшить?

Подсказка: Ложные положительные — на что они влияют?

Ответ:

- **Повысить порог принятия положительного решения**
- Увеличить долю положительных в обучающей выборке
- Уменьшить регуляризацию
- Понизить порог принятия положительного решения

Шаг 04:

Миссия 4

🔗 Команда развёртывает модель, и проект-менеджер спрашивает: «Что такое F1-метрика?»
Выбираем самое точное определение:

Подсказка: F1 — гармоническое среднее precision и recall.

Ответ:

- **$F1 = 2 \cdot (\text{precision} \times \text{recall}) / (\text{precision} + \text{recall})$**
- $F1 = \text{precision} + \text{recall}$
- F1 = разница между precision и recall

🔗 Шаг 05:

Миссия 5

👤 Павел сравнивает две модели. У первой precision = 0.9, recall = 0.2, у второй precision = 0.5, recall = 0.5. Какая может быть F1 лучше?

Подсказка: F1 зависит от баланса между precision и recall.

Ответ:

- **Вторая модель может иметь лучший F1**
- Первая обязательно лучше по F1
- Обе дадут одинаковый F1

🔗 Шаг 06:

Миссия 6

📖 Сопоставьте термин и его определение:

Подсказка: Вспоминайте каждую метрику и что она измеряет.

Ответ:

- Precision
 - Recall
 - F1-score
 - AUC
1. Доля правильно предсказанных положительных среди всех предсказанных положительных
 2. Доля правильно предсказанных положительных среди всех реальных положительных
 3. Гармоническое среднее precision и recall
 4. Площадь под ROC-кривой
- **Precision** — Доля правильно предсказанных положительных среди всех предсказанных положительных
 - **Recall** — Доля правильно предсказанных положительных среди всех реальных положительных
 - **F1-score** — Гармоническое среднее precision и recall
 - **AUC** — Площадь под ROC-кривой

🔗 Шаг 07:

Миссия 7

В лаборатории модель сделала следующее:

Ответ:

- True Positives = 80
- False Positives = 20

- False Negatives = 40
- True Negatives = 860

Сопоставьте метрику и её значение (округлите до трёх знаков):

1. Precision
2. Recall
3. F1-score
4. AUC (упрощённо считать ROC аппроксимированную как 0.9)

Подсказка: Используйте формулы $\text{precision} = \text{TP}/(\text{TP}+\text{FP})$, $\text{recall} = \text{TP}/(\text{TP}+\text{FN})$.

- **Precision** — 0.800
- **Recall** — 0.667
- **F1-score** — 0.727
- **AUC** — 0.900

Шаг 08:

Миссия 8

 В стартапе две модели. Модель A: precision высок, recall низок. Модель B: precision низок, recall высок. Что подходят для задач:

Ответ:

- Модель A
 - Модель B
 - Принцип «экономия ложных негативов»
 - Принцип «экономия ложных позитивов»
1. Когда важно минимизировать пропущенные положительные случаи
 2. Когда важно минимизировать ложные тревоги
 3. Более предпочтительна модель A
 4. Более предпочтительна модель B

Подсказка: Вспомните: ложные негативы опасны в медицине, ложноположительные — в спаме.

- **Модель A** — Более предпочтительна модель A
- **Модель B** — Более предпочтительна модель B
- **Принцип «экономия ложных негативов»** — Когда важно минимизировать пропущенные положительные случаи
- **Принцип «экономия ложных позитивов»** — Когда важно минимизировать ложные тревоги

Шаг 09:

Миссия 9

 В компании «DataGuard» вас просят объяснить: как связаны ROC и AUC. Сопоставьте пары:

Ответ:

- ROC-кривая
 - True Positive Rate
 - False Positive Rate
 - AUC
1. График зависимости TPR от FPR при разных порогах
 2. Доля истинно положительных при данном пороге

3. Доля ложноположительных при данном пороге
4. Интеграл под ROC-кривой

Подсказка: ROC показывает компромисс между TPR и FPR при всех порогах.

- **ROC-кривая** — График зависимости TPR от FPR при разных порогах
- **True Positive Rate** — Доля истинно положительных при данном пороге
- **False Positive Rate** — Доля ложноположительных при данном пороге
- **AUC** — Интеграл под ROC-кривой

🔮 Шаг 10:

Миссия 10

✳ Аналитик собрала результаты разных моделей:

Ответ:

- Model1: precision=0.9, recall=0.4
- Model2: precision=0.75, recall=0.6
- Model3: precision=0.6, recall=0.6
- Model4: precision=0.8, recall=0.5

Сопоставьте модель и её ожидаемый F1 (приблизительно):

1. 0.55
2. 0.68
3. 0.60
4. 0.72

Подсказка: $F1 = 2 \cdot PR / (P+R)$. Посчитайте приблизительно для каждой модели.

- **Model1** — 0.55
- **Model2** — 0.68
- **Model3** — 0.60
- **Model4** — 0.72

🔮 Шаг 11:

Миссия 11

📊 Упорядочите шаги построения ROC-кривой:

1. Изменение порога классификации
2. Расчёт TPR и FPR для каждого порога
3. Построение графика точек (FPR, TPR)
4. Интегрирование площади под кривой (AUC)

Подсказка: Начните с порога, потом метрики, затем визуализация, затем площадь.

Ответ:

- Изменение порога классификации
- Расчёт TPR и FPR для каждого порога
- Построение графика точек (FPR, TPR)
- Интегрирование площади под кривой (AUC)

🔮 Шаг 12:

Миссия 12

☞ Упорядочите модели по потенциальной пригодности, если задача — минимизировать ложные негативы (в порядке от наилучшей к худшей):

1. Model A: recall=0.95, precision=0.50
2. Model B: recall=0.85, precision=0.70
3. Model C: recall=0.98, precision=0.30
4. Model D: recall=0.60, precision=0.90

Подсказка: Для вашей задачи ключ — максимально высокий recall, даже ценой precision.

Ответ:

- Model C
- Model A
- Model B
- Model D

💡 Шаг 13:

Миссия 13

⚙ Упорядочите этапы оптимизации метрик для модели:

1. Выбрать нужную метрику (precision, recall или F1)
2. Подобрать порог классификации
3. Анализ кривых precision-recall или ROC
4. Изменение весов классов или выбор функции потерь

Подсказка: Сначала цель, потом порог, затем анализ, затем модификации.

Ответ:

- Выбрать нужную метрику (precision, recall или F1)
- Подобрать порог классификации
- Анализ кривых precision-recall или ROC
- Изменение весов классов или выбор функции потерь

💡 Шаг 14:

Миссия 14

🔧 Упорядочите этапы, как команда вводит мониторинг качества модели в продакшен:

1. Собирать реальные предсказания и истинные метки
2. Переоценивать precision, recall, AUC регулярно
3. Сравнивать метрики с базовой метрикой из валидации
4. **Реагировать** — ретренировать или корректировать порог

Подсказка: Контроль — итерационный, сначала сбор, затем оценка, затем решение.

Ответ:

- Собирать реальные предсказания и истинные метки
- Переоценивать precision, recall, AUC регулярно
- Сравнивать метрики с базовой метрикой из валидации
- **Реагировать** — ретренировать или корректировать порог

💡 Шаг 15:

Миссия 15

📋 Упорядочите шаги анализа ошибки модели, когда F1 хуже ожиданий:

1. **Посмотреть, где модель ошибается** — FP или FN
2. Проанализировать распределение вероятностей
3. Попробовать поменять порог
4. Изменить функцию потерь или веса классов

Подсказка: Выясните природу ошибок, исследуйте вероятности, затем экспериментируйте с порогом и с весами.

Ответ:

- **Посмотреть, где модель ошибается** — FP или FN
- Проанализировать распределение вероятностей
- Попробовать поменять порог
- Изменить функцию потерь или веса классов

🔮 Шаг 16:

Миссия 16

📋 Упорядочите этапы комплексной оценки моделей с точки зрения метрик:

1. Сравнить precision и recall для разных классов
2. Построить кривые Precision-Recall и ROC
3. Посчитать AUC и F1
4. Выбрать модель и порог, исходя из требований задачи

Подсказка: Оценка — это сначала метрики, потом кривые, потом агрегаты, потом выбор.

Ответ:

- Сравнить precision и recall для разных классов
- Построить кривые Precision-Recall и ROC
- Посчитать AUC и F1
- Выбрать модель и порог, исходя из требований задачи

🔮 Шаг 17:

Миссия 17

🌟 Финальный квест: упорядочите все ключевые шаги внедрения метрик (precision, recall, F1, AUC) в жизненный цикл модели:

1. Выбор метрики под задачу
2. Обучение модели
3. Оценка на валидации: precision, recall, AUC
4. Настройка порога и выбор модели
5. Внедрение в продакшен
6. **Мониторинг качества** — пересчёт метрик на живых данных
7. Ретренинг или корректировка при деградации

Подсказка: Жизненный цикл — это сначала выбор, потом обучение, оценка, внедрение, мониторинг и коррекция.

Ответ:

- Выбор метрики под задачу
- Обучение модели
- Оценка на валидации: precision, recall, AUC

- Настройка порога и выбор модели
- Внедрение в продакшен
- **Мониторинг качества** — пересчёт метрик на живых данных
- Ретренинг или корректировка при деградации

Модуль 5 - Балансировка и регуляризация

Урок 1 - Dropout, Label smoothing, Data augmentation

Шаг 01:

Миссия 1

Инженер Анна хочет добавить **dropout** в полносвязный слой, чтобы уменьшить переобучение. Какой параметр она должна настроить?

Подсказка: dropout — это вероятность «отключения» нейронов.

Ответ:

- **dropout rate (например, 0.2 или 0.5)**
- learning rate
- batch size

Шаг 02:

Миссия 2

Разработчик Борис хочет применить **label smoothing** при обучении классификатора. Какой эффект он ожидает?

Подсказка: метки «сглаживаются» – не быть чрезмерно уверенным.

Ответ:

- **Снижение overconfidence и улучшение обобщения**
- Ускорение сходимости к нулевой ошибке на обучении
- Полное устранение ошибок на валидации

Шаг 03:

Миссия 3

Девелопер Катя хочет использовать **data augmentation** для изображений. Что из следующего подходит как операция увеличения данных?

Подсказка: трансформации, не меняющие истинный класс изображения.

Ответ:

- **горизонтальное отражение (flip)**
- **слегка поворот (rotation)**
- замена всех пикселей на случайные
- **небольшой сдвиг (translation)**

Шаг 04:

Миссия 4

Алексей экспериментирует: он применил dropout = 0.9 (90 %). Что, скорее всего, произойдёт?

Подсказка: очень высокая доля отключения делает модель слишком слабой.

Ответ:

- **Модель будет недообучаться (слишком слабая)**
- Модель покажет выдающуюся точность
- Dropout не влияет на итоговую метрику

Шаг 05:

Миссия 5

Юлия вводит $\text{label smoothing} = 0.1$ для задачи с 5 классами. Что будет целевой вероятностью для правильного класса?

Подсказка: обычная метка 1.0 становится чуть меньше, остальные получают часть.

Ответ:

- **0.9**
- 0.1
- 0.2

Шаг 06:

Миссия 6

Инженер Павел сталкивается с терминами. Сопоставьте термин и его определение:

Термины:

Ответ:

- Dropout
- Label smoothing
- Data augmentation
- Overfitting

Определения:

1. Добавление изменённых версий данных (повороты, отражения и т.д.)
2. Случайное «отключение» нейронов в процессе обучения
3. Приведение меток к не «0 или 1», а слегка сглаженным значениям
4. Когда модель слишком точно подстраивается под тренировочные данные и плохо обобщает

Подсказка: подумайте, какая техника борется с уверенностью модели, какая — с недостатком данных.

- **Dropout** — Случайное «отключение» нейронов в процессе обучения
- **Label smoothing** — Приведение меток к не «0 или 1», а слегка сглаженным значениям
- **Data augmentation** — Добавление изменённых версий данных (повороты, отражения и т.д.)
- **Overfitting** — Когда модель слишком точно подстраивается под тренировочные данные и плохо обобщает

Шаг 07:

Миссия 7

 Машинный инженер Инна изучает, как label smoothing влияет на loss -функцию. Сопоставьте утверждение и результат:

Утверждения:

Ответ:

- Сглаженные метки уменьшают разрыв между истинной и предсказанной вероятностью
- Label smoothing полностью устраняет перекокс класса
- Сглаживание меток увеличивает кросс-энтропийный loss
- Label smoothing уменьшает переобучение

Результаты:

1. Мягкий переход между вероятностями классов
2. **Неверно** — перекокс остаётся
3. Иногда loss немного выше на обучении, но лучше на валидации
4. Улучшение обобщения модели

Подсказка: подумайте, что делает сглаживание меток и как это влияет на уверенность модели.

- **Сглаженные метки уменьшают разрыв между истинной и предсказанной вероятностью** — Мягкий переход между вероятностями классов
- **Label smoothing полностью устраняет перекокс класса** — Неверно — перекокс остаётся
- **Сглаживание меток увеличивает кросс-энтропийный loss** — Иногда loss немного выше на обучении, но лучше на валидации
- **Label smoothing уменьшает переобучение** — Улучшение обобщения модели

🔮 Шаг 08:

Миссия 8

🔮 Девелопер Илья строит пайплайн с data augmentation. Сопоставьте трансформацию и её потенциальный недостаток:

Трансформации:

Ответ:

- Случайный поворот 90°
- Изменение яркости
- Масштабирование (zoom)
- Искажение (warp)

Недостатки:

1. Класс может измениться (нарушить смысл)
2. Введён шум, ухудшающий обучение
3. Объекты могут быть обрезаны или уширены некорректно
4. Нарушение геометрии, неприродные формы

Подсказка: каждое преобразование потенциально может исказить фактическую информацию.

- **Случайный поворот 90°** — Класс может измениться (нарушить смысл)
- **Изменение яркости** — Введён шум, ухудшающий обучение
- **Масштабирование (zoom)** — Объекты могут быть обрезаны или уширены некорректно
- **Искажение (warp)** — Нарушение геометрии, неприродные формы

🔮 Шаг 09:

Миссия 9

🔧 Инженер Мария настраивает pipeline регуляризации. Расположите техники от **менее к более агрессивным**:

1. без регуляризации
2. label smoothing 0.1
3. dropout 0.2
4. dropout 0.5 + augmentation

Подсказка: чем сильнее регуляризация — тем активнее «усреднение» или «искажение» сигнала.

Ответ:

- без регуляризации
- label smoothing 0.1
- dropout 0.2
- dropout 0.5 + augmentation

🔧 Шаг 10:

Миссия 10

📊 Савелий делает серию экспериментов. У него есть эти комбинации (augment, dropout, label smoothing). Упорядочите по ожидаемой степени **снижения переобучения** (от слабого к сильному):

1. augmentation only
2. dropout 0.3 + label smoothing 0.1
3. dropout 0.5 only
4. augmentation + dropout 0.5 + label smoothing 0.2

Подсказка: комбинированные методы дают большую регуляризацию.

Ответ:

- augmentation only
- dropout 0.3 + label smoothing 0.1
- dropout 0.5 only
- augmentation + dropout 0.5 + label smoothing 0.2

🔧 Шаг 11:

Миссия 11

Инженер Олег хочет провести ступенчатую регуляризацию. Упорядочите этапы, как он будет их вводить:

1. introduce full augmentation + dropout + smoothing
2. тренировка без regularization
3. добавить dropout умеренный (0.2–0.3)
4. включить label smoothing

Подсказка: начинайте с базовой модели, потом постепенно добавляйте техники.

Ответ:

- тренировка без regularization
- добавить dropout умеренный (0.2–0.3)
- включить label smoothing
- introduce full augmentation + dropout + smoothing

🔗 Шаг 12:

Миссия 12

🕒 В каком случае **label smoothing** может навредить модели?

Подсказка: подумайте про классы-аномалии или редкие классы.

Ответ:

- Когда есть редкие важные классы, и сглаживание «размывает» различие
- Когда модель переобучается
- Когда данных много и разнообразных

🔗 Шаг 13:

Миссия 13

🔗 При data augmentation, какой приём менее рискован для сохранения истинной метки?

Подсказка: минимальные трансформации часто безопаснее.

Ответ:

- небольшой сдвиг или шум, не затрагивающий семантику
- объёмные искажения вручную
- удаление частей изображения

🔗 Шаг 14:

Миссия 14

Инженер Наташа использует Dropout + Label smoothing + Augmentation, но валидационная точность падает. Что стоит проверить первым?

Подсказка: иногда регуляризация чрезмерна.

Ответ:

- уменьшить коэффициенты регуляризации (dropout rate, smoothing)
- проверить корректность аугментаций (не разрушают ли данные)
- немедленно удалить все регуляризации

🔗 Шаг 15:

Миссия 15

При анализе результатов Павлу нужно интерпретировать метрики. Сопоставьте ситуацию и возможную диагностику:

Ситуации:

Ответ:

- Validation loss выше, чем training loss, и gap растёт
- Training loss стабильно низок, validation loss медленно снижается
- Обе метрики высокие, модель плохо учится
- Validation loss выше, но обе кривые близки

Диагностики:

1. недостаточная регуляризация или overfitting
2. слегка жесткая регуляризация, модель не может уложиться в ограничение
3. слишком агрессивная регуляризация / модель слишком слабая
4. регуляризация работает, но возможно слишком сильна

Подсказка: обратите внимание на форму кривых обучения.

- **Validation loss выше, чем training loss, и gap растёт** — недостаточная регуляризация или overfitting
- **Training loss стабильно низок, validation loss медленно снижается** — регуляризация работает, но возможно слишком сильна
- **Обе метрики высокие, модель плохо учится** — слишком агрессивная регуляризация / модель слишком слабая
- **Validation loss выше, но обе кривые близки** — слегка жесткая регуляризация, модель не может уложиться в ограничение

🔧 Шаг 16:

Миссия 16

🔧 Инженер Павел тестирует гиперпараметры. У него 4 конфигурации (augmentation, dropout, label smoothing). Расположите их по ожидаемой производительности (от худшей к лучшей):

1. без augmentation, dropout=0.5, smoothing=0.2
2. augmentation + dropout=0.3, smoothing=0.1
3. augmentation + dropout=0.5, smoothing=0.2
4. augmentation only, smoothing=0.05

Подсказка: часто умеренные комбинации дают лучший баланс между регуляризацией и обучением.

Ответ:

- без augmentation, dropout=0.5, smoothing=0.2
- augmentation + dropout=0.3, smoothing=0.1
- augmentation + dropout=0.5, smoothing=0.2
- augmentation only, smoothing=0.05

📖 Урок 2 - ✂ CutMix, MixUp, AugMix

🔧 Шаг 01:

Миссия 1

Анна внедряет CutMix при обучении классификатора изображений. Что из ниже приведённого — основной эффект CutMix?

Подсказка: Смешивать изображения и метки — это не просто аугментация.

Ответ:

- **Сгладить границу решения между классами**
- Полностью замаскировать объекты на изображении
- Увеличить яркость и контраст
- Добавить шум Гаусса

🔧 Шаг 02:

Миссия 2

Борис сравнивает MixUp и CutMix. Какое отличие MixUp от CutMix?

Подсказка: В MixUp смешиваются полностью изображения, а не фрагменты.

Ответ:

- **MixUp линейно смешивает два полных изображения и метки**

- MixUp вырезает кусочки и вставляет
- MixUp не использует метки
- **MixUp** — частная версия AugMix

🔮 Шаг 03:

Миссия 3

Катя читает про AugMix: что такое основная идея AugMix?

Подсказка: Это путь для создания разнообразия преобразований без разрушения семантики.

Ответ:

- **Комбинировать цепочки простых трансформаций и смешивать с оригиналом**
- Произвольно обрезать изображение
- Добавлять шум и размытие только
- Вращать на случайный угол более 90°

🔮 Шаг 04:

Миссия 4

Вадим записал четыре свойства техник смешивания. Сопоставьте технику с её ключевым эффектом.

Техники:

Ответ:

- CutMix
- MixUp
- AugMix
- Обычная аугментация (flip, rotate и др.)

Эффекты:

1. Слои регуляризуются через случайные сегменты изображения.
2. Сглаживание меток через линейную интерполяцию.
3. Комбинация цепочек простых трансформаций с микшированием.
4. Создание вариативности без смешения с другими примерами.

Подсказка: Подумайте, какая техника действительно смешивает два примера, а какая просто модифицирует один.

- **CutMix** — Слои регуляризуются через случайные сегменты изображения.
- **MixUp** — Сглаживание меток через линейную интерполяцию.
- **AugMix** — Комбинация цепочек простых трансформаций с микшированием.
- **Обычная аугментация (flip, rotate и др.)** — Создание вариативности без смешения с другими примерами.

🔮 Шаг 05:

Миссия 5

На практике Ирина хочет выбрать коэффициенты λ для MixUp и доли вырезаемого патча для CutMix. Сопоставьте параметр с эффектом на регуляризацию:

Параметры:

Ответ:

- λ в MixUp

- Размер патча в CutMix
- Кол-во цепочек в AugMix (width)
- Глубина цепочек в AugMix (depth)

Эффекты:

1. Увеличивает разнообразие трансформаций.
2. Сильнее размывает метки между классами.
3. Смещает акцент на кусочные регуляризации.
4. Увеличивает степень агрегации разных путей.

Подсказка: Про λ : ближе к 0.5 — сильная регуляризация. Про размер патча: больший — сильнее вмешательство.

- **λ в MixUp** — Сильнее размывает метки между классами.
- **Размер патча в CutMix** — Смещает акцент на кусочные регуляризации.
- **Кол-во цепочек в AugMix (width)** — Увеличивает разнообразие трансформаций.
- **Глубина цепочек в AugMix (depth)** — Увеличивает степень агрегации разных путей.

🔧 Шаг 06:

Миссия 6

Денис планирует включить техники смешивания в пайплайн обучения. Отсортируйте этапы в правильном порядке:

1. Применить CutMix или MixUp после базовой аугментации
2. Чтение и загрузка изображения
3. Нормализация и ресайз
4. Применить обычные трансформации (flip, rotate и др.)
5. Передать в модель

Подсказка: Сначала базовая подготовка, потом аугментации, после — смешивание, и только потом в модель.

Ответ:

- Чтение и загрузка изображения
- Нормализация и ресайз
- Применить обычные трансформации (flip, rotate и др.)
- Применить CutMix или MixUp после базовой аугментации
- Передать в модель

🔧 Шаг 07:

Миссия 7

Сергей тестирует, как AugMix влияет на устойчивость модели к шуму. Что он ожидает увидеть?

Подсказка: AugMix усиливает стабильность к мелким искажениям.

Ответ:

- **Меньшее падение точности при зашумлении входа**
- Улучшение на чистых данных, но падение на искажённых
- Полное игнорирование шума
- Падение точности и на чистых данных

🔮 Шаг 08:

Миссия 8

Наташа экспериментирует и записывает эффекты. Сопоставьте наблюдения с возможной причиной:

Наблюдения:

Ответ:

- Модель хуже определяет границы классов
- Сильное ухудшение на мелких деталях
- Улучшение обобщения на разнообразных примерах
- Увеличение времени обучения

Причины:

1. Слишком агрессивный размер патча в CutMix
2. Очень большое $\lambda \sim 0.5$ в MixUp
3. Большая глубина цепочек в AugMix
4. Увеличение вычислительных шагов аугментации

Подсказка: Связь между агрессивностью смешивания и потерей детализации важна.

- **Модель хуже определяет границы классов** — Слишком агрессивный размер патча в CutMix
- **Сильное ухудшение на мелких деталях** — Очень большое $\lambda \sim 0.5$ в MixUp
- **Улучшение обобщения на разнообразных примерах** — Большая глубина цепочек в AugMix
- **Увеличение времени обучения** — Увеличение вычислительных шагов аугментации

🔮 Шаг 09:

Миссия 9

Михаил хочет комбинировать AugMix с MixUp/CutMix. Сопоставьте стратегию комбинирования с её описанием:

Стратегии:

Ответ:

- Mix $\rightarrow$ AugMix
- AugMix $\rightarrow$ Mix
- Выбор случайной техники (Mix/CutMix/AugMix)
- Гибрид внутри батча (разные техники разным примерам)

Описание:

1. Перв сначала смешивание, затем трансформации
2. Сначала трансформации, потом смешивание
3. Случайный выбор каждой техники для каждого примера
4. Разные примеры получают разные техники одновременно

Подсказка: Подумайте, как порядок влияет на эффективность регуляризации.

- **Mix $\rightarrow$ AugMix** — Перв сначала смешивание, затем трансформации
- **AugMix $\rightarrow$ Mix** — Сначала трансформации, потом смешивание
- **Выбор случайной техники (Mix/CutMix/AugMix)** — Случайный выбор каждой техники для каждого примера

- **Гибрид внутри батча (разные техники разным примерам)** — Разные примеры получают разные техники одновременно

🔮 Шаг 10:

Миссия 10

Пётр проектирует обучение с несколькими техниками. Отсортируйте пайплайн по времени применения:

1. AugMix-трансформации
2. MixUp над результатом
3. Обычные аугментации (flip, rotate)
4. CutMix смешивание
5. Нормализация и ресайз

Подсказка: Какой порядок имеет смысл: базовые аугментации → смешивание → более сложные цепочки?

Ответ:

- Обычные аугментации (flip, rotate)
- Нормализация и ресайз
- AugMix-трансформации
- CutMix смешивание
- MixUp над результатом

🔮 Шаг 11:

Миссия 11

Ольга видит, что смешивание снижает наклон сети. Как объяснить это явление?

Подсказка: Смесь примеров заставляет модель быть более гладкой между классами.

Ответ:

- **Регуляризация заставляет границу разделения быть более сглаженной**
- Смешивание уменьшает размер модели
- Смешивание ускоряет обучение
- Смешивание увеличивает число слоёв

🔮 Шаг 12:

Миссия 12

Иван хочет избежать коллизий мячей в датасете (объекты частично пересекаются). Какой метод смешивания может помочь лучше?

Подсказка: В CutMix можно вырезать ненужные части, не смешивая весь объект.

Ответ:

- **CutMix использовать, чтобы вырезать непересекающиеся зоны**
- MixUp обязательно смешивать весь объект
- AugMix автоматически разрезает объекты
- Обычные аугментации решают проблему

🔮 Шаг 13:

Миссия 13

Катя фиксирует, как меняются метрики. Сопоставьте изменение метрики с вероятной техникой:

Изменения метрик:

Ответ:

- Увеличилась точность на редких классах
- Снизился recall на граничных образцах
- Улучшился robustness к шуму
- Снизилась variance между эпохами

Вероятная техника:

1. MixUp
2. CutMix
3. AugMix
4. Смешение нескольких техник

Подсказка: Подумайте, как каждая техника влияет на распределение ошибок и устойчивость.

- **Увеличилась точность на редких классах** — MixUp
- **Снизился recall на граничных образцах** — CutMix
- **Улучшился robustness к шуму** — AugMix
- **Снизилась variance между эпохами** — Смешение нескольких техник

Шаг 14:

Миссия 14

Максим готовит продакшн-pipeline. Отсортируйте логику обработки данных и смешивания при inference (или ТТА):

1. Вычислить предсказание базовой модели
2. Сгенерировать трансформированные версии через AugMix
3. Смешать предсказания (среднее или взвешенное)
4. Нормализация и ресайз

Подсказка: На этапе inference нельзя смешивать входные изображения, но можно усреднять прогнозы.

Ответ:

- Нормализация и ресайз
- Сгенерировать трансформированные версии через AugMix
- Вычислить предсказание базовой модели
- Смешать предсказания (среднее или взвешенное)

Шаг 15:

Миссия 15

Вера хочет комбинировать CutMix и MixUp, но боится дестабилизации обучения. Что можно сделать?

Подсказка: Снижать силу смешивания и динамически регулировать применение.

Ответ:

- **Использовать слабые λ и меньший патч + постепенно усиливать**
- Применять на полном обучении сразу сильный режим
- Применять только на последней эпохе

- Использовать без регуляризации

Шаг 16:

Миссия 16

Финальная задача: вы хотите оценить, какая техника (MixUp / CutMix / AugMix / их комбинация) дала лучший результат. Как вы это сравните объективно?

Подсказка: Сравнивать надо не только по ассигасу, но по устойчивости и стабильности.

Ответ:

- **Сравнить на отложенной выборке + статистически оценить variance и worst-case падение**
- Только по точности на обучающей выборке
- Сравнить по скорости обучения
- Выбрать случайно

Урок 3 - Квест победы переобучение на tiny-dataset

Шаг 01:

Миссия 1

Алексей тренируется на мини-дасете: что первое поможет избежать переобучения?

Подсказка: начинайте с самых простых изменений — «лечим» модель постепенно.

Ответ:

- Увеличить число слоёв
- **Добавить регуляризацию L2**
- Увеличить learning rate
- Удалить все dropout

Шаг 02:

Миссия 2

 Мария заметила, что модель слишком точно подгоняет тренировочные данные и плохо обобщается. Что разумно попробовать?

Подсказка: частичное выпадение нейронов (dropout) — классический приём в таких случаях.

Ответ:

- **Ввести dropout слоёв**
- Сделать mini-batch size равным 1
- Сделать hidden слои без активации
- Отключить оптимизацию

Шаг 03:

Миссия 3

 Павел пытается уменьшить переобучение: что из этого лучше добавить в функцию потерь?

Подсказка: регуляризация часто формулируется как штраф на величину весов.

Ответ:

- Отрицательный log-likelihood
- **L2-норма весов (регуляризация) в потере**
- Взвешенный cross-entropy без штрафа

- Абсолютная ошибка без штрафа

🔮 Шаг 04:

Миссия 4

🧩 Света уменьшает размер модели, чтобы бороться с переобучением. Что она может уменьшить?

Подсказка: параметров в модели меньше — шанс переобучения снижен.

Ответ:

- **Уменьшить число нейронов в слоях**
- Увеличить ширину слоёв
- Добавить побочный выход
- Сделать все веса одинаковыми

🔮 Шаг 05:

Миссия 5

⚙️ Сергей использует раннюю остановку (early stopping). Когда стоит останавливать обучение?

Подсказка: следите за валидационной ошибкой — как только она растёт, пора остановиться.

Ответ:

- **Как только валидационная ошибка начнёт расти**
- Когда тренировочная ошибка станет нулевой
- После фиксированного числа эпох, независимо от графика
- Когда модель перестанет сходиться

🔮 Шаг 06:

Миссия 6

🔊 Надя хочет добавить шум к входным данным как способ регуляризации. Что это даёт?

Подсказка: шум стимулирует устойчивость к вариациям.

Ответ:

- **Улучшение обобщения за счёт устойчивости**
- Уменьшение вычислительной нагрузки
- Ускорение сходимости
- Полное устранение переобучения

🔮 Шаг 07:

Миссия 7

🔍 Сопоставьте технику регуляризации с её основным эффектом на модель:

Методы:

Ответ:

- Dropout
- L1 регуляризация
- Batch Normalization
- Data Augmentation
- Early Stopping

Эффекты:

1. Остановка обучения при ухудшении на валидации
2. Добавление разнообразия входов
3. Случайное «отключение» нейронов во время обучения
4. Принудительное к обнулению части весов (разреживание)
5. Стабилизация распределения активаций

Подсказка: думайте о том, какая техника действует на веса, а какая — на данные или процесс обучения.

- **Dropout** — Случайное «отключение» нейронов во время обучения
- **L1 регуляризация** — Принудительное к обнулению части весов (разреживание)
- **Batch Normalization** — Стабилизация распределения активаций
- **Data Augmentation** — Добавление разнообразия входов
- **Early Stopping** — Остановка обучения при ухудшении на валидации

🔮 Шаг 08:

Миссия 8

Сопоставьте термин регуляризации с формулой / выражением:

Термины:

Ответ:

- L2 регуляризация
- L1 регуляризация
- Dropout
- Early Stopping
- Data Augmentation

Выражения / формулы:

1. Добавление $\lambda \sum w^2$ к функции потерь
2. Применение случайного обнуления нейронов
3. Дополнение тренировочных образцов шумом или трансформациями
4. Добавление $\lambda \sum |w|$ к функции потерь
5. Остановка обучения по условию валидации

Подсказка: L2 — квадрат весов, L1 — абсолютная величина.

- **L2 регуляризация** — Добавление $\lambda \sum w^2$ к функции потерь
- **L1 регуляризация** — Добавление $\lambda \sum |w|$ к функции потерь
- **Dropout** — Применение случайного обнуления нейронов
- **Early Stopping** — Остановка обучения по условию валидации
- **Data Augmentation** — Дополнение тренировочных образцов шумом или трансформациями

🔮 Шаг 09:

Миссия 9

✂ Сопоставьте проблему и подходящую регуляризацию:

Проблемы:

Ответ:

- Переобучение при малом датасете

- Очень большие веса в сети
- Неустойчивые градиенты
- Небольшое число примеров разных классов
- Медленный рост валидационной ошибки

Регуляризации / методы:

1. Dropout
2. L2 регуляризация
3. Batch Normalization
4. Data Augmentation
5. Early Stopping

Подсказка: соотнесите симптом (часто веса растут, градиенты скачут, мало данных) с методом борьбы.

- **Переобучение при малом датасете** — Dropout
- **Очень большие веса в сети** — L2 регуляризация
- **Неустойчивые градиенты** — Batch Normalization
- **Небольшое число примеров разных классов** — Data Augmentation
- **Медленный рост валидационной ошибки** — Early Stopping

Шаг 10:

Миссия 10

 Сопоставьте этап процесса настройки регуляризации с его описанием:

Этапы:

Ответ:

- Выбор гиперпараметров
- Оценка на валидации
- Анализ кривых обучения
- Настройка смещения методов
- Финальный тест

Описания:

1. Регулировка λ , dropout_rate и др.
2. Запуск на отложенной выборке
3. Сравнение тренировочной и валидационной ошибки
4. Использование комбинаций методов регуляризации
5. Оценка на независимом тестовом множестве

Подсказка: сначала подбираем, потом проверяем, потом анализируем графики, комбинируем, и в конце оцениваем окончательно.

- **Выбор гиперпараметров** — Регулировка λ , dropout_rate и др.
- **Оценка на валидации** — Запуск на отложенной выборке
- **Анализ кривых обучения** — Сравнение тренировочной и валидационной ошибки
- **Настройка смещения методов** — Использование комбинаций методов регуляризации
- **Финальный тест** — Оценка на независимом тестовом множестве

Шаг 11:

Миссия 11

📋 Расположите шаги борьбы с переобучением в логическом порядке:

1. Проводим baseline обучение без регуляризации
2. Добавляем L2 регуляризацию / dropout
3. Анализируем кривые тренда ошибок
4. Настраиваем гиперпараметры регуляризации
5. Проверяем на тестовой выборке
6. **Если ещё переобучение** — вводим data augmentation и комбинируем методы

Подсказка: подумайте, что логично делать сначала (без регуляризации), затем постепенно усложнять.

Ответ:

- Проводим baseline обучение без регуляризации
- Добавляем L2 регуляризацию / dropout
- Анализируем кривые тренда ошибок
- Настраиваем гиперпараметры регуляризации
- Проверяем на тестовой выборке
- **Если ещё переобучение** — вводим data augmentation и комбинируем методы

🔗 Шаг 12:

Миссия 12

📋 Расположите этапы эксперимента с ранней остановкой:

1. Запуск модели с мониторингом валиошибки
2. После каждой эпохи проверка валидационной ошибки
3. **Если валидационная ошибка растёт N эпох подряд** — остановка
4. Возврат к наилучшей модели с минимальной валидационной ошибкой
5. Оценка модели на тестовой выборке

Подсказка: при early stopping важно отслеживать ухудшение и вернуть лучшую модель.

Ответ:

- Запуск модели с мониторингом валиошибки
- После каждой эпохи проверка валидационной ошибки
- **Если валидационная ошибка растёт N эпох подряд** — остановка
- Возврат к наилучшей модели с минимальной валидационной ошибкой
- Оценка модели на тестовой выборке

🔗 Шаг 13:

Миссия 13

🔗 Расположите улучшения модели по возрастанию сложности их внедрения:

1. Добавить L2 регуляризацию
2. Добавить dropout
 - Уменьшить число нейронов
3. Добавить data augmentation
4. Ввести batch normalization
5. Комбинировать все методы и тонкая настройка гиперпараметров

Подсказка: сначала простые методы, потом более сложные и комбинированные.

Ответ:

- Добавить L2 регуляризацию
- Добавить dropout
- Уменьшить число нейронов
- Добавить data augmentation
- Ввести batch normalization
- Комбинировать все методы и тонкая настройка гиперпараметров

 Шаг 14:

Миссия 14

➡ Расположите шаги полного workflow при работе с маленьким датасетом:

1. Сбор и предобработка данных
2. Разделение на train / val / test
3. Обучение базовой модели без регуляризации
4. Оценка на валидации и анализ кривых ошибок
5. Введение регуляризаций (L2, dropout, noise)
6. Тонкая настройка гиперпараметров
7. Дополнительные методы (data augmentation, ensembling)
8. Оценка конечной модели на тестовой выборке

Подсказка: это полный путь от данных до финальной проверки.

Ответ:

- Сбор и предобработка данных
- Разделение на train / val / test
- Обучение базовой модель без регуляризации
- Оценка на валидации и анализ кривых ошибок
- Введение регуляризаций (L2, dropout, noise)
- Тонкая настройка гиперпараметров
- Дополнительные методы (data augmentation, ensembling)
- Оценка конечной модели на тестовой выборке

 Шаг 15:

Миссия 15

⚠ Расположите техники уменьшения сложности модели по «жёсткости» эффекта (от мягкого к жёсткому):

1. Добавление небольшого Gaussian шума к входам
2. Dropout с низким rate
3. L2 регуляризация
4. Dropout с высоким rate
5. Уменьшение числа нейронов
6. Принудительное обнуление весов через L1

Подсказка: от менее инвазивных к более радикальным методам.

Ответ:

- Добавление небольшого Gaussian шума к входам
- Dropout с низким rate
- L2 регуляризация

- Dropout с высоким rate
- Уменьшение числа нейронов
- Принудительное обнуление весов через L1

🔮 Шаг 16:

Миссия 16

🔧 Расположите стратегии комбинирования методов регуляризации:

1. Использование одной техники (например, только L2)
2. Добавление второго метода (например, L2 + dropout)
3. Добавление третьего (noise, augmentation и т.д.)
4. Тонкая настройка долей и коэффициентов
5. Проверка комбинации на валидации
6. Финальная проверка на тестовой выборке

Подсказка: стройте комбинации шаг за шагом и проверяйте каждую.

Ответ:

- Использование одной техники (например, только L2)
- Добавление второго метода (например, L2 + dropout)
- Добавление третьего (noise, augmentation и т.д.)
- Тонкая настройка долей и коэффициентов
- Проверка комбинации на валидации
- Финальная проверка на тестовой выборке

🔮 Шаг 17:

Миссия 17

🏁 Финал: упорядочьте этапы подготовки модели для продакшена после решения переобучения:

1. Выбор лучших гиперпараметров регуляризации
2. Обучение финальной модели на объединённом train+val
3. Оценка финальной модели на тестовом наборе
4. Сохранение весов и сериализация модели
5. Запуск валидаций на новых данных / мониторинг ошибок
6. Регулярные переобучения / обновления по мере поступления данных

Подсказка: от последнего шага разработки к эксплуатации.

Ответ:

- Выбор лучших гиперпараметров регуляризации
- Обучение финальной модели на объединённом train+val
- Оценка финальной модели на тестовом наборе
- Сохранение весов и сериализация модели
- Запуск валидаций на новых данных / мониторинг ошибок
- Регулярные переобучения / обновления по мере поступления данных

📖 Урок 4 - Регуляризация через L1, L2 и раннюю остановку

🔮 Шаг 01:

Миссия 1

Когда мы применяем ****L2-регуляризацию**** (Ridge) к весам нейросети, что происходит с большими весами?

Подсказка: Регуляризация стремится удерживать веса «маленькими», но не обнулять полностью.

Ответ:

- Они получают штраф и стремятся быть ближе к нулю
- Они усиливаются (увеличиваются)
- Они полностью обнуляются

🔮 Шаг 02:

Миссия 2

В чем ключевое отличие L1-регуляризации (Lasso) от L2-регуляризации с точки зрения поведения весов?

Подсказка: Один из методов может обнулять веса, другой — сглаживать их.

Ответ:

- **L1 может обнулять веса (делать жёсткую селекцию), L2** — растягивает их к нулю
- L2 обнуляет, а L1 делает веса плавными
- Оба обнуляют все веса

🔮 Шаг 03:

Миссия 3

Почему нельзя просто увеличить регуляризацию до огромной величины без оглядки?

Подсказка: Что произойдёт, если веса станут почти нулевыми?

Ответ:

- **Модель станет недообученной, потеряет способность к обобщению**
- Она станет идеальной на тренировочных данных
- Регуляризация не влияет на качество

🔮 Шаг 04:

Миссия 4

Разработчик Катя тренирует нейросеть и замечает, что ошибка на валидационном наборе начинает расти после некоторого числа эпох — что это может означать?

Подсказка: Подумайте о знаке переобучения.

Ответ:

- **Модель начала переобучаться** — пора остановить обучение
- Модель всё ещё недообучается
- **Ошибка валидации** — случайный шум, незначимый

🔮 Шаг 05:

Миссия 5

Какой из подходов наиболее прямолинейно предотвращает переобучение, когда вы видите рост валидационной ошибки во время обучения?

Подсказка: Один метод контролирует время, другой — величину весов.

Ответ:

- **Применить раннюю остановку (early stopping)**
- Увеличить число слоёв
- Убрать регуляризацию
- Снизить learning rate до 0

Шаг 06:

Миссия 6

Сопоставьте тип регуляризации с её особенностью:

Типы регуляризации:

Ответ:

- L1-регуляризация
- L2-регуляризация
- Dropout (вспомогательный метод)
- Ранняя остановка (early stopping)

Особенности:

1. Обнуляет некоторые веса, способствует разреженности
2. Снижает веса пропорционально квадрату
3. Случайно «выключает» нейроны во время обучения
4. Останавливает обучение при росте ошибки валидации

Подсказка: Подумайте, какой метод действует с весами жестко, а какой — динамически во времени.

- **L1-регуляризация** — Обнуляет некоторые веса, способствует разреженности
- **L2-регуляризация** — Снижает веса пропорционально квадрату
- **Dropout (вспомогательный метод)** — Случайно «выключает» нейроны во время обучения
- **Ранняя остановка (early stopping)** — Останавливает обучение при росте ошибки валидации

Шаг 07:

Миссия 7

Разработчик Иван отмечает, что L2-регуляризация не сильно уменьшает некоторые веса до нуля — какой тип регуляризации мог бы лучше подойти, если он хочет жёсткую селекцию?

Регуляризации:

Ответ:

- L1
- L2
- Elastic Net
- Dropout

Краткие описания:

1. Гибрид L1 и L2, баланс между обнулением и сглаживанием
2. Жёстко обнуляет некоторые веса
3. Сглаживает веса, но редко обнуляет
4. Выключает случайные нейроны на время обучения

Подсказка: Иван хочет, чтобы многие веса стали строго нулём, но также сохранить некоторые эффект сглаживания.

- **L1** — Жёстко обнуляет некоторые веса
- **L2** — Сглаживает веса, но редко обнуляет
- **Elastic Net** — Гибрид L1 и L2, баланс между обнулением и сглаживанием
- **Dropout** — Выключает случайные нейроны на время обучения

🔮 Шаг 08:

Миссия 8

Во время обучения сети Мария фиксирует, что часть весов стала нулевой при L1, но другие всё ещё велики — как можно комбинировать подходы?

Подходы:

Ответ:

- Увеличить коэффициент L1
- Добавить L2 вместе с L1 (Elastic Net)
- Использовать только dropout
- Применить раннюю остановку

Эффекты:

1. Смещение в сторону ограниченного роста весов
2. Создание разреженной модели + сглаживание
3. Мгновенное обнуление всех весов
4. Остановка обучения при росте ошибки

Подсказка: Elastic Net сочетает L1 и L2, давая комбинированный эффект.

- **Увеличить коэффициент L1** — Смещение в сторону ограниченного роста весов
- **Добавить L2 вместе с L1 (Elastic Net)** — Создание разреженной модели + сглаживание
- **Использовать только dropout** — Мгновенное обнуление всех весов
- **Применить раннюю остановку** — Остановка обучения при росте ошибки

🔮 Шаг 09:

Миссия 9

Сопоставьте этапы процесса обучения с применением регуляризации и ранней остановки:

Этапы:

Ответ:

- Инициализация весов
- Обучение на мини-батчах с регуляризацией
- Оценка на валидационном наборе после эпохи
- Решение: продолжать или остановить

Действия:

1. Применить штраф L1/L2 к градиентам перед обновлением
2. Сравнить текущую валидационную ошибку с лучшей
3. **Если нет улучшения** — остановить обучение
4. Установить начальные случайные веса

Подсказка: Следите за логикой: сначала инициализация, потом обновления, затем проверка, затем решение.

- **Инициализация весов** — Установить начальные случайные веса
- **Обучение на мини-батчах с регуляризацией** — Применить штраф L1/L2 к градиентам перед обновлением
- **Оценка на валидационном наборе после эпохи** — Сравнить текущую валидационную ошибку с лучшей
- **Решение: продолжать или остановить** — Если нет улучшения — остановить обучение

🔮 Шаг 10:

Миссия 10

Расположите шаги применения ранней остановки (early stopping) в правильном порядке:

1. Начать обучение модели на тренировочных данных с регуляризацией
2. После каждой эпохи оценивать модель на валидационном наборе
3. **Если ошибка валидации не улучшается определённое число эпох** — остановить обучение
4. Сохранить модель с наилучшим значением на валидации

Подсказка: Подумайте, что должно произойти сначала и что в конце.

Ответ:

- Начать обучение модели на тренировочных данных с регуляризацией
- После каждой эпохи оценивать модель на валидационном наборе
- **Если ошибка валидации не улучшается определённое число эпох** — остановить обучение
- Сохранить модель с наилучшим значением на валидации

🔮 Шаг 11:

Миссия 11

Программист Денис объясняет: «Я хочу реализовать механизм ранней остановки с *patience* = 3» — что должен делать алгоритм?

Упорядочьте шаги:

1. Запустить обучение модели
2. Считать, что не было улучшения на 1 эпохе
3. **Если три эпохи подряд нет улучшения** — остановить обучение
4. **Иначе** — сбросить счётчик и продолжить обучение

Подсказка: *patience* — это число безуспешных эпох подряд, после которых останавливают.

Ответ:

- Запустить обучение модели
- Считать, что не было улучшения на 1 эпохе
- **Если три эпохи подряд нет улучшения** — остановить обучение
- **Иначе** — сбросить счётчик и продолжить обучение

🔮 Шаг 12:

Миссия 12

Вам нужно встроить L1 и L2 регуляризацию в ваш цикл обучения вручную. Расположите основные шаги:

1. Вычислить градиенты без регуляризации
2. Добавить штрафы L1 и L2 к градиентам
3. Обновить веса с учётом поправок
4. Оценить модель на валидации и возможно остановиться

Подсказка: Регуляризация внедряется до обновления весов, а проверка early stopping — после.

Ответ:

- Вычислить градиенты без регуляризации
- Добавить штрафы L1 и L2 к градиентам
- Обновить веса с учётом поправок
- Оценить модель на валидации и возможно остановиться

Шаг 13:

Миссия 13

Алексей наблюдает, что при L2-регуляризации веса становятся малыми, но модель всё ещё переобучается. Он решает внедрить раннюю остановку и dropout. Расположите порядок включения методов:

1. Добавить L2-регуляризацию к весам
2. Включить dropout в архитектуре сети
3. При обучении пользоваться early stopping
4. Мониторить валидационную ошибку и останавливать, если нет улучшения

Подсказка: Архитектурные меры применяются до обучения, мониторинг — во время.

Ответ:

- Добавить L2-регуляризацию к весам
- Включить dropout в архитектуре сети
- При обучении пользоваться early stopping
- Мониторить валидационную ошибку и останавливать, если нет улучшения

Шаг 14:

Миссия 14

Вот четыре меры защиты от переобучения: L1, L2, dropout, ранняя остановка. В каком порядке вы бы их применили при разработке новой модели?

1. Добавить регуляризацию к функции потерь (L1 или L2)
2. Встроить dropout в слои нейросети
3. Обучать модель с мониторингом валидационной ошибки
4. Применить раннюю остановку при ухудшении результата

Подсказка: Сначала создаёте архитектуру и регуляризацию, потом следите за поведением при обучении.

Ответ:

- Добавить регуляризацию к функции потерь (L1 или L2)
- Встроить dropout в слои нейросети
- Обучать модель с мониторингом валидационной ошибки
- Применить раннюю остановку при ухудшении результата

Шаг 15:

Миссия 15

Команда работает над сервисом рекомендации фильмов. После первых экспериментов они добавили L2, но модель всё ещё начала переобучаться. Они решают применить серию шагов. Упорядочите:

1. Добавить L1-регуляризацию к некоторым слоям
2. Внедрить dropout между слоями
3. Мониторить валидационную ошибку и наилучшую модель
4. Применить раннюю остановку, если нет улучшения

Подсказка: Меры регуляризации включаются сначала, мониторинг — после, остановка — в конце.

Ответ:

- Добавить L1-регуляризацию к некоторым слоям
- Внедрить dropout между слоями
- Мониторить валидационную ошибку и наилучшую модель
- Применить раннюю остановку, если нет улучшения

Шаг 16:

Миссия 16

Вы хотите сравнить эффективность L1, L2 и ранней остановки в контролируемом эксперименте. Упорядочите шаги эксперимента:

1. Выбрать базовую архитектуру модели
2. Запустить обучение с L1, L2 и без регуляризации с early stopping
3. Сравнить результаты на тестовой выборке
4. Проанализировать, при каких условиях какая регуляризация лучше работала

Подсказка: Сначала ставите эксперимент, затем анализируете результаты.

Ответ:

- Выбрать базовую архитектуру модели
- Запустить обучение с L1, L2 и без регуляризации с early stopping
- Сравнить результаты на тестовой выборке
- Проанализировать, при каких условиях какая регуляризация лучше работала

Шаг 17:

Миссия 17

Вы проектируете пайплайн обучения в продакшене: хотите автоматизировать регуляризацию + раннюю остановку. Какой порядок логики вы бы заложили?

1. Инициализировать веса и метрики лучшей модели
2. В цикле по эпохам: обучать с регуляризацией и делать forward/backward
3. После каждой эпохи: оценивать на валидации и сравнивать с лучшей моделью
4. **Если улучшение** — обновить лучшую модель и сбросить счётчик, иначе инкрементировать счётчик
5. **Если счётчик без улучшения превысил порог** — остановить и вернуть лучшую модель

Подсказка: Это полная схема обучения с регуляризацией и early stopping, важно не перепутать порядок действий.

Ответ:

- Инициализировать веса и метрики лучшей модели
- В цикле по эпохам: обучать с регуляризацией и делать forward/backward
- После каждой эпохи: оценивать на валидации и сравнивать с лучшей моделью
- **Если улучшение** — обновить лучшую модель и сбросить счётчик, иначе инкрементировать счётчик
- **Если счётчик без улучшения превысил порог** — остановить и вернуть лучшую модель

Модуль 6 - 🧠 Обучение — как искусство

Урок 1 - ⚙️ AdamW, Lookahead, Lion

Шаг 01:

Миссия 1

Павел, исследователь моделей, пытается выбрать оптимизатор для новой нейросети. Он слышал об AdamW и хочет понять, что отличает его от обычного Adam. Что из приведённого наиболее характерно для AdamW?

Подсказка: Обратите внимание на роль регуляризации веса.

Ответ:

- **Decoupled weight decay (разделённая регуляризация)**
- Использует momentum как в SGD
- Не поддерживает адаптивные скорости
- Не требует градиентного спуска

Шаг 02:

Миссия 2

⚡ Анна внедряет Lookahead поверх AdamW: она хочет понять, как работает «fast» и «slow» обновления. Какой из вариантов верно описывает механизм Lookahead?

Подсказка: Один цикл — много быстрых шагов, затем усреднение с медленным.

Ответ:

- **После k быстрых шагов fast параметры усредняются с slow**
- Slow полностью заменяет fast без усреднения
- Fast делает шаг только после slow делает обновление
- Lookahead отменяет веса на каждом шаге

Шаг 03:

Миссия 3

🦁 Борис экспериментирует с оптимизатором Lion („EvoLved sign momentum optimizer“). Что из следующего соответствует ключевой идее Lion?

Подсказка: Подумайте про знак градиента и более простую формулу обновления.

Ответ:

- **Обновления по знаку градиента и меньше гиперпараметров**
- Полностью вычитаются квадраты градиентов
- Использует адаптивные learning rates как Adam
- Не допускает весовой распад (weight decay)

🔮 Шаг 04:

Миссия 4

🔪 Дарья пытается совместить AdamW + Lookahead + Lion и задаётся вопросом: возможно ли это? Что наиболее корректно?

Подсказка: Оптимизаторы можно комбинировать, если они совместимы по формуле обновления.

Ответ:

- Lion нельзя объединять с AdamW
- **Можно применять Lookahead поверх AdamW или Lion**
- Нельзя использовать weight decay вместе с Lookahead
- Объединение даёт всегда худший результат

🔮 Шаг 05:

Миссия 5

🔪 Команда развёртывает модель в продакшене. Важен вопрос: какой оптимизатор вы бы предпочли для финальной фазы дообучения через transfer learning? Почему?

Подсказка: Стабильность — ключ на финальном этапе.

Ответ:

- **AdamW + Lookahead (более стабильные обновления)**
- Стандартный SGD без моментов
- Lion без регуляризации
- Любой оптимизатор, скорость не важна

🔮 Шаг 06:

Миссия 6

🔪 Сопоставьте метод с его ключевым свойством:

Методы:

Ответ:

- AdamW
- Lookahead
- Lion
- Stochastic Gradient Descent (SGD) + momentum

Свойства:

1. Обновления на основе знака градиента
2. Разделённый weight decay
3. Медленный усреднённый шаг поверх быстрых шагов
4. Классический моментумный спуск

Подсказка: Вспомните, что для AdamW важна регуляризация, для Lion — знак.

- **AdamW** — Разделённый weight decay
- **Lookahead** — Медленный усреднённый шаг поверх быстрых шагов
- **Lion** — Обновления на основе знака градиента
- **SGD + momentum** — Классический моментумный спуск

🔧 Шаг 07:

Миссия 7

🔗 При имплементации оптимизатора программист сталкивается с компонентами. Сопоставьте термин и описание:

Термины:

Ответ:

- fast weights
- slow weights
- k steps
- weight decay

Описание:

1. Количество быстрых шагов перед усреднением
2. Регуляризация весов (L2) отдельно от адаптивного шага
3. Параметры, которые меняются медленно в Lookahead
4. Параметры, которые обновляются на каждом шаге

Подсказка: fast — быстрые обновления, slow — медленные.

- **fast weights** — Параметры, которые обновляются на каждом шаге
- **slow weights** — Параметры, которые меняются медленно в Lookahead
- **k steps** — Количество быстрых шагов перед усреднением
- **weight decay** — Регуляризация весов (L2) отдельно от адаптивного шага

🔧 Шаг 08:

Миссия 8

🔗 В ходе отладки оптимизаторов появились следующие свойства — сопоставьте с соответствующим оптимизатором:

Оптимизаторы:

Ответ:

- Adam (без W)
- AdamW
- Lookahead(AdamW)
- Lion

Свойства:

1. Использует decoupled weight decay
2. Стабилизирует поведение ансамблированием быстрых шагов
3. Обновления по знаку градиента
4. Применяет адаптивные learning rates без decoupled регуляризации

Подсказка: Свойство adaptivity характерно для Adam, но не обязательно для Lion.

- **Adam (без W)** — Применяет адаптивные learning rates без decoupled регуляризации
- **AdamW** — Использует decoupled weight decay
- **Lookahead(AdamW)** — Стабилизирует поведение ансамблированием быстрых шагов
- **Lion** — Обновления по знаку градиента

🔧 Шаг 09:

Миссия 9

📖 При мониторинге лосса команда заметила “шумные” колебания. Какой механизм из списка может помочь сгладить это поведение?

Механизмы:

Ответ:

- Lookahead
- Большой learning rate
- Lion без momentum
- Отключить weight decay

Эффекты:

1. Усреднение шагов уменьшает шум
2. Увеличение шага увеличит колебания
3. Удаление momentum приводит к нестабильности
4. Отключение регуляризации может ухудшить обобщение

Подсказка: Усреднение — это стабилизация.

- **Lookahead** — Усреднение шагов уменьшает шум
- **Большой learning rate** — Увеличение шага увеличит колебания
- **Lion без momentum** — Удаление momentum приводит к нестабильности
- **Отключить weight decay** — Отключение регуляризации может ухудшить обобщение

🔧 Шаг 10:

Миссия 10

Разработчики записали эти формулы и хотят понять, к какому механизму они относятся — сопоставьте:

Формулы / выражения:

Ответ:

- $\theta \leftarrow \theta - \eta \cdot \text{sign}(m + g)$
- $\theta_{\text{fast}} \leftarrow \theta_{\text{fast}} - \eta \cdot \nabla L$
- $\theta_{\text{slow}} \leftarrow \theta_{\text{slow}} + \alpha (\theta_{\text{fast}} - \theta_{\text{slow}})$
- $\theta \leftarrow \theta - \eta \cdot (m / (\sqrt{v} + \epsilon) + \lambda \theta)$

Механизмы / оптимизаторы:

1. Lion update
2. fast шаг Lookahead
3. slow update Lookahead
4. AdamW update

Подсказка: Последняя формула похожа на Adam с weight decay.

- $\theta \leftarrow \theta - \eta \cdot \text{sign}(m + g)$ — Lion update
- $\theta_{\text{fast}} \leftarrow \theta_{\text{fast}} - \eta \cdot \nabla L$ — fast шаг Lookahead
- $\theta_{\text{slow}} \leftarrow \theta_{\text{slow}} + \alpha (\theta_{\text{fast}} - \theta_{\text{slow}})$ — slow update Lookahead
- $\theta \leftarrow \theta - \eta \cdot (m / (\sqrt{v} + \epsilon) + \lambda \theta)$ — AdamW update

🔔 Шаг 11:

Миссия 11

Расположите этапы применения Lookahead к основному оптимизатору (например, AdamW):

1. Усреднить fast и slow ($\text{slow} \leftarrow \text{slow} + \alpha(\text{fast} - \text{slow})$)
2. Использовать оптимизатор (например, AdamW) на fast шаги
3. Запустить несколько (k) быстрых шагов
4. Начать новый цикл

Подсказка: Сначала fast делает много шагов, затем усреднение, потом повтор.

Ответ:

- Усреднить fast и slow ($\text{slow} \leftarrow \text{slow} + \alpha(\text{fast} - \text{slow})$)
- Использовать оптимизатор (например, AdamW) на fast шаги
- Запустить несколько (k) быстрых шагов
- Начать новый цикл

🔔 Шаг 12:

Миссия 12

🔗 Расположите этапы одного цикла обучения с Lion:

1. Вычислить градиенты ∇L
2. Обновить параметры по знаку ($\theta \leftarrow \theta - \eta \cdot \text{sign}(m + g)$)
3. Обновить моментум m
4. Следующий шаг

Подсказка: Первым всегда — вычислить градиенты, затем обновление, затем моментум.

Ответ:

- Вычислить градиенты ∇L
- Обновить параметры по знаку ($\theta \leftarrow \theta - \eta \cdot \text{sign}(m + g)$)
- Обновить моментум m
- Следующий шаг

🔔 Шаг 13:

Миссия 13

📦 При комбинировании AdamW + Lookahead + Lion — какой порядок вложенности будет логичен? (от внутреннего к внешнему)

1. Lion (на уровень fast)
2. Lookahead (усреднитель slow/fast)
3. AdamW (внутренний оптимизатор для fast)
4. Внешний управляющий цикл обучения

Подсказка: Fast слой — это Lion или AdamW, затем усреднение Lookahead, затем цикл.

Ответ:

- Lion (на уровень fast)
- Lookahead (усреднитель slow/fast)
- AdamW (внутренний оптимизатор для fast)
- Внешний управляющий цикл обучения

🔮 Шаг 14:

Миссия 14

🔪 При отладке комбинированного оптимизатора команда хочет диагностировать поведение: в каком порядке логично проверять?

1. Проверить значения градиентов и их знаки
2. Проверить, что fast-шаги корректно применяются (AdamW или Lion)
3. Проверить усреднение slow/fast (Lookahead)
4. Оценить влияние weight decay на параметры

Подсказка: Сначала — внутренние шаги, затем усреднение, затем регуляризация.

Ответ:

- Проверить значения градиентов и их знаки
- Проверить, что fast-шаги корректно применяются (AdamW или Lion)
- Проверить усреднение slow/fast (Lookahead)
- Оценить влияние weight decay на параметры

🔮 Шаг 15:

Миссия 15

📊 При сравнении трёх вариантов оптимизации (AdamW, AdamW+Lookahead, Lion+Lookahead) команда хочет упорядочить их по устойчивости обучения (от менее устойчивого к более устойчивому):

1. AdamW
2. Lion+Lookahead
3. AdamW+Lookahead

Подсказка: Усреднение обычно усиливает устойчивость.

Ответ:

- AdamW
- Lion+Lookahead
- AdamW+Lookahead

🔮 Шаг 16:

Миссия 16

🔪 В финальной миссии команды — упорядочите шаги от концепции к продакшену, применяя изученные методы:

1. Собрать лог экспериментов (loss, accuracy, поведение градиентов)
2. Выбрать базовый оптимизатор (AdamW / Lion)
3. Добавить Lookahead при необходимости Запустить дообучение на финальной задаче
4. Оценить стабильность обучения и скорректировать гиперпараметры

Подсказка: Сначала эксперимент, потом дизайн, потом финализация.

Ответ:

- Собрать лог экспериментов (loss, accuracy, поведение градиентов)
- Выбрать базовый оптимизатор (AdamW / Lion)
- Добавить Lookahead при необходимости
- Запустить дообучение на финальной задаче

- Оценить стабильность обучения и скорректировать гиперпараметры

Шаг 17:

Миссия 17

🕒 Последняя миссия: как бы вы расположили этапы отладки и запуска нейросети с сочетанным оптимизатором на продакшен?

1. Малые контрольные прогоны на валидации
2. Мониторинг скачков градиентов / взрывов и Nan'ов
3. Стресс-тестирование на больших батчах
4. Запуск в пользовательском окружении (продакшен)
5. Откат при проблемах

Подсказка: Без проверки стабильности нельзя сразу в продакшен.

Ответ:

- Малые контрольные прогоны на валидации
- Мониторинг скачков градиентов / взрывов и Nan'ов
- Стресс-тестирование на больших батчах
- Запуск в пользовательском окружении (продакшен)
- Откат при проблемах

Урок 2 - Gradient Clipping и Accumulation

Шаг 01:

Миссия 1

Вы — инженер по машинному обучению, и ваш градиент внезапно «взорвался» — его норма стала экстремально большой. Что вы делаете первым?

Подсказка: Подумайте: какое средство ограничивает величину градиента?

Ответ:

- **Применить gradient clipping (усечь градиенты)**
- Увеличить learning rate
- Добавить больше слоёв в сеть

Шаг 02:

Миссия 2

Вы замечаете, что при большом batch-размере память GPU переполняется. Какой трюк помогает снизить пиковые затраты памяти без уменьшения эффективного batch-размера?

Подсказка: Это техника, где вы делаете несколько мини-шагов и аккумулируете изменения.

Ответ:

- **Gradient accumulation (аккумуляция градиентов)**
- Dropout увеличивать
- Уменьшить размер слоёв сети

Шаг 03:

Миссия 3

Ваша команда хочет комбинировать gradient clipping и accumulation. Какой порядок операций правильный при каждом шаге?

Подсказка: Сначала вы `compute_gradients`, потом ...?

Ответ:

- **Compute → clip → accumulate → apply**
- Clip → compute → accumulate → apply
- Compute → accumulate → clip → apply

🔧 Шаг 04:

Миссия 4

Когда вы используете gradient accumulation со скоростью `accumulation_steps > 1`, как это влияет на эффективное learning rate при постоянной scheduler?

Подсказка: Подумайте: если вы делаете больше шагов до обновления, какое «весовое» влияние имеет каждый мини-градиент?

Ответ:

- **Эффективно learning rate становится больше для каждого обновления**
- Learning rate уменьшается
- Learning rate остаётся точно тем же

🔧 Шаг 05:

Миссия 5

Вы тренируете огромную модель, и после clipping видите, что многие градиенты обрезаются. Что вы рассматриваете как возможное решение?

Подсказка: Можно либо ослабить ограничение, либо изменить стратегию.

Ответ:

- **Увеличить clip-threshold (порог)**
- **Применить адаптивное clipping на основе норм**
- Отключить accumulation

🔧 Шаг 06:

Миссия 6

🧠 Сопоставьте проблему и стратегию её решения с градиентами:

Подсказка: Иногда градиенты взрываются, иногда их слишком мало.

Проблемы:

Ответ:

- Взрыв градиентов
- Затухание градиентов
- Память переполняется
- Нестабильное обучение

Стратегии:

1. Gradient clipping или norm clip
 2. Использование residual-связей или нормализации
 3. Gradient accumulation
 4. Адаптивные оптимизаторы (Adam, RMSProp) или grad clipping
- **Взрыв градиентов** — Gradient clipping или norm clip
 - **Затухание градиентов** — Использование residual-связей или нормализации

- **Память переполняется** — Gradient accumulation
- **Нестабильное обучение** — Адаптивные оптимизаторы (Adam, RMSProp) или grad clipping

🔮 Шаг 07:

Миссия 7

🧠 Сопоставьте термин и характерный эффект:

Подсказка: Подумайте, что именно изменяется при каждом методе.

Термины:

Ответ:

- Clip by global norm
- Clip by local norm per layer
- Accumulate then clip
- Clip after accumulation

Эффекты:

1. Ограничение нормы всего градиентного вектора сети
 2. Ограничение нормы на уровне слоя или тензора отдельно
 3. Сложение мини-градиентов перед усечением
 4. Усечение уже после накопления всех мини-градиентов
- **Clip by global norm** — Ограничение нормы всего градиентного вектора сети
 - **Clip by local norm per layer** — Ограничение нормы на уровне слоя или тензора отдельно
 - **Accumulate then clip** — Сложение мини-градиентов перед усечением
 - **Clip after accumulation** — Усечение уже после накопления всех мини-градиентов

🔮 Шаг 08:

Миссия 8

🧠 Сопоставьте реальный сценарий с решением:

Подсказка: Вспомните случаи, когда градиенты «взрывались» или память не хватала.

Сценарии:

Ответ:

- Градиенты слишком велики, модель нестабильна
- RAM GPU исчерпана при большом batch'e
- Уже накопленные градиенты слишком большие
- Обновление слишком редкое из-за accumulation

Решения:

1. Clip before apply
 2. Разбить batch на части + accumulate
 3. Clip после accumulation
 4. Сбалансировать accumulation_steps и learning rate
- **Градиенты слишком велики, модель нестабильна** — Clip before apply
 - **RAM GPU исчерпана при большом batch'e** — Разбить batch на части + accumulate
 - **Уже накопленные градиенты слишком большие** — Clip после accumulation
 - **Обновление слишком редкое из-за accumulation** — Сбалансировать accumulation_steps и learning rate

🔮 Шаг 09:

Миссия 9

🧠 Сопоставьте аббревиатуру и её роль:

Подсказка: Вспомните, какие аббревиатуры встречаются при обучении.

Аббревиатуры:

Ответ:

- LR
- RMSProp
- Adam
- GRAD_LEN

Роли/значения:

1. Скорость обучения
 2. Адаптивный оптимизатор, учитывающий квадраты градиентов
 3. Адаптивный оптимизатор, комбинирующий моменты и RMS
 4. Параметр, который может контролировать длину градиента
- **LR** — Скорость обучения
 - **RMSProp** — Адаптивный оптимизатор, учитывающий квадраты градиентов
 - **Adam** — Адаптивный оптимизатор, комбинирующий моменты и RMS
 - **GRAD_LEN** — Параметр, который может контролировать длину градиента

🔮 Шаг 10:

Миссия 10

🧠 Сопоставьте режим обучения и поведение градиентов:

Подсказка: Учтите, как dropout, batchnorm и другие факторы влияют.

Режимы:

Ответ:

- Без clipping / accumulation
- Только clipping
- Только accumulation
- Clipping + accumulation

Поведения:

1. Риск взрыва, большой шум в градиентах
 2. Стабильность, но возможно сильное усечение
 3. Экономия памяти, но редкие обновления
 4. Баланс между ограничением и накоплением
- **Без clipping / accumulation** — Риск взрыва, большой шум в градиентах
 - **Только clipping** — Стабильность, но возможно сильное усечение
 - **Только accumulation** — Экономия памяти, но редкие обновления
 - **Clipping + accumulation** — Баланс между ограничением и накоплением

🔮 Шаг 11:

Миссия 11

Упорядочите шаги реализации gradient accumulation с clipping:

1. Применить градиентное обновление (optimizer.step)
2. Вычислить градиенты (loss.backward)
3. **Если нужно** — усечь градиенты
4. Накопить градиенты, не обновляя веса сразу
5. Обнулить градиенты (optimizer.zero_grad)

Подсказка: Подумайте, где именно вы обнуляете и когда применяете шаг optimizer.

Ответ:

- Вычислить градиенты (loss.backward)
- **Если нужно** — усечь градиенты
- Накопить градиенты, не обновляя веса сразу
- Применить градиентное обновление (optimizer.step)
- Обнулить градиенты (optimizer.zero_grad)

Шаг 12:

Миссия 12

Упорядочите события при обучении с clipping без accumulation:

1. Zero_grad
2. Forward pass, compute loss
3. Backward pass, compute gradients
4. Clip gradients
5. Optimizer.step()

Подсказка: Где играет роль операция clip: перед применением градиентов.

Ответ:

- Zero_grad
- Forward pass, compute loss
- Backward pass, compute gradients
- Clip gradients
- Optimizer.step()

Шаг 13:

Миссия 13

Упорядочите стратегии по возрастанию риска нестабильности (от менее рискованной к более):

1. Accumulation + adaptive clip
2. Adaptive clip без accumulation
3. Fixed clip без accumulation
4. Без clip и accumulation

Подсказка: Без защиты риск взрыва — самый высокий.

Ответ:

- Accumulation + adaptive clip
- Adaptive clip без accumulation
- Fixed clip без accumulation
- Без clip и accumulation

Шаг 14:

Миссия 14

Упорядочите сценарии по усилию, которое требуется от инженера (от простого к сложному):

1. Добавить статический clipping
2. Добавить accumulation
3. Настроить адаптивный clip per layer
4. Исследовать динамическое изменение clip в зависимости от эпохи
5. Внедрить адаптивное clip + accumulation пер слой

Подсказка: Сложность растёт с комбинаторикой и адаптивностью.

Ответ:

- Добавить статический clipping
- Добавить accumulation
- Настроить адаптивный clip per layer
- Исследовать динамическое изменение clip в зависимости от эпохи
- Внедрить адаптивное clip + accumulation пер слой

Шаг 15:

Миссия 15

Упорядочите этапы эксперимента, когда вы тестируете разные threshold для clipping с accumulation:

1. Запуск обучения с threshold = a
2. Оценка валидационной метрики после каждой эпохи
3. Логирование нормы градиентов и числа обрезанных градиентов
4. Сравнение поведения при разных threshold'ax
5. Запуск обучения с threshold = b

Подсказка: Эксперимент — это серия запусков и анализ их результатов.

Ответ:

- Запуск обучения с threshold = a
- Оценка валидационной метрики после каждой эпохи
- Логирование нормы градиентов и числа обрезанных градиентов
- Сравнение поведения при разных threshold'ax
- Запуск обучения с threshold = b

Шаг 16:

Миссия 16

Упорядочите стек действий, если вы хотите в продакшене динамически адаптировать accumulation и clip:

1. Наблюдать статистики градиентов (нормы, частоты обрезания)
2. Выбрать правило адаптации (например, увеличивать clip, если много обрезаний)
3. Обновить accumulation_steps или clip на лету
4. Продолжить обучение с новыми параметрами
5. Логировать эффект и подтверждать, что обучение стабильно

Подсказка: В продакшене контроль + адаптация важнее жёстких фиксированных правил.

Ответ:

- Наблюдать статистики градиентов (нормы, частоты обрезания)
- Выбрать правило адаптации (например, увеличивать clip, если много обрезаний)
- Обновить accumulation_steps или clip на лету
- Продолжить обучение с новыми параметрами
- Логировать эффект и подтверждать, что обучение стабильно

Урок 3 - Квест стабилизируй обучение нестабильной модели

Шаг 01:

Миссия 1

Вы запустили обучение модели, но видите, что loss быстро взлетает и потом падает хаотически. Что первое стоит проверить?

Подсказка: Часто причина — слишком большая скорость обучения.

Ответ:

- **Уменьшить learning rate**
- Добавить больше слоёв
- Сразу менять архитектуру
- Прекратить обучение полностью

Шаг 02:

Миссия 2

⚠ Ваша модель выдаёт «nan» градиенты после нескольких итераций. Что вы проверите первым?

Подсказка: Иногда виноваты нестабильные активации или деление на ноль.

Ответ:

- **Где происходит деление (делить ли на вероятность)**
- Размер батча сделать 1
- Увеличить глубину модели
- Игнорировать nan и продолжать

Шаг 03:

Миссия 3

📊 Вы наблюдаете резкие скачки обучения на определённых батчах. Какая техника может помочь сгладить поведение?

Подсказка: Сглаживание часто достигается усреднением изменений.

Ответ:

- **Использовать экспоненциальное сглаживание градиентов (gradient smoothing)**
- Сразу остановить обучение
- Увеличить dropout до 0.9
- Удалить нормализацию

Шаг 04:

Миссия 4

🔧 Вы пробовали уменьшать learning rate вручную, но улучшений нет. Какую стратегию scheduler стоит применить?

Подсказка: Часто хороший вариант — адаптивный reduce-on-plateau.

Ответ:

- **ReduceLROnPlateau (уменьшать lr при застоях)**
- Увеличивать lr экспоненциально
- Сброс lr к нулю раз в эпоху
- Не менять lr вообще

Шаг 05:

Миссия 5

✂ Вы решили добавить градиентный клиппинг, но используете слишком большое значение клипа. Что может произойти?

Подсказка: Если клиппинг слишком слабый, он почти не влияет.

Ответ:

- **Эффект будет почти незаметным** — нестабильность не уменьшится
- Модель переобучится мгновенно
- Обратный градиент станет отрицательным
- Вы получите nan сразу же

Шаг 06:

Миссия 6

🔍 Сопоставьте методы стабилизации обучения с их ключевой целью:

Методы:

Ответ:

- Dropout
- Batch Normalization
- Gradient Clipping
- Weight Decay

Цели:

1. Ограничение величины градиентов
2. Снижение переобучения за счёт регуляризации
3. Сглаживание статистики активаций между слоями
4. Случайное отключение нейронов для устойчивости

Подсказка: Подумайте, какой метод отвечает именно за контроль величин и какой — за регуляризацию.

- **Dropout** — Случайное отключение нейронов для устойчивости
- **Batch Normalization** — Сглаживание статистики активаций между слоями
- **Gradient Clipping** — Ограничение величины градиентов
- **Weight Decay** — Снижение переобучения за счёт регуляризации

Шаг 07:

Миссия 7

🕒 Сопоставьте виды scheduler'ов с их характерным поведением:

Scheduler:

Ответ:

- Step LR
- Cosine Annealing
- Exponential LR
- Cyclic LR

Поведенческий шаблон:

1. Снижение с периодическими скачками вверх
2. Плавное уменьшение по косинусу от max к min
3. Уменьшение экспоненциально во времени
4. Понижение через фиксированные шаги каждые N эпох

Подсказка: Подумайте, какие графики вы видели при использовании этих scheduler'ов.

- **Step LR** — Понижение через фиксированные шаги каждые N эпох
- **Cosine Annealing** — Плавное уменьшение по косинусу от max к min
- **Exponential LR** — Уменьшение экспоненциально во времени
- **Cyclic LR** — Снижение с периодическими скачками вверх

Шаг 08:

Миссия 8

 Сопоставьте типы нормализации и их преимущество:

Тип нормализации:

Ответ:

- Layer Normalization
- Instance Normalization
- Group Normalization
- Batch Normalization

Преимущества:

1. Независимость от размера батча
2. Нормализация внутри отдельных каналов изображения
3. **Баланс между BN и LN** — нормализация по подгруппам каналов
4. Используется статистика по батчу для стабилизации

Подсказка: Если батч очень маленький — какой тип нормализации будет предпочтителен?

- **Layer Normalization** — Независимость от размера батча
- **Instance Normalization** — Нормализация внутри отдельных каналов изображения
- **Group Normalization** — Баланс между BN и LN — нормализация по подгруппам каналов
- **Batch Normalization** — Используется статистика по батчу для стабилизации

Шаг 09:

Миссия 9

 Сопоставьте методы регуляризации с эффектом, который они дают моделям:

Метод:

Ответ:

- DropConnect

- Early Stopping
- Data Augmentation
- Label Smoothing

Эффект:

1. Остановка обучения при отсутствии улучшений на валидации
2. Плавность целевых распределений для классификации
3. Отключение связей весов随机 во время обучения
4. Искусственное увеличение разнообразия данных

Подсказка: Подумайте, что делает DropConnect и как Label Smoothing влияет на цели.

- **DropConnect** — Отключение связей весов随机 во время обучения
- **Early Stopping** — Остановка обучения при отсутствии улучшений на валидации
- **Data Augmentation** — Искусственное увеличение разнообразия данных
- **Label Smoothing** — Плавность целевых распределений для классификации

Шаг 10:

Миссия 10

 Упорядочьте шаги внедрения градиентного клиппинга в пайплайн обучения:

1. Проверка границ клипа (min, max)
2. Вычисление градиентов
3. Ограничение (clip) градиентов по норме или значению
4. Применение оптимизатора (update weights)

Подсказка: Клиппинг должен происходить после вычисления градиентов, но до обновления весов.

Ответ:

- Вычисление градиентов
- Проверка границ клипа (min, max)
- Ограничение (clip) градиентов по норме или значению
- Применение оптимизатора (update weights)

Шаг 11:

Миссия 11

 Упорядочите события, которые часто происходят при нестабильном обучении:

1. Loss резко возрастает
2. Появляются nan в весах или градиентах
3. Обучение «взрывается»
4. Модель перестаёт сходиться

Подсказка: Как часто начинаются проблемы — с потерь, а затем усиливаются?

Ответ:

- Loss резко возрастает
- Появляются nan в весах или градиентах
- Обучение «взрывается»
- Модель перестаёт сходиться"

Шаг 12:

Миссия 12

 Упорядочите по этапам отладку нестабильного обучения:

1. Снизить learning rate
2. Добавить градиентный клиппинг
3. Включить нормализацию (batch / group / layer)
4. Применить регуляризацию (weight decay, label smoothing)
5. Добавить scheduler или адаптивный lr

Подсказка: Начинать стоит с самых простых мер, затем добавлять дополнительные способы стабилизации.

Ответ:

- Снизить learning rate
- Добавить градиентный клиппинг
- Включить нормализацию (batch / group / layer)
- Применить регуляризацию (weight decay, label smoothing)
- Добавить scheduler или адаптивный lr

Шаг 13:

Миссия 13

 Упорядочите эксперименты, которые вы проведёте для стабилизации обучения:

1. Попробовать уменьшить batch size
2. Включить градиентный клиппинг
3. Применить learning rate scheduler
4. Добавить нормализацию слоёв
5. Ввести регуляризацию (weight decay, dropout)

Подсказка: Начинаем с гиперпараметров, затем архитектурные меры.

Ответ:

- Попробовать уменьшить batch size
- Включить градиентный клиппинг
- Применить learning rate scheduler
- Добавить нормализацию слоёв
- Ввести регуляризацию (weight decay, dropout)

Шаг 14:

Миссия 14

 Упорядочите порядок действий, когда вы получаете nan в весах модели:

1. Прекратить обучение
2. Проверить границы градиентов и раскрыть места деления
3. Добавить градиентный клиппинг или уменьшить lr
4. Повторить эксперимент с сохранённой стабильной моделью

Подсказка: Не просто остановить — сначала диагностировать, затем улучшить, потом восстановиться.

Ответ:

- Прекратить обучение
- Проверить границы градиентов и раскрыть места деления
- Добавить градиентный клиппинг или уменьшить lr
- Повторить эксперимент с сохранённой стабильной моделью

 Шаг 15:

Миссия 15

 Упорядочите подходы по уровню вмешательства (от слабого к сильному):

1. Применить scheduler lr
2. Добавить weight decay / label smoothing
3. Включить нормализацию слоёв
4. Ввести градиентный клиппинг
5. Значительно уменьшить learning rate вручную

Подсказка: Слабое вмешательство — это изменение стратегии обучения, сильное — прямое ограничение градиентов или агрессивный lr.

Ответ:

- Применить scheduler lr
- Добавить weight decay / label smoothing
- Включить нормализацию слоёв
- Ввести градиентный клиппинг
- Значительно уменьшить learning rate вручную

 Шаг 16:

Миссия 16

 Упорядочите этапы подготовительной диагностики перед поднятием обучения модели в продакшен:

1. Стабилизировать обучение на небольших наборах данных
2. Проверить поведение на новых данных (валид. / тест)
3. Убедиться, что метрики не «скачут» от одной партии данных к другой
4. Протестировать модель на стрессовых сценариях
5. Внедрить финальный вариант в пайплайн

Подсказка: Сначала стабилизируйте, потом тестируйте, затем внедряйте.

Ответ:

- Стабилизировать обучение на небольших наборах данных
- Проверить поведение на новых данных (валид. / тест)
- Убедиться, что метрики не «скачут» от одной партии данных к другой
- Протестировать модель на стрессовых сценариях
- Внедрить финальный вариант в пайплайн

 Шаг 17:

Миссия 17

🗒 Упорядочите полный чек-лист действий при прогоне модели в продакшен, чтобы избежать нестабильности:

1. Подготовка стабильной версии модели на dev / small scale
2. Отладка и устранение nan / перепадов loss
3. Тестирование на валидации и стрессовых сценариях
4. Канареечный релиз / постепенное развёртывание
5. Мониторинг в продакшене и fallback при аномалиях

Подсказка: В продакшене не сразу выкладываете всем — начните осторожно, с мониторингом.

Ответ:

- Подготовка стабильной версии модели на dev / small scale
- Отладка и устранение nan / перепадов loss
- Тестирование на валидации и стрессовых сценариях
- Канареечный релиз / постепенное развёртывание
- Мониторинг в продакшене и fallback при аномалиях

📖 Урок 4 - ⌚ Scheduler'ы StepLR, CosineAnnealing, OneCycle

🔮 Шаг 01:

Миссия 1

Вы — инженер ML в стартапе, и ваш код начинает «залипать» из-за слишком высокого learning rate — что вы делаете?

Подсказка: Иногда плавное уменьшение lr помогает стабилизировать обучение.

Ответ:

- **Начать с Schedule: StepLR, CosineAnnealing или OneCycle**
- Оставить lr как есть и надеяться
- Сразу снизить lr до нуля

🔮 Шаг 02:

Миссия 2

Ваш коллега предлагает использовать StepLR с шагом в 10 эпох. В каком случае это оправдано?

Подсказка: Подумайте, когда модель учится стабильно, и нужен «шаговый» сброс.

Ответ:

- **Когда ошибка стабильно убывает, но потом застаивается**
- Когда обучение хаотичное с сильными флуктуациями
- Когда модель уже перейдет в overfit

🔮 Шаг 03:

Миссия 3

Вы пробуете CosineAnnealingLR без рестартов — что вы ожидаете от поведения learning rate?

Подсказка: Представьте cos-волны, убывающую до минимального значения.

Ответ:

- **Плавное снижение lr к минимальному значению**
- Много сильных скачков вверх вниз
- Увеличение lr к концу

🔮 Шаг 04:

Миссия 4

При использовании OneCycle ваш lr сначала растет, потом падает — зачем такое поведение?

Подсказка: Это схема «warm-up и annealing» для более быстрого старта.

Ответ:

- Чтобы постепенно прогреть веса и затем снизить lr
- Чтобы запутать оптимизатор
- Чтобы lr менялся хаотично

🔮 Шаг 05:

Миссия 5

Вы внедряете StepLR с $\text{gamma} = 0.5$, $\text{step_size} = 5$. Как изменится lr через 10 эпох?

Подсказка: Через два шага ($10 / 5 = 2$) lr будет уменьшен дважды.

Ответ:

- lr станет $\text{lr_initial} * 0.5 * 0.5$
- lr станет $\text{lr_initial} * 0.5$
- lr останется lr_initial

🔮 Шаг 06:

Миссия 6

Сопоставьте Scheduler с его характерным поведением lr:

Подсказка: Вспомните графики изменения lr.

Schedulers:

Ответ:

- StepLR
- CosineAnnealingLR
- OneCycleLR

Поведение:

1. lr сначала растёт, потом плавно падает
 2. резкие ступенчатые падения через фиксированные эпохи
 3. плавное cos-спад от начального к минимальному
- **StepLR** — резкие ступенчатые падения через фиксированные эпохи
 - **CosineAnnealingLR** — плавное cos-спад от начального к минимальному
 - **OneCycleLR** — lr сначала растёт, потом плавно падает

🔮 Шаг 07:

Миссия 7

Ваш эксперимент с CosineAnnealing с restarts: зачем делать перезапуски?

Подсказка: Иногда хочется возвращать lr к большей величине для «ре-исследования» пространства.

Ответ:

- Чтобы периодически возвращать lr вверх и избегать попадания в локальный минимум

- Чтобы lr прыгал случайно
- Чтобы lr сбрасывался до нуля

🔮 Шаг 08:

Миссия 8

У вас ограниченный бюджет вычислений — хотите минимизировать колебания lr , при этом всё ещё адаптироваться. Какой scheduler выбрать?

Подсказка: OneCycle проще и часто работает без настроек.

Ответ:

- **OneCycleLR**
- StepLR
- CosineAnnealingLR с множественными restarts

🔮 Шаг 09:

Миссия 9

Разработчик Пётр объясняет схему OneCycle: сопоставьте фазу с её описанием:

Подсказка: В OneCycle есть фаза «growth», максимума, затем спада.

Фазы:

Ответ:

- Фаза возрастания lr
- Фаза высокой plateau или максимум
- Фаза убывания lr

Описание:

1. lr постепенно повышается к peak
2. lr держится около максимума или чуть падает незначительно
3. lr плавно уменьшается к низкому значению

- **Фаза возрастания lr** — lr постепенно повышается к peak
- **Фаза высокой plateau или максимум** — lr держится около максимума или чуть падает незначительно
- **Фаза убывания lr** — lr плавно уменьшается к низкому значению

🔮 Шаг 10:

Миссия 10

Вы тестируете StepLR и заметили, что lr слишком падает — вы хотите сделать шаги реже. Как модифицировать?

Подсказка: step_size отвечает за период между уменьшениями.

Ответ:

- **Увеличить step_size**
- Уменьшить gamma
- Понизить начальный lr

🔮 Шаг 11:

Миссия 11

Вы хотите комбинировать warm-up и CosineAnnealing: какой подход разумен?

Подсказка: Можно добавить линейный рост перед косинусным спадом.

Ответ:

- **Сначала линейно повысить lr, затем запустить CosineAnnealing**
- Сначала CosineAnnealing, потом линейно повысить
- Просто применить CosineAnnealing без warm-up

💡 Шаг 12:

Миссия 12

Вы хотите понять, как gamma влияет на StepLR: сопоставьте параметр с эффектом:

Подсказка: $\gamma < 1$ уменьшает lr, $\gamma = 1$ — не меняет.

Параметры:

Ответ:

- $\gamma = 0.1$
- $\gamma = 0.5$
- $\gamma = 1.0$

Эффект:

1. lr уменьшится в 10 раз
 2. lr уменьшится в 2 раза
 3. lr останется без изменений
- **$\gamma = 0.1$** — lr уменьшится в 10 раз
 - **$\gamma = 0.5$** — lr уменьшится в 2 раза
 - **$\gamma = 1.0$** — lr останется без изменений

💡 Шаг 13:

Миссия 13

Вы запустили OneCycle, но замечаете, что lr никогда не достигает ожидаемого пика — что проверите первым?

Подсказка: Peak зависит от max_lr и доли warm-up.

Ответ:

- **Настройки max_lr или процента роста (warm-up proportion)**
- Количество слоёв модели
- Шаг регуляризации

💡 Шаг 14:

Миссия 14

Ваша модель начала хуже обучаться после слишком агрессивного OneCycle. Что скорректировать?

Подсказка: Возможно, рост или спад lr чересчур крутые.

Ответ:

- **Уменьшить max_lr или замедлить спад**
- Увеличить init_lr
- Убрать регуляризацию

🔧 Шаг 15:

Миссия 15

Расположите этапы при интеграции scheduler в тренировочный цикл:

1. Объявление scheduler после оптимизатора
2. Вызов scheduler.step() (после эпохи или шага)
3. Определение параметров lr, step_size, gamma, max_lr и пр.
4. Запуск обучения по эпохам

Подсказка: Сначала задаются параметры, потом связывают с optimizer, после этого вызывают в цикле.

Ответ:

- Определение параметров lr, step_size, gamma, max_lr и пр.
- Объявление scheduler после оптимизатора
- Запуск обучения по эпохам
- Вызов scheduler.step() (после эпохи или шага)

🔧 Шаг 16:

Миссия 16

У вас есть три варианта поведения lr: StepLR, CosineAnnealing и OneCycle. Расставьте их по сложности настройки (от простой к сложной):

1. OneCycle
2. StepLR
3. CosineAnnealing

Подсказка: Подумайте, куда нужно подбирать параметры вручную.

Ответ:

- StepLR
- CosineAnnealing
- OneCycle

🔧 Шаг 17:

Миссия 17

Окончательная проверка: расставьте стадии работы scheduler от инициации до завершения тренировки:

1. Создание scheduler с заданными параметрами
2. Связывание scheduler с optimizer
3. Внутри цикла обучения: вызовы scheduler.step()
4. Оценка метрик на валидации и возможный рестарт scheduler (для CosineAnnealing)

Подсказка: Подумайте: можно ли сделать рестарт в середине цикла.

Ответ:

- Создание scheduler с заданными параметрами
- Связывание scheduler с optimizer
- Внутри цикла обучения: вызовы scheduler.step()
- Оценка метрик на валидации и возможный рестарт scheduler (для CosineAnnealing)

Модуль 7 - Распараллеливание и AMP

Урок 1 - AMP и mixed precision

Шаг 01:

Миссия 1

Иван написал модель на PyTorch и хочет ускорить обучение с помощью AMP (Automatic Mixed Precision). Что он должен сделать в первую очередь?

Подсказка: Начните с изменений, минимально влияющих на код.

Ответ:

- Ручной перевод всех тензоров в float16
- **Обернуть модель и оптимизатор в `amp.autocast()` / `GradScaler`**
- Переход к TensorRT сразу

Шаг 02:

Миссия 2

Мария замечает, что при включённом AMP модель иногда выдаёт NaN градиенты. Что может быть причиной?

Подсказка: Подумайте про переполнение или недополнение при операции в низкой точности.

Ответ:

- **Слишком большой learning rate при float16**
- Неправильный batch size, равный 1
- Негативный вес regularization

Шаг 03:

Миссия 3

Алексей хочет смешивать precision, но некоторые слои модели плохо работают в float16 (например, softmax с большой экспонентой). Что делать?

Подсказка: Не обязательно весь модель переводить в float16.

Ответ:

- **Оставить такие слои в float32 (mixed precision)**
- Приводить всё к float8
- Отказаться от AMP

Шаг 04:

Миссия 4

Как использовать GradScaler в PyTorch с AMP?

Подсказка: Рассмотрите порядок масштабирования градиентов и оптимизации.

Ответ:

- Сначала unscale, затем backward, затем step
- **Сначала backward с scaled loss, затем unscale и step**
- **Никаких специальных действий** — просто обычный backward

Шаг 05:

Миссия 5

Как убедиться, что при использовании AMP модель остаётся числительно стабильной?

Подсказка: Используйте методы мониторинга в ходе обучения.

Ответ:

- Игнорировать предупреждения
- **Логгировать частичные градиенты и проверять NaN/Inf**
- Отключать AMP при первом сбое

Шаг 06:

Миссия 6

Сопоставьте термин и его роль в AMP:

Подсказка: Подумайте о функции каждого компонента.

Термины:

Ответ:

- autocast
- GradScaler
- scale (масштабирование)
- unscale (обратное масштабирование)

Роли:

1. Временное повышение точности для операций
 2. Масштабирование loss для сохранения чувствительности градиентов
 3. Возврат градиентов к исходному масштабу перед оптимизацией
 4. Снижение риска переполнения в float16
- **autocast** — Временное повышение точности для операций
 - **GradScaler** — Масштабирование loss для сохранения чувствительности градиентов
 - **unscale (обратное масштабирование)** — Возврат градиентов к исходному масштабу перед оптимизацией
 - **scale (масштабирование)** — Снижение риска переполнения в float16

Шаг 07:

Миссия 7

Сопоставьте сценарий и действие, которое нужно предпринять при AMP:

Подсказка: Внимательно прочитайте сценарий.

Сценарии:

Ответ:

- Обнаружен Inf в градиентах
- Обнаружен NaN в loss
- Отсутствует выигрыш по скорости
- Модель падает в точности при переводе в float16

Действия:

1. Понизить lr или применить clip_grad_norm
2. Отключить автокастинг для проблемных слоёв

3. Добавить проверку градиентов или переход к float32
4. Проверить, правильно ли включён autograd и GradScaler
 - **Обнаружен Inf в градиентах** — Понизить lr или применить clip_grad_norm
 - **Обнаружен NaN в loss** — Добавить проверку градиентов или переход к float32
 - **Отсутствует выигрыш по скорости** — Проверить, правильно ли включён autograd и GradScaler
 - **Модель падает в точности при переводе в float16** — Отключить автокастинг для проблемных слоёв

🔮 Шаг 08:

Миссия 8

Сопоставьте тип операции с рекомендацией при mixed precision:

Подсказка: Не все операции устойчивы к низкой точности.

Типы операций:

Ответ:

- Сложение $a + b$
- Матрица $\times$ вектор
- Softmax / экспоненциальная функция
- Сумма снизу (reduce_sum)

Рекомендации:

1. Оставить в float32
 2. Можно безопасно выполнять в float16
 3. Использовать fp32 для стабильности
 4. Можно в float16 при аккуратности
- **Сложение $a + b$** — Можно безопасно выполнять в float16
 - **Матрица $\times$ вектор** — Можно в float16 при аккуратности
 - **Softmax / экспоненциальная функция** — Оставить в float32
 - **Сумма снизу (reduce_sum)** — Использовать fp32 для стабильности

🔮 Шаг 09:

Миссия 9

Сопоставьте этап обучения с AMP и его цель:

Подсказка: Подумайте, где именно происходит масштабирование и unscale.

Этапы:

Ответ:

- Forward pass
- Backward pass
- unscale перед step
- Обновление параметров (optimizer.step)

Цели:

1. Применение градиентов к весам
2. Расчёт потерь и активаций
3. Возврат градиентов к нужному масштабу
4. Расчёт градиентов на основе loss

- **Forward pass** — Расчёт потерь и активаций
- **Backward pass** — Расчёт градиентов на основе loss
- **unscale перед step** — Возврат градиентов к нужному масштабу
- **Обновление параметров (optimizer.step)** — Применение градиентов к весам

🔮 Шаг 10:

Миссия 10

Сопоставьте тип ошибки при mixed precision и наиболее вероятную причину:

Подсказка: Ошибки часто связаны с диапазоном значений.

Типы ошибок:

Ответ:

- Overflow (Inf)
- Underflow (0)
- Precision loss (шум)
- Inconsistent outputs

Причины:

1. Слишком большая magnitude тензоров
 2. Слишком малые значения теряются в float16
 3. Смешение float16 и float32 без приведения
 4. Неправильный выбор слоёв в low precision
- **Overflow (Inf)** — Слишком большая magnitude тензоров
 - **Underflow (0)** — Слишком малые значения теряются в float16
 - **Precision loss (шум)** — Смешение float16 и float32 без приведения
 - **Inconsistent outputs** — Неправильный выбор слоёв в low precision

🔮 Шаг 11:

Миссия 11

Сопоставьте приём оптимизации при AMP с его эффектом:

Подсказка: Подумайте, что помогает уменьшить переполнение.

Приёмы:

Ответ:

- clip_grad_norm
- gradient accumulation
- dynamical loss scaling
- переход к float32 на отдельных слоях

Эффекты:

1. Уменьшение риска overflow в отдельных шагах
 2. Уменьшение Peak memory usage
 3. Стабилизация градиентов при разных масштабах loss
 4. Улучшение точности для чувствительных слоёв
- **clip_grad_norm** — Уменьшение риска overflow в отдельных шагах
 - **gradient accumulation** — Уменьшение Peak memory usage
 - **dynamical loss scaling** — Стабилизация градиентов при разных масштабах loss

- **переход к float32 на отдельных слоях** — Улучшение точности для чувствительных слоёв

Шаг 12:

Миссия 12

Расставьте шаги обучения с AMP в правильном порядке:

1. Backward pass (град вычисляются)
2. optimizer.step()
3. autocast-блок forward
4. unscale + проверка NaN/Inf
5. scale loss через GradScaler

Подсказка: Подумайте, где происходит масштабирование и в какой момент проверка градиентов.

Ответ:

- autocast-блок forward
- scale loss через GradScaler
- Backward pass (град вычисляются)
- unscale + проверка NaN/Inf
- optimizer.step()

Шаг 13:

Миссия 13

Внедрение mixed precision требует особого порядка действий. Расположите этапы:

1. инициализация GradScaler
2. вход в autocast контекст
3. loss.backward() через scaler.scale()
4. scaler.unscale_(optimizer)
5. scaler.step(optimizer) и scaler.update()
6. отключение autocast по выходу из блока

Подсказка: Обратите внимание: этапы unscale и step связаны с GradScaler.

Ответ:

- инициализация GradScaler
- вход в autocast контекст
- loss.backward() через scaler.scale()
- scaler.unscale_(optimizer)
- scaler.step(optimizer) и scaler.update()
- отключение autocast по выходу из блока

Шаг 14:

Миссия 14

При распараллеливании (DDP + AMP) нужно встроить два механизма. Расположите правильный порядок операций внутри шага обучения:

1. optimizer.zero_grad()
2. autocast forward
3. scaler.scale(loss).backward()

4. scaler.unscale_(optimizer)
5. scaler.step(optimizer)
6. scaler.update()
7. DDP all-reduce градиентов (синхронизация)

Подсказка: Синхронизация градиентов DDP должна произойти до обновления параметров.

Ответ:

- optimizer.zero_grad()
- autocast forward
- scaler.scale(loss).backward()
- DDP all-reduce градиентов (синхронизация)
- scaler.unscale_(optimizer)
- scaler.step(optimizer)
- scaler.update()

Шаг 15:

Миссия 15

Инженер Никита описывает pipeline обучения с AMP и multiple devices. Упорядочите стадии:

1. broadcast начальных весов на все устройства
2. вход в autocast на каждом устройстве
3. сбор градиентов (all-reduce)
4. применение optimizer.step с unscale
5. scaler.update()
6. exit autocast

Подсказка: Подумайте, что должно быть синхронизировано до шага обновления.

Ответ:

- broadcast начальных весов на все устройства
- вход в autocast на каждом устройстве
- сбор градиентов (all-reduce)
- применение optimizer.step с unscale
- scaler.update()
- exit autocast

Шаг 16:

Миссия 16

В развернутой среде продакшена нужно встроить контроль стабильности mixed precision. Расположите блоки мониторинга по порядку внутри training loop:

1. логирование потерь и метрик
2. проверка градиентов на NaN / Inf
3. scale loss / backward / unscale / step / update
4. снижение learning rate при застрявших градиентах
5. **если частые переполнения** — отключить AMP для слоёв
6. архивация чекпоинтов при стабильности

Подсказка: Контроль ошибок и корректировка стратегии — ключевые этапы.

Ответ:

- логирование потерь и метрик
- проверка градиентов на NaN / Inf
- scale loss / backward / unscale / step / update
- снижение learning rate при застрявших градиентах
- **если частые переполнения** — отключить AMP для слоёв
- архивация чекпоинтов при стабильности

 Шаг 17:

Миссия 17

Финальный этап: упорядочить интеграцию AMP + DDP + контроль ошибок перед продакшеном:

1. инициализация DDP и разбивка данных по устройствам
2. инициализация GradScaler после DDP
3. внутри цикла: zero_grad на optimizer
4. в autocast: forward + loss
5. scaler.scale(loss).backward()
6. DDP all-reduce синхронизация градиентов
7. scaler.unscale_(optimizer)
8. scaler.step(optimizer) / scaler.update()
9. **логика fallback: если переполнения** — откат для слоёв
10. логирование и сохранение checkpoint

Подсказка: Следите, чтобы синхронизация и unscale были перед optimizer.step, а fallback — после ошибок.

Ответ:

- инициализация DDP и разбивка данных по устройствам
- инициализация GradScaler после DDP
- внутри цикла: zero_grad на optimizer
- в autocast: forward + loss
- scaler.scale(loss).backward()
- DDP all-reduce синхронизация градиентов
- scaler.unscale_(optimizer)
- scaler.step(optimizer) / scaler.update()
- **логика fallback: если переполнения** — откат для слоёв
- логирование и сохранение checkpoint

 Урок 2 - Multi-GPU и distributed training

 Шаг 01:

Миссия 1

Иван запускает тренировку на двух GPU, но видит, что вторая карта почти не используется. Что наиболее вероятно?

Подсказка: Проверьте, как распределяется мини-батч и синхронизацию градиентов.

Ответ:

- **Мини-батч не разбивается по устройствам**
- Слишком маленький learning rate
- Неправильная инициализация весов

- Ошибка в функции активации

🔮 Шаг 02:

Миссия 2

Мария хочет использовать автоматическое смешивание точности (AMP) для ускорения. Какой подход выбрать?

Подсказка: Есть встроенные библиотеки, поддерживающие AMP.

Ответ:

- **Использовать `torch.cuda.amp.autocast` и `GradScaler`**
- Писать вручную float16 свертки
- Перевести всю модель на float16 без контроля
- Отключить нормализацию батчей

🔮 Шаг 03:

Миссия 3

Пётр пытается запустить DDP (DistributedDataParallel), но получает ошибку, что модель не на GPU. Что могло произойти?

Подсказка: DDP требует, чтобы модель была перемещена на правильное устройство перед обёрткой.

Ответ:

- **`model.to(device)` вызван после обёртки DDP**
- `batch_size` слишком велик
- `learning rate` равен нулю
- `optimizer` не задан

🔮 Шаг 04:

Миссия 4

Анна хочет безопасно сохранить чекпоинты при обучении на множестве GPU. Что важно учесть?

Подсказка: Нужно сохранить только главный процесс (rank 0).

Ответ:

- **Сохранять модель только из процесса rank 0**
- Сохранять из всех процессов одновременно
- Игнорировать `optimizer state`
- Не сохранять `scaler` при AMP

🔮 Шаг 05:

Миссия 5

Сопоставьте термин с его назначением в multi-GPU обучении:

Термины:

Ответ:

- All-reduce
- Scatter
- Gather
- Barrier

Назначения:

1. Объединение градиентов со всех устройств
2. Разделение данных между устройствами
3. Синхронизация точек перед продолжением
4. Сбор результатов с устройств в главный

Подсказка: Think, что нужно делать с тензорами: разделить, собрать, совместно синхронизировать.

- **All-reduce** — Объединение градиентов со всех устройств
- **Scatter** — Разделение данных между устройствами
- **Barrier** — Синхронизация точек перед продолжением
- **Gather** — Сбор результатов с устройств в главный

Шаг 06:

Миссия 6

Сопоставьте класс из PyTorch с задачей при distributed training:

Классы:

Ответ:

- DistributedSampler
- DistributedDataParallel
- torch.cuda.amp.GradScaler
- torch.cuda.amp.autocast

Задачи:

1. Автоматическое масштабирование градиентов в AMP
2. Оборачивание модели для распределённой тренировки
3. Подбор сэмплов так, чтобы без перекрытия между GPU
4. Контекст для вычислений с микшированной точностью

Подсказка: AMP компоненты работают вместе, DDP — отдельно, а sampler отвечает за данные.

- **DistributedSampler** — Подбор сэмплов так, чтобы без перекрытия между GPU
- **DistributedDataParallel** — Оборачивание модели для распределённой тренировки
- **torch.cuda.amp.GradScaler** — Автоматическое масштабирование градиентов в AMP
- **torch.cuda.amp.autocast** — Контекст для вычислений с микшированной точностью

Шаг 07:

Миссия 7

Упорядочите этапы, как правильно настроить training loop с DDP и AMP:

1. Backward pass и scaler.step()
2. Оборачивание модели DDP после model.to(device)
3. Начало автокаста с autocast()
4. Оптимизатор zero_grad перед forward
5. Запуск forward внутри autocast
6. Communicate градиенты (all-reduce) автоматически DDP

Подсказка: Сначала модель и DDP, затем каждый шаг: zero → forward → backward → step.

Ответ:

- Оборачивание модели DDP после `model.to(device)`
- Оптимизатор `zero_grad` перед `forward`
- Запуск `forward` внутри `autocast`
- Backward pass и `scaler.step()`
- Communicate градиенты (`all-reduce`) автоматически DDP

🔗 **Шаг 08:**

Миссия 8

Сергей заметил, что при запуске на 4 GPU обучение стало медленнее, чем на 2. Возможная причина?

Подсказка: Посмотрите на накладные расходы синхронизации.

Ответ:

- **Синхронизация градиентов даёт большую задержку при многих устройствах**
- GPU слишком сильно нагреваются
- learning rate слишком мал
- batch_size меньше, чем число GPU

🔗 **Шаг 09:**

Миссия 9

Когда использовать `torch.nn.SyncBatchNorm` в multi-GPU тренинге?

Подсказка: BatchNorm по-умолчанию считает статистики по локальному батчу, что может быть плохо.

Ответ:

- **Когда хотите усреднять статистику BatchNorm по всем GPU**
- Когда модель без BatchNorm
- Когда у вас только один GPU
- Когда вы не используете DDP

🔗 **Шаг 10:**

Миссия 10

Сопоставьте проблему при распределённом обучении с её возможным симптомом:

Проблемы:

Ответ:

- Плохой баланс данных
- Отключённый `gradient synchronization`
- Неправильный `seed` на каждом процессе
- Memory fragmentation

Симптомы:

1. Отклоняющиеся потери (`loss`) на разных GPU
2. Градиенты не сливаются, веса расходятся
3. Непоследовательные результаты при повторных запусках
4. `OutOfMemory` на одной GPU, но не на других

Подсказка: Подумайте: что относится к данным, что к `seed`, что к синхронизации, что к памяти.

- **Плохой баланс данных** — Отклоняющиеся потери (loss) на разных GPU
- **Отключённый gradient synchronization** — Градиенты не сливаются, веса расходятся
- **Неправильный seed на каждом процессе** — Непоследовательные результаты при повторных запусках
- **Memory fragmentation** — OutOfMemory на одной GPU, но не на других

🔮 Шаг 11:

Миссия 11

Сопоставьте стратегию распределения данных с её особенностью:

Стратегии:

Ответ:

- Data parallelism
- Model parallelism
- Pipeline parallelism
- Hybrid parallelism

Особенности:

1. Разделение слоёв или частей модели между устройства
2. Чередование стадий (forward/backward) по пайплайну
3. Копирование всей модели на каждое устройство и разделение данных
4. Сочетание нескольких подходов

Подсказка: Data parallel = копия модели, Model = часть модели, pipeline = стадийно, hybrid = смесь.

- **Data parallelism** — Копирование всей модели на каждое устройство и разделение данных
- **Model parallelism** — Разделение слоёв или частей модели между устройства
- **Pipeline parallelism** — Чередование стадий (forward/backward) по пайплайну
- **Hybrid parallelism** — Сочетание нескольких подходов

🔮 Шаг 12:

Миссия 12

Упорядочите сценарии по возрастанию сложности реализации:

1. Data parallelism
2. Model parallelism
3. Pipeline parallelism
4. Hybrid parallelism

Подсказка: Сначала проще всего — копирование модели, потом части модели, затем конвейеры и гибриды.

Ответ:

- Data parallelism
- Model parallelism
- Pipeline parallelism
- Hybrid parallelism

🔮 Шаг 13:

Миссия 13

При запуске на кластере с несколькими нодами, что нужно задать обязательно?

Подсказка: DDP требует информацию о ранге и адрес мастера.

Ответ:

- **MASTER_ADDR, MASTER_PORT, world_size, rank**
- learning rate и weight decay
- dropout и batch_norm momentum
- optimizer type

🔮 Шаг 14:

Миссия 14

В распределённой среде обучение застопорилось на этапе backward. Что возможно причина?

Подсказка: Проверьте, нет ли неконсистентности в вызове backward на одном из процессов.

Ответ:

- **В одном процессе не вызван backward, нарушена синхронизация**
- learning rate слишком велик
- переопределена loss функция
- optimizer.step вызван до backward

🔮 Шаг 15:

Миссия 15

Сопоставьте режим управления точностью (precision) с его преимуществом:

Режимы:

Ответ:

- FP32
- FP16 (mixed precision)
- BFLOAT16
- FP64

Преимущества:

1. Максимальная точность для чисел двойной точности
2. Ускорение и экономия памяти с контролем точности
3. Универсальность и стабильность обучения
4. Баланс между точностью и скоростью на аппаратуре, поддерживающей BFLOAT16

Подсказка: Mixed precision часто означает FP16 + масштабирование. BFLOAT16 — альтернатива с лучшей динамикой.

- **FP32** — Универсальность и стабильность обучения
- **FP16 (mixed precision)** — Ускорение и экономия памяти с контролем точности
- **BFLOAT16** — Баланс между точностью и скоростью на аппаратуре, поддерживающей BFLOAT16
- **FP64** — Максимальная точность для чисел двойной точности

Шаг 16:

Миссия 16

Упорядочите события при восстановлении модели с AMP и DDP после сбоя:

1. Загрузка state_dict модели
2. Загрузка state_dict optimizer
3. Перемещение модели на устройство и оборачивание DDP
4. Загрузка state_dict scaler для AMP
5. Продолжение тренировки

Подсказка: Сначала модель, затем optimizer, затем scaler, затем обёртка и старт снова.

Ответ:

- Загрузка state_dict модели
- Перемещение модели на устройство и оборачивание DDP
- Загрузка state_dict optimizer
- Загрузка state_dict scaler для AMP
- Продолжение тренировки

Шаг 17:

Миссия 17

Финальная проверка: команда хочет выйти в продакшен с multi-GPU моделью и AMP. Что обязательно протестировать перед запуском?

Подсказка: Проверьте воспроизводимость, масштабируемость и стабильность.

Ответ:

- **Как модель ведёт себя на разных числах GPU**
- **Сохранение и загрузка чекпоинтов с AMP и DDP**
- Только точность FP32 без AMP
- **Воспроизводимость при фиксированных seed**

Урок 3 - Квест настрой обучение на 2xGPU с минимальным VRAM

Шаг 01:

Миссия 1

Вы и ваша команда готовитесь к запуску обучения модели на двух GPU в едином узле. Какой шаг нужно сделать ****первым****?

Подсказка: Перед распределением важно удостовериться, что устройство обнаружено.

Ответ:

- Запустить training loop сразу на два GPU
- **Проверить, что оба GPU распознаны (torch.cuda.device_count())**
- Разбить данные вручную по GPU
- Отключить проверку CUDA

Шаг 02:

Миссия 2

 После проверки системы вы хотите использовать PyTorch DataParallel или DistributedDataParallel? Что выбрать?

Подсказка: Подумайте, где масштабируется — внутри одного узла или на многоузловой кластер.

Ответ:

- DataParallel
- **DistributedDataParallel**
- nn.ModuleList
- RPC-based parallelism

🔧 Шаг 03:

Миссия 3

✨ Вы хотите максимально ускорить обучение на 2 GPU и уменьшить нагрузку на память. Какую технологию стоит внедрить?

Подсказка: Это оптимизация на уровне операций смешанной точности.

Ответ:

- DataParallel
- **AMP (Automatic Mixed Precision)**
- Model pruning
- Gradient clipping

🔧 Шаг 04:

Миссия 4

🔊 При включении AMP вы получаете предупреждение о нехватке памяти на одном из GPU. Что можно попробовать?

Подсказка: Один из режимов AMP слабее, но безопаснее по памяти.

Ответ:

- **Перейти с opt_level O2 на O1 (более щадящий)**
- **Отключить autocast для самых больших слоёв**
- Увеличить batch size
- Отключить gradient scaling

🔧 Шаг 05:

Миссия 5

🔍 Теперь вы хотите измерить, сколько памяти фактически используется на каждом GPU. Какой инструмент стоит использовать?

Подсказка: В PyTorch часто применяют «слежение» за использованием CUDA.

Ответ:

- nvidia-smi лишь извне
- **torch.cuda.memory_allocated() / memory_reserved()**
- perf profiler
- gpustat CLI

🔧 Шаг 06:

Миссия 6

🔗 Сопоставьте термин и его значение в контексте обучения на нескольких GPU:

Термины:

Ответ:

- DDP (DistributedDataParallel)
- AMP autocast
- Gradient accumulation
- Отключение градиентов (no_grad)

Значения:

1. Техника, позволяющая суммировать градиенты за несколько мини-батчей перед шагом оптимизатора.
2. Ограничение области в коде, где не вычисляются градиенты для ускорения вычислений.
3. Обёртка, которая распределяет модель по GPU и синхронизирует градиенты.
4. Контекст, при котором часть операций вычисляются в смешанной точности (FP16/FP32).

Подсказка: Подумайте, что делает каждая техника: распределение, накопление, режим без градиентов, автокаст.

- **DDP (DistributedDataParallel)** — Обёртка, которая распределяет модель по GPU и синхронизирует градиенты.
- **AMP autocast** — Контекст, при котором часть операций вычисляются в смешанной точности (FP16/FP32).
- **Gradient accumulation** — Техника, позволяющая суммировать градиенты за несколько мини-батчей перед шагом оптимизатора.
- **Отключение градиентов (no_grad)** — Ограничение области в коде, где не вычисляются градиенты для ускорения вычислений.

 Шаг 07:

Миссия 7

🔍 Сопоставьте тип ошибки и её причина, характерную при двух-GPU + AMP:

Тип ошибки:

Ответ:

- CUDA OOM (Out of Memory)
- Unexpected NaN в loss
- Различие синхронизации градиентов
- Неопределённый DeviceMismatch

Возможная причина:

1. Непреднамеренное накопление градиентов без очистки перед backward
2. Некорректное указание device для части тензоров на разных GPU
3. Overflow при вычислении в FP16 без масштабирования градиентов
4. Слишком большой batch size на один GPU с учётом бонусной памяти из AMP

Подсказка: Подумайте, что может вызвать NaN в смешанной точности, и почему может возникнуть mismatch.

- **CUDA OOM (Out of Memory)** — Слишком большой batch size на один GPU с учётом бонусной памяти из AMP
- **Unexpected NaN в loss** — Overflow при вычислении в FP16 без масштабирования градиентов
- **Различие синхронизации градиентов** — Непреднамеренное накопление градиентов без очистки перед backward

- **Неопределённый DeviceMismatch** — Некорректное указание device для части тензоров на разных GPU

🔮 Шаг 08:

Миссия 8

🧩 Сопоставьте часть кода с её назначением при AMP + DDP:

Фрагменты:

Ответ:

- `scaler = GradScaler()`
- `with autocast(): loss = model(input)`
- `scaler.scale(loss).backward()`
- `scaler.step(optimizer); scaler.update()`

Назначения:

1. Автоматически масштабировать градиенты для предотвращения underflow/overflow
2. Включить смешанную точность при вычислении forward
3. Запустить шаг оптимизации с учётом масштабирования
4. Подготовка скалера перед backward

Подсказка: Пройдите путь от forward до backward и оптимизатора, отслеживая, что делает каждый вызов.

- **`scaler = GradScaler()`** — Подготовка скалера перед backward
- **`with autocast(): loss = model(input)`** — Включить смешанную точность при вычислении forward
- **`scaler.scale(loss).backward()`** — Автоматически масштабировать градиенты для предотвращения underflow/overflow
- **`scaler.step(optimizer); scaler.update()`** — Запустить шаг оптимизации с учётом масштабирования

🔮 Шаг 09:

Миссия 9

Сопоставьте стратегию распределения градиентов и её характеристику:

Стратегии:

Ответ:

- All-reduce
- Reduce-scatter + All-gather
- Tree reduce
- Bucketed gradient sync

Характеристики:

1. Разбиение градиентов на чанки и синхронизация частями
2. Полное суммирование всех градиентов по всем GPU
3. Синхронизация по древовидной схеме
4. Сочетание редукции и обратного сбора для минимизации трафика

Подсказка: Стратегии отличаются порядком передачи данных и разбиением градиентов на части.

- **All-reduce** — Полное суммирование всех градиентов по всем GPU

- **Reduce-scatter + All-gather** — Сочетание редукции и обратного сбора для минимизации трафика
- **Tree reduce** — Синхронизация по древовидной схеме
- **Bucketed gradient sync** — Разбиение градиентов на чанки и синхронизация частями

🔮 Шаг 10:

Миссия 10

🔗 Сопоставьте проблему и решение при 2×GPU + AMP:

Проблема:

Ответ:

- Градиенты стали слишком малы (underflow)
- Часть параметров не обновляется
- Память переполняется в backward
- Разная скорость обучения на разных GPU

Решение:

1. Использовать gradient accumulation или увеличить precision масштабы
2. Возврат к синхронной оптимизации и включение find_unused_parameters=True
3. Уменьшить batch size, включить checkpointing или отключить лишние буферы
4. Убедиться, что все процессы имеют согласованный seed и lr scheduler

Подсказка: Проблемы в распределённом режиме часто связаны с синхронизацией, неиспользуемыми параметрами и переполнением памяти.

- **Градиенты стали слишком малы (underflow)** — Использовать gradient accumulation или увеличить precision масштабы
- **Часть параметров не обновляется** — Возврат к синхронной оптимизации и включение find_unused_parameters=True
- **Память переполняется в backward** — Уменьшить batch size, включить checkpointing или отключить лишние буферы
- **Разная скорость обучения на разных GPU** — Убедиться, что все процессы имеют согласованный seed и lr scheduler

🔮 Шаг 11:

Миссия 11

🔗 Разместите шаги по отладке OOM на 2 GPU + AMP в правильном порядке:

1. Измерить память на каждом GPU (memory_allocated, memory_reserved)
2. Уменьшить batch size или включить gradient accumulation
3. Отключить autocast для тяжёлых блоков
4. Включить fp32 для некоторых слоёв (mixed precision hybrid)
5. Повторно запустить обучение и проверку на OOM

Подсказка: Начинайте с измерения, затем — шаги по уменьшению нагрузки, затем проверка.

Ответ:

- Измерить память на каждом GPU (memory_allocated, memory_reserved)
- Уменьшить batch size или включить gradient accumulation
- Отключить autocast для тяжёлых блоков
- Включить fp32 для некоторых слоёв (mixed precision hybrid)
- Повторно запустить обучение и проверку на OOM

Шаг 12:

Миссия 12

✂ Правильный порядок включения DDP + AMP + профилирования:

1. Проинициализировать process group (init_process_group)
2. Обёрнуть модель в DDP
3. Создать GradScaler перед training loop
4. Вести forward + backward в autocast контексте
5. Проводить шаг optimizer через scaler и обновление

Подсказка: Сначала идет инициализация распределения, затем оборачивание, затем подготовка AMP и выполнение итерации.

Ответ:

- Проинициализировать process group (init_process_group)
- Обёрнуть модель в DDP
- Создать GradScaler перед training loop
- Вести forward + backward в autocast контексте
- Проводить шаг optimizer через scaler и обновление

Шаг 13:

Миссия 13

📊 Какой порядок профилирования при обучении на 2 GPU с AMP?

1. Запустить baseline без AMP и измерить скорость и память
2. Добавить DDP, повторить измерения
3. Включить AMP (autocast + GradScaler), повторить измерения
4. Внедрить оптимизации (gradient accumulation, layer striping и т.д.)
5. Сравнить результаты и выбрать конфигурацию

Подсказка: Начинайте с базового, постепенно добавляя усложнения и сравнивая.

Ответ:

- Запустить baseline без AMP и измерить скорость и память
- Добавить DDP, повторить измерения
- Включить AMP (autocast + GradScaler), повторить измерения
- Внедрить оптимизации (gradient accumulation, layer striping и т.д.)
- Сравнить результаты и выбрать конфигурацию

Шаг 14:

Миссия 14

🕒 Упорядочите этапы внедрения обучения на 2 GPU с AMP в продакшн пайплайн:

1. Локальное тестирование и профилирование
2. Интеграция в CI / автоматические тесты
3. Запуск на staging-сервере с 2 GPU
4. Мониторинг памяти, скорости, графиков потерь
5. Деплой на продакшн кластер

Подсказка: Включайте компоненты постепенно, следите за метриками перед деплоем.

Ответ:

- Локальное тестирование и профилирование
- Интеграция в CI / автоматические тесты
- Запуск на staging-сервере с 2 GPU
- Мониторинг памяти, скорости, графиков потерь
- Деплой на продакшн кластер

 Шаг 15:

Миссия 15

 Перед деплоем вы хотите гарантировать, что модель ведёт себя одинаково при 1 и при 2 GPU. Что сделать?

Подсказка: Проверьте консистентность вывода и градиентов в обоих режимах.

Ответ:

- Запустить только на 2 GPU и считать, что с 1 будет эквивалентно
- **Сравнить loss и градиенты при запуске на 1 и 2 GPU с одинаковым seed**
- Использовать более высокий learning rate на одном GPU
- Отключить DDP для сравнения

 Шаг 16:

Миссия 16

 Финальная проверка: на двух GPU всё работает, но вы хотите ещё сэкономить VRAM. Какой метод стоит добавить **в конце**?

Подсказка: Это метод, связанный с временным сохранением активаций.

Ответ:

- Применить модель дизертирование
- Увеличить lr scheduler шаг
- **Чекпоинтинг слоёв (activation checkpointing)**
- Использовать gRPC коммуникации

 Урок 4 - Работа с TPU и облачными платформами

 Шаг 01:

Миссия 1

Инженер Анна запускает первую тренировку модели на TPU, но получает ошибку OOM. Что разумнее всего сделать в первую очередь?

Подсказка: На TPU важно контролировать размеры батча и точность.

Ответ:

- **Уменьшить batch size**
- Добавить ещё слоёв модели
- Переключиться на GPU

 Шаг 02:

Миссия 2

 Разработчик Борис хочет ускорить обучение модели на TPU. Какой приём из списка может помочь?

Подсказка: Используйте смешанную точность (mixed precision).

Ответ:

- **Включить AMP / mixed precision**
- Добавить dropout
- Увеличить learning rate

🔧 Шаг 03:

Миссия 3

🔧 Во время обучения на TPU Лена замечает, что загрузка ядра (TPU utilization) низкая. Что первым проверит Лена?

Подсказка: Часто узким местом становится загрузка данных.

Ответ:

- **Проблемы с input pipeline / data loader**
- Неправильная архитектура сети
- Слишком маленький learning rate

🔧 Шаг 04:

Миссия 4

🔧 Разработчик Семён хочет запустить TPU-кластер в облаке (например, GCP или Kaggle). Что сначала нужно сделать?

Подсказка: Нужно обеспечить окружение и доступ.

Ответ:

- **Настроить сервисный аккаунт и роли доступа к TPU**
- Скачать готовую модель
- Сразу запустить обучение
- Игнорировать авторизацию

🔧 Шаг 05:

Миссия 5

➔ Девушка-разработчица Мария хочет перенести модель, протестированную локально, в облачный TPU. Что важно учесть при переходе?

Подсказка: Среда окружения может отличаться.

Ответ:

- **Убедиться, что версии библиотек (TensorFlow, PyTorch/XLA) совпадают**
- Сразу увеличить batch size
- Игнорировать логирование
- Удалить контрольные точки

🔧 Шаг 06:

Миссия 6

🔧 Разработчик Иван хочет понять, какие препятствия возникают при работе с TPU и облачными платформами.

Параметры:

Ответ:

- Ограниченное количество TPU-устройств
- Передача данных между хостом и TPU
- Различия версий библиотек
- Ограничения по памяти

Последствия:

1. Конфликты пакетов и некорректная работа
2. Большая задержка и узкие места
3. Невозможность распределить полностью
4. OOM и отказ выполнения

 Подсказка:

Подумайте, какое последствие логично (версия → конфликт; передача данных → задержка и т.д.).

- **Ограниченное количество TPU-устройств** — Невозможность распределить полностью
- **Передача данных между хостом и TPU** — Большая задержка и узкие места
- **Различия версий библиотек** — Конфликты пакетов и некорректная работа
- **Ограничения по памяти** — OOM и отказ выполнения

 Шаг 07:

Миссия 7

 Инженер Олег изучает методы масштабирования модели на TPU и хочет сопоставить методы с их описаниями.

Методы:

Ответ:

- Data parallelism
- Model parallelism
- Pipeline parallelism
- Tensor parallelism

Описания:

1. Разбиение входных батчей между устройствами
2. Разбиение слоёв или частей модели по устройствам
3. Конвейерное исполнение последовательных стадий модели
4. Параллельное деление операций внутри слоёв (на уровне тензоров)

 Подсказка:

Подумайте, что именно разделяется: данные, слои, этапы, операции внутри слоёв.

- **Data parallelism** — Разбиение входных батчей между устройствами
- **Model parallelism** — Разбиение слоёв или частей модели по устройствам
- **Pipeline parallelism** — Конвейерное исполнение последовательных стадий модели
- **Tensor parallelism** — Параллельное деление операций внутри слоёв (на уровне тензоров)

 Шаг 08:

Миссия 8

✳ Разработчица Ирина разбирает ошибки при запуске TPU в облаке. Сопоставьте ошибку с её типичной причиной.

Ошибки:

Ответ:

- PERMISSION_DENIED
- RESOURCE_EXHAUSTED
- INVALID_ARGUMENT
- DEADLINE_EXCEEDED

Причины:

1. Неправильные аргументы API (напр. некорректный формат)
2. Недостаток квот или ресурсов
3. Превышено время ожидания операции
4. Нет прав доступа к ресурсу

💡 **Подсказка:**

Подумайте: PERMISSION = права, RESOURCE = ресурсы, DEADLINE = время, INVALID_ARGUMENT = аргументы.

- **PERMISSION_DENIED** — Нет прав доступа к ресурсу
- **RESOURCE_EXHAUSTED** — Недостаток квот или ресурсов
- **INVALID_ARGUMENT** — Неправильные аргументы API (напр. некорректный формат)
- **DEADLINE_EXCEEDED** — Превышено время ожидания операции

💡 **Шаг 09:**

Миссия 9

Архитектор Павел хочет подобрать сервисы облака под TPU-использование. Сопоставьте облачный сервис с его свойством:

Сервисы:

Ответ:

- TPU Pods
- Preemptible TPU режим
- On-demand TPU
- TPU в узле с GPU

Свойства:

1. Снижение стоимости, но возможна прерываемость
2. Высокая масштабируемость и плотность ресурсов
3. Быстрый доступ без ожидания резервирования
4. Гибридные возможности и комбинирование с GPU

💡 **Подсказка: Подумайте, что означает Pod (масштаб), preemptible (прерывается), on-demand (по запросу), смешанный узел.**

- **TPU Pods** — Высокая масштабируемость и плотность ресурсов
- **Preemptible TPU режим** — Снижение стоимости, но возможна прерываемость
- **On-demand TPU** — Быстрый доступ без ожидания резервирования
- **TPU в узле с GPU** — Гибридные возможности и комбинирование с GPU

Шаг 10:

Миссия 10

 Инженер Юлия хочет подготовить deployment pipeline модели на TPU. Расположите этапы по порядку:

1. Запуск тестов интеграции и заданий inference
2. Упаковка модели в контейнер (Docker/TensorFlow Serving и т.д.)
3. Производственный деплой в облако с TPU
4. Валидация модели на контрольных данных в облаке
5. Настройка сетевых и IAM разрешений для доступа к TPU

Подсказка:

Подумайте: сначала упаковать, затем дать доступ, затем развернуть, затем проверить, затем запустить.

Ответ:

- Запуск тестов интеграции и заданий inference
- Упаковка модели в контейнер (Docker/TensorFlow Serving и т.д.)
- Настройка сетевых и IAM разрешений для доступа к TPU
- Производственный деплой в облако с TPU
- Валидация модели на контрольных данных в облаке

Шаг 11:

Миссия 11

Разработчик Сергей интегрирует AMP и распределённое обучение. Расположите шаги в правильном порядке:

1. Инициализировать распределённый контекст / стратегии (TPU strategy)
2. Обёрнуть оптимизатор в AMP/GradScaler (если требуется)
3. Обёртка модели для распределения (например, strategy.scope)
4. Разделить данные по устройствам (shard / replicate batches)
5. Вызывать backward / step через AMP

Подсказка:

Подумайте, что сначала нужно настроить стратегию, затем распределить модель и данные, затем обрабатывать шаг обучения.

Ответ:

- Инициализировать распределённый контекст / стратегии (TPU strategy)
- Обёрнуть оптимизатор в AMP/GradScaler (если требуется)
- Обёртка модели для распределения (например, strategy.scope)
- Разделить данные по устройствам (shard / replicate batches)
- Вызывать backward / step через AMP

Шаг 12:

Миссия 12

 Девушка-инженер Катя деплоит модель через CI/CD. Упорядочите этапы:

1. Проверка кода на правила (lint, форматирование)
2. Проведение unit / интеграционных тестов локально
3. Сборка контейнера с моделью и зависимостями

4. Деплой на staging / тестовый TPU кластер
5. Запуск smoke-тестов в staging и проверка корректности
6. Деплой на production TPU

 Подсказка:

Стандартный CI/CD: сначала проверка и тесты, затем сборка, затем staging, затем проверка, затем продакшн.

Ответ:

- Проверка кода на правила (lint, форматирование)
- Проведение unit / интеграционных тестов локально
- Сборка контейнера с моделью и зависимостями
- Деплой на staging / тестовый TPU кластер
- Запуск smoke-тестов в staging и проверка корректности
- Деплой на production TPU

 Шаг 13:

Миссия 13

 Разработчик Алиса строит систему обновлений модели на облачном TPU. Расположите действия по порядку:

1. Сохранить текущую контрольную точку модели
2. Развернуть новую версию модели параллельно (канареечный режим)
3. Провести A/B тестирование на части трафика
4. Мониторить метрики качества и производительности
5. Откатить или завершить перевод трафика на новую модель

 Подсказка:

Подумайте: сначала сохранить стабильную версию, затем запускать новую параллельно, проводить тест, мониторить, затем решать.

Ответ:

- Сохранить текущую контрольную точку модели
- Развернуть новую версию модели параллельно (канареечный режим)
- Провести A/B тестирование на части трафика
- Мониторить метрики качества и производительности
- Откатить или завершить перевод трафика на новую модель

 Шаг 14:

Миссия 14

 Инженер Виктор анализирует лог-файлы TPU и хочет выстроить порядок этапов от low-level к high-level:

1. Собрать логи TPU runtime (XLA, HLO)
2. Агрегировать логи в облачное хранилище
3. Построить дашборды по задержкам и узким местам
4. Интерпретировать горячие операции (hotspots)
5. Принять оптимизационные решения (pipeline, сплитование и т.д.)

 Подсказка:

Сначала низкоуровневые логи, затем агрегация, визуализация, анализ, решение.

Ответ:

- Собрать логи TPU runtime (XLA, HLO)
- Агрегировать логи в облачное хранилище
- Построить дашборды по задержкам и узким местам
- Интерпретировать горячие операции (hotspots)
- Принять оптимизационные решения (pipeline, сплитование и т.д.)

 Шаг 15:

Миссия 15

 Инженер Марк хочет диагностировать падение производительности после миграции на TPU. Упорядочите шаги:

1. Сравнить профили CPU, GPU и TPU на одинаковой задаче
2. Проверить загрузку input pipeline (I/O, prefetch)
3. Измерить размеры передачи данных между хостом и TPU
4. Проверить использование mixed precision и AMP везде
5. Проанализировать распределение нагрузки между TPU-ядрами

 **Подсказка:**Начинайте с простых измерений (pipeline, ввод-вывод), затем углубляйтесь.

Ответ:

- Сравнить профили CPU, GPU и TPU на одинаковой задаче
- Проверить загрузку input pipeline (I/O, prefetch)
- Измерить размеры передачи данных между хостом и TPU
- Проверить использование mixed precision и AMP везде
- Проанализировать распределение нагрузки между TPU-ядрами

 Шаг 16:

Миссия 16

 Разработчица Наташа проектирует резервирование TPU-ресурсов. Расположите этапы по логике:

1. Оценить среднюю и пиковую загрузку TPU
2. Забронировать дополнительные ресурсы для пиков
3. Настроить авто-скейлинг и мониторинг
4. Ограничить расходы (budget caps)
5. Периодически пересматривать ставку резервирования

 **Подсказка:**Сначала нужно понять нагрузку, затем страховаться, затем автоматизировать, затем контролировать расходы, затем ревизовать.

Ответ:

- Оценить среднюю и пиковую загрузку TPU
- Забронировать дополнительные ресурсы для пиков
- Настроить авто-скейлинг и мониторинг
- Ограничить расходы (budget caps)
- Периодически пересматривать ставку резервирования

 Шаг 17:

Миссия 17

🕒 **Финальный испытательный квест!** Разработчик Даниил строит конвейер от ядра до продакшена с TPU. Упорядочите ключевые фазы этапа от обучения до финального деплоя:

1. Профилирование и оптимизация модели (XLA, HLO, трассировки)
2. Обучение и валидация модели на TPU с AMP и распределением
3. Сохранение контрольных точек и версии моделей
4. Упаковка модели и зависимостей в контейнер / сервер
5. Развёртывание на облачный TPU и проверка инференса
6. Мониторинг производительности и корректности в продакшене

💡 **Подсказка:** Подумайте: сначала профиль, потом обучение, потом сохранить, затем упаковать, затем развернуть, затем мониторить.

Ответ:

- Профилирование и оптимизация модели (XLA, HLO, трассировки)
- Обучение и валидация модели на TPU с AMP и распределением
- Сохранение контрольных точек и версии моделей
- Упаковка модели и зависимостей в контейнер / сервер
- Развёртывание на облачный TPU и проверка инференса
- Мониторинг производительности и корректности в продакшене

📁 **Модуль 8 - 🔍 Внутренности модели**

📖 **Урок 1 - 🔗 Grad-CAM, attention map, активации**

💡 **Шаг 01:**

Миссия 1

Вася запускает сверточную нейросеть и хочет визуализировать, **какие регионы входного изображения активируются** при классификации. Какой метод он выберет?

Подсказка: Нужно визуализировать *тепловую карту активаций* на уровне слоёв.

Ответ:

- **Grad-CAM**
- Dropout Visualization
- Weight Decay Map
- BatchNorm Heatmap

💡 **Шаг 02:**

Миссия 2

👩 Лена экспериментирует с attention-сетями и хочет **отобразить, на какие части последовательности модель обращает внимание** при переводе. Какой подход она применит?

Подсказка: Внимание — это веса, распределённые по элементам входа.

Ответ:

- **Attention heatmap / attention weights visualization**
- Gradient clipping map
- Activation maximization
- Dropconnect visualizer

🔗 Шаг 03:

Миссия 3

Иван хочет визуализировать активации ****по каналам**** в сверточном слое для анализа поведения сети. Что он сделает?

Подсказка: Представьте массив активаций размером (C × H × W).

Ответ:

- **Визуализировать каждый канал как отдельное изображение**
- **Вычислить среднее или сумму по каналам и отобразить 2D**
- Отобразить веса слоя вместо активаций
- Игнорировать spatial размер и просто взять первый пиксель каждого канала

🔗 Шаг 04:

Миссия 4

Оля замечает, что heatmap от Grad-CAM совпадает с центральным объектом на изображении, хотя модель ошибается. Что это может означать?

Подсказка: Акцент на наиболее «важной» области не гарантирует корректную классификацию.

Ответ:

- **Модель могла «зафокусироваться» на неверной области, дав ложную атрибуцию**
- Grad-CAM всегда показывает правильный объект
- Heatmap показывает только фон
- Модель не использует активации

🔗 Шаг 05:

Миссия 5

Максим использует Grad-CAM в продакшене и хочет ускорить вычисление. Какое упрощение может он применить?

Подсказка: Можно ограничиться лишь одним слоем ближе к выходу.

Ответ:

- **Берёт один наиболее поздний сверточный слой вместо всех**
- Применяет Grad-CAM ко всем слоям и усредняет
- Запускает Grad-CAM на каждый пиксель входа
- Игнорирует backward pass и использует случайные градиенты

🔗 Шаг 06:

Миссия 6

Архитектор модели Настя хочет сопоставить ключевые термины визуализации нейросетей и их определения.

Термины:

Ответ:

- Grad-CAM
- Guided Backpropagation
- Attention Map
- Feature Map Activation

Определения:

1. Веса внимания, распределённые по элементам входной последовательности
2. Карта, построенная на основе градиентов и активаций
3. Активации одного слоя (обычно сверточного) для каждого входа
4. Модификация обратного распространения с удержанием знаков градиентов

Подсказка: Grad-CAM комбинирует градиенты и активации; attention map — это веса внимания.

- **Grad-CAM** — Карта, построенная на основе градиентов и активаций
- **Guided Backpropagation** — Модификация обратного распространения с удержанием знаков градиентов
- **Attention Map** — Веса внимания, распределённые по элементам входной последовательности
- **Feature Map Activation** — Активации одного слоя (обычно сверточного) для каждого входа

Шаг 07:

Миссия 7

 Артём тестирует три варианта визуализации. Сопоставьте метод с его особенностью.

Методы:

Ответ:

- Grad-CAM++
- Score-CAM
- Layer-CAM
- Eigen-CAM

Особенности:

1. Учитывает положительные градиенты и агрегирует их по каналам
2. Не использует градиенты напрямую, а делает forward-пассы с масками
3. Смешивает идеи Grad-CAM и методы Eigenvector-деф., линейная комбинация каналов
4. Полностью линейная комбинация активаций с компрессией по пространству

Подсказка: Score-CAM — маскирующие forward-проходы; ++ вариант усиливает позитивные градиенты.

- **Grad-CAM++** — Учитывает положительные градиенты и агрегирует их по каналам
- **Score-CAM** — Не использует градиенты напрямую, а делает forward-пассы с масками
- **Layer-CAM** — Смешивает идеи Grad-CAM и методы Eigenvector-деф., линейная комбинация каналов
- **Eigen-CAM** — Полностью линейная комбинация активаций с компрессией по пространству

Шаг 08:

Миссия 8

 Кира хочет понять, что означает каждое из свойств визуализации активаций. Сопоставьте свойство и его значение.

Свойства:

Ответ:

- Локальность
- Стабильность

- Интерпретируемость
- Разрешение

Значения:

1. Способность выделить точные области на изображении
2. Устойчивость к мелким изменённым входам
3. Лёгкость понимания человеком
4. Способность локализовать внимание на небольшом участке

Подсказка: «Разрешение» — это детализация, «стабильность» — устойчивость.

- **Локальность** — Способность локализовать внимание на небольшом участке
- **Стабильность** — Устойчивость к мелким изменённым входам
- **Интерпретируемость** — Лёгкость понимания человеком
- **Разрешение** — Способность выделить точные области на изображении

🔮 Шаг 09:

Миссия 9

👤 Полина работает с трансформером и хочет объяснить, что означают три типа attention: сопоставьте тип и описание.

Типы внимания:

Ответ:

- Self-attention
- Cross-attention
- Multihead attention
- Scaled dot-product attention

Описание:

1. Делится на несколько «голов», которые обучаются независимо
2. Проводится внимание между разными представлениями (encoder→decoder)
3. Вычисляет внимание внутри одной последовательности
4. Использует масштабирование перед softmax для стабильности

Подсказка: Scaled dot-product — базовый механизм внимания; multihead — композиция голов.

- **Self-attention** — Вычисляет внимание внутри одной последовательности
- **Cross-attention** — Проводится внимание между разными представлениями (encoder→decoder)
- **Multihead attention** — Делится на несколько «голов», которые обучаются независимо
- **Scaled dot-product attention** — Использует масштабирование перед softmax для стабильности

🔮 Шаг 10:

Миссия 10

👤 Дима хочет построить визуализацию attention в трансформере. Расположите шаги **в правильном порядке**:

1. Получить attention weights из соответствующего слоя
2. Нормализовать веса (если требуется) и интерполировать к размеру входа
3. Наложить heatmap на вход (визуализация поверх изображения или над токенами)
4. Сделать forward pass через модель с нужным вводом

Подсказка: Сначала inference, потом извлечение, затем нормализация и визуализация.

Ответ:

- Получить attention weights из соответствующего слоя
- Нормализовать веса (если требуется) и интерполировать к размеру входа
- Наложить heatmap на вход (визуализация поверх изображения или над токенами)
- Сделать forward pass через модель с нужным вводом

Шаг 11:

Миссия 11

 Катя анализирует пулы Grad-CAM и Guided Backprop, она хочет создать «Guided Grad-CAM». В каком порядке она объединит процессы?

1. Вычислить Grad-CAM карту
2. Выполнить Guided Backpropagation, получив градиенты по входу
3. Элемент-по-элемент умножить Grad-CAM карту с guided backprop картой
4. Наложить итоговую карту на исходное изображение

Подсказка: Сначала нужно получить каждую карту отдельно, затем объединить и визуализировать.

Ответ:

- Вычислить Grad-CAM карта
- Выполнить Guided Backpropagation, получив градиенты по входу
- Элемент-по-элемент умножить Grad-CAM карту с guided backprop картой
- Наложить итоговую карта на исходное изображение

Шаг 12:

Миссия 12

 Света хочет визуализировать активации нескольких слоёв и объединить их по значимости. Каков правильный порядок?

1. Выбрать слои интереса (напр., ранние, средние, поздние)
2. Нормализовать активации в каждом слое
3. Интерполировать активации к общему разрешению (например, размеру входа)
4. Усреднить или взвесить карты и комбинировать их в единый heatmap
5. Наложить итоговый heatmap на изображение

Подсказка: Сначала обработать каждый слой, затем агрегация и визуализация.

Ответ:

- Выбрать слои интереса (напр., ранние, средние, поздние)
- Нормализовать активации в каждом слое
- Интерполировать активации к общему разрешению (например, размеру входа)
- Усреднить или взвесить карты и комбинировать их в единый heatmap
- Наложить итоговый heatmap на изображение

Шаг 13:

Миссия 13

🔧 Павел хочет сравнить поведение Grad-CAM и Score-CAM на одном входе. Какой порядок действий верен?

1. Forward pass и получить предсказание
2. Grad-CAM: обратно распространить градиенты до слоя, вычислить карту
3. Score-CAM: маскировать карту по каналам и делать forward pass для каждой маски
4. Нормализовать обе карты и наложить на вход для визуализации

Подсказка: Score-CAM требует множества forward-проходов; Grad-CAM — особенно backward.

Ответ:

- Forward pass и получить предсказание
- Grad-CAM: обратно распространить градиенты до слоя, вычислить карту
- Score-CAM: маскировать карту по каналам и делать forward pass для каждой маски
- Нормализовать обе карты и наложить на вход для визуализации

🔧 **Шаг 14:**

Миссия 14

Злата хочет исследовать устойчивость attention maps: в каком порядке она сделает эксперименты с шумами на входе?

1. Добавить слабый шум к входу
2. Произвести forward pass и извлечь attention map
3. Увеличить силу шума постепенно и повторить
4. Сравнить изменения attention map с базовой версией
5. Визуализировать разницу карт

Подсказка: Устойчивость измеряется через постепенные воздействия.

Ответ:

- Добавить слабый шум к входу
- Произвести forward pass и извлечь attention map
- Увеличить силу шума постепенно и повторить
- Сравнить изменения attention map с базовой версией
- Визуализировать разницу карт

🔧 **Шаг 15:**

Миссия 15

🔧 Юлия строит pipeline для объясняющей визуализации: в каком порядке этапов должна быть система?

1. Запустить inference модели
2. Извлечь активации и/или attention weights
3. Построить heatmap / attention map / активационные карты
4. Интерполировать карты к размеру входа
5. Наложить визуализацию на исходные данные для пользователя

Подсказка: Это полный pipeline: от модели до визуализации.

Ответ:

- Запустить inference модели
- Извлечь активации и/или attention weights
- Построить heatmap / attention map / активационные карты

- Интерполировать карты к размеру входа
- Наложить визуализацию на исходные данные для пользователя

🔧 Шаг 16:

Миссия 16

⚙ Борис хочет встроить объясняющую визуализацию в API: упорядочите задачи интеграции:

1. Добавить endpoint для запроса explain-map
2. Принимать входной объект (изображение / последовательность)
3. Запросить модель и получить её активации / attention weights
4. Сгенерировать визуализацию (heatmap / overlay)
5. Вернуть пользователю визуализацию в понятном формате (например, PNG или JSON)

Подсказка: API строится от HTTP-уровня вниз к внутренним вычислениям.

Ответ:

- Добавить endpoint для запроса explain-map
- Принимать входной объект (изображение / последовательность)
- Запросить модель и получить её активации / attention weights
- Сгенерировать визуализацию (heatmap / overlay)
- Вернуть пользователю визуализацию в понятном формате (например, PNG или JSON)

🔧 Шаг 17:

Миссия 17

🔗 Финальный вызов: Мария хочет автоматизировать выбор лучшего метода визуализации (Grad-CAM, Score-CAM, Eigen-CAM и др.) по задаче. Расположите шаги её алгоритма по сложности:

1. Собрать метрики качества визуализации (например, локальность, стабильность)
2. Протестировать все методы визуализации на валидационном наборе
3. Оценить визуализации по метрикам
4. Выбрать метод с лучшим средним результатом
5. Динамически переключать метод в продакшене в зависимости от типа данных

Подсказка: Начать с измерений, потом тестирование, потом выбор и адаптация.

Ответ:

- Собрать метрики качества визуализации (например, локальность, стабильность)
- Протестировать все методы визуализации на валидационном наборе
- Оценить визуализации по метрикам
- Выбрать метод с лучшим средним результатом
- Динамически переключать метод в продакшене в зависимости от типа данных

📖 Урок 2 - 🛠 Debugging в PyTorch и TensorBoard

🔧 Шаг 01:

Миссия 1

Вы работаете над проектом, и в процессе тренировки модели в PyTorch столкнулись с ошибкой при загрузке данных. Какой из вариантов поможет вам понять проблему?

Подсказка: Попробуйте посмотреть на вывод из TensorBoard.

Ответ:

- **Посмотреть на графики в TensorBoard, связанные с загрузкой данных**
- Посмотреть на логи консоли, чтобы найти ошибку
- Просмотреть метрики модели в ходе тренировки
- Проверить все слои модели на ошибки

Шаг 02:

Миссия 2

На вашем проекте возникли проблемы с обучением модели в PyTorch. Какую ошибку наиболее вероятно вы получите, если забыли передать нужные данные в функцию оптимизации?

Подсказка: Оптимизатор должен знать, с чем работать.

Ответ:

- **Ошибка при вычислении градиентов**
- Ошибка с размерностью входных данных
- Ошибка при инициализации модели
- Ошибка в коде активации слоя

Шаг 03:

Миссия 3

Вы заметили, что время обучения модели в PyTorch сильно увеличилось. Что из следующего стоит проверить в первую очередь?

Подсказка: Время обучения можно ускорить, если правильно настроить устройство.

Ответ:

- **Проверьте использование GPU для обучения модели**
- Проверьте настройки оптимизатора
- Уменьшите размер пакета данных
- Проверьте правильность работы функции потерь

Шаг 04:

Миссия 4

 Задача: Сопоставьте ошибки PyTorch с возможными решениями.

Ошибки:

Ответ:

- Ошибка градиента
- Ошибка в размере тензора
- Ошибка на устройстве
- Ошибка с перемещением модели

Решения:

1. Убедитесь, что модель и данные на одном устройстве (GPU/CPU).
 2. Используйте метод `.backward()` для корректного вычисления градиентов.
 3. Проверьте размеры входных и выходных тензоров.
 4. Переместите модель и данные на одно устройство: GPU или CPU.
- **Ошибка градиента** — Используйте метод `.backward()` для корректного вычисления градиентов.

- **Ошибка в размере тензора** — Проверьте размеры входных и выходных тензоров.
- **Ошибка на устройстве** — Убедитесь, что модель и данные на одном устройстве (GPU/CPU).
- **Ошибка с перемещением модели** — Переместите модель и данные на одно устройство: GPU или CPU.

🔗 Шаг 05:

Миссия 5

При работе с TensorBoard, вам нужно настроить отображение градиентов для отслеживания их поведения. Какую команду следует использовать для этого?

Подсказка: Подумайте, какой метод необходим для логирования градиентов.

Ответ:

- **writer.add_scalar('gradients', gradients)**
- writer.add_histogram('gradients', gradients)
- writer.add_image('gradients', gradients)
- writer.add_text('gradients', gradients)

🔗 Шаг 06:

Миссия 6

В ходе разработки модели вам нужно отлаживать её работу. В какой ситуации будет полезно использовать команду torch.autograd.set_detect_anomaly(True) ?

Подсказка: Этот инструмент полезен для выявления ошибок, связанных с градиентами.

Ответ:

- **Когда вы хотите отслеживать аномалии в вычислениях градиентов**
- Когда модель не обучается из-за ошибки в архитектуре
- Когда данные неправильно загружаются
- Когда нет информации о прогрессе обучения

🔗 Шаг 07:

Миссия 7

Вы заметили, что градиенты модели слишком большие и это вызывает проблему с обучением. Какую команду можно использовать, чтобы отследить такие проблемы?

Подсказка: Проверьте, где можно ограничить значения градиентов.

Ответ:

- **torch.nn.utils.clip_grad_norm_()**
- torch.optim.lr_scheduler.StepLR()
- torch.autograd.detect_anomaly()
- torch.tensor.normal_()

🔗 Шаг 08:

Миссия 8

Вы решаете проблему, связанную с ошибками градиента в PyTorch. Какую функцию стоит использовать, чтобы найти все возможные аномалии в процессе вычислений?

Подсказка: Ошибки часто возникают при расчетах, связанных с градиентами.

Ответ:

- `torch.autograd.set_detect_anomaly(True)`
- `torch.nn.functional.softmax()`
- `torch.autograd.Variable()`
- `torch.cuda.is_available()`

Шаг 09:

Миссия 9

 Задача: Сопоставьте методы отладки с их применением в контексте PyTorch.

Методы:

Ответ:

- `torch.autograd.set_detect_anomaly()`
- `torch.nn.utils.clip_grad_norm_()`
- `TensorBoard`
- `torch.cuda.empty_cache()`

Применение:

1. Поиск аномалий в градиентах.
2. Ограничение максимальных значений градиентов.
3. Мониторинг метрик и графиков во время тренировки.
4. Очистка памяти на GPU.

- **`torch.autograd.set_detect_anomaly()`** — Поиск аномалий в градиентах.
- **`torch.nn.utils.clip_grad_norm_()`** — Ограничение максимальных значений градиентов.
- **`TensorBoard`** — Мониторинг метрик и графиков во время тренировки.
- **`torch.cuda.empty_cache()`** — Очистка памяти на GPU.

Шаг 10:

Миссия 10

Расставьте шаги для эффективной отладки модели в PyTorch в правильном порядке:

1. Активировать обнаружение аномалий в градиентах
2. Использовать TensorBoard для анализа метрик
3. Ограничить значения градиентов с помощью `clip_grad_norm_`
4. Очистить кэш GPU

 **Подсказка:**

Ответ:

- Первым делом нужно найти аномалии.
- Затем важно отслеживать метрики для анализа.
- Не забывайте о памяти GPU в процессе тренировки.
- Активировать обнаружение аномалий в градиентах
- Использовать TensorBoard для анализа метрик
- Ограничить значения градиентов с помощью `clip_grad_norm_`
- Очистить кэш GPU

🔗 Шаг 11:

Миссия 11

Вы анализируете проблему с памятью в процессе обучения модели. Какая из следующих команд может помочь вам освободить память на GPU?

Подсказка: Иногда модель может потребовать очистки памяти, особенно при работе с большими наборами данных.

Ответ:

- **`torch.cuda.empty_cache()`**
- `torch.autograd.set_detect_anomaly(True)`
- `torch.optim.lr_scheduler.StepLR()`
- `torch.nn.utils.clip_grad_norm_()`

🔗 Шаг 12:

Миссия 12

Вы хотите исследовать метрики модели в реальном времени с помощью TensorBoard. Что для этого нужно сделать?

Подсказка: Нужно правильно настроить логирование данных.

Ответ:

- **Использовать `SummaryWriter` для записи метрик в TensorBoard**
- Использовать `backward()` для записи градиентов
- Настроить `DataLoader` для вывода изображений
- Использовать `cuda.empty_cache()` для очистки памяти GPU

🔗 Шаг 13:

Миссия 13

В ходе обучения вашей модели возникла ошибка, связанная с неправильным размером входных данных. Что из перечисленного может помочь отследить этот тип ошибок?

Подсказка: Проблемы с размерами часто связаны с недосмотрами в батчах или слоях модели.

Ответ:

- **`torchsummary.summary()`**
- `torch.optim.Adam()`
- `torch.nn.CrossEntropyLoss()`
- `torch.utils.data.DataLoader()`

🔗 Шаг 14:

Миссия 14

Вы хотите ускорить процесс тренировки, однако заметили, что время тренировки слишком велико. Какую функцию стоит использовать для диагностики производительности модели?

Подсказка: Профилирование позволяет детально узнать о времени работы разных частей модели.

Ответ:

- **`torch.profiler.profile()`**
- `torch.nn.Conv2d()`
- `torch.optim.lr_scheduler()`
- `torch.tensor().mean()`

🔗 Шаг 15:

Миссия 15

📖 Задача: Сопоставьте функции с их назначением в контексте профилирования и отладки в PyTorch.

Функции:

Ответ:

- `torch.profiler.profile()`
- `torchsummary.summary()`
- `torch.autograd.set_detect_anomaly()`
- `torch.cuda.empty_cache()`

Назначение:

1. Отслеживание аномалий в градиентах.
 2. Вывод информации о слоях модели и размерах тензоров.
 3. Диагностика производительности модели.
 4. Освобождение памяти на GPU.
- **`torch.profiler.profile()`** — Диагностика производительности модели.
 - **`torchsummary.summary()`** — Вывод информации о слоях модели и размерах тензоров.
 - **`torch.autograd.set_detect_anomaly()`** — Отслеживание аномалий в градиентах.
 - **`torch.cuda.empty_cache()`** — Освобождение памяти на GPU.

🔗 Шаг 16:

Миссия 16

Расставьте шаги для эффективного профилирования и отладки модели в PyTorch в правильном порядке:

1. Использовать `torch.profiler.profile()` для диагностики производительности
2. Анализировать размер входных данных с помощью `torchsummary.summary()`
3. Активировать поиск аномалий в градиентах с помощью `torch.autograd.set_detect_anomaly()`
4. Освободить память на GPU с помощью `torch.cuda.empty_cache()`

🔗 Подсказка:

Ответ:

- Сначала нужно диагностировать производительность модели.
 - Затем можно будет найти проблемы с входными данными.
 - Поиск аномалий в градиентах и освобождение памяти на GPU можно выполнить в конце.
- Использовать `torch.profiler.profile()` для диагностики производительности
 - Анализировать размер входных данных с помощью `torchsummary.summary()`
 - Активировать поиск аномалий в градиентах с помощью `torch.autograd.set_detect_anomaly()`
 - Освободить память на GPU с помощью `torch.cuda.empty_cache()`

🔗 Шаг 17:

Миссия 17

При использовании TensorBoard вы столкнулись с проблемой: графики метрик не обновляются. Что из перечисленного может быть причиной?

Подсказка: Возможно, проблема в том, как вы записываете данные в TensorBoard.

Ответ:

- **Не вызван метод `flush()` для записи данных**
- Модель не обучается корректно
- Используется устаревшая версия TensorBoard
- Неправильный формат данных в TensorBoard

 **Шаг 18:**

Миссия 18

При исследовании проблемы с производительностью в PyTorch вы заметили, что модель часто тормозит на одном из слоев. Что из следующего может помочь найти причину?

Подсказка: Возможно, вам стоит провести детальный анализ этого слоя.

Ответ:

- **Использовать `torch.profiler.profile()` для детального анализа слоя**
- Применить оптимизацию в слое с помощью `torch.quantization`
- Уменьшить размер модели
- Уменьшить размер входных данных

Урок 3 - 🕒 Квест найди, где захлебнулась сеть

 **Шаг 01:**

Миссия 1

Вы загрузили датасет и начали тренировать свою нейросеть, но результаты её работы не совсем соответствуют ожиданиям. Что первым делом стоит проверить?

Подсказка: Проблемы с результатами могут быть связаны с подготовкой данных.

Ответ:

- **Проверить правильность предобработки данных**
- Изменить архитектуру нейросети
- Добавить больше слоёв в нейросеть
- Проверить гиперпараметры модели

 **Шаг 02:**

Миссия 2

В процессе обучения вашей нейросети появляется слишком большое значение функции потерь. Что из этого может быть причиной?

Подсказка: Чрезмерное увеличение функции потерь может быть связано с неправильным выбором шага градиентного спуска.

Ответ:

- **Слишком большой `learning rate`**
- Низкая скорость обучения
- Недостаточная регуляризация
- Неправильный формат данных

 **Шаг 03:**

Миссия 3

 **Задача:** Сопоставьте ошибку с её возможной причиной.

Ошибки:

Ответ:

- Низкая точность модели
- Переполнение градиента
- Проблемы с регуляризацией

Причины:

1. Слишком высокая скорость обучения
2. Отсутствие нормализации данных
3. Переобучение модели

 **Подсказка:**

Обратите внимание на проблему переобучения, которая возникает при слишком сложной модели или малом количестве данных.

- **Низкая точность модели** — Переобучение модели
- **Переполнение градиента** — Слишком высокая скорость обучения
- **Проблемы с регуляризацией** — Отсутствие нормализации данных

 **Шаг 04:**

Миссия 4

После многомесячной работы вы наконец-то настроили свою нейросеть, но в продакшене она начинает вести себя странно. Что первое нужно сделать?

Подсказка: Проблемы на продакшене часто бывают связаны с некорректной обработкой новых данных.

Ответ:

- **Проверить, как модель обрабатывает новые данные**
- Перепроверить гиперпараметры обучения
- Уменьшить размер модели
- Изменить архитектуру нейросети

 **Шаг 05:**

Миссия 5

 **Задача:** Сопоставьте типы регуляризации с их назначением.

Типы регуляризации:

Ответ:

- Dropout
- L2-регуляризация
- Раннее прекращение

Назначения:

1. Предотвращает переобучение, игнорируя некоторые нейроны во время тренировки
2. Останавливает обучение, если метрика на валидации не улучшилась
3. Наказание за большие веса в модели

 **Подсказка:**

Использование регуляризации помогает избежать переобучения, но важно выбрать правильный тип для вашей задачи.

- **Dropout** — Предотвращает переобучение, игнорируя некоторые нейроны во время тренировки
- **L2-регуляризация** — Наказание за большие веса в модели
- **Раннее прекращение** — Останавливает обучение, если метрика на валидации не улучшилась

💡 Шаг 06:

Миссия 6

В процессе обучения ваша модель начинает показывать хороший результат на обучающих данных, но на валидации — значительно хуже. Что это может означать?

Подсказка: Если модель хорошо работает на обучающем наборе, но не на валидационном, скорее всего, она переобучается.

Ответ:

- **Модель переобучается на обучающих данных**
- Модель не выучила необходимые паттерны
- Недостаточно данных для обучения модели
- Архитектура модели слишком простая

💡 Шаг 07:

Миссия 7

📖 Задача: Сопоставьте типы ошибок с возможными методами их устранения.

Типы ошибок:

Ответ:

- Недообучение
- Переполнение градиента
- Переобучение

Методы устранения:

1. Добавить слои в нейросеть или использовать более сложные архитектуры
2. Использовать регуляризацию или раннее прекращение
3. Уменьшить скорость обучения или использовать нормализацию данных

💡 Подсказка:

Каждая ошибка требует уникального подхода к решению. Например, переполнение градиента можно исправить с помощью изменения скорости обучения.

- **Недообучение** — Добавить слои в нейросеть или использовать более сложные архитектуры
- **Переполнение градиента** — Уменьшить скорость обучения или использовать нормализацию данных
- **Переобучение** — Использовать регуляризацию или раннее прекращение

💡 Шаг 08:

Миссия 8

Ваша нейросеть плохо обучается на ограниченном наборе данных. Какую стратегию стоит использовать для улучшения результатов?

Подсказка: Попробуйте извлечь больше информации из уже имеющихся данных.

Ответ:

- **Использовать аугментацию данных**
- Добавить больше слоёв в модель
- Использовать меньший learning rate
- Уменьшить размер датасета

Шаг 09:

Миссия 9

 Задача: Сопоставьте типы обучения с их характеристиками.

Типы обучения:

Ответ:

- Обучение с учителем
- Обучение без учителя
- Обучение с подкреплением

Характеристики:

1. Модель обучается на основе заранее размеченных данных
2. Модель обучается на данных без меток
3. Модель учится взаимодействовать с окружающей средой и получать награды

Подсказка:

Подумайте, какой тип обучения используется для задач, где нужно принимать решения на основе полученных наград.

- **Обучение с учителем** — Модель обучается на основе заранее размеченных данных
- **Обучение без учителя** — Модель обучается на данных без меток
- **Обучение с подкреплением** — Модель учится взаимодействовать с окружающей средой и получать награды

Шаг 10:

Миссия 10

Ваша модель имеет слишком много слоёв и требует огромного времени на обучение. Что стоит сделать в первую очередь?

Подсказка: Иногда проще сделать модель менее сложной, чтобы уменьшить время обучения.

Ответ:

- **Уменьшить количество слоёв в модели**
- Уменьшить количество данных для обучения
- Использовать более сложные гиперпараметры
- Добавить больше слоёв для улучшения обучения

Шаг 11:

Миссия 11

Расположите шаги для оптимизации гиперпараметров нейросети в правильном порядке:

1. Провести анализ результатов текущей модели
2. Провести настройку гиперпараметров
3. Оценить модель на тестовом наборе данных
4. Повторить настройку гиперпараметров при необходимости

Подсказка:

Не забывайте, что гиперпараметры модели нужно подбирать на основе результатов её работы.

Ответ:

- Провести анализ результатов текущей модели
- Провести настройку гиперпараметров
- Оценить модель на тестовом наборе данных
- Повторить настройку гиперпараметров при необходимости

Шаг 12:

Миссия 12

 **Задача:** Сопоставьте методы оптимизации с их характеристиками.

Методы оптимизации:

Ответ:

- SGD
- Adam
- RMSprop

Характеристики:

1. Использует моментум и корректирует скорость обучения в зависимости от предыдущих шагов
2. Адаптивная скорость обучения для каждого параметра
3. Использует среднеквадратичное значение градиента для корректировки скорости обучения

Подсказка:

Выбор оптимизатора может существенно повлиять на скорость сходимости и точность модели.

- **SGD** — Использует моментум и корректирует скорость обучения в зависимости от предыдущих шагов
- **Adam** — Адаптивная скорость обучения для каждого параметра
- **RMSprop** — Использует среднеквадратичное значение градиента для корректировки скорости обучения

Шаг 13:

Миссия 13

Вы заметили, что нейросеть плохо обрабатывает данные с пропущенными значениями. Как поступить?

Подсказка: Порой простое заполнение пропусков может решить проблему.

Ответ:

- **Заполнить пропущенные значения средним или медианой**
- Удалить все строки с пропусками
- Заменить пропуски на случайные значения
- Не обращать внимания на пропуски в данных

Шаг 14:

Миссия 14

Расположите этапы процесса развертывания нейросети на продакшн-сервере в правильном порядке:

1. Подготовить инфраструктуру и развернуть контейнеры
2. Обучить и протестировать модель
3. Интегрировать модель в продакшн-систему
4. Проверить модель на реальных данных

Подсказка:

Не забывайте, что подготовка среды важна для корректного развертывания.

Ответ:

- Подготовить инфраструктуру и развернуть контейнеры
- Обучить и протестировать модель
- Интегрировать модель в продакшн-систему
- Проверить модель на реальных данных

Урок 4 - Анализ градиентов и визуализация потерь

Шаг 01:

Миссия 1

 Разработчик Пётр визуализирует loss-функцию и видит множество локальных минимумов. Какое свойство сети помогает выйти из таких ловушек?

Подсказка: Некоторые функции активации и оптимизаторы специально создавались, чтобы бороться с этим эффектом.

Ответ:

- **Стохастичность оптимизатора (например, Adam или SGD)**
- Глубина сети
- Плоская архитектура
- Отсутствие dropout

Шаг 02:

Миссия 2

 Маша сравнивает loss-функции разных моделей. Какой график говорит о корректном обучении без переобучения?

Подсказка: Ищи, где валидационный loss снижается вместе с обучающим и потом стабилизируется.

Ответ:

- **Train loss снижается, validation loss тоже снижается и выходит на плато**
- Train loss растёт, validation падает
- Train loss падает, validation резко растёт
- Обе функции колеблются хаотично

Шаг 03:

Миссия 3

Андрей изучает градиенты разных слоёв своей CNN и замечает, что первые слои почти не обновляются. Что можно предпринять?

Подсказка: Иногда помогает менять активации или нормализацию, чтобы градиенты лучше проходили назад.

Ответ:

- **Использовать Batch Normalization**
- Удалить последние слои
- Снизить learning rate
- Увеличить размер батча

🔮 **Шаг 04:**

Миссия 4

🕒 Сопоставь тип ошибки при обучении нейросети с её возможной причиной.

Подсказка: Ошибки часто проявляются в графиках: ищи закономерности между loss и метриками.

Ответ:

- **Train loss падает, validation loss растёт** — Переобучение
- **Train и validation loss не меняются** — Нулевые градиенты
- **Loss колеблется хаотично** — Слишком большой learning rate
- **Train loss резко падает в начале** — Хорошая инициализация весов

🔮 **Шаг 05:**

Миссия 5

Алексей анализирует функции активации. Сопоставь их с особенностями влияния на градиенты.

Подсказка: Обрати внимание: ReLU помогает избегать затухания, а sigmoid часто вызывает его.

Ответ:

- **Sigmoid** — Может вызвать исчезающие градиенты
- **ReLU** — Сохраняет большие градиенты для положительных значений
- **Leaky ReLU** — Позволяет небольшие отрицательные градиенты
- **Tanh** — Симметричная, но подвержена затуханию

🔮 **Шаг 06:**

Миссия 6

🔍 Лена изучает оптимизаторы. Сопоставь каждый оптимизатор с его особенностью работы с градиентами.

Подсказка: Подумай, какие используют момент, а какие — адаптивные скорости обучения.

Ответ:

- **SGD** — Использует фиксированный learning rate
- **SGD с моментом** — Добавляет инерцию к изменению весов
- **Adam** — Адаптивно подстраивает learning rate под каждый параметр
- **RMSProp** — Использует экспоненциальное усреднение квадратов градиентов

🔮 **Шаг 07:**

Миссия 7

⚡ Настя экспериментирует с разными функциями потерь. Сопоставь функцию с типичной задачей, где она применяется.

Подсказка: Вспомни: MSE — регрессия, CrossEntropy — классификация.

Ответ:

- **MSE (Mean Squared Error)** — Регрессия (например, прогноз температуры)
- **CrossEntropyLoss** — Классификация (например, распознавание цифр)
- **Hinge Loss** — SVM и задачи разделения классов
- **Binary CrossEntropy** — Двоичная классификация

💡 Шаг 08:

Миссия 8

Данил анализирует ландшафт потерь. Сопоставь понятие с его описанием.

Подсказка: Визуализация помогает понять, почему обучение идёт медленно.

Ответ:

- **Локальный минимум** — Точка, где loss меньше, чем у соседей, но не минимален глобально
- **Глобальный минимум** — Точка с наименьшим возможным значением функции потерь
- **Плато** — Область, где градиенты почти нулевые
- **Седловая точка** — Градиент по одним направлениям положителен, по другим отрицателен

💡 Шаг 09:

Миссия 9

🔗♂ Расставь этапы анализа градиентов в правильном порядке.

Подсказка: Сначала ты считаешь градиенты, потом визуализируешь, а уже после ищешь проблемы.

Ответ:

- Вычислить градиенты параметров
- Построить визуализацию распределения
- Проанализировать наличие исчезающих или взрывающихся градиентов
- Принять решение о корректировке обучения

💡 Шаг 10:

Миссия 10

📖 Расставь шаги при настройке функции потерь для новой задачи.

Подсказка: Сначала пойми тип задачи, затем выбери функцию, а потом настрой веса и метрики.

Ответ:

- Определить тип задачи (регрессия, классификация)
- Выбрать подходящую функцию потерь
- Настроить дополнительные параметры (веса классов, нормализацию)
- Проверить корректность снижения loss при обучении

💡 Шаг 11:

Миссия 11

Расположи этапы визуализации ландшафта потерь.

Подсказка: Сначала собери данные о loss, потом построй поверхность и анализируй структуру.

Ответ:

- Собрать значения функции потерь при разных параметрах
- Построить поверхность потерь
- Выделить локальные минимумы и седловые точки
- Сделать вывод о стабильности обучения

 Шаг 12:

Миссия 12

 Определи последовательность действий при отладке проблемы исчезающих градиентов.

Подсказка: Действуй от диагностики к изменению архитектуры и параметров.

Ответ:

- Проверить распределение градиентов по слоям
- Изменить функцию активации (например, на ReLU)
- Добавить BatchNorm
- Перезапустить обучение и проверить улучшение

 Шаг 13:

Миссия 13

 Расположи шаги анализа переобучения по графикам потерь.

Подсказка: Порядок: наблюдение → сравнение → вывод → действие.

Ответ:

- Наблюдать динамику train/validation loss
- Сравнить поведение обеих кривых
- Сделать вывод о переобучении
- Добавить регуляризацию или dropout

 Шаг 14:

Миссия 14

Финальный квест! Расставь шаги полного цикла анализа и визуализации потерь.

Подсказка: Заверши всё не просто визуализацией, а улучшением модели.

Ответ:

- Собрать данные о loss во время обучения
- Построить графики и тепловые карты
- Интерпретировать результаты визуализации
- Внести изменения в архитектуру или параметры обучения

Модуль 9 - ⚙ Автоматизация и гиперпараметры

Урок 1 - 🕒 Optuna, Weights&Biases

 Шаг 01:

Миссия 1

Ваша команда только начинает работу с Optuna. Какой метод вы выберете для запуска экспериментов оптимизации гиперпараметров?

Подсказка: Подумайте, хотите ли вы просто пробовать наугад или использовать разумную стратегию поиска.

Ответ:

- **Использовать TPE-сэмплер (Tree-structured Parzen Estimator)**
- Перебирать вручную все комбинации
- Выбирать случайные точки из равномерного распределения

🔮 Шаг 02:

Миссия 2

🧐 Вы используете Weights & Biases (W&B) для отслеживания экспериментов. Какой объект вам нужно создать первым в коде, чтобы логировать параметры и метрики?

Подсказка: В документации W&B первым шагом обычно создание сессии эксперимента.

Ответ:

- **wandb.init()**
- wandb.finish()
- wandb.log_params()

🔮 Шаг 03:

Миссия 3

🧐 Вы хотите задать ограничение общего времени на оптимизацию в Optuna. Какой параметр вы используете при запуске `study.optimize()`?

Подсказка: В функции `optimize` есть аргументы, связанные с временем и количеством итераций.

Ответ:

- **timeout**
- time_limit
- max_time

🔮 Шаг 04:

Миссия 4

🧐 У вас есть проблема переобучения. Какую стратегию в ходе гиперпараметрической оптимизации вы добавите, чтобы бороться с этим в W&B?

Подсказка: W&B поддерживает раннюю остановку и контроль за метриками в реальном времени.

Ответ:

- **Включить W&B раннюю остановку (early stopping)**
- Записать все эпохи, даже если ухудшение
- Отключить логирование метрик
- Увеличить batch size в процессе логирования

🔮 Шаг 05:

Миссия 5

Сопоставьте термины Optuna / W&B с их описаниями:

Термины:

Ответ:

- Study
- Trial
- wandb.run
- Семплер (Sampler)

Определения:

1. Одна попытка выбора набора гиперпараметров и обучение модели
2. Класс W&B, представляющий конкретный прогон эксперимента
3. Объект, управляющий выбором новых точек в поиске
4. Коллекция всех треков оптимизации (экспериментов)

Подсказка: Study содержит много trial, а семплер отвечает за стратегию пробирования.

- **Study** — Коллекция всех треков оптимизации (экспериментов)
- **Trial** — Одна попытка выбора набора гиперпараметров и обучение модели
- **wandb.run** — Класс W&B, представляющий конкретный прогон эксперимента
- **Семплер (Sampler)** — Объект, управляющий выбором новых точек в поиске

 Шаг 06:

Миссия 6

 Сопоставьте типы семплеров в Optuna с их характеристиками:

Типы семплеров:

Ответ:

- GridSampler
- RandomSampler
- TPE (Tree-structured Parzen Estimator)
- CmaEsSampler

Характеристики:

1. Исследует пространство по сетке
2. Выбор точек случайно
3. Байесовская оптимизация с распределениями Парзенов
4. Эволюционный подход, особенно для непрерывных пространств

Подсказка: Grid и Random — базовые, TPE — байесовский, CmaEs — эволюционный.

- **GridSampler** — Исследует пространство по сетке
- **RandomSampler** — Выбор точек случайно
- **TPE (Tree-structured Parzen Estimator)** — Байесовская оптимизация с распределениями Парзенов
- **CmaEsSampler** — Эволюционный подход, особенно для непрерывных пространств

 Шаг 07:

Миссия 7

 Сопоставьте ключевые сущности W&B с их назначениями:

Элементы:

Ответ:

- Run

- Project
- Artifact
- Config

Назначения:

1. Сохранение версии данных/моделей для повторного использования
2. Настройки гиперпараметров, фиксированные до запуска
3. Конкретный эксперимент, содержащий логи
4. Группа экспериментов объединённая по смыслу

Подсказка: Artifact — это как файлы и модели, Run — одна попытка, Project — контейнер, Config — параметры.

- **Run** — Конкретный эксперимент, содержащий логи
- **Project** — Группа экспериментов объединённая по смыслу
- **Artifact** — Сохранение версии данных/моделей для повторного использования
- **Config** — Настройки гиперпараметров, фиксированные до запуска

Шаг 08:

Миссия 8

✂ Отсортируйте шаги при запуске проекта, который сочетает Optuna + Logging W&B:

1. Определить пространство гиперпараметров
2. Запустить `wandb.init()` и задать конфигурацию
3. Запуск ``study.optimize()`` с логированием внутри trial
4. Закончить W&B run через `wandb.finish()`
5. Сохранить лучшую модель и зарегистрировать как артефакт

Подсказка: Нужно сначала инициализировать логирование, затем оптимизация, потом сохранение и финиш.

Ответ:

- Определить пространство гиперпараметров
- Запустить `wandb.init()` и задать конфигурацию
- Запуск ``study.optimize()`` с логированием внутри trial
- Закончить W&B run через `wandb.finish()`
- Сохранить лучшую модель и зарегистрировать как артефакт

Шаг 09:

Миссия 9

🔗 Упорядочите этапы логирования метрик внутри одного `trial` в оптимизации:

1. Обучение модели на эпохе
2. Вычисление метрик (loss, accuracy и т.д.)
3. Вызов `wandb.log()` или `trial.report()`
4. Проверка условия остановки (early stopping или прерывание trial)

Подсказка: Каждая эпоха: сначала обучение, потом метрики, затем лог, затем проверка условий прерывания.

Ответ:

- Обучение модели на эпохе
- Вычисление метрик (loss, accuracy и т.д.)

- Вызов `wandb.log()` или `trial.report()`
- Проверка условия остановки (early stopping или прерывание trial)

🔮 Шаг 10:

Миссия 10

📋 Упорядочите стадии интеграции W&B в pipeline ML-проекта:

1. Добавить логирование гиперпараметров
2. Использовать артефакты для хранения модели и данных
3. Включить раннюю остановку и callback
4. Настроить дашборды (метрики, визуализации)
5. Завершить run и очистить ресурсы

Подсказка: Логика: сначала включаем параметры, затем артефакты и остановки, дальше визуализации и наконец завершение.

Ответ:

- Добавить логирование гиперпараметров
- Использовать артефакты для хранения модели и данных
- Включить раннюю остановку и callback
- Настроить дашборды (метрики, визуализации)
- Завершить run и очистить ресурсы

🔮 Шаг 11:

Миссия 11

🔗 Вы запускаете масштабные эксперименты. Какой режим запуска Optuna вы примените, чтобы выдержать параллелизм?

Подсказка: Optuna поддерживает распределённые запуски через определённый режим хранения.

Ответ:

- **RDB (Relational DB) backend**
- In-memory storage
- CSV файл как хранилище

🔮 Шаг 12:

Миссия 12

📋 Вы заметили, что некоторые тройки гиперпараметров часто приводят к ошибкам (NaN). Что стоит сделать при вызове `trial.report()` / `wandb.log()`?

Подсказка: Иногда стоит установить специальное условие или исключение.

Ответ:

- **Пропускать этот trial с `trial.should_prune()` или прерывать его**
- Логировать NaN как 0
- Игнорировать и продолжать как обычно

🔮 Шаг 13:

Миссия 13

🔗 Сопоставьте методы `trial` с их функциями в Optuna:

Методы:

Ответ:

- `trial.suggest_float`
- `trial.suggest_int`
- `trial.should_prune`
- `trial.report`

Функции:

1. Сообщить промежуточную метрику (для ранней остановки)
2. Запросить сэмпл непрерывного гиперпараметра
3. Запросить сэмпл целочисленного гиперпараметра
4. Определить, стоит ли прервать текущий trial

Подсказка: `suggest` — выбор, `report` — лог, `should_prune` — проверка.

- **`trial.suggest_float`** — Запросить сэмпл непрерывного гиперпараметра
- **`trial.suggest_int`** — Запросить сэмпл целочисленного гиперпараметра
- **`trial.report`** — Сообщить промежуточную метрику (для ранней остановки)
- **`trial.should_prune`** — Определить, стоит ли прервать текущий trial

🔮 Шаг 14:

Миссия 14

🗄️ Сопоставьте типы хранилищ Optuna с их характеристиками:

Типы хранилищ:

Ответ:

- `InMemoryStorage`
- `RDBStorage`
- `RedisStorage`
- `SQLiteStorage`

Характеристики:

1. **Хранилище в оперативной памяти** — не сохраняется между запусками
2. СУБД с поддержкой параллельного доступа (PostgreSQL, MySQL и др.)
3. Хранилище через Redis, подходит для распределённой работы
4. Файловое хранение через SQLite, простой вариант на диск

Подсказка: `RDBStorage` — для баз данных, `Redis` — быстрый in-memory сервер, `SQLite` — файл, `InMemory` — быстро, но временно.

- **`InMemoryStorage`** — Хранилище в оперативной памяти — не сохраняется между запусками
- **`RDBStorage`** — СУБД с поддержкой параллельного доступа (PostgreSQL, MySQL и др.)
- **`RedisStorage`** — Хранилище через Redis, подходит для распределённой работы
- **`SQLiteStorage`** — Файловое хранение через SQLite, простой вариант на диск

🔮 Шаг 15:

Миссия 15

📋 Упорядочите этапы деплоя процесса оптимизации гиперпараметров в продакшен:

1. Обёртывание `study.optimize()` в API или микросервис
2. Настройка параметров времени, параллелизма и хранилища

3. Интеграция логирования (W&B) и артефактов в сервис
4. Мониторинг, алертинг и контроль пределов
5. Запуск автозапуска или триггерного этапа

Подсказка: Сначала конфигурация, затем обёртка, логирование внутри, мониторинг и автозапуск.

Ответ:

- Обёртывание ``study.optimize()`` в API или микросервис
- Настройка параметров времени, параллелизма и хранилища
- Интеграция логирования (W&B) и артефактов в сервис
- Мониторинг, алертинг и контроль пределов
- Запуск автозапуска или триггерного этапа

Шаг 16:

Миссия 16

 Упорядочите шаги сохранения и публикации лучшей модели после завершения ``study.optimize()``:

1. Получить лучший trial и гиперпараметры
2. Обучить модель на полном наборе данных с лучшими параметрами
3. Сохранить модель/весы как артефакт W&B
4. Зарегистрировать артефакт в проекте W&B / опубликовать
5. Вернуть артефакт пользователю или системе

Подсказка: Сначала выбираем, затем дообучаем, сохраняем, публикуем, отдаём.

Ответ:

- Получить лучший trial и гиперпараметры
- Обучить модель на полном наборе данных с лучшими параметрами
- Сохранить модель/весы как артефакт W&B
- Зарегистрировать артефакт в проекте W&B / опубликовать
- Вернуть артефакт пользователю или системе

Шаг 17:

Миссия 17

★ Вы хотите комбинировать оптимизацию гиперпараметров и автоматическое масштабирование. Какой подход наиболее практичен?

Подсказка: Подумайте, как можно запускать оптимизацию адаптивно, реагируя на нагрузку.

Ответ:

- **Запуск ``study.optimize()`` в автоскейлируемых контейнерах по нагрузке**
- Ограничиться ручным масштабированием серверов
- Отключить параллельность, запускать поочерёдно

Урок 2 - Reproducibility seed, init, save

Шаг 01:

Миссия 1

Разработчик Алексей обнаружил, что при каждом запуске его модели результаты немного отличаются. Что ему нужно сделать, чтобы добиться воспроизводимости?

Подсказка: Подумай, что помогает фиксировать случайность.

Ответ:

- **Установить фиксированный seed для всех библиотек**
- Запустить обучение несколько раз и усреднить результат
- Использовать разные random seed на каждом запуске
- Выключить GPU

🔮 Шаг 02:

Миссия 2

👤 Инженер Марина запускает PyTorch-модель и хочет, чтобы результат не зависел от случайной инициализации весов. Что ей нужно сделать?

Подсказка: Есть функция, которая задаёт случайность для torch.

Ответ:

- **torch.manual_seed(42)**
- randomize_weights()
- torch.set_default_tensor_type('float64')
- torch.reset_rng()

🔮 Шаг 03:

Миссия 3

👤 Команда DataLab замечает, что при запуске на GPU результаты отличаются от CPU. Почему так происходит?

Подсказка: GPU оптимизации иногда жертвуют точностью ради скорости.

Ответ:

- **Некоторые GPU-операции выполняются недетерминированно**
- CPU использует другой seed
- GPU случайно меняет веса модели
- CPU использует float16, а GPU float64

🔮 Шаг 04:

Миссия 4

👤 Младший инженер Ким сохраняет веса модели с помощью torch.save(). Что произойдет, если он забудет зафиксировать seed перед сохранением?

Подсказка: Сохранённые веса фиксируются, но обучение перед этим — нет.

Ответ:

- **Результаты обучения могут отличаться при повторном запуске**
- Файл модели будет повреждён
- torch.save не работает
- Seed сохранится автоматически

🔮 Шаг 05:

Миссия 5

👤 Программист Олег сохраняет и загружает модель PyTorch. Как правильно восстановить обучение?

Подсказка: Сохраняй не только веса, но и состояние оптимизатора.

Ответ:

- **Сохранять state_dict модели и оптимизатора**
- Сохранять только веса модели
- Сохранять только loss
- Использовать pickle.dump() для всего объекта

🔗 Шаг 06:

Миссия 6

Инженер Саша использует NumPy и PyTorch в одном проекте. Что нужно сделать, чтобы результаты были стабильными?

Подсказка: Каждая библиотека имеет свой генератор случайных чисел.

Ответ:

- **np.random.seed(42)**
- **torch.manual_seed(42)**
- time.sleep(42)
- torch.disable_randomness()

🔗 Шаг 07:

Миссия 7

Исследователь Анна тестирует несколько моделей. Что поможет ей убедиться, что эксперименты воспроизводимы между запусками?

Подсказка: Сохрани не только seed, но и конфигурации.

Ответ:

- **Фиксировать seed**
- **Сохранять параметры модели и данных**
- Менять seed для каждого запуска
- Удалять старые результаты

🔗 Шаг 08:

Миссия 8

📦 Сопоставь библиотеку и способ установки случайного seed.

Подсказка: Каждая библиотека имеет собственную функцию.

Ответ:

- **NumPy** — np.random.seed(42)
- **PyTorch** — torch.manual_seed(42)
- **Python random** — random.seed(42)
- **TensorFlow** — tf.random.set_seed(42)

🔗 Шаг 09:

Миссия 9

🔧 Разработчик Вика готовит модель к сохранению. Сопоставь действие и его правильный метод.

Подсказка: Сохраняем с save , загружаем с load .

Ответ:

- **Сохранить веса модели** — torch.save(model.state_dict(), 'model.pt')

- **Загрузить веса модели** — `model.load_state_dict(torch.load('model.pt'))`
- **Сохранить оптимизатор** — `torch.save(optimizer.state_dict(), 'opt.pt')`
- **Загрузить оптимизатор** — `optimizer.load_state_dict(torch.load('opt.pt'))`

🔗 Шаг 10:

Миссия 10

🔗 У программиста Ильи три эксперимента с разными seed. Сопоставь seed и ожидаемое поведение.

Подсказка: Разные seed → разные результаты, один и тот же seed → идентичность.

Ответ:

- **seed=42** — Результаты повторяются при каждом запуске
- **seed=random()** — Результаты случайны
- **seed фиксируется вручную** — Можно воспроизвести эксперименты
- **seed не указан** — Различие между сессиями

🔗 Шаг 11:

Миссия 11

🔗 Сопоставь термин и его значение в контексте воспроизводимости.

Подсказка: Инициализация — начало, seed — ключ к повторению.

Ответ:

- **Инициализация весов** — Процесс назначения начальных значений параметрам модели
- **Случайный seed** — Начальное значение генератора случайных чисел
- **Детерминизм** — Повторяемость результатов при одинаковых условиях
- **Воспроизводимость** — Возможность получить те же результаты эксперимента

🔗 Шаг 12:

Миссия 12

Инженер Катя хочет сделать обучение детерминированным в PyTorch. Сопоставь настройку и её назначение.

Подсказка: Некоторые опции отвечают за контроль случайных операций на GPU.

Ответ:

- **torch.use_deterministic_algorithms(True)** — Отключает недетерминированные алгоритмы
- **torch.backends.cudnn.deterministic = True** — Обеспечивает детерминированность для cudnn
- **torch.backends.cudnn.benchmark = False** — Отключает динамический подбор оптимизаций
- **torch.manual_seed(0)** — Фиксирует seed для CPU и GPU

🔗 Шаг 13:

Миссия 13

🔗 Расставь шаги для воспроизводимого эксперимента в правильном порядке.

Подсказка: Сначала настрой окружение, потом фиксируй случайность.

Ответ:

- Импортировать все нужные библиотеки

- Задать seed для Python, NumPy, PyTorch
- Настроить детерминированные алгоритмы
- Запустить обучение модели
- Сохранить результаты и конфигурацию

🔮 Шаг 14:

Миссия 14

⚡ Определи правильный порядок сохранения и восстановления модели.

Подсказка: Вначале сохранение, потом загрузка.

Ответ:

- Обучить модель
- Сохранить state_dict модели и оптимизатора
- Загрузить state_dict при новом запуске
- Продолжить обучение или инференс

🔮 Шаг 15:

Миссия 15

📋 Расположи шаги эксперимента для гарантии одинаковых результатов.

Подсказка: Логирование и контроль seed важны в конце и начале.

Ответ:

- Установить фиксированные seed
- Подготовить данные
- Обучить модель
- Залогировать конфигурацию
- Проверить идентичность результатов

🔮 Шаг 16:

Миссия 16

Упорядочи действия для работы с GPU, чтобы добиться детерминизма.

Подсказка: Настройки cudnn должны идти после установки seed.

Ответ:

- `torch.manual_seed(42)`
- `torch.cuda.manual_seed_all(42)`
- `torch.backends.cudnn.deterministic = True`
- `torch.backends.cudnn.benchmark = False`
- Запуск обучения

🔮 Шаг 17:

Миссия 17

📋 Расположи этапы вывода модели в продакшен с сохранением воспроизводимости.

Подсказка: Конфигурация и контроль версий — не последний шаг.

Ответ:

- Фиксировать seed и версию библиотек
- Сохранить обученную модель и оптимизатор

- Экспортировать в формат для продакшена
- Тестировать идентичность вывода
- Задеплоить в продакшен

Урок 3 - Квест сделай автотюнинг loss и lr

Шаг 01:

Миссия 1

Тимлид Артём заметил, что обучение модели идёт слишком медленно. Он хочет подобрать оптимальный learning rate (lr). Какое действие стоит сделать первым?

Подсказка: Начинать стоит с анализа динамики loss — иначе вы будете наощупь.

Ответ:

- Построить график зависимости loss от эпох при разных lr
- Сразу увеличить lr в 10 раз
- Удалить все регуляризации
- Увеличить количество эпох без анализа

Шаг 02:

Миссия 2

🛠 Инженер Маша автоматизирует подбор гиперпараметров. Какой инструмент лучше использовать для автотюнинга learning rate?

Подсказка: Этот инструмент часто применяют для байесовской оптимизации и поиска по сетке.

Ответ:

- Optuna
- TensorBoard
- Matplotlib
- NumPy

Шаг 03:

Миссия 3

👨‍💻 Разработчик Илья запустил автотюнинг loss-функции. Что ему поможет избежать переобучения при подборе гиперпараметров?

Подсказка: Иногда нужно остановиться раньше, чем кажется логичным.

Ответ:

- Ранняя остановка (early stopping)
- Увеличение batch size до максимума
- Удаление dropout
- Повторное обучение без валидации

Шаг 04:

Миссия 4

👥 В команде решили протестировать разные loss-функции для задачи регрессии. Какой из вариантов подойдёт лучше всего?

Подсказка: Подумай, что измеряет ошибку между реальным и предсказанным числом.

Ответ:

- **Mean Squared Error (MSE)**
- Cross Entropy Loss
- Hinge Loss
- Categorical Accuracy

🔮 **Шаг 05:**

Миссия 5

📖 Алексей использует Scheduler для изменения learning rate во время обучения. Зачем это может быть нужно?

Подсказка: Иногда модель учится быстро вначале, но позже нужно замедлить шаги.

Ответ:

- **Чтобы ускорить сходимость и стабилизировать обучение**
- Чтобы постоянно увеличивать lr
- Чтобы выключить оптимизатор
- Чтобы сохранить веса в файл

🔮 **Шаг 06:**

Миссия 6

Соотнеси методы оптимизации с их особенностями.

Подсказка: Один из них использует момент, другой — адаптирует шаг для каждого параметра.

Ответ:

- **SGD** — Использует фиксированный шаг без адаптации
- **Adam** — Адаптирует шаг обучения по каждому параметру
- **RMSprop** — Использует экспоненциальное сглаживание квадратов градиента
- **Momentum** — Добавляет инерцию к направлению градиента

🔮 **Шаг 07:**

Миссия 7

🔧 Помоги Соне сопоставить loss-функции с задачами, где они применяются.

Подсказка: Подумай, где важно различать классы, а где — считать числовую ошибку.

Ответ:

- **Cross Entropy Loss** — Классификация
- **Mean Absolute Error** — Регрессия с устойчивостью к выбросам
- **Binary Cross Entropy** — Двоичная классификация
- **Huber Loss** — Регрессия, компромисс между L1 и L2

🔮 **Шаг 08:**

Миссия 8

Соотнеси стратегии подбора learning rate с их описанием.

Подсказка: Каждая стратегия управляет скоростью обучения по-своему — одни снижают, другие поднимают её волнами.

Ответ:

- **StepLR** — Снижает lr через заданное количество эпох

- **ExponentialLR** — Постепенно уменьшает lr экспоненциально
- **CyclicLR** — Меняет lr по циклу: рост и спад
- **ReduceLROnPlateau** — Снижает lr при остановке улучшений

💡 Шаг 09:

Миссия 9

🔗 Разработчик Паша изучает библиотеки для автоматического подбора гиперпараметров. Соотнеси библиотеку с её особенностью.

Подсказка: У каждой свой подход — от случайного поиска до байесовской магии.

Ответ:

- **Optuna** — Байесовская оптимизация с Tree-structured Parzen Estimator
- **Ray Tune** — Распределённый автотюнинг
- **Hyperopt** — Использует случайный и байесовский поиск
- **Keras Tuner** — Интегрируется с TensorFlow для простого поиска параметров

💡 Шаг 10:

Миссия 10

🔗 Ассистент Лена настраивает логирование автотюнинга. Соотнеси метрику с тем, зачем она нужна.

Подсказка: Одна помогает видеть точность, другая — сходимость.

Ответ:

- **loss** — Измеряет ошибку модели
- **val_loss** — Показывает ошибку на валидации
- **accuracy** — Измеряет долю верных предсказаний
- **lr** — Отслеживает текущий шаг обучения

💡 Шаг 11:

Миссия 11

🔗 Сопоставь тип проблемы с нужной стратегией обучения.

Подсказка: Иногда помогает уменьшить lr, иногда — поменять функцию потерь.

Ответ:

- **Модель не сходится** — Снизить lr или нормализовать данные
- **Переобучение** — Использовать регуляризацию и early stopping
- **Loss скачет** — Сделать lr scheduler или уменьшить шаг
- **Слишком медленное обучение** — Увеличить lr или оптимизировать архитектуру

💡 Шаг 12:

Миссия 12

Расставь шаги настройки автотюнинга lr в правильном порядке.

Подсказка: Сначала определяем диапазон, потом оцениваем метрику.

Ответ:

- Определить диапазон значений learning rate
- Запустить эксперимент с разными lr
- Собрать метрики loss и accuracy
- Выбрать оптимальное значение lr

🔧 Шаг 13:

Миссия 13

🔧 Упорядочь шаги по настройке loss-функции под задачу классификации.

Подсказка: Всё начинается с анализа данных и заканчивается тестированием модели.

Ответ:

- Определить тип задачи (классификация/регрессия)
- Выбрать подходящую loss-функцию
- Настроить оптимизатор и lr
- Проверить качество модели на валидации

🔧 Шаг 14:

Миссия 14

🔧 Упорядочь шаги для настройки ReduceLROnPlateau.

Подсказка: Сначала определяем метрику, затем действие при её стагнации.

Ответ:

- Выбрать метрику для отслеживания (например, val_loss)
- Указать порог улучшения (patience)
- Настроить коэффициент уменьшения lr
- Активировать scheduler во время обучения

🔧 Шаг 15:

Миссия 15

🔧 Упорядочь процесс применения Optuna для подбора lr.

Подсказка: Начни с цели и закончи визуализацией результатов.

Ответ:

- Определить целевую функцию для минимизации loss
- Задать диапазон параметров (например, lr)
- Запустить исследование (study.optimize)
- Анализировать лучшие результаты

🔧 Шаг 16:

Миссия 16

🔧 Расставь этапы интеграции автотюнера в пайплайн обучения.

Подсказка: Автоматизация начинается с конфигурации и заканчивается сохранением лучшей модели.

Ответ:

- Настроить конфигурацию гиперпараметров
- Запустить процесс автотюнинга
- Собрать статистику и метрики
- Сохранить лучшую модель и отчёт

🔧 Шаг 17:

Миссия 17

🏁 Финальный квест! Расположи шаги полного цикла автотюнинга loss и lr в правильном порядке.

Подсказка: От анализа задачи до мониторинга результатов.

Ответ:

- Определить тип задачи и подходящую loss-функцию
- Настроить диапазон значений lr
- Запустить автотюнер для оптимизации
- Выбрать лучшую комбинацию параметров
- Мониторить качество и повторять при необходимости

📖 Урок 4 - 🎁 Создание кастомных callback-ов

💡 Шаг 01:

Миссия 1

🖥️ Маша создаёт кастомный callback для отслеживания точности модели во время обучения. Какой метод callback-а нужно переопределить, чтобы выполнить действие после завершения каждой эпохи?

Подсказка: Вспомни, что эпоха — это полный проход по данным. Есть метод, который срабатывает именно в этот момент.

Ответ:

- **on_epoch_end**
- on_batch_end
- on_train_begin
- on_predict_end

💡 Шаг 02:

Миссия 2

👤 Иван хочет добавить логирование потерь в TensorBoard через свой callback. Где лучше разместить логику записи метрик?

Подсказка: Потери обновляются на каждом шаге обучения, а не только в конце эпохи.

Ответ:

- **В методе on_batch_end**
- В методе on_epoch_begin
- В методе on_train_end
- В методе on_test_begin

💡 Шаг 03:

Миссия 3

🔧 Анна пишет callback, который должен прервать обучение, если метрика accuracy достигнет 0.95. Где лучше поместить эту проверку?

Подсказка: Проверка должна выполняться регулярно — после каждого обновления метрики.

Ответ:

- **В on_epoch_end**
- В on_train_begin
- В on_test_end
- В on_batch_begin

🔮 Шаг 04:

Миссия 4

Алексей добавил callback для сохранения лучших весов модели, но файл не обновляется. Что он забыл сделать?

Подсказка: Callback должен знать, когда метрика улучшилась и где сохранить модель.

Ответ:

- **Передать параметр monitor и путь для сохранения**
- Увеличить learning rate
- Добавить больше эпох
- Изменить размер батча

🔮 Шаг 05:

Миссия 5

📖 София хочет, чтобы callback сохранял модель только если loss уменьшился. Какой флаг отвечает за это поведение в ModelCheckpoint?

Подсказка: Вспомни параметр, который сравнивает текущее и лучшее значение метрики.

Ответ:

- **save_best_only=True**
- overwrite=True
- save_weights_only=False
- monitor_mode='auto'

🔮 Шаг 06:

Миссия 6

Даниил написал кастомный callback и хочет доступ к модели внутри него. Где хранится ссылка на модель внутри callback?

Подсказка: После компиляции и fit() callback получает специальный атрибут.

Ответ:

- **self.model**
- self.net
- self.trainer
- self.session

🔮 Шаг 07:

Миссия 7

👤 Мария хочет добавить задержку в обучении между эпохами для симуляции сложных вычислений. В каком методе стоит разместить паузу?

Подсказка: Задержка нужна после завершения каждой эпохи.

Ответ:

- **on_epoch_end**
- on_train_end
- on_batch_end
- on_predict_begin

🔧 Шаг 08:

Миссия 8

👤 Артём изучает сигналы callback-ов. Сопоставьте методы callback-а с моментом их вызова.

Подсказка: Вспомни: начало — begin, конец — end, а batch и epoch — ключевые этапы.

Ответ:

- **on_train_begin** — Перед стартом обучения
- **on_batch_end** — После обработки каждого батча
- **on_epoch_begin** — В начале каждой эпохи
- **on_train_end** — После завершения обучения

🔧 Шаг 09:

Миссия 9

Катя хочет понять, как связаны типы callback-ов с их задачами. Сопоставьте их правильно.

Подсказка: Подумай, какой callback логирует метрики, а какой — сохраняет модель.

Ответ:

- **ModelCheckpoint** — Сохраняет веса при улучшении метрики
- **EarlyStopping** — Останавливает обучение при плато
- **TensorBoard** — Логирует метрики в визуализатор
- **ReduceLROnPlateau** — Уменьшает скорость обучения при застое

🔧 Шаг 10:

Миссия 10

👤 Андрей пишет callback для контроля за GPU. Сопоставь действия с нужными методами.

Подсказка: Мониторинг ресурсов чаще делают в начале и конце эпохи.

Ответ:

- **on_epoch_begin** — Вывод информации о текущей загрузке GPU
- **on_epoch_end** — Подсчёт среднего времени за эпоху
- **on_train_begin** — Инициализация счётчиков времени
- **on_train_end** — Сброс измерителей и вывод итогов

🔧 Шаг 11:

Миссия 11

👤 Сергей анализирует, какие callback-и могут работать вместе. Сопоставьте комбинации callback-ов с результатом.

Подсказка: Часто один отвечает за логирование, другой — за контроль метрики.

Ответ:

- **TensorBoard + ModelCheckpoint** — Логирование и сохранение лучших моделей
- **EarlyStopping + ReduceLROnPlateau** — Гибкая остановка и адаптация скорости обучения
- **CSVLogger + TensorBoard** — Текстовые и визуальные логи
- **LambdaCallback** — Используется для простых одноразовых действий

🔧 Шаг 12:

Миссия 12

✳ Дмитрий создаёт свой CustomCallback. Сопоставь методы с их назначением внутри класса.
Подсказка: Помни, что callback — это класс, и методы соответствуют событиям обучения.

Ответ:

- **def on_train_begin(self, logs=None)** — Выполняется перед обучением
- **def on_epoch_end(self, epoch, logs=None)** — Вызывается после каждой эпохи
- **def on_batch_begin(self, batch, logs=None)** — Перед началом каждой мини-партии
- **def on_train_end(self, logs=None)** — После завершения всего обучения

🔮 Шаг 13:

Миссия 13

📖 Ирина комбинирует callback-и. Сопоставь описание ситуации с правильным инструментом.
Подсказка: Логи — это не только TensorBoard.

Ответ:

- **Хочет сохранить метрики в CSV** — CSVLogger
- **Нужно визуализировать логи в браузере** — TensorBoard
- **Хочет сохранить только веса модели** — ModelCheckpoint(save_weights_only=True)
- **Нужно просто вывести сообщение в консоль** — LambdaCallback

🔮 Шаг 14:

Миссия 14

📦 Расположи шаги создания кастомного callback-а в правильном порядке.
Подсказка: Начни с импорта, закончи использованием при fit().

Ответ:

- Импортировать `tf.keras.callbacks`
- Создать класс, наследующий от `Callback`
- Переопределить нужные методы (например, `on_epoch_end`)
- Передать экземпляр callback в `model.fit()`

🔮 Шаг 15:

Миссия 15

🕒 Упорядочьте шаги для реализации EarlyStopping вручную через кастомный callback.
Подсказка: Нужно хранить лучшую метрику и проверять улучшения после каждой эпохи.

Ответ:

- Инициализировать значение `best_metric`
- В `on_epoch_end` сравнивать текущую метрику с `best_metric`
- **Если улучшения нет** — увеличить счётчик без улучшений
- **Если счётчик превышает patience** — вызвать `self.model.stop_training = True`

🔮 Шаг 16:

Миссия 16

🔗 Расположи этапы интеграции нескольких callback-ов в проекте.
Подсказка: Важно сначала объявить все callback-и, а потом передать их при обучении.

Ответ:

- Импортировать нужные callback-и

- Создать экземпляры callback-ов
- Добавить их в список callbacks
- Передать список в `model.fit()`

Шаг 17:

Миссия 17

 Размести шаги подготовки callback-а для мониторинга GPU-использования.

Подсказка: Нужно импортировать нужные библиотеки, собрать статистику и встроить её в обучение.

Ответ:

- Импортировать `psutil` или `nvidia-smi`
- Создать класс `GPUCallback`
- Реализовать методы `on_epoch_begin` и `on_epoch_end`
- Передать `GPUCallback` в `model.fit()`

Модуль 10 - Подготовка к продакшену

Урок 1 - TorchScript, ONNX, TFLite

Шаг 01:

Миссия 1

Алексей готовит модель PyTorch к продакшену. Он слышал про TorchScript. Что делает `torch.jit.trace()` ?

Подсказка: Этот метод наблюдает за выполнением модели и записывает вычислительный граф.

Ответ:

- **Создаёт оптимизированное представление модели на основе выполнения с примером входа**
- Удаляет все тензоры из модели
- Переводит модель в NumPy формат
- Конвертирует модель в TensorFlow GraphDef

Шаг 02:

Миссия 2

 Программистка Нина хочет ускорить модель на сервере, но сомневается, какой способ сериализации TorchScript выбрать. Какой вариант предпочтителен для переноса модели между Python и C++?

Подсказка: Вспомни: C++ не умеет читать обычные питоновские pickle-файлы.

Ответ:

- **Использовать сохранение через `torch.jit.save()`**
- Использовать `pickle.dump()`
- Использовать `torch.save(model.state_dict())`
- Сохранять модель в формате `.pt` как обычный Python-объект

Шаг 03:

Миссия 3

👤 Инженер Артём тестирует TorchScript-модель на сервере без GPU. Что нужно сделать, чтобы принудительно выполнить инференс на CPU?

Подсказка: Вспомни про метод `.to()` .

Ответ:

- **`model.to('cpu')`**
- `model.cuda(False)`
- `torch.jit.toCPU(model)`
- `model.eval('cpu')`

🔧 Шаг 04:

Миссия 4

👤 Инна хочет конвертировать PyTorch-модель в ONNX. Какой ключевой аргумент обязателен в функции `torch.onnx.export()` ?

Подсказка: Без него PyTorch не знает форму входных данных.

Ответ:

- **`example_input` (пример входного тензора)**
- `output_file`
- `optimizer`
- `device`

🔧 Шаг 05:

Миссия 5

Сергей загрузил ONNX-модель в `onnxruntime`, но она работает медленнее, чем ожидалось. Что может помочь ускорить выполнение?

Подсказка: Попробуй подумать о разных `execution providers` .

Ответ:

- **Использовать GPU Execution Provider (CUDA или TensorRT)**
- **Включить Graph Optimization Level**
- Перейти на интерпретатор Python 3.6
- Добавить `print()` в каждую функцию для отладки

🔧 Шаг 06:

Миссия 6

👤 Маша готовит модель для мобильного приложения и хочет уменьшить размер. Какой формат она должна использовать?

Подсказка: Этот формат используется в TensorFlow и отлично подходит для телефонов.

Ответ:

- **TensorFlow Lite (TFLite)**
- ONNX
- TorchScript
- Keras HDF5

🔧 Шаг 07:

Миссия 7

Дмитрий хочет оптимизировать TFLite-модель. Что даст наибольшее уменьшение размера без сильной потери точности?

Подсказка: Это связано с переводом float32 в int8.

Ответ:

- **Пост-тренировочная квантизация (Post-training quantization)**
- Data augmentation
- L2-регуляризация
- Dropout

🔮 Шаг 08:

Миссия 8

🔗 Разработчик Павел сравнивает три подхода: TorchScript, ONNX и TFLite. Сопоставьте каждую технологию с её типичным применением.

Подсказка: Один формат универсален, другой мобильный, а третий родом из PyTorch.

Ответ:

- **TorchScript** — Оптимизация и выполнение PyTorch-моделей без Python
- **ONNX** — Обмен моделями между фреймворками
- **TFLite** — Инференс на мобильных устройствах и IoT
- **ONNX Runtime** — Высокопроизводительное выполнение моделей ONNX

🔮 Шаг 09:

Миссия 9

⚙️ Инженер Юля анализирует ошибки при экспорте в ONNX. Сопоставьте ошибку с её вероятной причиной.

Подсказка: Подумайте о неподдерживаемых операциях и неправильных аргументах.

Ответ:

- **Unsupported operator** — Модель использует операцию, отсутствующую в ONNX
- **Shape inference failed** — Не удалось вывести размерность входов или выходов
- **Invalid graph** — Ошибка структуры вычислительного графа
- **Export params missing** — Не переданы веса модели в экспорт

🔮 Шаг 10:

Миссия 10

Антон разбирается с TorchScript API. Сопоставьте методы с их описанием.

Подсказка: Два способа компиляции, один — сохранение, один — загрузка.

Ответ:

- **torch.jit.script** — Компиляция на основе анализа исходного кода
- **torch.jit.trace** — Компиляция на основе выполнения примера входа
- **torch.jit.save** — Сохранение оптимизированной модели в файл
- **torch.jit.load** — Загрузка сохранённой модели

🔮 Шаг 11:

Миссия 11

👧 Катя хочет применить ONNXRuntime. Сопоставьте понятие с описанием.

Подсказка: Некоторые понятия связаны с ускорением, другие — с API.

Ответ:

- **InferenceSession** — Основной объект для загрузки и выполнения модели
- **Execution Provider** — Бэкенд, обеспечивающий вычисления (CPU, CUDA, TensorRT)
- **Run()** — Метод для выполнения инференса
- **Optimization Level** — Уровень оптимизации графа модели

Шаг 12:

Миссия 12

📱 Игорь работает над мобильным приложением. Сопоставьте этап подготовки модели для TFLite с правильным шагом.

Подсказка: Последовательность от тренировки до оптимизации.

Ответ:

- **Training model** — Обучение модели в TensorFlow или PyTorch
- **Convert to SavedModel** — Сохранение в формате TensorFlow
- **TFLite Converter** — Преобразование SavedModel в TFLite
- **Quantization** — Уменьшение размера модели за счёт понижения точности

Шаг 13:

Миссия 13

Расставьте шаги экспорта PyTorch-модели в ONNX в правильном порядке.

Подсказка: Сначала подготовь модель и пример входа, потом экспортируй.

Ответ:

- Загрузить обученную модель
- Создать пример входных данных
- Вызвать `torch.onnx.export()`
- Проверить модель с помощью `onnx.checker.check_model()`

Шаг 14:

Миссия 14

⚙️ Упорядочите шаги подготовки TorchScript-модели к деплою на сервере C++.

Подсказка: Сначала компиляция, потом сохранение, потом загрузка в приложении.

Ответ:

- Компиляция модели через `torch.jit.trace` или `script`
- Сохранение с помощью `torch.jit.save()`
- Импорт модели в C++ приложение через `torch::jit::load()`
- Выполнение инференса

Шаг 15:

Миссия 15

Расставьте шаги оптимизации TFLite модели в правильной последовательности.

Подсказка: Оптимизация обычно происходит после конвертации.

Ответ:

- Конвертировать модель в формат TFLite
- Применить пост-тренировочную квантизацию

- Тестировать точность на валидационных данных
- Развернуть модель в мобильном приложении

🔧 Шаг 16:

Миссия 16

🔧 Упорядочьте шаги проверки ONNX-модели после экспорта.

Подсказка: Проверять модель до того, как запускать на runtime.

Ответ:

- Открыть модель с помощью `onnx.load()`
- Выполнить `onnx.checker.check_model()`
- Проверить визуально граф через Netron
- Запустить инференс через `onnxruntime.InferenceSession`

🔧 Шаг 17:

Миссия 17

🔧 Разработчик готовит пайплайн конверсии модели от обучения до деплоя. Расставьте этапы в порядке выполнения.

Подсказка: Подумайте о пути от PyTorch до смартфона.

Ответ:

- Обучить модель в PyTorch
- Экспортировать модель в ONNX
- Конвертировать в TensorFlow формат
- Преобразовать в TFLite
- Выполнить квантизацию и тестирование на устройстве

📖 Урок 2 - 🌐 FastAPI, Flask и обёртки

🔧 Шаг 01:

Миссия 1

Разработчик Алексей решил быстро задеплоить свой первый API. Какой фреймворк подойдёт лучше для простого REST API без сложной асинхронности?

Подсказка: Подумайте о классическом инструменте, с которого начинали многие Python-разработчики.

Ответ:

- **Flask**
- FastAPI
- Django
- Tornado

🔧 Шаг 02:

Миссия 2

🌐 Разработчица Мария решила перейти на асинхронный API, чтобы ускорить отклик при большом количестве запросов. Какой фреймворк ей подойдёт?

Подсказка: Он был создан с нуля для `asyncio` и Pydantic.

Ответ:

- **FastAPI**
- Flask
- Bottle
- CherryPy

💡 **Шаг 03:**

Миссия 3

✦ Тимлид Иван объясняет джунам, что у FastAPI есть встроенная поддержка документации. Что именно создаёт OpenAPI спецификацию автоматически?

Подсказка: Это не шаблон HTML и не middleware, а особенность, встроенная в сам фреймворк.

Ответ:

- **Автоматическая генерация OpenAPI-схемы**
- Ручное описание API через JSON
- Использование Swagger UI как отдельного пакета
- Генерация документации через Jinja2

💡 **Шаг 04:**

Миссия 4

🐞 Тимофей настраивает запуск FastAPI через Uvicorn. Какой правильный способ запуска приложения в консоли?

Подсказка: Команда начинается с имени сервера и заканчивается указанием пути до приложения.

Ответ:

- **uvicorn main:app --reload**
- python main.py runserver
- fastapi serve app
- flask run main:app

💡 **Шаг 05:**

Миссия 5

Команда внедряет в FastAPI CORS, чтобы разрешить фронтенду обращаться к API. Какой пакет используется для этой настройки?

Подсказка: Это расширение из экосистемы Starlette.

Ответ:

- **CORSMiddleware**
- Flask-CORS
- aiohttp.CORS
- CORSManager

💡 **Шаг 06:**

Миссия 6

📖 Ваня хочет в FastAPI добавить обработку POST-запросов. Какой параметр декоратора используется для этого?

Подсказка: Подумайте о типе HTTP-метода, а не о функции.

Ответ:

- **@app.post()**
- **@app.method('POST')**
- **@router.push()**
- **@app.route('POST')**

 **Шаг 07:**

Миссия 7

🔪 Разработчик Андрей изучает различия между Flask и FastAPI. Сопоставьте характеристики фреймворков с правильными описаниями.

Подсказка: Один из них — асинхронный и использует типизацию, другой — проще и старше.

Ответ:

- **Flask** — Синхронный фреймворк с минимальным ядром.
- **FastAPI** — Асинхронный фреймворк, использующий Pydantic и type hints.
- **Flask Blueprints** — Механизм модульного построения приложения.
- **FastAPI Depends** — Система внедрения зависимостей.

 **Шаг 08:**

Миссия 8

📁 Артём настраивает структуру проекта FastAPI. Сопоставьте папки с их содержимым.

Подсказка: Логика маршрутов — не там, где модели, а шаблоны — не рядом с API.

Ответ:

- **routers/** — Маршруты и эндпоинты API.
- **models/** — Pydantic-схемы и ORM-модели.
- **templates/** — HTML-шаблоны (если используются).
- **core/** — Конфигурации и настройки приложения.

 **Шаг 09:**

Миссия 9

⚡ Егор хочет добавить асинхронную работу с базой. Сопоставьте библиотеку и её назначение.

Подсказка: Одна из них работает через SQLAlchemy, а другая — через asyncpg.

Ответ:

- **SQLAlchemy** — ORM для синхронного взаимодействия с БД.
- **SQLModel** — Совместимая с FastAPI ORM с типизацией.
- **Databases** — Асинхронный слой над SQLAlchemy.
- **asyncpg** — Асинхронный драйвер PostgreSQL.

 **Шаг 10:**

Миссия 10

Настя добавляет обработку ошибок в FastAPI. Сопоставьте тип ошибки и корректную обработку.

Подсказка: Для ошибок валидации и HTTP-ответов используются разные подходы.

Ответ:

- **HTTPException** — Используется для генерации кастомных HTTP-ответов с кодом и сообщением.
- **RequestValidationError** — Выбрасывается при ошибке проверки входных данных.
- **ValueError** — Обычное исключение Python, может быть перехвачено глобально.
- **MiddlewareException** — Редко используется, применяется в пользовательских middleware.

🔮 Шаг 11:

Миссия 11

👤 Тимлид Артём хочет объяснить команде, как использовать зависимости в FastAPI. Сопоставьте зависимость и её назначение.

Подсказка: Обратите внимание, что Depends может вызывать функции, возвращающие подключение, токен и т.п.

Ответ:

- **Depends(get_db)** — Получение подключения к базе данных.
- **Depends(get_current_user)** — Авторизация текущего пользователя.
- **Depends(get_settings)** — Доступ к конфигурации приложения.
- **Depends(get_cache)** — Работа с кэшем или Redis.

🔮 Шаг 12:

Миссия 12

👤 Команда готовит Flask-приложение к продакшену. Сопоставьте инструменты и их предназначение.

Подсказка: Один отвечает за WSGI-сервер, другой за окружение.

Ответ:

- **Gunicorn** — WSGI-сервер для запуска Flask-приложения.
- **dotenv** — Загрузка переменных окружения из .env файла.
- **Flask-RESTful** — Создание REST API на Flask.
- **Werkzeug** — Базовый инструмент для маршрутизации и WSGI в Flask.

🔮 Шаг 13:

Миссия 13

Расположите шаги создания базового FastAPI-приложения в правильном порядке:

Подсказка: Вспомните: сначала создаётся файл, потом приложение, маршруты и запуск.

Ответ:

- Создать файл main.py
- Инициализировать объект FastAPI()
- Определить маршрут с декоратором @app.get()
- Запустить приложение через uvicorn main:app --reload

🔮 Шаг 14:

Миссия 14

👤 Расположите этапы подготовки API к продакшену в правильном порядке:

Подсказка: Перед запуском стоит убедиться, что окружение настроено и тесты пройдены.

Ответ:

- Настройка переменных окружения (.env)
- Проведение автоматических тестов
- Запуск через Gunicorn/Uvicorn
- Настройка CORS и логирования

 Шаг 15:

Миссия 15

Расположите шаги интеграции FastAPI с фронтендом на React в логическом порядке:
Подсказка: Начинаем с настройки CORS и заканчиваем запуском сервера.

Ответ:

- Установить CORSMiddleware
- Создать API эндпоинты
- Подключить фронтенд к API
- Запустить сервер FastAPI

 Шаг 16:

Миссия 16

 Расположите шаги деплоя FastAPI-приложения на сервер в правильном порядке:
Подсказка: Сначала настройка окружения, потом запуск процесса и мониторинг.

Ответ:

- Подготовка виртуального окружения и зависимостей
- Настройка Gunicorn/Uvicorn
- Запуск через systemd или supervisor
- Мониторинг логов и состояния приложения

 Урок 3 - Квест оберни модель и замерь latency

 Шаг 01:

Миссия 1

Разработчик Саша только что обучил модель классификации изображений и хочет обернуть её в API. Что нужно сделать первым делом перед запуском сервера?

Подсказка: Прежде чем что-то отдавать наружу — убедись, что модель готова к использованию.

Ответ:

- **Загрузить и протестировать модель локально**
- Сразу задеплоить модель на сервер
- Открыть порты и ждать запросы
- Добавить случайный эндпоинт для проверки

 Шаг 02:

Миссия 2

 Инженер Миша использует FastAPI для обёртки модели. Какой HTTP-метод логичнее выбрать для запроса с изображением?

Подсказка: Метод должен позволять передавать данные тела запроса (payload).

Ответ:

- **POST**
- GET
- DELETE
- PATCH

 **Шаг 03:**

Миссия 3

 Дизайнер данных Катя хочет измерить время отклика (latency) своего API. Что будет наиболее корректным способом?

Подсказка: Не измеряй скорость глазами — используй таймер или инструмент.

Ответ:

- **Замерить время до получения ответа через `time.time()`**
- Посчитать среднее время руками
- Добавить `print` в код модели
- Использовать логирование без тайминга

 **Шаг 04:**

Миссия 4

 Разработчик Алекс запускает FastAPI-приложение с моделью на сервере. Какой способ измерения latency будет наиболее репрезентативным?

Подсказка: Локальные замеры не всегда отражают реальную задержку при сетевом взаимодействии.

Ответ:

- **Посылать запросы через `curl` или `requests` с другого клиента**
- Мерить внутри функции предсказания
- Считать время запуска Python
- Добавить `sleep()` и проверять

 **Шаг 05:**

Миссия 5

 Настя сравнивает latency модели в CPU и GPU окружении. Где задержка, как правило, больше?

Подсказка: Учитывай время передачи данных между устройствами.

Ответ:

- **На GPU при частых маленьких запросах**
- На CPU всегда
- Зависит только от версии Python
- На GPU всегда быстрее

 **Шаг 06:**

Миссия 6

 Инженер Пётр хочет уменьшить latency модели. Что ему стоит сделать в первую очередь?

Подсказка: Иногда проблема не в модели, а в лишних шагах вокруг неё.

Ответ:

- **Убрать ненужные преобразования данных перед инференсом**
- Добавить print для дебага
- Разбить модель на части
- Сжать данные в zip перед каждым вызовом

🔗 Шаг 07:

Миссия 7

Никита хочет вычислить среднюю latency по 100 запросам. Какой способ наиболее точен?

Подсказка: Один замер — не статистика.

Ответ:

- **Выполнить 100 запросов и взять среднее время отклика**
- Посмотреть время первого запроса
- Умножить время одного запроса на 100
- Замерить один запрос и умножить на два для запаса

🔗 Шаг 08:

Миссия 8

🔗 Команда тестирует разные способы измерения latency. Сопоставьте инструмент и его описание.

Подсказка: Некоторые инструменты встроены в Python, другие внешние.

Ответ:

- **time.time()** — Простая функция Python для замера начала и конца операции.
- **ab (Apache Benchmark)** — Инструмент командной строки для нагрузочного тестирования HTTP.
- **curl -w** — Позволяет вывести время отклика запроса в консоль.
- **requests + time** — Ручной способ измерения latency через Python.

🔗 Шаг 09:

Миссия 9

🔗 Разработчица Оля измеряет latency нейросети. Сопоставьте фактор и его влияние на задержку.

Подсказка: Одни факторы связаны с сетью, другие — с вычислениями.

Ответ:

- **Batch size** — Большие значения увеличивают время ответа, но повышают throughput.
- **Сетевые задержки** — Зависит от расстояния между клиентом и сервером.
- **IO операции** — Чтение/запись файлов может тормозить инференс.
- **Оптимизация модели** — Может существенно уменьшить latency без потери качества.

🔗 Шаг 10:

Миссия 10

⚡ Команда использует ONNX Runtime и TensorRT. Сопоставьте технологию и её преимущество.

Подсказка: Эти инструменты нужны не для обучения, а для ускорения инференса.

Ответ:

- **ONNX Runtime** — Позволяет выполнять модели на разных фреймворках с оптимизациями.
- **TensorRT** — Глубокая оптимизация вычислений на GPU от NVIDIA.
- **TorchScript** — Позволяет сериализовать PyTorch-модель для быстрого запуска.
- **OpenVINO** — Оптимизация моделей под Intel CPU и VPU.

Шаг 11:

Миссия 11

Миша строит пайплайн инференса. Сопоставьте этап и его роль.

Подсказка: Вспомните, что происходит от входных данных до ответа модели.

Ответ:

- **Preprocessing** — Подготовка входных данных (resize, normalize).
- **Model inference** — Запуск предсказания модели.
- **Postprocessing** — Преобразование вывода модели в удобный формат.
- **Response** — Отправка результата клиенту.

Шаг 12:

Миссия 12

🔧 Разработчик Дима анализирует latency. Сопоставьте метрику и её значение.

Подсказка: Не все метрики отражают одно и то же: среднее и перцентили — разные истории.

Ответ:

- **Average latency** — Среднее время отклика за множество запросов.
- **P95 latency** — 95% запросов укладываются в это время.
- **P99 latency** — Почти все запросы, кроме редких медленных, быстрее этого порога.
- **Max latency** — Самое медленное время отклика из всех.

Шаг 13:

Миссия 13

Команда оптимизирует latency. Сопоставьте приём и его эффект.

Подсказка: Некоторые оптимизации выполняются на уровне модели, другие — инфраструктурные.

Ответ:

- **Quantization** — Уменьшает размер модели и ускоряет вычисления.
- **Batching** — Обрабатывает несколько запросов за один проход модели.
- **Caching** — Повторно использует результаты предыдущих запросов.
- **Model pruning** — Удаляет незначимые веса, снижая сложность модели.

Шаг 14:

Миссия 14

📋 Расположите шаги измерения latency в правильном порядке:

Подсказка: Не забудьте — сначала запрос, потом замер, потом результат.

Ответ:

- Отправить запрос на API
- Замерить время начала

- Дождаться ответа
- Вычислить разницу между началом и концом

💡 Шаг 15:

Миссия 15

Расположите шаги оптимизации latency модели в логическом порядке:

Подсказка: Сначала измеряем, потом оптимизируем, а не наоборот.

Ответ:

- Измерить текущую latency
- Найти узкие места
- Применить оптимизацию (quantization, batching и т.д.)
- Перемерить результат

💡 Шаг 16:

Миссия 16

🔧 Расположите шаги развертывания API с моделью в продакшене в правильном порядке:

Подсказка: Деплой без тестов — путь к катастрофе.

Ответ:

- Подготовить Docker-образ с моделью
- Протестировать API локально
- Настроить CI/CD
- Развернуть контейнер на сервере

💡 Шаг 17:

Миссия 17

🌐 Расположите шаги анализа latency после продакшена в правильной последовательности:

Подсказка: Не всё измеряется синтетикой — важно смотреть реальные данные.

Ответ:

- Собрать логи с реальных запросов
- Рассчитать P95 и P99 latency
- Сравнить с контрольными значениями
- Принять решение об оптимизации

📖 Урок 4 - 🔗 CI/CD с GitHub Actions и Jenkins

💡 Шаг 01:

Миссия 1

🔧 Тимлид Маша хочет добавить автоматическую проверку тестов при каждом пуше в GitHub. Какое решение подойдёт лучше всего?

Подсказка: Это инструмент, встроенный прямо в GitHub и не требует внешнего сервера.

Ответ:

- **GitHub Actions**
- Docker Compose
- Git Submodules
- VS Code Tasks

🔗 Шаг 02:

Миссия 2

Разработчик Пётр настраивает CI для проекта на Python. Какой шаг ему нужно добавить в workflow, чтобы установить зависимости перед тестами?

Подсказка: Без этого команда pytest просто не запустится.

Ответ:

- **pip install -r requirements.txt**
- python setup.py uninstall
- pytest --skip-install
- mkdir deps

🔗 Шаг 03:

Миссия 3

👤 Настя настроила GitHub Actions, но pipeline не запускается. Что из этого может быть причиной?

Подсказка: Действие не может сработать без определённого события в репозитории.

Ответ:

- **Не указано событие trigger (например, push или pull_request)**
- Файл workflow имеет расширение .json
- Файл pipeline слишком большой
- Репозиторий приватный

🔗 Шаг 04:

Миссия 4

Команда DevOps хочет запускать тесты только при изменении кода в папке src/. Как этого добиться в GitHub Actions?

Подсказка: Для этого в разделе on: можно указать определённые пути.

Ответ:

- **Использовать параметр paths: ['src/**']**
- Добавить тег 'src' к коммиту
- Внести правило в .gitignore
- Переименовать workflow в src.yaml

🔗 Шаг 05:

Миссия 5

🔒 DevOps-инженер Андрей хочет хранить токен доступа к DockerHub в GitHub Actions безопасно. Где ему это сделать?

Подсказка: Там, где GitHub автоматически шифрует значения переменных.

Ответ:

- **В GitHub Secrets**
- В README.md
- В workflow.yaml в открытом виде
- В config.json внутри репозитория

🔧 Шаг 06:

Миссия 6

В Jenkins pipeline команда хочет, чтобы тесты и линтеры выполнялись параллельно. Какой тип блока нужно использовать?

Подсказка: Этот блок позволяет запускать несколько стадий одновременно.

Ответ:

- **parallel**
- sequential
- wait
- mutex

🔧 Шаг 07:

Миссия 7

Инженер Лена замечает, что Jenkins job не видит репозиторий. Что нужно проверить в первую очередь?

Подсказка: Jenkins должен иметь право доступа к исходникам.

Ответ:

- **Корректность Git credentials или SSH ключей**
- Размер репозитория
- Количество веток
- Тип лицензии

🔧 Шаг 08:

Миссия 8

🗋 Сопоставьте основные элементы Jenkins Pipeline с их назначением.

Подсказка: Подумайте, что отвечает за стадии, а что — за агента, выполняющего код.

Ответ:

- **agent** — Указывает, где будет выполняться pipeline
- **stages** — Содержит последовательность шагов сборки
- **steps** — Описывает конкретные команды внутри стадии
- **environment** — Хранит переменные окружения для job

🔧 Шаг 09:

Миссия 9

💬 Сопоставьте команды GitHub Actions с их назначением.

Подсказка: Подумайте, какая команда отвечает за настройку среды, а какая — за тестирование.

Ответ:

- **actions/checkout** — Клонировать репозиторий в рабочую среду
- **actions/setup-python** — Устанавливает нужную версию Python
- **pytest** — Запускает тесты
- **flake8** — Проверяет стиль кода

🔧 Шаг 10:

Миссия 10

⚙️ Сопоставьте типы триггеров GitHub Actions с их описанием.

Подсказка: Обратите внимание, какое событие в репозитории вызывает процесс CI/CD.

Ответ:

- **push** — Запуск при отправке изменений в репозиторий
- **pull_request** — Запуск при создании или обновлении PR
- **schedule** — Автоматический запуск по расписанию
- **workflow_dispatch** — Ручной запуск пользователем

🔮 Шаг 11:

Миссия 11

Сопоставьте стадии типичного CI/CD pipeline с их описанием.

Подсказка: Вспомните, какие шаги идут от тестирования к деплою.

Ответ:

- **Build** — Сборка артефакта или контейнера
- **Test** — Проверка качества и корректности кода
- **Deploy** — Развёртывание на сервере или в облаке
- **Notify** — Отправка уведомлений о результате

🔮 Шаг 12:

Миссия 12

📦 Сопоставьте инструменты CI/CD с их основной функцией.

Подсказка: Некоторые инструменты специализируются на тестировании, другие — на развёртывании.

Ответ:

- **GitHub Actions** — Автоматизация задач в GitHub
- **Jenkins** — Гибкий сервер CI с плагинами
- **Docker** — Изоляция окружений для деплоя
- **Terraform** — Управление инфраструктурой как кодом

🔮 Шаг 13:

Миссия 13

📋 Расположите шаги настройки GitHub Actions workflow в правильном порядке.

Подсказка: Сначала создают файл, потом добавляют событие и шаги.

Ответ:

- Создание файла `.github/workflows/main.yml`
- Определение события запуска (on:)
- Настройка job с шагами
- Коммит и пуш изменений

🔮 Шаг 14:

Миссия 14

Упорядочьте этапы развёртывания приложения с помощью Jenkins.

Подсказка: Перед деплоем обязательно тестируют сборку.

Ответ:

- Сборка проекта
- Запуск тестов
- Создание артефакта
- Деплой на сервер

 Шаг 15:

Миссия 15

 Упорядочьте действия при отладке неудачного CI build.

Подсказка: Сначала анализ логов, потом правка кода, затем новый пуш.

Ответ:

- Просмотр логов сборки
- Определение причины ошибки
- Исправление кода
- Пуш исправлений и повторный запуск

 Шаг 16:

Миссия 16

 Расположите шаги настройки уведомлений CI/CD в Slack.

Подсказка: Нужно сначала создать вебхук, потом подключить его к pipeline.

Ответ:

- Создание Slack webhook
- Добавление секрета в репозиторий
- Настройка шага уведомления в pipeline
- Тестирование отправки уведомления

 Шаг 17:

Миссия 17

 Расположите этапы перехода CI/CD-процесса от тестовой среды к продакшену.

Подсказка: Продакшен всегда последняя остановка.

Ответ:

- CI тестирование
- Сборка Docker-образа
- Деплой на staging
- Деплой на production

 Модуль 11 - Защита и оптимизация на проде

 Урок 1 - Защита от adversarial-атак

 Шаг 01:

Миссия 1

 Младший инженер Алексей услышал, что существуют быстрые методы генерации adversarial-примеров, использующие градиенты. Какой из них самый базовый?

Подсказка: Метод включает добавление знака градиента к исходным данным.

Ответ:

- **FGSM (Fast Gradient Sign Method)**
- DeepFool
- PGD (Projected Gradient Descent)
- Carlini & Wagner Attack

💡 **Шаг 02:**

Миссия 2

Исследователь Ирина проводит анализ robustness. Она хочет измерить, насколько модель устойчива к adversarial-примерам. Что обычно используется для оценки этой устойчивости? Подсказка: Чем выше значение, тем труднее обмануть модель минимальными изменениями.

Ответ:

- **Robust accuracy**
- Precision
- Recall
- ROC-AUC

💡 **Шаг 03:**

Миссия 3

🔒 Инженер Сергей добавил шум в данные и замечает, что атаки стали менее эффективны. Как называется такая стратегия защиты?

Подсказка: Этот метод основан на рандомизации входных данных для снижения точности атакующего.

Ответ:

- **Input randomization**
- Weight decay
- Gradient masking
- Pruning

💡 **Шаг 04:**

Миссия 4

Сопоставьте типы adversarial-атак с их характеристиками.

Подсказка: Некоторые атаки требуют доступа к модели, а некоторые — нет.

Ответ:

- **White-box attack** — Атакующий имеет полный доступ к архитектуре и градиентам модели.
- **Black-box attack** — Атакующий не имеет доступа к модели, использует только входы и выходы.
- **Transfer attack** — Использует примеры, сгенерированные на другой модели.
- **Physical attack** — Проявляется в реальном мире, например, изменением изображения дорожного знака.

💡 **Шаг 05:**

Миссия 5

Сопоставьте методы защиты с их основными идеями.

Подсказка: Один метод использует обучение, другой — фильтрацию входных данных.

Ответ:

- **Adversarial training** — Обучение модели на adversarial-примерах.
- **Input preprocessing** — Фильтрация или нормализация входных изображений перед подачей в модель.
- **Gradient masking** — Скрытие информации о градиентах от атакующего.
- **Defensive distillation** — Использование мягких меток для обучения второй модели.

🔗 Шаг 06:

Миссия 6

🔗 Команда ML-разработчиков сопоставляет популярные библиотеки для adversarial-исследований и их особенности.

Подсказка: Одна из них от CleverHans, другая от Facebook AI Research.

Ответ:

- **CleverHans** — Библиотека для создания и тестирования adversarial-атак.
- **Foolbox** — Инструмент с поддержкой PyTorch и TensorFlow для оценки устойчивости моделей.
- **Advertorch** — Набор adversarial-методов для PyTorch от Facebook AI Research.
- **RobustBench** — Бенчмарк для сравнения устойчивости различных моделей.

🔗 Шаг 07:

Миссия 7

🔗 Сопоставьте уровни доступа атакующего и возможные типы атак.

Подсказка: Чем больше информации у злоумышленника, тем сильнее атака.

Ответ:

- **Full access to model** — White-box attack
- **Limited API access** — Black-box attack
- **Access only to outputs** — Query-based attack
- **No access at all** — Transfer-based attack

🔗 Шаг 08:

Миссия 8

Сопоставьте типы adversarial-примеров с их описаниями.

Подсказка: Некоторые атакуют классификатор, другие — сегментацию или детекцию.

Ответ:

- **Targeted attack** — Изменяет вход так, чтобы модель предсказала конкретный класс.
- **Untargeted attack** — Любое изменение, приводящее к ошибке классификации.
- **Universal attack** — Один шум, который работает для множества изображений.
- **Patch attack** — Маленький участок изображения, вызывающий сбой модели.

🔗 Шаг 09:

Миссия 9

Разработчик Тимофей сопоставляет стратегии защиты и их типичные проблемы.

Подсказка: Некоторые методы лишь создают иллюзию защиты.

Ответ:

- **Gradient masking** — Может быть легко обойдена другими методами.
- **Adversarial training** — Требуется большого количества вычислительных ресурсов.
- **Input denoising** — Может ухудшить точность модели на обычных данных.
- **Defensive distillation** — Уменьшает эффективность атак, но не гарантирует защиту.

Шаг 10:

Миссия 10

Расположите шаги создания adversarial-примера методом FGSM в правильном порядке.
Подсказка: Начинается всё с вычисления градиента, а заканчивается добавлением шума к изображению.

Ответ:

- Вычислить градиент функции потерь по входу
- Взять знак градиента
- Умножить на малый коэффициент ϵ
- Добавить полученное возмущение к исходному изображению

Шаг 11:

Миссия 11

 Расположите этапы построения защиты с помощью adversarial training.
Подсказка: Сначала нужны adversarial-примеры, потом обучение и проверка.

Ответ:

- Генерация adversarial-примеров
- Объединение их с исходным датасетом
- Обучение модели на смешанном наборе
- Оценка устойчивости модели

Шаг 12:

Миссия 12

Расположите шаги проведения атаки типа PGD.
Подсказка: Этот метод итерационный — изменения вносятся постепенно.

Ответ:

- Инициализация случайным шумом
- Вычисление градиента потерь
- Обновление входа с учётом ограничения ϵ
- Повтор итераций до достижения результата

Шаг 13:

Миссия 13

 Расположите шаги построения системы мониторинга adversarial-атак в продакшене.
Подсказка: Наблюдение, анализ, реакция — три кита защиты.

Ответ:

- Сбор статистики о входных данных
- Анализ аномалий и подозрительных паттернов

- Оповещение и логирование событий
- Автоматическая реакция (например, блокировка запроса)

🔧 Шаг 14:

Миссия 14

🔧 Расположите шаги тестирования устойчивости модели перед релизом.

Подсказка: Прежде чем выпускать модель, нужно проверить всё — от атак до отчётов.

Ответ:

- Создание набора adversarial-примеров
- Прогон модели на тестах с атаками
- Анализ падения точности
- Документирование результатов

🔧 Шаг 15:

Миссия 15

🔧 Расположите шаги интеграции защиты от adversarial-атак в продакшен-пайплайн.

Подсказка: От разработки до CI/CD — всё должно быть автоматизировано.

Ответ:

- Разработка стратегии защиты
- Интеграция adversarial-тестов в CI/CD
- Настройка мониторинга и алертов
- Периодический аудит и обновление моделей

📖 Урок 2 - 🧮 Квантование и pruning

🔧 Шаг 01:

Миссия 1

Тимлид Саша заметил, что модель для детекции дефектов на конвейере грузится слишком долго. Он предлагает применить квантование. Что произойдет после квантования модели?

Подсказка: Подумайте о том, что квантование уменьшает размер весов и ускоряет вычисления.

Ответ:

- **Модель станет меньше и будет работать быстрее, с незначительной потерей точности**
- Модель станет больше, но точнее
- Модель потеряет способность к обучению
- Модель будет требовать больше оперативной памяти

🔧 Шаг 02:

Миссия 2

🔧 Разработчик Илья пытается ускорить инференс модели ResNet. Он применяет динамическое квантование. В чем отличие этого метода от статического?

Подсказка: Вспомните, когда именно происходит калибровка весов и активаций.

Ответ:

- **При динамическом квантовании веса квантованы заранее, а активации — во время инференса**

- Динамическое квантование требует калибровочного набора
- Статическое квантование выполняется только на GPU
- Разница только в размере модели

🔮 Шаг 03:

Миссия 3

🛠 Инженер Катя внедряет int8-квантование для модели NLP. Она замечает снижение точности. Что нужно сделать, чтобы минимизировать потери?

Подсказка: Можно использовать специальный процесс обучения, чтобы адаптировать модель к новому формату весов.

Ответ:

- **Использовать квантованное дообучение (QAT)**
- Снизить размер батча
- Применить dropout
- Увеличить количество эпох обучения

🔮 Шаг 04:

Миссия 4

🏢 Команда оптимизирует модель для встраивания в микроконтроллер. Какой формат данных чаще всего выбирают при квантовании под такие устройства?

Подсказка: Ограниченные ресурсы → минимальные биты на значение.

Ответ:

- **int8**
- float32
- float16
- bfloat16

🔮 Шаг 05:

Миссия 5

🔍 Инженер по MLOps Артём анализирует лог после pruning модели. Что означает параметр “sparsity=0.8”?

Подсказка: Вспомните, что pruning удаляет часть весов, чтобы сделать сеть разреженной.

Ответ:

- **Удалено 80% весов, оставлено 20% активных параметров**
- 80% весов были увеличены
- Сеть уменьшена на 80% по слоям
- Сеть стала на 80% быстрее

🔮 Шаг 06:

Миссия 6

🧑 Дата-сайентист Оля применяет структурное pruning. Чем оно отличается от неструктурного?

Подсказка: Иногда удаляют отдельные веса, а иногда — целые блоки или фильтры.

Ответ:

- **Структурное удаляет целые фильтры или нейроны, а не отдельные веса**
- Структурное применяется только к рекуррентным сетям

- Неструктурное быстрее, чем структурное
- Структурное нельзя применять к CNN

🔮 Шаг 07:

Миссия 7

⚡ Разработчик Денис внедрил pruning, но ускорения не увидел. Почему?

Подсказка: Не все фреймворки умеют эффективно использовать разреженные матрицы.

Ответ:

- **Потому что удалённые веса просто заменены нулями, но вычисления выполняются как раньше**
- Потому что pruning увеличил количество слоёв
- Потому что модель была слишком маленькая
- Потому что не применён dropout

🔮 Шаг 08:

Миссия 8

Сопоставьте типы квантования с их особенностями:

Подсказка: Вспомните, какие методы требуют калибровки, а какие — нет.

Ответ:

- **Динамическое квантование** — Квантование весов заранее, активаций — во время инференса
- **Статическое квантование** — Квантование весов и активаций после калибровки на данных
- **Квантованное дообучение (QAT)** — Имитация квантования в процессе обучения для минимизации потерь точности
- **Пост-тренировочное квантование** — Квантование уже обученной модели без повторного обучения

🔮 Шаг 09:

Миссия 9

🔗 Соотнесите инструменты с их поддержкой квантования:

Подсказка: Один из инструментов принадлежит TensorFlow, другой — PyTorch.

Ответ:

- **TensorFlow Lite** — Поддерживает int8 и float16 квантование для мобильных устройств
- **PyTorch FX Graph Mode** — Используется для статического квантования моделей PyTorch
- **ONNX Runtime** — Умеет выполнять инференс квантованных моделей на разных бэкендах
- **OpenVINO** — Оптимизирует модели для Intel CPU и VPU с поддержкой int8

🔮 Шаг 10:

Миссия 10

Соотнесите тип pruning с его кратким описанием:

Подсказка: Разные методы удаляют разные элементы сети — от отдельных весов до целых фильтров.

Ответ:

- **Unstructured pruning** — Удаляются отдельные веса независимо от структуры слоя

- **Structured pruning** — Удаляются целые фильтры или нейроны
- **Global pruning** — Удаляются веса по всей сети согласно глобальному порогу
- **Layer-wise pruning** — Удаляются веса в каждом слое по индивидуальному порогу

🔗 Шаг 11:

Миссия 11

🎧 Сопоставьте проблему с возможным решением при оптимизации:

Подсказка: Иногда помогает комбинировать методы — например, pruning и квантование.

Ответ:

- **Медленный инференс** — Применить квантование модели
- **Большой размер модели** — Применить pruning
- **Потеря точности после квантования** — Использовать квантованное дообучение (QAT)
- **Медленная загрузка модели** — Конвертировать в оптимизированный формат (например, TorchScript)

🔗 Шаг 12:

Миссия 12

📊 Сопоставьте метрику с её применением при анализе после pruning:

Подсказка: Одна метрика оценивает структуру, другая — точность.

Ответ:

- **Accuracy** — Проверка, насколько хорошо модель классифицирует после оптимизации
- **Sparsity** — Измеряет долю нулевых весов после pruning
- **Latency** — Время отклика при инференсе
- **Model size** — Размер модели на диске

🔗 Шаг 13:

Миссия 13

Соотнесите библиотеку с её ключевой функцией оптимизации:

Подсказка: Некоторые библиотеки специализируются на inference optimization, другие — на edge deployment.

Ответ:

- **TensorRT** — Оптимизация и ускорение инференса на GPU
- **Core ML** — Оптимизация моделей для iOS
- **TVM** — Компиляция моделей для разных аппаратных платформ
- **TFLite Micro** — Запуск квантованных моделей на микроконтроллерах

🔗 Шаг 14:

Миссия 14

📋 Расположите шаги квантования модели в правильном порядке:

Подсказка: Вспомните: сначала обучение, потом калибровка, потом экспорт.

Ответ:

- Обучить модель в float32
- Собрать калибровочные данные
- Выполнить квантование весов и активаций
- Проверить точность и экспортировать квантованную модель

Шаг 15:

Миссия 15

Расположите этапы pruning по логическому порядку:

Подсказка: Обычно сначала идёт анализ важности весов, затем обнуление, а потом дообучение.

Ответ:

- Вычислить важность весов
- Удалить наименее важные веса
- Дообучить модель для восстановления точности
- Оценить метрики sparsity и accuracy

Шаг 16:

Миссия 16

 Упорядочьте действия инженера при комбинировании pruning и квантования:

Подсказка: Не стоит квантовать до того, как модель станет разреженной.

Ответ:

- Обучить исходную модель
- Применить pruning и удалить неважные веса
- Дообучить модель после pruning
- Применить квантование
- Оценить итоговую точность и скорость

Шаг 17:

Миссия 17

 Расположите шаги вывода квантованной модели в продакшен:

Подсказка: Сначала подготовка модели, потом конвертация, затем тестирование и деплой.

Ответ:

- Подготовить обученную модель
- Выполнить квантование
- Проверить точность и производительность
- Экспортировать модель в формат для inference
- Развернуть на целевом устройстве

Урок 3 - Квест сожми модель на 70% без потерь

Шаг 01:

Миссия 1

Разработчик Миша обнаружил, что его модель BERT слишком тяжелая для продакшена. Какой первый шаг он должен сделать, чтобы оценить возможности сжатия без потери точности?

Подсказка: Прежде чем оптимизировать, нужно измерить.

Ответ:

- **Оценить метрики и размер модели до оптимизации**
- Сразу применить квантование весов
- Переписать архитектуру модели
- Удалить слои наугад для ускорения

💡 Шаг 02:

Миссия 2

🛠 Инженер Анна хочет уменьшить размер модели, не изменяя точность. Какой из способов подходит лучше всего?

Подсказка: Подумай о методе, который сохраняет весовую информацию, но хранит её компактнее.

Ответ:

- Удалить половину слоёв
- **Использовать квантование весов (int8 вместо float32)**
- Обучить модель заново
- Сжать данные входного датасета

💡 Шаг 03:

Миссия 3

🧠 Тимур экспериментирует с Pruning. Какой вариант pruning уменьшит модель без потери точности при достаточной избыточности слоёв?

Подсказка: Представь, что некоторые нейроны почти не активируются — с ними можно что-то сделать.

Ответ:

- **Удаление весов с малыми значениями (magnitude pruning)**
- Удаление случайных весов
- Добавление новых нейронов
- Повторное обучение без слоёв BatchNorm

💡 Шаг 04:

Миссия 4

🔧 Разработчик Паша использует distillation. Какой компонент обязателен в процессе knowledge distillation?

Подсказка: Один обучает другого.

Ответ:

- **Teacher-модель, генерирующая soft-метки**
- Случайная инициализация всех весов
- Удаление слоя внимания
- Применение L2-регуляризации

💡 Шаг 05:

Миссия 5

Команда внедряет TinyTransformer в мобильное приложение. Какой критерий укажет, что модель успешно оптимизирована без потери качества?

Подсказка: Подумай о сочетании размера и метрик точности.

Ответ:

- **Размер модели уменьшился, а точность не снизилась более чем на 0.1%**
- Время обучения сократилось
- Код стал компактнее
- Выросла скорость предсказания без оценки метрик

🔮 Шаг 06:

Миссия 6

📦 Дима использует ONNX Runtime. Какую оптимизацию можно включить для ускорения inference без потери точности?

Подсказка: Она связана с объединением вычислительных узлов.

Ответ:

- **Graph optimization (fusion, constant folding)**
- Удаление BatchNorm
- Рандомизация весов
- Увеличение dropout

🔮 Шаг 07:

Миссия 7

✂ Лена хочет проверить, изменилась ли точность модели после квантования. Что она должна сделать?

Подсказка: Только метрики скажут правду.

Ответ:

- **Прогнать валидационный набор до и после оптимизации и сравнить метрики**
- Посмотреть размер файла модели
- Посчитать количество параметров
- Оценить визуально вывод модели

🔮 Шаг 08:

Миссия 8

Сопоставь методы оптимизации с их основным эффектом:

Подсказка: Один уменьшает размер, другой ускоряет работу, а третий делает обучение быстрее.

Ответ:

- **Pruning** — Удаляет незначимые веса, уменьшая количество параметров.
- **Quantization** — Понижает точность представления весов для ускорения.
- **Knowledge Distillation** — Передаёт знания от большой модели к меньшей.
- **Graph Optimization** — Объединяет операции для ускорения вычислений.

🔮 Шаг 09:

Миссия 9

📦 Сопоставь формат модели с его типичным применением:

Подсказка: Мобильные устройства любят компактные форматы.

Ответ:

- **TensorFlow SavedModel** — Разработка и обучение в TensorFlow.
- **TorchScript** — Оптимизация PyTorch-моделей для продакшена.
- **ONNX** — Кроссплатформенный экспорт и совместимость фреймворков.
- **TFLite** — Деплой на мобильных устройствах.

🔗 Шаг 10:

Миссия 10

🔍 Сопоставь инструмент и его назначение:

Подсказка: Одни анализируют граф, другие измеряют производительность.

Ответ:

- **TensorBoard** — Визуализация метрик и структуры модели.
- **Netron** — Просмотр архитектуры нейросетей в разных форматах.
- **TorchProfiler** — Измерение времени выполнения операций.
- **NNCF** — Автоматизация сжатия и оптимизации моделей.

🔗 Шаг 11:

Миссия 11

Сопоставь задачу и подход к оптимизации:

Подсказка: Не всё требует сложных методов — иногда хватает pruning.

Ответ:

- **Большая CNN для изображений** — Использовать pruning по фильтрам.
- **Модель для микроконтроллера** — Применить post-training quantization.
- **NLP-трансформер** — Использовать distillation с teacher-моделью.
- **Модель для GPU inference** — Применить graph fusion оптимизации.

🔗 Шаг 12:

Миссия 12

⚙️ Сопоставь тип потери качества и возможную причину:

Подсказка: Часто виноват перекося между точностью весов и стабильностью вычислений.

Ответ:

- **Падение точности после квантования** — Недостаточная калибровка данных при int8-конверсии.
- **Медленный inference после pruning** — Неподдерживаемые sparse-операции.
- **Рост потерь при distillation** — Слишком высокая температура softmax при обучении.
- **Ошибки при экспорте ONNX** — Несовместимость операторов между фреймворками.

🔗 Шаг 13:

Миссия 13

📋 Расположи шаги post-training quantization в правильном порядке:

Подсказка: Сначала анализ, потом преобразование, потом проверка.

Ответ:

- Собрать статистику активаций на валидационном наборе
- Выполнить квантование весов и активаций
- Экспортировать квантованную модель
- Проверить точность модели после преобразования

🔗 Шаг 14:

Миссия 14

Расположи шаги distillation процесса:

Подсказка: Сначала обучается teacher, потом student перенимает знания.

Ответ:

- Обучить teacher-модель
- Сгенерировать soft-метки на данных
- Обучить student-модель с использованием soft-меток
- Оценить точность student-модели

🔮 Шаг 15:

Миссия 15

⚡ Расположи этапы внедрения оптимизированной модели в продакшен:

Подсказка: Сначала готовим модель, потом проверяем, потом деплоим.

Ответ:

- Оптимизация модели
- Тестирование точности
- Создание inference-пайплайна
- Развёртывание на сервере

🔮 Шаг 16:

Миссия 16

🌀 Расположи шаги при применении pruning в правильном порядке:

Подсказка: Убираем лишнее постепенно, а не сразу.

Ответ:

- Определить порог обнуления весов
- Применить pruning
- Дообучить модель для восстановления точности
- Проверить результат

🔮 Шаг 17:

Миссия 17

🕒 Расположи шаги комплексного сжатия модели на 70% без потерь:

Подсказка: Сначала выбираем подход, потом применяем, затем проверяем и деплоим.

Ответ:

- Выбрать подходящую стратегию сжатия (pruning + quantization)
- Применить оптимизацию к модели
- Оценить точность и размер
- Развернуть обновлённую модель в продакшене

📖 Урок 4 - 🗡️ Оптимизация latency и throughput

🔮 Шаг 01:

Миссия 1

Тимлид Саша заметил, что модель для детекции дефектов на конвейере грузится слишком долго. Он предлагает применить квантование. Что произойдет после квантования модели?

Подсказка: Подумайте о том, что квантование уменьшает размер весов и ускоряет вычисления.

Ответ:

- **Модель станет меньше и будет работать быстрее, с незначительной потерей точности**
- Модель станет больше, но точнее
- Модель потеряет способность к обучению
- Модель будет требовать больше оперативной памяти

🔮 Шаг 02:

Миссия 2

👤 Разработчик Илья пытается ускорить инференс модели ResNet. Он применяет динамическое квантование. В чем отличие этого метода от статического?

Подсказка: Вспомните, когда именно происходит калибровка весов и активаций.

Ответ:

- **При динамическом квантовании веса квантованы заранее, а активации — во время инференса**
- Динамическое квантование требует калибровочного набора
- Статическое квантование выполняется только на GPU
- Разница только в размере модели

🔮 Шаг 03:

Миссия 3

🔧 Инженер Катя внедряет int8-квантование для модели NLP. Она замечает снижение точности. Что нужно сделать, чтобы минимизировать потери?

Подсказка: Можно использовать специальный процесс обучения, чтобы адаптировать модель к новому формату весов.

Ответ:

- **Использовать квантованное дообучение (QAT)**
- Снизить размер батча
- Применить dropout
- Увеличить количество эпох обучения

🔮 Шаг 04:

Миссия 4

📋 Команда оптимизирует модель для встраивания в микроконтроллер. Какой формат данных чаще всего выбирают при квантовании под такие устройства?

Подсказка: Ограниченные ресурсы → минимальные биты на значение.

Ответ:

- **int8**
- float32
- float16
- bfloat16

🔮 Шаг 05:

Миссия 5

🔍 Инженер по MLOps Артём анализирует лог после pruning модели. Что означает параметр "sparsity=0.8"?

Подсказка: Вспомните, что pruning удаляет часть весов, чтобы сделать сеть разреженной.

Ответ:

- **Удалено 80% весов, оставлено 20% активных параметров**
- 80% весов были увеличены
- Сеть уменьшена на 80% по слоям
- Сеть стала на 80% быстрее

🔮 Шаг 06:

Миссия 6

🧐 Дата-сайентист Оля применяет структурное pruning. Чем оно отличается от неструктурного?

Подсказка: Иногда удаляют отдельные веса, а иногда — целые блоки или фильтры.

Ответ:

- **Структурное удаляет целые фильтры или нейроны, а не отдельные веса**
- Структурное применяется только к рекуррентным сетям
- Неструктурное быстрее, чем структурное
- Структурное нельзя применять к CNN

🔮 Шаг 07:

Миссия 7

🔗 Разработчик Денис внедрил pruning, но ускорения не увидел. Почему?

Подсказка: Не все фреймворки умеют эффективно использовать разреженные матрицы.

Ответ:

- **Потому что удалённые веса просто заменены нулями, но вычисления выполняются как раньше**
- Потому что pruning увеличил количество слоёв
- Потому что модель была слишком маленькая
- Потому что не применён dropout

🔮 Шаг 08:

Миссия 8

Сопоставьте типы квантования с их особенностями:

Подсказка: Вспомните, какие методы требуют калибровки, а какие — нет.

Ответ:

- **Динамическое квантование** — Квантование весов заранее, активаций — во время инференса
- **Статическое квантование** — Квантование весов и активаций после калибровки на данных
- **Квантованное дообучение (QAT)** — Имитация квантования в процессе обучения для минимизации потерь точности
- **Пост-тренировочное квантование** — Квантование уже обученной модели без повторного обучения

🔮 Шаг 09:

Миссия 9

🔗 Соотнесите инструменты с их поддержкой квантования:

Подсказка: Один из инструментов принадлежит TensorFlow, другой — PyTorch.

Ответ:

- **TensorFlow Lite** — Поддерживает int8 и float16 квантование для мобильных устройств
- **PyTorch FX Graph Mode** — Используется для статического квантования моделей PyTorch
- **ONNX Runtime** — Умеет выполнять инференс квантованных моделей на разных бэкендах
- **OpenVINO** — Оптимизирует модели для Intel CPU и VPU с поддержкой int8

Шаг 10:

Миссия 10

Соотнесите тип pruning с его кратким описанием:

Подсказка: Разные методы удаляют разные элементы сети — от отдельных весов до целых фильтров.

Ответ:

- **Unstructured pruning** — Удаляются отдельные веса независимо от структуры слоя
- **Structured pruning** — Удаляются целые фильтры или нейроны
- **Global pruning** — Удаляются веса по всей сети согласно глобальному порогу
- **Layer-wise pruning** — Удаляются веса в каждом слое по индивидуальному порогу

Шаг 11:

Миссия 11

 Сопоставьте проблему с возможным решением при оптимизации:

Подсказка: Иногда помогает комбинировать методы — например, pruning и квантование.

Ответ:

- **Медленный инференс** — Применить квантование модели
- **Большой размер модели** — Применить pruning
- **Потеря точности после квантования** — Использовать квантованное дообучение (QAT)
- **Медленная загрузка модели** — Конвертировать в оптимизированный формат (например, TorchScript)

Шаг 12:

Миссия 12

 Сопоставьте метрику с её применением при анализе после pruning:

Подсказка: Одна метрика оценивает структуру, другая — точность.

Ответ:

- **Accuracy** — Проверка, насколько хорошо модель классифицирует после оптимизации
- **Sparsity** — Измеряет долю нулевых весов после pruning
- **Latency** — Время отклика при инференсе
- **Model size** — Размер модели на диске

Шаг 13:

Миссия 13

Соотнесите библиотеку с её ключевой функцией оптимизации:

Подсказка: Некоторые библиотеки специализируются на inference optimization, другие — на edge deployment.

Ответ:

- **TensorRT** — Оптимизация и ускорение инференса на GPU
- **Core ML** — Оптимизация моделей для iOS
- **TVM** — Компиляция моделей для разных аппаратных платформ
- **TFLite Micro** — Запуск квантованных моделей на микроконтроллерах

🔮 Шаг 14:

Миссия 14

📋 Расположите шаги квантования модели в правильном порядке:

Подсказка: Вспомните: сначала обучение, потом калибровка, потом экспорт.

Ответ:

- Обучить модель в float32
- Собрать калибровочные данные
- Выполнить квантование весов и активаций
- Проверить точность и экспортировать квантованную модель

🔮 Шаг 15:

Миссия 15

Расположите этапы pruning по логическому порядку:

Подсказка: Обычно сначала идёт анализ важности весов, затем обнуление, а потом дообучение.

Ответ:

- Вычислить важность весов
- Удалить наименее важные веса
- Дообучить модель для восстановления точности
- Оценить метрики sparsity и accuracy

🔮 Шаг 16:

Миссия 16

📋 Упорядочьте действия инженера при комбинировании pruning и квантования:

Подсказка: Не стоит квантовать до того, как модель станет разреженной.

Ответ:

- Обучить исходную модель
- Применить pruning и удалить неважные веса
- Дообучить модель после pruning
- Применить квантование
- Оценить итоговую точность и скорость

🔮 Шаг 17:

Миссия 17

📋 Расположите шаги вывода квантованной модели в продакшен:

Подсказка: Сначала подготовка модели, потом конвертация, затем тестирование и деплой.

Ответ:

- Подготовить обученную модель
- Выполнить квантование

- Проверить точность и производительность
- Экспортировать модель в формат для inference
- Развернуть на целевом устройстве

Модуль 12 - Интерпретируемость и объяснимость

Урок 1 - SHAP и LIME для нейросетей

Шаг 01:

Миссия 1

 Аналитик Даша впервые применяет LIME к своей модели для предсказания эмоций в тексте. Она задаётся вопросом: что делает LIME при объяснении предсказания?

Подсказка: Вспомните, что LIME пытается локально аппроксимировать сложную модель.

Ответ:

- **Создаёт локальную линейную модель вокруг точки предсказания**
- Применяет глубокое обучение для объяснения каждого слоя
- Изменяет веса нейронов для интерпретации
- Добавляет шум для улучшения устойчивости

Шаг 02:

Миссия 2

 Разработчик Тимур использует SHAP, чтобы понять вклад признаков в предсказание модели. Что лежит в основе метода SHAP?

Подсказка: Этот метод заимствует идею из теории игр.

Ответ:

- **Значения Шепли из кооперативной теории игр**
- Градиентные методы оптимизации
- Байесовские вероятности
- Параметрические распределения

Шаг 03:

Миссия 3

Инженер Алекс анализирует выводы LIME и замечает, что объяснение нестабильно при разных запусках. Что, скорее всего, является причиной?

Подсказка: Вспомните, что LIME использует случайные точки вокруг исходного примера.

Ответ:

- **Случайность в генерации выборки точек для аппроксимации**
- Ошибки в обучении нейросети
- Проблемы с нормализацией признаков
- Плохой выбор функции активации

Шаг 04:

Миссия 4

 Исследователь Анна хочет использовать SHAP для модели CNN, классифицирующей изображения кошек и собак. Что ей нужно сделать, чтобы применить SHAP корректно?

Подсказка: Некоторые методы SHAP требуют совместимости с конкретными фреймворками, такими как TensorFlow или PyTorch.

Ответ:

- **Использовать DeepExplainer или GradientExplainer для нейросетей**
- Включить dropout для интерпретируемости
- Понизить размерность данных PCA
- Применить batch normalization к входным данным

🔮 Шаг 05:

Миссия 5

🔍 Команда ML-инженеров сравнивает LIME и SHAP. Какое основное отличие между ними?

Подсказка: Один метод глобально объясняет модель, а другой — локально.

Ответ:

- **LIME** — локальный, SHAP — глобальный и консистентный
- LIME и SHAP одинаково глобальные методы
- SHAP использует случайные шумы для аппроксимации
- LIME основан на деревьях решений

🔮 Шаг 06:

Миссия 6

Сопоставьте методы с их типичными сценариями применения.

Подсказка: Один метод больше подходит для табличных данных, другой — для изображений или нейросетей.

Ответ:

- **LIME** — Хорошо подходит для табличных данных и текстов
- **SHAP** — Используется для сложных нейросетей и моделей с нелинейными зависимостями
- **DeepSHAP** — Адаптирован под глубокие модели в TensorFlow/PyTorch
- **KernelSHAP** — Универсальный метод, не требующий структуры модели

🔮 Шаг 07:

Миссия 7

Разработчик Вика разбирает компоненты SHAP. Соотнесите элементы с их описаниями.

Подсказка: Вспомните, что SHAP основывается на идее вклада каждого признака в итоговое решение.

Ответ:

- **SHAP value** — Вклад отдельного признака в предсказание
- **Baseline value** — Среднее предсказание без признаков
- **Feature importance** — Суммарная оценка значимости признаков
- **Explanation model** — Аппроксимирующая модель, описывающая влияние признаков

🔮 Шаг 08:

Миссия 8

📋 Сопоставьте типы объяснений с их характеристиками.

Подсказка: Некоторые объяснения относятся к отдельным примерам, другие — к всей модели.

Ответ:

- **Локальное объяснение** — Показывает вклад признаков для конкретного примера
- **Глобальное объяснение** — Описывает общее поведение модели
- **Post-hoc объяснение** — Создаётся после обучения модели
- **Intrinsic объяснение** — Модель изначально интерпретируема

🔮 Шаг 09:

Миссия 9

Дата-сайентист Артём оценивает различные методы интерпретации. Соотнесите метод и его возможный недостаток.

Подсказка: Подумайте о стабильности и вычислительной сложности методов.

Ответ:

- **LIME** — Может давать нестабильные результаты при разных запусках
- **SHAP** — Требуется значительных вычислительных ресурсов
- **Integrated Gradients** — Не всегда интерпретируем при шумных данных
- **Permutation Importance** — Не учитывает взаимодействие признаков

🔮 Шаг 10:

Миссия 10

🔮 Вика готовит презентацию об интерпретации моделей. Сопоставьте фразы и соответствующие визуализации.

Подсказка: Вспомните, какие графики чаще всего сопровождают SHAP и LIME.

Ответ:

- **SHAP summary plot** — Отображает распределение вкладов признаков
- **LIME bar chart** — Показывает важность признаков для одного предсказания
- **Dependence plot** — Связывает значение признака с его вкладом
- **Force plot** — Иллюстрирует смещение предсказания от базовой линии

🔮 Шаг 11:

Миссия 11

Расположите шаги применения LIME к модели классификации текста в правильном порядке.

Подсказка: Сначала нужно обучить модель, затем выбрать пример, потом провести аппроксимацию.

Ответ:

- Обучить модель на данных
- Выбрать конкретный пример для объяснения
- Сгенерировать случайные вариации этого примера
- Построить локальную линейную модель и визуализировать веса

🔮 Шаг 12:

Миссия 12

🔮 Расположите шаги вычисления SHAP значений в верном порядке.

Подсказка: Сначала определяют базовую линию, а затем оценивают вклад признаков.

Ответ:

- Определить базовое предсказание без признаков
- Вычислить предсказания с разными комбинациями признаков
- Оценить вклад каждого признака (значение Шепли)
- Агрегировать результаты для всех признаков

 Шаг 13:

Миссия 13

 Расположите шаги анализа SHAP summary plot.

Подсказка: Сначала смотрим важность, затем направление влияния.

Ответ:

- Определить признаки с наибольшими абсолютными значениями SHAP
- Посмотреть, как значения признаков влияют на предсказание
- Понять общую тенденцию (положительное или отрицательное влияние)
- Сделать выводы о важнейших признаках

 Шаг 14:

Миссия 14

Расположите шаги в процессе интерпретации модели с помощью LIME для изображений.

Подсказка: LIME для изображений выделяет суперпиксели и оценивает их влияние.

Ответ:

- Разделить изображение на суперпиксели
- Создать вариации с маскированием суперпикселей
- Получить предсказания модели для каждой вариации
- Построить линейную модель и визуализировать важные области

 Шаг 15:

Миссия 15

 Расположите шаги внедрения SHAP в продакшн-пайплайн.

Подсказка: Сначала собираем данные, потом обучаем, объясняем и визуализируем.

Ответ:

- Собрать входные данные и обучить модель
- Интегрировать SHAP для расчёта вкладов признаков
- Сохранить объяснения в базе данных
- Создать визуализацию для пользователей

 Шаг 16:

Миссия 16

 Расположите шаги, которые поможет выполнить ML-инженеру Наде для автоматизации отчётов по интерпретации.

Подсказка: Последовательность — от расчёта значений до генерации отчёта.

Ответ:

- Вычислить SHAP значения для новых данных
- Агрегировать результаты по сегментам

- Сформировать визуализации и таблицы
- Сгенерировать PDF-отчёт

🔮 Шаг 17:

Миссия 17

🔮 Финальная миссия! Упорядочите шаги полного цикла: от интерпретации до принятия решений в бизнесе.

Подсказка: Последний шаг всегда связан с действием на основе интерпретации.

Ответ:

- Провести интерпретацию модели с помощью SHAP/LIME
- Обсудить результаты с бизнес-аналитиками
- Выделить признаки, влияющие на ключевые метрики
- Принять решение по улучшению модели или продукта

📖 Урок 2 - 📊 Feature importance и визуализация

🔮 Шаг 01:

Миссия 1

🔮 Вы работаете в команде стартапа, которая разрабатывает модель для прогнозирования оттока клиентов. Ваш коллега говорит: «Давайте посмотрим feature importance, чтобы понять, какие признаки влияют сильнее всего». Что он имеет в виду?

Подсказка: Это связано с тем, как модель оценивает вклад каждого признака в итоговое предсказание.

Ответ:

- **Измерение вклада каждого признака в итоговое предсказание модели**
- Изменение гиперпараметров для увеличения точности модели
- Удаление неиспользуемых признаков из датасета
- Анализ корреляции признаков с целевой переменной

🔮 Шаг 02:

Миссия 2

📊 Вы обучили модель Random Forest и вызвали метод `feature_importances_`. Что вернёт этот метод?

Подсказка: Этот атрибут помогает понять, какие признаки оказались наиболее значимыми при принятии решений деревьями.

Ответ:

- **Массив значений важности признаков, сумма которых равна 1**
- Список имён признаков без их весов
- Массив вероятностей классов для каждого признака
- Оценку важности только для категориальных признаков

🔮 Шаг 03:

Миссия 3

🔮 Вы заметили, что модель даёт странные результаты. Ваш тимлид советует: «Попробуй permutation importance». Что делает этот метод?

Подсказка: Он связан с перестановкой значений признаков и измерением изменения метрики.

Ответ:

- **Измеряет изменение качества модели при случайной перестановке значений признака**
- Удаляет признак из обучающего набора
- Добавляет шум к признакам для борьбы с переобучением
- Переставляет слои нейросети для оптимизации

🔮 Шаг 04:

Миссия 4

В ходе хакатона вы визуализируете feature importance для модели XGBoost. Какой график подойдёт лучше всего для отображения этих данных?

Подсказка: Попробуйте вспомнить графики, где можно легко сравнить величины между признаками.

Ответ:

- **Горизонтальная столбчатая диаграмма**
- Круговая диаграмма
- Диаграмма рассеяния
- Тепловая карта

🔮 Шаг 05:

Миссия 5

⚙ В команде идёт спор: один разработчик говорит, что feature importance из дерева и из permutation importance совпадают. Другой — что нет. Кто прав?

Подсказка: Один метод основан на структуре модели, другой — на экспериментах с метрикой.

Ответ:

- **Они могут различаться, так как используют разные принципы оценки важности**
- Они всегда совпадают, если модель обучена правильно
- Permutation importance используется только для категориальных признаков
- Feature importance не может быть рассчитан в деревьях решений

🔮 Шаг 06:

Миссия 6

🔍 Сопоставьте тип визуализации с её описанием в контексте объяснимости моделей.

Подсказка: Каждая визуализация помогает понять разные аспекты модели — от важности признаков до влияния отдельных наблюдений.

Ответ:

- **Bar chart** — Показывает важность признаков с помощью длин горизонтальных полос.
- **Partial Dependence Plot** — Показывает, как изменение одного признака влияет на предсказание.
- **SHAP summary plot** — Отображает распределение влияния признаков для всех наблюдений.
- **Feature correlation heatmap** — Позволяет увидеть взаимосвязь признаков между собой.

🔮 Шаг 07:

Миссия 7

🕒 Вы внедряете explainability в продакшен. Сопоставьте инструмент с его назначением.

Подсказка: Некоторые инструменты работают с конкретными моделями, а другие — универсальны.

Ответ:

- **LIME** — Локальное объяснение предсказаний путём аппроксимации модели.
- **SHAP** — Интерпретация модели на основе теории Шепли из игровой теории.
- **Eli5** — Библиотека для вывода важности признаков и весов моделей sklearn.
- **Skater** — Фреймворк для визуализации и интерпретации моделей в Python.

Шаг 08:

Миссия 8

 Сопоставьте тип модели с корректным методом оценки важности признаков.

Подсказка: Не все методы подходят для каждой модели — у некоторых встроенные средства, у других — внешние.

Ответ:

- **Decision Tree** — Использует снижение неопределённости (impurity decrease).
- **Random Forest** — Агрегирует важности признаков по всем деревьям ансамбля.
- **Linear Regression** — Использует абсолютные значения коэффициентов признаков.
- **Neural Network** — Часто требует внешние методы вроде SHAP или LIME.

Шаг 09:

Миссия 9

Вы работаете с SHAP. Сопоставьте элемент SHAP-графика с его значением.

Подсказка: Цвета, направления и длины несут смысл: внимательно интерпретируйте оси.

Ответ:

- **Ось X** — Показывает влияние признака на итоговое предсказание.
- **Цвет точки** — Показывает значение признака (высокое или низкое).
- **Длина точки по оси** — Отражает силу влияния признака.
- **Сумма SHAP значений** — Равна разнице между базовым и фактическим предсказанием модели.

Шаг 10:

Миссия 10

Вы анализируете модель кредитного скоринга. Сопоставьте объяснение признака с типом зависимости.

Подсказка: Подумайте, какие признаки увеличивают или уменьшают вероятность отказа в кредите.

Ответ:

- **Возраст клиента** — Обратная зависимость: чем старше, тем меньше риск.
- **Доход** — Прямая зависимость: чем выше доход, тем выше вероятность одобрения.
- **Количество просрочек** — Обратная зависимость: больше просрочек — ниже балл.
- **Наличие недвижимости** — Прямая зависимость: наличие залога повышает надёжность.

Шаг 11:

Миссия 11

 Ваш коллега внедряет LIME в пайплайн. Сопоставьте шаг процесса LIME с его описанием.

Подсказка: LIME упрощает модель локально — вспомните, какие шаги для этого нужны.

Ответ:

- **Выбор конкретного наблюдения** — Точка, вокруг которой строится локальное объяснение.
- **Создание синтетических данных** — Моделирование похожих примеров с изменёнными признаками.
- **Обучение локальной модели** — Аппроксимация поведения сложной модели вокруг точки.
- **Построение визуализации** — Отображение вклада признаков в локальное предсказание.

Шаг 12:

Миссия 12

 Расположите шаги анализа feature importance в правильном порядке.

Подсказка: Сначала модель, потом важность, потом визуализация.

Ответ:

- Обучить модель на данных
- Рассчитать важность признаков
- Отсортировать признаки по убыванию важности
- Построить график важности

Шаг 13:

Миссия 13

 Расположите шаги работы permutation importance в правильной последовательности.

Подсказка: Метод основан на перестановке и измерении метрики.

Ответ:

- Измерить базовую метрику модели
- Переставить значения одного признака
- Измерить новую метрику
- Оценить разницу в качестве как важность признака

Шаг 14:

Миссия 14

Расположите шаги построения SHAP summary plot.

Подсказка: Этот график требует предварительного вычисления SHAP-значений.

Ответ:

- Обучить модель
- Рассчитать SHAP значения
- Отсортировать признаки по сумме абсолютных SHAP значений
- Построить summary plot

Шаг 15:

Миссия 15

 Расположите шаги интеграции explainability в продакшен.

Подсказка: Сначала эксперимент, потом API, потом визуализация.

Ответ:

- Выбрать метод объяснимости (SHAP, LIME и др.)
- Интегрировать его в API модели
- Проверить корректность объяснений
- Добавить визуализацию в интерфейс пользователя

🔮 Шаг 16:

Миссия 16

⚡ Расположите шаги построения Partial Dependence Plot.

Подсказка: PDP показывает влияние изменения признака на прогноз модели.

Ответ:

- Выбрать интересующий признак
- Задать диапазон его значений
- Вычислить средние предсказания модели
- Построить зависимость на графике

🔮 Шаг 17:

Миссия 17

🕒 Расположите шаги при сравнении важности признаков из разных моделей.

Подсказка: Важно обеспечить сопоставимость шкал и моделей.

Ответ:

- Обучить несколько моделей
- Нормализовать значения важности
- Объединить данные в общую таблицу
- Построить сравнительный график

📖 Урок 3 - 🎯 Квест реализуй interpretability pipeline

🔮 Шаг 01:

Миссия 1

Разработчик Антон только начал изучать интерпретируемость. Он слышал, что SHAP объясняет решения моделей через теорию игр. Что именно оценивает SHAP?

Подсказка: Вспомни, что каждый признак вносит свой вклад, как игрок в кооперативной игре.

Ответ:

- **Вклад каждого признака в итоговое предсказание**
- Только влияние ошибки обучения
- Скорость сходимости модели
- Размерность данных

🔮 Шаг 02:

Миссия 2

👩 Инженер Юлия использует LIME для объяснения предсказаний модели. Она замечает, что объяснение зависит от конкретного объекта. Почему?

Подсказка: LIME не анализирует всю модель, а строит локальную аппроксимацию вокруг точки.

Ответ:

- Потому что LIME анализирует модель локально вокруг одного примера
- Потому что LIME использует случайные веса
- Потому что LIME обучает новую сеть
- Потому что LIME зависит от числа эпох

🔮 **Шаг 03:**

Миссия 3

🕒 Тимур применяет Grad-CAM к своей CNN, чтобы понять, что видит модель. На какой слой он должен смотреть?

Подсказка: Grad-CAM анализирует активации последнего сверточного слоя перед классификацией.

Ответ:

- На последний сверточный слой
- На слой softmax
- На слой ввода
- На слой нормализации

🔮 **Шаг 04:**

Миссия 4

⚙️ Команда анализирует важность признаков в модели RandomForest. Почему простой показатель feature_importances_ может вводить в заблуждение?

Подсказка: Если признаки сильно коррелируют, то распределение важности будет искажено.

Ответ:

- Потому что метод чувствителен к коррелирующим признакам
- Потому что он основан на градиентах
- Потому что он использует невалидные данные
- Потому что он вычисляется только для линейных моделей

🔮 **Шаг 05:**

Миссия 5

👤 Дата-сайентист Илья создаёт отчёт по интерпретации модели. Он хочет показать, какие признаки влияли на решение. Какой подход ему подойдёт?

Подсказка: Подумай о методах, которые дают численную оценку вклада признаков в конкретное предсказание.

Ответ:

- SHAP
- PCA
- Dropout
- BatchNorm

🔮 **Шаг 06:**

Миссия 6

Этичный инженер Миша проверяет модель на предмет дискриминации. Почему интерпретируемость здесь важна?

Подсказка: Если мы не можем объяснить решение модели, мы не можем проверить его справедливость.

Ответ:

- **Потому что прозрачность помогает обнаружить предвзятые решения**
- Потому что это ускоряет обучение
- Потому что это улучшает дизайн интерфейса
- Потому что это уменьшает размер модели

🔮 Шаг 07:

Миссия 7

Сопоставьте методы интерпретации с их ключевыми идеями.

Подсказка: LIME — локальные объяснения, SHAP — теория игр.

Ответ:

- **LIME** — Локальная линейная аппроксимация модели
- **SHAP** — Распределение вклада признаков по теории Шепли
- **Grad-CAM** — Визуализация активаций сверточных слоёв
- **Permutation Importance** — Перемешивание признаков и оценка влияния

🔮 Шаг 08:

Миссия 8

📋 Сопоставьте модели с наиболее подходящими методами интерпретации.

Подсказка: Для CNN — Grad-CAM, для табличных данных — LIME или SHAP.

Ответ:

- **Сверточная нейросеть** — Grad-CAM
- **Табличная модель** — LIME
- **Случайный лес** — SHAP
- **Регрессия** — Коэффициенты весов

🔮 Шаг 09:

Миссия 9

Соотнесите уровни интерпретации и их описание.

Подсказка: Глобальный — для всей модели, локальный — для отдельного предсказания.

Ответ:

- **Глобальный уровень** — Объяснение поведения всей модели
- **Локальный уровень** — Интерпретация одного предсказания
- **Визуальный уровень** — Карта внимания или Grad-CAM
- **Этический уровень** — Оценка справедливости решений

🔮 Шаг 10:

Миссия 10

🔗 Сопоставьте тип данных с подходящим методом интерпретации.

Подсказка: Grad-CAM для изображений, Attention — для текста.

Ответ:

- **Изображения** — Grad-CAM

- **Текст** — Attention heatmap
- **Табличные данные** — LIME
- **Временные ряды** — SHAP

🔮 Шаг 11:

Миссия 11

Сопоставьте модель и уровень её интерпретируемости.

Подсказка: Линейные модели проще объяснять, чем глубокие сети.

Ответ:

- **Линейная регрессия** — Высокая интерпретируемость
- **Дерево решений** — Хорошая интерпретируемость
- **Нейросеть** — Низкая интерпретируемость
- **Случайный лес** — Средняя интерпретируемость

🔮 Шаг 12:

Миссия 12

Сопоставьте этап пайплайна интерпретации с его задачей.

Подсказка: Начинаем с данных, заканчиваем визуализацией и отчётом.

Ответ:

- **Предобработка данных** — Очистка и нормализация признаков
- **Выбор метода** — Определение подходящего инструмента интерпретации
- **Интерпретация** — Оценка вкладов признаков
- **Визуализация** — Отображение результатов на графиках

🔮 Шаг 13:

Миссия 13

🔗 Расставь шаги построения интерпретируемого пайплайна.

Подсказка: Сначала обучение, затем выбор метода, потом объяснение и визуализация.

Ответ:

- Обучение модели
- Выбор метода интерпретации
- Получение объяснений
- Визуализация результатов

🔮 Шаг 14:

Миссия 14

🌀 Расположи этапы работы метода LIME в правильном порядке.

Подсказка: Вспомни, что LIME создаёт локальные данные, обучает линейную модель и оценивает вклады признаков.

Ответ:

- Выбор объекта для объяснения
- Генерация соседних данных
- Обучение локальной линейной модели
- Интерпретация вкладов признаков

Шаг 15:

Миссия 15

 Упорядочи шаги типичного процесса SHAP.

Подсказка: Сначала базовое значение, потом вычисляем SHAP-значения, суммируем и визуализируем.

Ответ:

- Вычисление базового значения
- Получение SHAP значений
- Суммирование вкладов признаков
- Визуализация результатов

Шаг 16:

Миссия 16

 Расположи шаги генерации Grad-CAM карты внимания.

Подсказка: Сначала вычисляем градиенты, потом применяем ReLU и накладываем карту.

Ответ:

- Вычисление градиентов по выходу
- Взвешивание активаций градиентами
- Суммирование и ReLU
- Наложение карты на изображение

Шаг 17:

Миссия 17

Заверши пайплайн интерпретации проекта в правильном порядке.

Подсказка: После интерпретации следует анализ, визуализация и отчёт.

Ответ:

- Загрузка модели
- Применение метода интерпретации
- Анализ и визуализация
- Создание итогового отчёта

Урок 4 - Анализ ошибок и отладка сложных примеров

Шаг 01:

Миссия 1

Разработчик Артём обучил модель классификации изображений, но она путает кошек и собак. Что первым делом стоит проверить?

Подсказка: Иногда проблема не в модели, а в данных. Вспомните про баланс классов и разметку.

Ответ:

- **Качество и корректность обучающих данных**
- Добавить больше слоёв в модель
- Изменить оптимизатор на AdamW
- Снизить размер батча

🔮 Шаг 02:

Миссия 2

🗯 У Марии модель сегментации работает идеально на валидации, но плохо на новых данных. Что это значит?

Подсказка: Это типичная ситуация, когда модель выучивает конкретные детали обучающего набора.

Ответ:

- **Модель переобучилась**
- Модель недообучилась
- Модель неправильно инициализирована
- Произошёл data leakage

🔮 Шаг 03:

Миссия 3

🗯 Андрей замечает, что loss падает, но accuracy не растёт. Что это может означать?

Подсказка: Иногда функция потерь и метрика не отражают одно и то же качество модели.

Ответ:

- **Несоответствие между метрикой и функцией потерь**
- Ошибка в подсчёте accuracy
- Слишком большой learning rate
- Проблема с логированием

🔮 Шаг 04:

Миссия 4

🗯 Елена использует Grad-CAM для объяснения предсказаний CNN. Что именно показывает эта визуализация?

Подсказка: Grad-CAM помогает понять, какие участки входного изображения были важны для решения сети.

Ответ:

- **Активные области изображения, влияющие на предсказание**
- Ошибочные пиксели, которые нужно удалить
- Точки, где градиент равен нулю
- Средние значения весов свёрточных фильтров

🔮 Шаг 05:

Миссия 5

Тимур замечает, что модель даёт неверные результаты при маленьких изображениях. Какое действие стоит предпринять?

Подсказка: Иногда важно обеспечить одинаковый масштаб входных данных.

Ответ:

- **Привести изображения к единому размеру и нормализации**
- Снизить learning rate
- Добавить больше dropout
- Заменить функцию активации на ReLU6

🔮 Шаг 06:

Миссия 6

Сопоставьте методы интерпретируемости с их описаниями.

Подсказка: Вспомните, какие методы показывают важность признаков, а какие — визуализируют активации.

Ответ:

- **LIME** — Локальное объяснение предсказаний с помощью линейной модели
- **SHAP** — Определяет вклад каждого признака в предсказание
- **Grad-CAM** — Визуализирует активные области изображения
- **Feature Importance** — Показывает значимость признаков по весам модели

🔮 Шаг 07:

Миссия 7

🔮 Сопоставьте тип ошибок с возможной причиной.

Подсказка: Подумайте, что означает каждая ошибка: иногда виновата разметка, иногда данные.

Ответ:

- **False Positive** — Модель выдала положительный результат, где его не было
- **False Negative** — Модель не обнаружила положительный пример
- **Data Leakage** — Информация из теста попала в обучение
- **Overfitting** — Модель выучила детали обучающего набора

🔮 Шаг 08:

Миссия 8

🔮 Сопоставьте инструмент с его основным назначением при анализе ошибок.

Подсказка: Один инструмент визуализирует, другой показывает значения метрик, третий помогает искать зависимости.

Ответ:

- **TensorBoard** — Визуализация метрик и графиков обучения
- **Confusion Matrix** — Показывает распределение ошибок между классами
- **SHAP Summary Plot** — Анализ вклада признаков в предсказание
- **Weights & Biases** — Отслеживание экспериментов и параметров

🔮 Шаг 09:

Миссия 9

Сопоставьте описание поведения модели с вероятной причиной проблемы.

Подсказка: Подумайте, что может вызвать колебания метрик или низкую обобщающую способность.

Ответ:

- **Метрики сильно колеблются от эпохи к эпохе** — Слишком большой learning rate
- **Модель выдаёт одинаковые ответы** — Ошибка в предобработке данных
- **Высокий train accuracy, низкий test accuracy** — Переобучение
- **Loss не уменьшается** — Ошибочная нормализация входных данных

🔗 Шаг 10:

Миссия 10

🔗 Сопоставьте тип визуализации с её применением при интерпретации нейросетей.

Подсказка: Обратите внимание, какие из них применяются для CNN, а какие — для табличных моделей.

Ответ:

- **Heatmap** — Визуализация областей внимания на изображении
- **Bar Plot** — Отображение важности признаков
- **Scatter Plot** — Исследование зависимости признаков
- **Partial Dependence Plot** — Показывает влияние одного признака при фиксированных других

🔗 Шаг 11:

Миссия 11

🔗 Сопоставьте понятие с объяснением в контексте анализа ошибок.

Подсказка: Подумайте, что связано с несбалансированными классами и смещением выборки.

Ответ:

- **Class Imbalance** — Преобладание одного класса над другими
- **Data Drift** — Изменение распределения данных со временем
- **Outlier** — Аномальные значения, не характерные для набора
- **Sampling Bias** — Ошибочный выборка, не отражающая реальную ситуацию

🔗 Шаг 12:

Миссия 12

🔗 Сопоставьте метод отладки с его задачей.

Подсказка: Некоторые методы позволяют понять, что происходит в слоях модели, другие — корректируют данные.

Ответ:

- **Layer-wise Inspection** — Анализ промежуточных активаций
- **Data Sanity Check** — Проверка целостности и корректности данных
- **Gradient Clipping** — Предотвращение взрывов градиентов
- **Model Debugger** — Поэтапный запуск и отслеживание ошибок

🔗 Шаг 13:

Миссия 13

Расположите шаги анализа ошибок модели в правильном порядке.

Подсказка: Начните с визуализации, затем анализируйте закономерности и только после — корректируйте модель.

Ответ:

- Собрать примеры с ошибками
- Построить confusion matrix
- Найти закономерности ошибок
- Внести корректировки в данные или архитектуру

🔗 Шаг 14:

Миссия 14

🔍 Расположите этапы работы с SHAP в правильной последовательности.

Подсказка: Сначала нужен обученный модель, потом расчёт значений и визуализация.

Ответ:

- Обучить модель
- Рассчитать SHAP значения
- Построить summary plot
- Интерпретировать результаты

🔗 Шаг 15:

Миссия 15

🔍 Расположите шаги при обнаружении переобучения.

Подсказка: Обычно начинают с анализа метрик, затем проверяют данные и только потом корректируют модель.

Ответ:

- Сравнить метрики train и test
- Проверить баланс классов
- Добавить регуляризацию
- Переобучить модель

🔗 Шаг 16:

Миссия 16

⚡ Расположите шаги при обнаружении exploding gradients.

Подсказка: Диагностика, проверка весов, исправление — в этом порядке.

Ответ:

- Обнаружить резкое увеличение loss
- Проверить значения весов и градиентов
- Применить gradient clipping
- Перезапустить обучение

🔗 Шаг 17:

Миссия 17

🔍 Расположите шаги подготовки отчёта по интерпретируемости модели.

Подсказка: Сначала собираются данные, потом создаются визуализации и только в конце пишется отчёт.

Ответ:

- Собрать результаты анализа интерпретаций
- Построить визуализации (SHAP, Grad-CAM и др.)
- Добавить выводы и рекомендации
- Оформить итоговый отчёт

Модуль 13 - Обработка и генерация данных

Урок 1 - Аугментация текста, изображений и аудио

Шаг 01:

Миссия 1

Разработчица Анна работает над моделью распознавания рукописных цифр. Какой способ аугментации изображений поможет улучшить обобщающую способность модели?

Подсказка: Подумайте о том, как человек пишет цифры: они могут быть наклонены, повернуты или немного смещены.

Ответ:

- **Повороты и смещения изображений**
- Изменение цветовой палитры
- Добавление случайных символов
- Использование только исходных данных без изменений

Шаг 02:

Миссия 2

 Артём обучает модель классификации картин. Он замечает, что при изменении яркости результаты становятся лучше. Какой тип аугментации он использует?

Подсказка: Этот вид трансформации помогает модели быть устойчивой к разной освещённости изображений.

Ответ:

- **Фотометрическая аугментация**
- Геометрическая аугментация
- Текстовая аугментация
- Шумовая регуляризация

Шаг 03:

Миссия 3

 Звукорежиссёр Иван обучает модель распознавания речи и добавляет к записям фоновый шум кафе. Зачем он это делает?

Подсказка: Подумайте, как улучшить устойчивость модели к шумным условиям записи.

Ответ:

- **Для повышения устойчивости модели к шуму**
- Чтобы проверить скорость модели
- Для синхронизации аудио и текста
- Чтобы улучшить сегментацию речи

Шаг 04:

Миссия 4

 Ольга применяет аугментацию текста с заменой слов на синонимы. Какую цель она преследует?

Подсказка: Замена слов помогает модели понимать смысл, а не только конкретные формулировки.

Ответ:

- **Увеличение разнообразия формулировок в обучении**
- Сокращение размера словаря
- Удаление редко встречающихся слов
- Фильтрация неинформативных предложений

🔮 **Шаг 05:**

Миссия 5

👤 Сергей экспериментирует с аугментацией изображений для модели детекции объектов. Что из перечисленного не относится к корректной аугментации?

Подсказка: Аугментация должна сохранять смысл изображения, не искажая структуру объектов.

Ответ:

- **Перестановка фрагментов изображения случайным образом**
- Отражение по горизонтали
- Небольшие повороты
- Изменение контраста

🔮 **Шаг 06:**

Миссия 6

Сопоставьте тип аугментации с примером её применения.

Подсказка: Геометрические — меняют форму, фотометрические — свет, текстовые — слова.

Ответ:

- **Геометрическая аугментация** — Поворот, отражение, масштабирование
- **Фотометрическая аугментация** — Изменение яркости и контраста
- **Текстовая аугментация** — Замена слов на синонимы
- **Аудио-аугментация** — Добавление фонового шума

🔮 **Шаг 07:**

Миссия 7

👤 Сопоставьте инструмент с его использованием в аугментации данных.

Подсказка: Разные библиотеки специализируются на своих типах данных.

Ответ:

- **Albumentations** — Аугментация изображений для CV
- **TextAttack** — Аугментация текстов для NLP
- **Audiomentations** — Аугментация звука
- **AugLy** — Универсальная библиотека мультимодальной аугментации

🔮 **Шаг 08:**

Миссия 8

👤 Сопоставьте метод аудио-аугментации с его эффектом.

Подсказка: Вспомните, какие операции меняют громкость, а какие — скорость воспроизведения.

Ответ:

- **Time Stretch** — Изменение скорости аудио без изменения тона

- **Pitch Shift** — Изменение высоты тона
- **Add Noise** — Добавление фонового шума
- **Gain** — Изменение громкости сигнала

🔮 Шаг 09:

Миссия 9

Сопоставьте технику аугментации с её задачей в NLP.

Подсказка: Некоторые методы заменяют слова, другие — изменяют структуру предложений.

Ответ:

- **Synonym Replacement** — Замена слов на синонимы
- **Back Translation** — Перевод текста на другой язык и обратно
- **Random Insertion** — Вставка случайных слов
- **Random Swap** — Перестановка слов местами

🔮 Шаг 10:

Миссия 10

🖼️ Сопоставьте задачу компьютерного зрения с наиболее полезной аугментацией.

Подсказка: Для детекции — важно положение, для классификации — освещение.

Ответ:

- **Классификация изображений** — Изменение яркости и контраста
- **Детекция объектов** — Сдвиги и повороты
- **Сегментация** — Повороты и отражения
- **OCR (распознавание текста)** — Искажения и размытия

🔮 Шаг 11:

Миссия 11

💬 Сопоставьте подход к аугментации текста с его особенностью.

Подсказка: Одни методы требуют моделей, другие — простые операции со словами.

Ответ:

- **Rule-based аугментация** — Основана на простых заменах слов
- **Model-based аугментация** — Использует языковые модели для генерации
- **Back Translation** — Перевод текста через промежуточный язык
- **EDA (Easy Data Augmentation)** — Комбинация простых текстовых техник

🔮 Шаг 12:

Миссия 12

🌀 Сопоставьте тип искажения изображения с его назначением при обучении.

Подсказка: Одни искажения тренируют устойчивость к положению, другие — к освещению.

Ответ:

- **Rotation** — Изменение угла изображения
- **Brightness** — Имитирует разные условия освещения
- **Crop** — Фокусировка на разных частях изображения
- **Flip** — Создание зеркальных копий

Шаг 13:

Миссия 13

Расположите шаги создания пайплайна аугментации изображений.

Подсказка: Сначала данные загружаются, потом применяются трансформации, и только потом модель обучается.

Ответ:

- Загрузка набора изображений
- Определение набора трансформаций
- Применение аугментаций
- Передача данных в модель

Шаг 14:

Миссия 14

 Расположите шаги генерации новых аудиосемплов.

Подсказка: Обратите внимание на порядок: анализ → модификация → сохранение → проверка.

Ответ:

- Загрузка исходного аудио
- Применение трансформаций (шум, pitch shift)
- Сохранение изменённых файлов
- Прослушивание и проверка качества

Шаг 15:

Миссия 15

 Расположите этапы работы с текстовой аугментацией.

Подсказка: Нельзя аугментировать, не подготовив токены и словарь.

Ответ:

- Токенизация текстов
- Выбор метода аугментации
- Применение трансформаций
- Сохранение расширенного датасета

Шаг 16:

Миссия 16

 Расположите шаги валидации эффективности аугментации.

Подсказка: Важно измерять, насколько улучшились метрики на валидации.

Ответ:

- Разделить данные на train и validation
- Применить аугментацию к train
- Обучить модель
- Сравнить результаты на validation

Шаг 17:

Миссия 17

 Расположите шаги проектирования кастомной аугментации изображений.

Подсказка: Сначала нужно определить задачу, потом описать функцию и интегрировать в пайплайн.

Ответ:

- Определить цель аугментации
- Написать функцию трансформации
- Протестировать на примерах
- Интегрировать в обучающий конвейер

Урок 2 - ⚙ Генерация синтетических данных и GAN

Шаг 01:

Миссия 1

Разработчик Алекс столкнулся с проблемой: данных для обучения мало, и модель переобучается. Что первым делом ему стоит попробовать?

Подсказка: Иногда лучше не искать новые данные, а создать их искусственно.

Ответ:

- **Попробовать генерацию синтетических данных (например, с помощью GAN)**
- Добавить больше слоёв в модель
- Уменьшить количество эпох
- Прекратить обучение и начать заново

Шаг 02:

Миссия 2

Инженер Марина обучает GAN для генерации изображений. Но дискриминатор учится слишком быстро и подавляет генератор. Что можно предпринять?

Подсказка: Иногда баланс важнее скорости — особенно в дуэлях нейросетей.

Ответ:

- **Замедлить обучение дискриминатора (уменьшить learning rate)**
- Добавить больше данных
- Удалить часть архитектуры
- Изменить функцию потерь генератора на MSE

Шаг 03:

Миссия 3

Тимур хочет обучить GAN для создания рукописных цифр, но видит артефакты. Что, скорее всего, поможет?

Подсказка: Иногда шум в начале обучения — не баг, а фича.

Ответ:

- **Добавить нормализацию батчей (BatchNorm) в генератор**
- Уменьшить размер скрытого слоя дискриминатора
- Удалить шум на входе
- Использовать ReLU вместо LeakyReLU

Шаг 04:

Миссия 4

Команда разрабатывает GAN для синтеза голосов. На первых итерациях аудио искажается. Почему?

Подсказка: GAN не любят высокую нестабильность — особенно при сложных сигналах.

Ответ:

- **Слишком большая скорость обучения генератора**
- Недостаточное количество итераций
- Слишком маленький размер батча
- Неправильный формат данных (mp3 вместо wav)

🔮 Шаг 05:

Миссия 5

Разработчик Евгений хочет использовать GAN для синтетических данных в медицине. Какое преимущество он получает?

Подсказка: В медицине важно не раскрывать реальных пациентов.

Ответ:

- **Можно обучать модели, не раскрывая реальные данные пациентов**
- Ускоряет сбор реальных данных
- Гарантирует идеальную точность модели
- Сокращает вычислительные затраты

🔮 Шаг 06:

Миссия 6

GAN обучается, но выходы генератора стали одинаковыми. Что произошло?

Подсказка: Когда генератор находит лёгкий способ обмануть дискриминатор — это не победа, а коллапс.

Ответ:

- **Mode collapse** — генератор производит однотипные выходы
- **Overfitting** — генератор выучил все данные
- Vanishing gradients
- Проблема в активации tanh

🔮 Шаг 07:

Миссия 7

Сопоставьте тип GAN с его особенностью.

Подсказка: Каждый вариант решает конкретную проблему классического GAN.

Ответ:

- **DCGAN** — Использует сверточные слои для изображений
- **WGAN** — Устраняет нестабильность с помощью расстояния Вассерштейна
- **CycleGAN** — Преобразует изображения между доменами без пар
- **StyleGAN** — Контролирует стиль и детали с помощью отдельного пространства

🔮 Шаг 08:

Миссия 8

Помоги Диме понять, какие элементы входят в архитектуру GAN.

Подсказка: Всегда помни, что GAN — это дуэль между двумя сетями.

Ответ:

- **Генератор** — Создаёт фальшивые данные из шума
- **Дискриминатор** — Отличает реальные данные от поддельных
- **Функция потерь** — Оценивает результат противостояния
- **Шумовой вектор** — Служит источником случайности

🔮 **Шаг 09:**

Миссия 9

Сопоставь проблему с её возможным решением при обучении GAN.

Подсказка: Не все проблемы решаются добавлением данных — иногда важно сбалансировать обучение.

Ответ:

- **Mode collapse** — Добавить шум в вход дискриминатора
- **Неустойчивое обучение** — Использовать WGAN
- **Переобучение дискриминатора** — Снизить learning rate
- **Отсутствие реалистичных выходов** — Добавить BatchNorm

🔮 **Шаг 10:**

Миссия 10

Сопоставьте тип данных и форму GAN, которая лучше подходит для их генерации.

Подсказка: Тип данных определяет архитектуру — звук, изображение, текст требуют разных подходов.

Ответ:

- **Изображения** — DCGAN или StyleGAN
- **Аудио** — WaveGAN
- **Текст** — TextGAN или GPT-GAN
- **Видео** — TGAN или MoCoGAN

🔮 **Шаг 11:**

Миссия 11

Сопоставьте часть GAN с её ролью в обучении.

Подсказка: Генератор и дискриминатор учатся друг у друга, а не в одиночку.

Ответ:

- **Генератор** — Учит создавать данные, похожие на настоящие
- **Дискриминатор** — Учит отличать подделки от реальных
- **Функция потерь генератора** — Мотивирует обмануть дискриминатор
- **Функция потерь дискриминатора** — Мотивирует выявлять фальшивые данные

🔮 **Шаг 12:**

Миссия 12

Сопоставь улучшения GAN с их преимуществами.

Подсказка: Новые версии GAN решают старые проблемы — вспомни, какая из них за что отвечает.

Ответ:

- **WGAN-GP** — Добавляет штраф градиента для стабильности
- **Pix2Pix** — Реализует условное преобразование изображений
- **LSGAN** — Использует квадратичную функцию потерь
- **BigGAN** — Масштабируется на большие наборы данных

🔮 Шаг 13:

Миссия 13

Расположите шаги обучения GAN в правильном порядке.

Подсказка: Подумай о цикле дуэли: сначала дискриминатор, потом генератор, и снова — проверка.

Ответ:

- Создать шумовой вектор
- Сгенерировать данные с помощью генератора
- Обновить дискриминатор на реальных и поддельных данных
- Обновить генератор, чтобы обмануть дискриминатор

🔮 Шаг 14:

Миссия 14

Расположите шаги настройки WGAN по порядку.

Подсказка: Основное отличие — другая функция расстояния и ограничение весов.

Ответ:

- Заменить функцию потерь на расстояние Вассерштайна
- Добавить weight clipping
- Обучить дискриминатор несколько раз за итерацию
- Обучить генератор

🔮 Шаг 15:

Миссия 15

Упорядочите шаги создания синтетического набора данных с помощью GAN.

Подсказка: Сначала нужно обучить модель, а потом использовать её для генерации новых данных.

Ответ:

- Собрать обучающий набор
- Обучить GAN
- Сгенерировать синтетические данные
- Объединить с реальными данными

🔮 Шаг 16:

Миссия 16

Расположи шаги улучшения генератора при возникновении mode collapse.

Подсказка: Иногда помогает чуть больше хаоса — в хорошем смысле.

Ответ:

- Добавить шум в дискриминатор

- Изменить learning rate генератора
- Добавить dropout или batch noise
- Перезапустить обучение с новыми гиперпараметрами

🔧 Шаг 17:

Миссия 17

Расположи этапы генерации изображения с помощью обученного GAN.

Подсказка: Всё начинается с случайного вектора и заканчивается красивой картинкой.

Ответ:

- Создать случайный шумовой вектор
- Передать его генератору
- Получить тензор изображения
- Преобразовать тензор в видимое изображение

📖 Урок 3 - 🎯 Квест создай генератор данных для обучения

🔧 Шаг 01:

Миссия 1

Программистка Лена начала разрабатывать генератор данных для обучения нейросети, которая распознаёт эмоции по фото. С чего ей стоит начать?

Подсказка: Хороший генератор начинается не с кода, а с понимания источников данных и целей модели.

Ответ:

- **Определить тип данных и формат, нужный для модели**
- Сразу писать код генератора
- Скачать случайные картинки из интернета
- Добавить шум в данные без анализа

🔧 Шаг 02:

Миссия 2

👤 Алекс пишет генератор для обучения модели, предсказывающей цены квартир. Что важно учесть при генерации признаков?

Подсказка: Некоторые признаки могут иметь нелинейную зависимость — важно не потерять структуру реальных данных.

Ответ:

- **Корреляции между признаками**
- **Диапазоны реальных значений**
- Порядок следования строк в CSV
- Цвет интерфейса генератора

🔧 Шаг 03:

Миссия 3

👤 Марина тестирует генератор данных для NLP-модели. Какой шаг поможет ей избежать переобучения?

Подсказка: Важно, чтобы модель не запомнила конкретные тексты из обучающего набора.

Ответ:

- **Разделить данные на train/validation/test до генерации**
- Увеличить число эпох обучения
- Перемешать предложения вручную
- Добавить случайные опечатки

🔗 **Шаг 04:**

Миссия 4

👨‍💻 Команда разработчиков создает генератор изображений для модели, классифицирующей фрукты. Что будет ошибкой?

Подсказка: Подумай, как аугментации могут изменить суть объекта, а не только внешний вид.

Ответ:

- **Превращение яблок в абстрактные формы с фильтрами**
- Случайное вращение и отражение изображений
- Легкое изменение яркости
- Добавление случайного шума

🔗 **Шаг 05:**

Миссия 5

Никита пишет генератор числовых данных. Что означает параметр «seed»?

Подсказка: Этот параметр часто нужен для воспроизводимости.

Ответ:

- **Начальное значение для генератора случайных чисел**
- Количество примеров для обучения
- Максимальная глубина модели
- Порог точности

🔗 **Шаг 06:**

Миссия 6

Свяжите типы данных с примерами, которые может сгенерировать генератор данных.

Подсказка: Представьте, какие данные подойдут для разных задач — изображения, текст, табличные признаки.

Ответ:

- **Изображения** — Фотографии кошек и собак
- **Текст** — Отзывы пользователей о товарах
- **Табличные данные** — Столбцы с ценами и площадью квартир
- **Звук** — Аудиозаписи голоса

🔗 **Шаг 07:**

Миссия 7

🧠 Сопоставьте термин с его значением в контексте генерации данных.

Подсказка: Обратите внимание на связь между случайностью, закономерностью и балансом классов.

Ответ:

- **Аугментация** — Создание новых данных на основе существующих
- **Синтетические данные** — Искусственно сгенерированные образцы
- **Дисбаланс классов** — Преобладание примеров одного типа
- **Рандомизация** — Внесение случайных вариаций

🔗 **Шаг 08:**

Миссия 8

📖 Разработчик Сергей улучшает генератор данных. Сопоставьте инструмент с его функцией.

Подсказка: Некоторые библиотеки больше подходят для изображений, другие — для текста или чисел.

Ответ:

- **NumPy** — Генерация числовых массивов
- **Pandas** — Создание и обработка табличных данных
- **OpenCV** — Аугментация изображений
- **NLTK** — Работа с текстами

🔗 **Шаг 09:**

Миссия 9

Лаборатория AICorp анализирует поток данных. Сопоставьте этап с его описанием.

Подсказка: Вспомни стандартный pipeline обработки данных перед обучением модели.

Ответ:

- **Сбор данных** — Получение исходных файлов из разных источников
- **Очистка** — Удаление пропусков и ошибок
- **Преобразование** — Нормализация и кодирование
- **Генерация** — Создание новых данных на основе существующих

🔗 **Шаг 10:**

Миссия 10

📖 Сопоставьте тип генератора данных с примером использования.

Подсказка: Одни генераторы нужны для синтетических изображений, другие — для текстов или числовых таблиц.

Ответ:

- **ImageDataGenerator** — Создание новых изображений для CNN
- **DataLoader (PyTorch)** — Подача батчей данных в модель
- **TextVectorizer** — Преобразование слов в векторы
- **FeatureGenerator** — Синтетические признаки для обучения

🔗 **Шаг 11:**

Миссия 11

Сопоставьте параметр генерации данных с его влиянием.

Подсказка: Некоторые параметры контролируют случайность, другие — масштаб или форму данных.

Ответ:

- **Seed** — Фиксирует случайность для воспроизводимости
- **Noise** — Добавляет вариации в данные
- **Batch size** — Определяет размер пакета данных
- **Shuffle** — Перемешивает порядок данных

 Шаг 12:

Миссия 12

Расставьте этапы создания генератора данных в правильной последовательности.

Подсказка: Сначала нужно спланировать данные, затем реализовать, потом протестировать и оптимизировать.

Ответ:

- Определить цели и тип данных
- Реализовать код генератора
- Протестировать корректность и распределения
- Оптимизировать производительность

 Шаг 13:

Миссия 13

 Упорядочьте шаги подготовки данных перед подачей в генератор.

Подсказка: Сначала идут действия по сбору и чистке, а затем преобразования.

Ответ:

- Сбор данных
- Очистка и фильтрация
- Преобразование признаков
- Разделение на train/test

 Шаг 14:

Миссия 14

 Упорядочьте действия при создании аугментации изображений.

Подсказка: От анализа к реализации — важно соблюдать логику процесса.

Ответ:

- Определить виды и диапазоны аугментаций
- Настроить параметры генератора
- Применить к набору изображений
- Проверить баланс и реалистичность

 Шаг 15:

Миссия 15

Расставьте шаги в правильном порядке для тестирования генератора данных.

Подсказка: Проверь корректность, воспроизводимость и качество распределения.

Ответ:

- Создать тестовые данные
- Запустить генератор

- Проверить распределения признаков
- Оценить воспроизводимость

Шаг 16:

Миссия 16

⚙ Упорядочьте этапы интеграции генератора данных в пайплайн обучения.

Подсказка: Генератор нужно добавить после подготовки данных, но до обучения модели.

Ответ:

- Подготовить исходные данные
- Подключить генератор
- Проверить формат и батчи
- Запустить обучение модели

Шаг 17:

Миссия 17

⚙ Расставьте шаги генерации синтетического датасета для модели предсказания спроса.

Подсказка: Начните с анализа существующих данных, завершите сохранением результата.

Ответ:

- Анализ существующих данных
- Выбор метода генерации
- Настройка параметров случайности
- Генерация и сохранение датасета

Урок 4 - Очистка и балансировка датасетов

Шаг 01:

Миссия 1

Вы — дата-инженер, и у вас есть датасет с логами пользователей. Примерно 5 % строк — с пустыми значениями в ключевом столбце «user_id». Что вы сделаете?

Подсказка: Пустые user_id обычно мешают связыванию данных.

Ответ:

- **Удалю эти строки, если они малы по количеству**
- Оставлю как есть и проигнорирую
- Заполню случайным ID
- Объединю их все под одним ID

Шаг 02:

Миссия 2

Ваша коллега прислала CSV с колонками «age», «income», но в некоторых строках возраст записан как «twenty five». Как поступить?

Подсказка: Строка в числовом поле — признак ошибки парсинга.

Ответ:

- Оставить как есть
- **Попытаться преобразовать строковое значение к числу или отвергнуть строку**
- Заменить на 0

- Удалить все такие строки без анализа

🔮 Шаг 03:

Миссия 3

В наборе изображений для классификации один класс содержит в 10 раз больше примеров, чем другие. Какое действие наиболее оправдано?

Подсказка: Дисбаланс часто ухудшает обобщение модели.

Ответ:

- Ничего не делать
- **Сделать undersampling или oversampling**
- Удалить класс с большим количеством
- Удвоить данные из меньших классов без контроля

🔮 Шаг 04:

Миссия 4

Вы замечаете, что в датасете дублируются записи клиентов — полностью совпадают все поля. Что делать?

Подсказка: Полные дубли бизону не нужны.

Ответ:

- **Удалить полные копии, оставив одну запись**
- Оставить дубли как есть
- Оставить одну и две для теста
- Сделать смешение полей для разнообразия

🔮 Шаг 05:

Миссия 5

Сопоставьте реальную проблему и метод балансировки:

Проблемы:

Ответ:

- Класс А слишком редкий
- Класс В доминирует
- Искаженные распределения признаков
- Случайные шумовые метки

Способы балансировки / очистки:

1. Undersampling
2. Oversampling (SMOTE и др.)
3. Удаление шумовых меток
4. Нормализация / трансформация

Подсказка: Match предполагает, что виды проблем и методы взаимосвязаны.

- **Класс А слишком редкий** — Oversampling (SMOTE и др.)
- **Класс В доминирует** — Undersampling
- **Искаженные распределения признаков** — Нормализация / трансформация
- **Случайные шумовые метки** — Удаление шумовых меток

🔗 Шаг 06:

Миссия 6

Сопоставьте тип ошибки в данных и возможную причину:

Типы ошибок:

Ответ:

- Пропуски в столбце
- Строковые значения в числовом поле
- Дубли строк
- Неправильные категории

Причины:

1. Ошибки ввода
2. Плохой парсер
3. Неполная очистка
4. Неправильная кодировка / маппинг

Подсказка: На подумайте, как каждая проблема может возникнуть от цепочки обработки.

- **Пропуски в столбце** — Ошибки ввода
- **Строковые значения в числовом поле** — Плохой парсер
- **Дубли строк** — Неполная очистка
- **Неправильные категории** — Неправильная кодировка / маппинг

🔗 Шаг 07:

Миссия 7

Сопоставьте стратегию балансировки с её особенностью:

Стратегии:

Ответ:

- Random oversample
- SMOTE
- ADASYN
- Cluster-based undersample

Характеристики:

1. Генерация новых точек вдоль линий
2. Обращение внимания на плотность классов
3. Случайное копирование объектов
4. Удаление точек с кластерным подходом

Подсказка: Подумайте, какая стратегия меняет распределение данных более изящно.

- **Random oversample** — Случайное копирование объектов
- **SMOTE** — Генерация новых точек вдоль линий
- **ADASYN** — Обращение внимания на плотность классов
- **Cluster-based undersample** — Удаление точек с кластерным подходом

🔗 Шаг 08:

Миссия 8

Сопоставьте задачи очистки с методами:

Задачи:

Ответ:

- Нормализация признаков
- Обработка категориальных переменных
- Работа с пропусками
- Формирование новых признаков

Методы:

1. One-Hot Encoding / Label Encoding
2. MinMax / StandardScaler
3. Imputation (median, mean, model)
4. Feature engineering (полиномиальные, комбинации)

Подсказка: Каждый метод подходит ровно к одной задаче.

- **Нормализация признаков** — MinMax / StandardScaler
- **Обработка категориальных переменных** — One-Hot Encoding / Label Encoding
- **Работа с пропусками** — Imputation (median, mean, model)
- **Формирование новых признаков** — Feature engineering (полиномиальные, комбинации)

Шаг 09:

Миссия 9

✂ Отсортируйте этапы предобработки данных перед обучением модели:

1. Импутация пропусков
2. Балансировка классов
3. Кодирование категориальных признаков
4. Масштабирование признаков
5. Формирование новых признаков

Подсказка: Подумайте, что должно идти раньше кодирования, а что — после.

Ответ:

- Импутация пропусков
- Кодирование категориальных признаков
- Формирование новых признаков
- Масштабирование признаков
- Балансировка классов

Шаг 10:

Миссия 10

📋 Отсортируйте шаги работы со сбалансированным датасетом:

1. Проверка распределений после балансировки
2. Балансировка (oversample / undersample и др.)
3. Анализ качества модели на валидации
4. Разделение на train / validation / test

Подсказка: Сначала разбиваем, потом балансируем, потом проверяем.

Ответ:

- Разделение на train / validation / test
- Балансировка (oversample / undersample и др.)
- Проверка распределений после балансировки
- Анализ качества модели на валидации

 Шаг 11:

Миссия 11

🔑 Расположите методы борьбы с выбросами в порядке от наименее агрессивного к наиболее:

1. Winsorizing
2. Clipping по квантилям
3. Удаление выбросных записей
4. Замена медианой / усреднением

Подсказка: Агрессивность — насколько кардинально мы меняем или удаляем данные.

Ответ:

- Winsorizing
- Clipping по квантилям
- Замена медианой / усреднением
- Удаление выбросных записей

 Шаг 12:

Миссия 12

Расположите этапы pipeline по очистке данных (end-to-end):

1. Удаление дублирующихся строк
2. Обработка пропусков
3. Кодирование категориальных признаков
4. Масштабирование / нормализация признаков
5. Балансировка классов

Подсказка: Цепочка должна идти логично от малых трансформаций к глобальным.

Ответ:

- Удаление дублирующихся строк
- Обработка пропусков
- Кодирование категориальных признаков
- Масштабирование / нормализация признаков
- Балансировка классов

 Шаг 13:

Миссия 13

Расположите инструменты (или методы), которые вы бы применяли к сырым данным, в порядке, как вы их примените:

1. SMOTE / oversampling
2. Pandas drop_duplicates / dedupe
3. scikit-learn SimpleImputer / IterativeImputer
4. OneHotEncoder / OrdinalEncoder

5. StandardScaler / MinMaxScaler

Подсказка: Выберите порядок инструментов, а не случайный.

Ответ:

- Pandas drop_duplicates / dedupe
- scikit-learn SimpleImputer / IterativeImputer
- OneHotEncoder / OrdinalEncoder
- StandardScaler / MinMaxScaler
- SMOTE / oversampling

Шаг 14:

Миссия 14

 Расположите методы балансировки по возрастанию риска переобучения:

1. Undersampling
2. Random oversampling
3. SMOTE
4. ADASYN

Подсказка: Чем больше искусственных точек — тем выше риск переобучения.

Ответ:

- Undersampling
- Random oversampling
- SMOTE
- ADASYN

Шаг 15:

Миссия 15

 Отсортируйте этапы оценки качества модели при использовании скорректированного датасета:

1. Оценка метрик на тестовом наборе
2. Выполнение кросс-валидации
3. Проверка распределения классов в train / val / test
4. Обучение модели на train (уже сбалансированном)
5. Тюнинг гиперпараметров (Grid / Random Search)

Подсказка: Сначала убеждаемся, что распределения хороши, затем учимся и валидируем.

Ответ:

- Проверка распределения классов в train / val / test
- Выполнение кросс-валидации
- Обучение модели на train (уже сбалансированном)
- Тюнинг гиперпараметров (Grid / Random Search)
- Оценка метрик на тестовом наборе

Шаг 16:

Миссия 16

✧ Отсортируйте ключевые шаги отладки баланса и очистки кода до продакшена:

1. Мониторинг распределений новых входных данных
2. Версионирование данных / схемы
3. Автоматизация очистки и балансировки в пайплайне
4. Тестирование на краевых случаях и аномалиях
5. Отслеживание дрефта данных в продакшене

Подсказка: Начинаем с разработки, завершаем мониторингом.

Ответ:

- Версионирование данных / схемы
- Автоматизация очистки и балансировки в пайплайне
- Тестирование на краевых случаях и аномалиях
- Мониторинг распределений новых входных данных
- Отслеживание дрефта данных в продакшене

📖 **Модуль 14 - Современные методы transfer learning**

📖 **Урок 1 - 🐦 Fine-tuning и LoRA**

🔮 **Шаг 01:**

Миссия 1

Вы догнали коллегу после вечеринки, и она говорит: «Я хочу начать fine-tuning модели большего размера, чем раньше». Что вы ей порекомендуете как первое действие?

Подсказка: Не сразу трогайте все слои — сначала оцените, с чего можно начать.

Ответ:

- **Заморозить большую часть слоёв и дообучить только верхние**
- Обучать все слои сразу
- Удалить последние слои и начать с нуля
- Подключить другой предобученный вес и начать с него

🔮 **Шаг 02:**

Миссия 2

Во время fine-tuning вы получили ошибку OOM на GPU при батче 8. Что логично попробовать первым?

Подсказка: Уменьшение ресурсов — самый прямой путь устранения OOM.

Ответ:

- **Уменьшить batch size**
- Увеличить learning rate
- Разморозить больше слоёв
- Добавить ещё слоёв

🔮 **Шаг 03:**

Миссия 3

Ваша команда планирует использовать LoRA для дообучения модели для генерации текста. Какое ключевое преимущество LoRA?

Подсказка: Подумайте о размере параметров, которые вы фактически обновляете.

Ответ:

- Обучает все веса быстрее
- **Обновляет лишь небольшие матрицы ранга низкого порядка**
- Ограничивает длину входа
- Автоматически генерирует новые слои

🔮 Шаг 04:

Миссия 4

Вы настраиваете оптимизатор для fine-tuning. Что разумно учесть?

Подсказка: Часто в fine-tuning применяют низкий learning rate к базовым весам.

Ответ:

- **Использовать разный learning rate для базовых слоёв и новых**
- Очень высокий learning rate по всем слоям
- Заморозить оптимизатор
- Использовать SGD без momentum

🔮 Шаг 05:

Миссия 5

Во время fine-tuning вы замечаете, что validation loss перестал уменьшаться, но train loss продолжает падать. Что это за симптом?

Подсказка: Модель всё лучше запоминает тренинг, но не обобщает.

Ответ:

- **Переобучение (overfitting)**
- Недообучение
- Ошибка расчёта loss
- Отсутствие регуляризации

🔮 Шаг 06:

Миссия 6

Сопоставьте термин с его значением в контексте transfer learning / LoRA:

Термины:

Ответ:

- Adapter modules
- Rank decomposition
- Frozen layers
- Delta weights

Значения:

1. Обновляемые параметры небольшой размерности, добавляемые к базовой модели
2. Разложение матриц при обновлении весов с низким рангом
3. Слои, параметры которых остаются неизменными во время тренировки
4. Отличие между дообученной моделью и оригинальной моделью

Подсказка: Подумайте, как LoRA или адаптеры влияют на модель весов.

- **Adapter modules** — Обновляемые параметры небольшой размерности, добавляемые к базовой модели

- **Rank decomposition** — Разложение матриц при обновлении весов с низким рангом
- **Frozen layers** — Слои, параметры которых остаются неизменными во время тренировки
- **Delta weights** — Отличие между дообученной моделью и оригинальной моделью

🔮 Шаг 07:

Миссия 7

Сопоставьте технику fine-tuning с её свойством:

Техники:

Ответ:

- Full fine-tuning
- Layer-wise fine-tuning
- LoRA / low-rank adaptation
- Prompt tuning / prefix tuning

Свойства:

1. Обучаются все веса модели
2. Обучаются только верхние слои постепенно вниз
3. Минимальное изменение параметров, малый размер памяти
4. Меняют только входной контекст без затрагивания слоёв модели

Подсказка: Подумайте, сколько параметров меняется при каждом подходе.

- **Full fine-tuning** — Обучаются все веса модели
- **Layer-wise fine-tuning** — Обучаются только верхние слои постепенно вниз
- **LoRA / low-rank adaptation** — Минимальное изменение параметров, малый размер памяти
- **Prompt tuning / prefix tuning** — Меняют только входной контекст без затрагивания слоёв модели

🔮 Шаг 08:

Миссия 8

Сопоставьте шаг fine-tuning / LoRA с его целью:

Шаги:

Ответ:

- Инициализация LoRA-матриц
- Настройка learning rate schedule
- Замораживание базовых слоёв
- Регуляризация (dropout, weight decay)

Цели:

1. Обеспечить стабильность обновлений
2. Ограничить переобучение малых параметров
3. Обновлять только вспомогательные модули
4. Обеспечить адаптацию к задаче без больших изменений

Подсказка: Задача сопоставления — подумать, зачем настраивают каждый элемент.

- **Инициализация LoRA-матриц** — Обеспечить адаптацию к задаче без больших изменений

- **Настройка learning rate schedule** — Обеспечить стабильность обновлений
- **Замораживание базовых слоёв** — Обновлять только вспомогательные модули
- **Регуляризация (dropout, weight decay)** — Ограничить переобучение малых параметров

🔮 Шаг 09:

Миссия 9

Сопоставьте проблему и контрмеру при fine-tuning:

Проблемы:

Ответ:

- Catastrophic forgetting
- Градиентный взрыв / затухание
- Утечка памяти GPU
- Слишком быстрый дрейф в весах

Контрмеры:

1. Мелкий learning rate + регуляризация
2. Gradient clipping
3. Удаление ненужных слоёв, очистка кэшей
4. Замораживание некоторых слоёв или постепенное размораживание

Подсказка: Подумайте, какая проблема связана с «забыванием», а какая — с переполняемыми градиентами.

- **Catastrophic forgetting** — Замораживание некоторых слоёв или постепенное размораживание
- **Градиентный взрыв / затухание** — Gradient clipping
- **Утечка памяти GPU** — Удаление ненужных слоёв, очистка кэшей
- **Слишком быстрый дрейф в весах** — Мелкий learning rate + регуляризация

🔮 Шаг 10:

Миссия 10

✂ Расположите шаги при реализации LoRA-файна в pipeline до обучения:

1. Заморозить базовые слои
2. Инициализировать LoRA-матрицы
3. Подключить LoRA в нужные слои
4. Настроить optimizer и scheduler
5. Произвести warm-up (небольшие шаги)

Подсказка: Подумайте, что сначала — интеграция, а что — настройка.

Ответ:

- Заморозить базовые слои
- Инициализировать LoRA-матрицы
- Подключить LoRA в нужные слои
- Настроить optimizer и scheduler
- Произвести warm-up (небольшие шаги)

🔮 Шаг 11:

Миссия 11

🔧 Отсортируйте этапы fine-tuning модели после загрузки предобученной базы:

1. Заморозить базовые слои
2. Добавить адаптеры / LoRA слои
Настроить разные learning rate для разных частей модели
3. Обучить модель (несколько эпох)
4. Выполнить тонкую настройку всех слоёв (если нужно)

Подсказка: Не торопитесь разморозкой всех слоёв — сперва протестируйте мягкий режим.

Ответ:

- Заморозить базовые слои
- Добавить адаптеры / LoRA слои
- Настроить разные learning rate для разных частей модели
- Обучить модель (несколько эпох)
- Выполнить тонкую настройку всех слоёв (если нужно)

💡 Шаг 12:

Миссия 12

🛡️ Расположите этапы защиты модели от переобучения при fine-tuning:

1. Добавление регуляризации (weight decay, dropout)
2. Gradient clipping
3. Раннее завершение (early stopping)
4. Контроль learning rate scheduling
5. Мониторинг validation loss и метрик

Подсказка: Эти меры взаимно дополняют друг друга.

Ответ:

- Добавление регуляризации (weight decay, dropout)
- Gradient clipping
- Раннее завершение (early stopping)
- Контроль learning rate scheduling
- Мониторинг validation loss и метрик

💡 Шаг 13:

Миссия 13

📊 Расположите стадии отладки LoRA-модуля перед запуском в продакшен:

1. Тестирование на edge-кейсах
2. Проверка влияния на память и latency
3. Оценка качества генерации на контрольных примерах
4. Проверка масштабируемости на разных batch size
5. Сравнение с full fine-tuning baseline

Подсказка: Начните с проверки качества, затем нагрузочные аспекты.

Ответ:

- Тестирование на edge-кейсах
- Оценка качества генерации на контрольных примерах
- Сравнение с full fine-tuning baseline
- Проверка влияния на память и latency

- Проверка масштабируемости на разных batch size

Шаг 14:

Миссия 14

 Расположите этапы lifecycle от идеи fine-tuning до мониторинга в продакшене:

1. Идея и постановка задачи
2. Выбор части модели для дообучения (LoRA / адаптеры / слои)
3. Обучение и валидация
4. Интеграция в inference pipeline
5. Мониторинг дрейфа и производительности

Подсказка: Это полный цикл, от идеи до эксплуатации.

Ответ:

- Идея и постановка задача
- Выбор части модели для дообучения (LoRA / адаптеры / слои)
- Обучение и валидация
- Интеграция в inference pipeline
- Мониторинг дрейфа и производительности

Шаг 15:

Миссия 15

 Расположите этапы отладки модели после запуска, чтобы обеспечить устойчивость:

1. Сбор живых данных и обратной связи
2. Регулярное переобучение / обновление LoRA
3. Анализ случаев генерации ошибок / артефактов
4. Сравнение рабочего поведения модели с базой
5. Откат к предыдущим версиям, если деградация

Подсказка: Мониторинг и адаптация — ключ к стабильно работающей системе.

Ответ:

- Сбор живых данных и обратной связи
- Регулярное переобучение / обновление LoRA
- Анализ случаев генерации ошибок / артефактов
- Сравнение рабочего поведения модели с базой
- Откат к предыдущим версиям, если деградация

Урок 2 - Обучение с учителем и без учителя

Шаг 01:

Миссия 1

Вы — ML-инженер, и вам дали размеченный датасет с фотографиями кошек и собак. Какой подход вы сразу примените?

Подсказка: У вас есть метки для каждого примера — классическая supervised задача.

Ответ:

- Обучить классификатор (supervised learning)
- Применить кластеризацию (unsupervised learning)

- Использовать reinforcement learning
- Использовать методика без учителя (unsupervised)

🔮 Шаг 02:

Миссия 2

Коллега из NLP команды хочет обучить языковую модель на неразмеченном корпусе текстов. Какой подход наиболее логичен?

Подсказка: Нет меток, только тексты — что останется?

Ответ:

- **Обучение без учителя (self-supervised / unsupervised)**
- Классификация по заранее заданным меткам
- Обучение с подкреплением
- Регрессия

🔮 Шаг 03:

Миссия 3

Вы сочетаете размеченные и неразмеченные данные. Какой гибридный подход вам подойдет?

Подсказка: Смешанные задачи часто используют semi-supervised методы.

Ответ:

- **Semi-supervised learning**
- Метод без учителя только
- Метод лишь с учителем
- Обучение с подкреплением

🔮 Шаг 04:

Миссия 4

Ваша модель подбирает темы статей. Она обучена на размеченном корпусе, но вы хотите расширить знания за счёт большого неразмеченного корпуса. Что логично внедрить?

Подсказка: Подумайте о методах, которые обучаются сначала на неразмеченных данных, потом дообучаются.

Ответ:

- **Преобучение (pretraining) → fine-tuning**
- Кластеризация только
- Случайная инициализация
- RL-fine tuning

🔮 Шаг 05:

Миссия 5

Во время обучения кластеризации вы увидели, что все объекты попадают в один кластер. Что может быть не так?

Подсказка: Проверьте гиперпараметры и масштаб признаков.

Ответ:

- **Плохо задана метрика расстояния или параметры k**
- Все объекты одинаковые
- Ошибка только в интерфейсе

- Обучение supervised неправильно

🔮 Шаг 06:

Миссия 6

Сопоставьте подход (supervised / unsupervised / semi-supervised) с его ключевым свойством:

Подходы:

Ответ:

- Supervised learning
- Unsupervised learning
- Semi-supervised learning
- Self-supervised learning

Свойства:

1. Использует только размеченные данные
2. Работает без меток, ищет структуру в данных
3. Сочетает немного меток и много неразмеченных данных
4. Учится прогнозировать части данных на основе других частей без внешних меток

Подсказка: Обратите внимание, как конструкция self-supervised отличается от semi-supervised.

- **Supervised learning** — Использует только размеченные данные
- **Unsupervised learning** — Работает без меток, ищет структуру в данных
- **Semi-supervised learning** — Сочетает немного меток и много неразмеченных данных
- **Self-supervised learning** — Учится прогнозировать части данных на основе других частей без внешних меток

🔮 Шаг 07:

Миссия 7

Сопоставьте задачу из жизни ML-разработчика с подходящим типом обучения:

Ситуации:

Ответ:

- Классификация спама, есть метки для писем
- Группировка пользователей по интересам без меток
- У вас 100 размеченных примеров и 10 000 неразмеченных
- Автокодировщик для восстановления изображения

Тип обучения:

1. Supervised learning
2. Unsupervised learning
3. Semi-supervised learning
4. Self-supervised learning / autoencoder approach

Подсказка: Какие примеры требуют меток, а какие — просто структуры?

- **Классификация спама, есть метки для писем** — Supervised learning
- **Группировка пользователей по интересам без меток** — Unsupervised learning
- **У вас 100 размеченных примеров и 10 000 неразмеченных** — Semi-supervised learning
- **Автокодировщик для восстановления изображения** — Self-supervised learning / autoencoder approach

🔮 Шаг 08:

Миссия 8

Сопоставьте технику с её применением в рамках обучения без учителя или полуучителя:

Техники:

Ответ:

- k-means clustering
- PCA / SVD
- Consistency regularization
- Pseudo-labeling

Применения:

1. Обучение устойчивости к шуму для неразмеченных примеров
2. Присвоение псевдометок неразмеченным образцам
3. Снижение размерности, выявление латентных факторов
4. Разделение данных на группы без меток

Подсказка: Подумайте, какие техники требуют метки, какие — нет.

- **k-means clustering** — Разделение данных на группы без меток
- **PCA / SVD** — Снижение размерности, выявление латентных факторов
- **Consistency regularization** — Обучение устойчивости к шуму для неразмеченных примеров
- **Pseudo-labeling** — Присвоение псевдометок неразмеченным образцам

🔮 Шаг 09:

Миссия 9

Сопоставьте проблему и подходящий способ её решения в semi-supervised контексте:

Проблемы:

Ответ:

- Мало размеченных данных
- Шумные псевдометки
- Смещение распределения между размеченными и неразмеченными
- Переобучение на размеченной части

Решения:

1. Использовать consistency regularization
2. Фильтровать псевдометки по порогу уверенности
3. Domain adaptation / reweighting неразмеченных примеров
4. Регуляризация / dropouts / ранняя остановка

Подсказка: Подумайте, какие меры направлены на устойчивость, а какие — на корректность меток.

- **Мало размеченных данных** — Использовать consistency regularization
- **Шумные псевдометки** — Фильтровать псевдометки по порогу уверенности
- **Смещение распределения между размеченными и неразмеченными** — Domain adaptation / reweighting неразмеченных примеров
- **Переобучение на размеченной части** — Регуляризация / dropouts / ранняя остановка

Шаг 10:

Миссия 10

✂ Отсортируйте шаги pipeline для semi-supervised обучения:

1. Обучение на размеченных данных
2. Применение модели к неразмеченным примерам
3. Генерация псевдометок с порогом уверенности
4. Обучение с учителем + псевдометки вместе
5. Повторить этапы итеративно

Подсказка: Сначала нужно модель обучить на метках, потом “расширить” её на неразмеченные.

Ответ:

- Обучение на размеченных данных
- Применение модели к неразмеченным примерам
- Генерация псевдометок с порогом уверенности
- Обучение с учителем + псевдометки вместе
- Повторить этапы итеративно

Шаг 11:

Миссия 11

📋 Отсортируйте стадии от pretraining до итогового fine-tuning, если вы используете self-supervised предобучение:

1. Self-supervised pretraining на большом неразмеченном корпусе
2. Заморозка части слоёв / адаптация
3. Обучение на размеченном подмножестве
4. Тонкая настройка всех слоёв (fine-tuning)
5. Оценка на hold-out тестовом наборе

Подсказка: Подумайте, где полезно фиксировать веса, а где — давать гибкость.

Ответ:

- Self-supervised pretraining на большом неразмеченном корпусе
- Заморозка части слоёв / адаптация
- Обучение на размеченном подмножестве
- Тонкая настройка всех слоёв (fine-tuning)
- Оценка на hold-out тестовом наборе

Шаг 12:

Миссия 12

🔒 Расположите методы защиты от ошибок псевдометок в semi-supervised подходе:

1. Порог доверия (confidence threshold)
2. Скользящее усреднение меток (temporal ensembling)
3. Dropout / регуляризация
4. Фильтрация по консенсусу моделей (ensemble agreement)
5. Раннее завершение (early stopping)

Подсказка: Некоторые меры контролируют метки, другие — устойчивость модели.

Ответ:

- Порог доверия (confidence threshold)
- Скользящее усреднение меток (temporal ensembling)
- Dropout / регуляризация
- Фильтрация по консенсусу моделей (ensemble agreement)
- Раннее завершение (early stopping)

 Шаг 13:

Миссия 13

Отсортируйте уровни сложности задач по мере роста знаний студента от supervised к unsupervised:

1. Классификация с метками
2. Регрессия с метками
3. Semi-supervised с псевдометками
4. Кластеризация без меток
5. Обучение self-supervised (маскировка / автокодировщик)

Подсказка: Подумайте, как добавление неизвестности (метки или их отсутствие) влияет на сложность задачи.

Ответ:

- Классификация с метками
- Регрессия с метками
- Semi-supervised с псевдометками
- Кластеризация без меток
- Обучение self-supervised (маскировка / автокодировщик)

 Шаг 14:

Миссия 14

🔧 Расположите компоненты системы обучения с учителем и без в гибридном pipeline:

1. Self-supervised предобучение большого корпуса
2. Кластеризация / выявление структуры на неразмеченных данных
3. Генерация псевдометок для части данных
4. Тренировка supervised модели на расширенном наборе
5. Финальная тонкая настройка (fine-tuning)

Подсказка: Pipeline — это череда шагов, усиливающих друг друга.

Ответ:

- Self-supervised предобучение большого корпуса
- Кластеризация / выявление структуры на неразмеченных данных
- Генерация псевдометок для части данных
- Тренировка supervised модели на расширенном наборе
- Финальная тонкая настройка (fine-tuning)

 Шаг 15:

Миссия 15

📋 Расположите меры контроля качества модели в гибридной setting:

1. Оценка метрик на валидационном наборе
2. Анализ шума в псевдометках
3. Мониторинг divergence между размеченной и псевдо размеченной частями
4. Регулярное переобучение / обновление псевдометок
5. Откат параметров при деградации

Подсказка: Контроль начинается с оценки, затем углублённый анализ.

Ответ:

- Оценка метрик на валидационном наборе
- Анализ шума в псевдометках
- Мониторинг divergence между размеченной и псевдо размеченной частями
- Регулярное переобучение / обновление псевдометок
- Откат параметров при деградации

💡 **Шаг 16:**

Миссия 16

📋 Расположите этапы lifecycle от идеи до мониторинга в гибридной setting:

1. Формулировка задачи и сбор размеченных данных
2. Сбор большого неразмеченного корпуса
3. Self-supervised предобучение
4. Semi-supervised обучение / псевдометки
5. Fine-tuning и тестирование
6. Мониторинг и обновление модели

Подсказка: Это полный цикл от данных к эксплуатации.

Ответ:

- Формулировка задачи и сбор размеченных данных
- Сбор большого неразмеченного корпуса
- Self-supervised предобучение
- Semi-supervised обучение / псевдометки
- Fine-tuning и тестирование
- Мониторинг и обновление модели

💡 **Шаг 17:**

Миссия 17

📋 Расположите этапы от запуска гибридной модели к её устойчивости в продакшене:

1. Сбор обратной связи от пользователей / метки
2. Переподгонка псевдометок / корректировка
3. Регулярные переобучения с новыми данными
4. Мониторинг drift распределения признаков
5. Откат к предыдущим версиям при деградации модели

Подсказка: После запуска важны обратная связь и адаптация.

Ответ:

- Сбор обратной связи от пользователей / метки

- Переподгонка псевдометок / корректировка
- Регулярные переобучения с новыми данными
- Мониторинг drift распределения признаков
- Откат к предыдущим версиям при деградации модели

Урок 3 - Квест адаптируй модель под новую задачу

Шаг 01:

Миссия 1

Ваша команда получила готовую модель классификации фруктов. Клиент просит адаптировать её под распознавание цветов. С чего начать?

Подсказка: Не переписывайте всё сразу — подумайте об используемой архитектуре и о том, что менять безопасно.

Ответ:

- **Подключить новый классификационный слой и дообучить**
- Запустить обучение с нуля
- Оставить модель как есть
- Заморозить весь сверточный блок и не трогать ничего

Шаг 02:

Миссия 2

В процессе адаптации вы заметили, что в новых данных цветов всего 3 класса, а старая модель была настроена на 100 классов. Что логично сделать?

Подсказка: Выходной слой должен соответствовать числу целевых классов.

Ответ:

- **Заменить финальный слой на слой с 3 выходами**
- Оставить 100 выходов и игнорировать лишние
- Добавить ещё скрытые слои
- Использовать softmax с 100 классов всё равно

Шаг 03:

Миссия 3

Вы решили заморозить большую часть слоёв предобученной модели. Какая стратегия имеет смысл?

Подсказка: Можно размораживать постепенно.

Ответ:

- **Заморозить первые N слоёв, дообучать только последние**
- Заморозить всё кроме входного слоя
- Заморозить только последний слой
- Никогда ничего не размораживать

Шаг 04:

Миссия 4

При адаптации вы получаете переполнение градиентов на новых слоях. Что логично попробовать?

Подсказка: Контроль градиентов часто решает такие проблемы.

Ответ:

- Применить **gradient clipping**
- Увеличить learning rate
- Удалить все градиенты
- Использовать ReLU вместо LeakyReLU

🔗 **Шаг 05:**

Миссия 5

Во время адаптации validation accuracy не растёт, хотя train accuracy растёт. Что вероятнее всего происходит?

Подсказка: Подумайте о разнице между обучением и обобщением.

Ответ:

- **Модель переобучается на тренировочных данных**
- Модель недообучается
- Ошибка в валидационном скрипте
- Данные валидирования пустые

🔗 **Шаг 06:**

Миссия 6

Сопоставьте элементы адаптации модели и их роль при переводе на новую задачу:

Элементы:

Ответ:

- Замена выходного слоя
- Замораживание слоёв
- Размораживание сверху вниз
- Регуляризация новых слоёв

Роли:

1. Смена количества классов и активатора
2. Ограничить изменение весов базовой модели
3. Постепенный контроль обучения слоёв
4. Предотвращение переобучения новых параметров

Подсказка: Каждый элемент адаптации должен иметь логическую мотивацию.

- **Замена выходного слой** — Смена количества классов и активатора
- **Замораживание слоёв** — Ограничить изменение весов базовой модели
- **Размораживание сверху вниз** — Постепенный контроль обучения слоёв
- **Регуляризация новых слоёв** — Предотвращение переобучения новых параметров

🔗 **Шаг 07:**

Миссия 7

Сопоставьте проблемы адаптации и подходящие методы их решения:

Проблемы:

Ответ:

- Catastrophic forgetting

- Переобучение на новых данных
- Градиентный взрыв / затухание
- Несоответствие распределений старых и новых данных

Методы:

1. Заморозка части слоёв или регуляризация базовых весов
2. Регуляризация / dropout / ранняя остановка
3. Gradient clipping или нормализация градиентов
4. Domain adaptation, fine-tuning с весами или reweighting

Подсказка: Подумайте, какой метод прямо противоположен каждой проблеме.

- **Catastrophic forgetting** — Заморозка части слоёв или регуляризация базовых весов
- **Переобучение на новых данных** — Регуляризация / dropout / ранняя остановка
- **Градиентный взрыв / затухание** — Gradient clipping или нормализация градиентов
- **Несоответствие распределений старых и новых данных** — Domain adaptation, fine-tuning с весами или reweighting

🔮 Шаг 08:

Миссия 8

Сопоставьте стратегию fine-tuning под новую задачу и её сценарий:

Стратегии:

Ответ:

- Fine-tuning всех слоёв сразу
- Fine-tuning верхних слоёв
- LoRA / адаптеры
- Prompt tuning / prefix tuning

Сценарии:

1. У вас большой размеченный набор и можно менять всё
2. Небольшие изменения задачи, похожий домен
3. Не хочется менять большую модель, нужно лёгкое хранилище изменений
4. Хотите добавить мягкую адаптацию без переписывания слоёв вообще

Подсказка: Сравните, сколько изменения параметров требуется в каждой стратегии.

- **Fine-tuning всех слоёв сразу** — У вас большой размеченный набор и можно менять всё
- **Fine-tuning верхних слоёв** — Небольшие изменения задачи, похожий домен
- **LoRA / адаптеры** — Не хочется менять большую модель, нужно лёгкое хранилище изменений
- **Prompt tuning / prefix tuning** — Хотите добавить мягкую адаптацию без переписывания слоёв вообще

🔮 Шаг 09:

Миссия 9

Сопоставьте этап адаптации с его важной проверкой:

Этапы:

Ответ:

- Замена слоя вывода

- Дообучение новых слоёв
- Размораживание слоёв постепенно
- Инференс на новом наборе

Проверки:

1. Корректность формы выходов
2. Конвергенция loss на тренировочных данных
3. Небольшие скачки в метриках после размораживания
4. Оценка качества на hold-out новых примерах

Подсказка: На каждом этапе важно проверять именно ту часть, которая изменилась.

- **Замена слоя вывода** — Корректность формы выходов
- **Дообучение новых слоёв** — Конвергенция loss на тренировочных данных
- **Размораживание слоёв постепенно** — Небольшие скачки в метриках после размораживания
- **Инференс на новом наборе** — Оценка качества на hold-out новых примерах

Шаг 10:

Миссия 10

 Расположите шаги адаптации модели к новой задаче в правильном порядке:

1. Замена выходного слоя
2. Заморозка базовых слоёв
3. Дообучение новых слоёв
4. Размораживание некоторых слоёв
5. Финальное обучение всей модели (при необходимости)

Подсказка: Сначала измените структуру, затем обучайте постепенно.

Ответ:

- Замена выходного слоя
- Заморозка базовых слоёв
- Дообучение новых слоёв
- Размораживание некоторых слоёв
- Финальное обучение всей модели (при необходимости)

Шаг 11:

Миссия 11

 Отсортируйте проверочные этапы в адаптации при выпуске в продакшен:

1. Тестирование на hold-out новых классов
2. Мониторинг drift входных данных
3. Сравнение метрик старой и адаптированной модели
4. Проверка производительности (латентность, память)
5. Откат к предыдущей модели при деградации

Подсказка: Важно не только качество, но и надёжность и производительность.

Ответ:

- Тестирование на hold-out новых классов
- Мониторинг drift входных данных

- Сравнение метрик старой и адаптированной модели
- Проверка производительности (латентность, память)
- Откат к предыдущей модели при деградации

Шаг 12:

Миссия 12

📌 Расположите меры защиты от ошибок и деградации после адаптации:

1. Регулярный мониторинг метрик
2. Раннее завершение (early stopping) при обучении
3. Регуляризация новых параметров (dropout, weight decay)
4. Создание механизма отката моделей
5. Контроль drift и обновление модели при необходимости

Подсказка: Защита должна быть многослойной: и на этапе обучения, и в эксплуатации.

Ответ:

- Регулярный мониторинг метрик
- Раннее завершение (early stopping) при обучении
- Регуляризация новых параметров (dropout, weight decay)
- Создание механизма отката моделей
- Контроль drift и обновление модели при необходимости

Шаг 13:

Миссия 13

📌 Отсортируйте возможные сценарии адаптации модели по возрастанию сложности:

1. Замена выходного слоя и дообучение
2. Fine-tuning верхних слоёв
3. Полный fine-tuning всех слоёв
4. Использование адаптеров / LoRA
5. Prompt tuning / prefix tuning

Подсказка: Сложность зависит от объёма изменения параметров.

Ответ:

- Замена выходного слоя и дообучение
- Fine-tuning верхних слоёв
- Полный fine-tuning всех слоёв
- Использование адаптеров / LoRA
- Prompt tuning / prefix tuning

Шаг 14:

Миссия 14

📌 Расположите этапы адаптации + валидации при добавлении новых классов:

1. Дообучение на расширенной меточной части
2. Валидация на hold-out из новых классов
3. Проверка, что старая производительность не деградировала
4. Проверка, что новая модель не сильно отличается по представлениям
5. Интеграция в production pipeline

Подсказка: Убедитесь, что адаптация не сломала старое поведение.

Ответ:

- Дообучение на расширенной меточной части
- Валидация на hold-out из новых классов
- Проверка, что старая производительность не деградировала
- Проверка, что новая модель не сильно отличается по представлениям
- Интеграция в production pipeline

Шаг 15:

Миссия 15

 Расположите фазы доработки адаптированной модели перед выпуском:

1. Тестирование на краевых / редких случаях
2. Оценка латентности и памяти в сравнении со старой моделью
3. Сравнение метрик между старой и новой моделью
4. Нагрузочное тестирование (batch size, throughput)
5. Интеграционное тестирование с downstream системой

Подсказка: Наведите порядок: от функциональности к нагрузке и интеграции.

Ответ:

- Тестирование на краевых / редких случаях
- Оценка латентности и памяти в сравнении со старой моделью
- Сравнение метрик между старой и новой моделью
- Нагрузочное тестирование (batch size, throughput)
- Интеграционное тестирование с downstream системой

Шаг 16:

Миссия 16

 Отсортируйте жизненный цикл адаптированной модели в production:

1. Адаптация / дообучение
2. Тестирование и проверка качества
3. Развёртывание модели
4. Мониторинг производительности и drift
5. Обновление / перераспределение / откат при необходимости

Подсказка: Жизненный цикл — это непрерывный цикл от тренировки к эксплуатации и обратно.

Ответ:

- Адаптация / дообучение
- Тестирование и проверка качества
- Развёртывание модели
- Мониторинг производительности и drift
- Обновление / перераспределение / откат при необходимости

Шаг 17:

Миссия 17

🔍 Расположите этапы от запуска адаптированной модели до её устойчивости:

1. Мониторинг метрик на продакшене
2. Сбор примеров ошибок и анализ случаев падения
3. Переобучение или корректировка параметров модели
4. Откат на предыдущую версию при ухудшении
5. Регулярное обновление с новыми данными / адаптация

Подсказка: После запуска важно не отпускать модель — следить и корректировать.

Ответ:

- Мониторинг метрик на продакшене
- Сбор примеров ошибок и анализ случаев падения
- Переобучение или корректировка параметров модели
- Откат на предыдущую версию при ухудшении
- Регулярное обновление с новыми данными / адаптация

📖 Урок 4 - Few-shot и zero-shot learning

🔮 Шаг 01:

Миссия 1

Алиса создает модель few-shot классификации для распознавания новых эмоций на основе всего 5 примеров. Какой подход лучше всего использовать в самом начале?

Подсказка: Поиск модели, уже обученной на большом корпусе, и дообучение её на ваших примерах.

Ответ:

- **Использовать предварительно обученную модель и дообучить несколько слоёв**
- Обучить модель с нуля на 5 примерах
- Применить обычный fine-tuning без адаптации к задачам

🔮 Шаг 02:

Миссия 2

Боб имеет задачу zero-shot: модель должна классифицировать тексты на темы, которых не было в тренировочных данных. Что стоит заложить в архитектуру?

Подсказка: Использовать языковую модель с богатым семантическим представлением и метки как текстовое описание.

Ответ:

- **Вербальное представление меток и сравнение эмбедингов**
- Обычный softmax по фиксированному набору классов
- Использовать свёрточную сеть без текстовых дескрипторов

🔮 Шаг 03:

Миссия 3

Карина работает над системой prompt-архитектуры: к какому из вариантов относится few-shot prompt?

Подсказка: Подумайте: сколько примеров вы вставляете в prompt перед запросом.

Ответ:

- **В prompt вставлены несколько примеров «вопрос → ответ»**

- Prompt содержит только инструкцию, без примеров
- Prompt содержит тысячи примеров для обучения

🔮 Шаг 04:

Миссия 4

Сопоставьте термин и описание в контексте few-shot / zero-shot:

Термины:

Ответ:

- Prompt learning
- Meta-learning
- Embedding similarity
- Label verbalizer

Описания:

1. Сравнение эмбедингов примеров и входного запроса
2. Привязка текстовых описаний классов к выходным токенам
3. Обучение стратегии адаптации задач в небольшом количестве шагов
4. Использование шаблонов prompt для управления поведением модели

Подсказка: Обратите внимание: verbalizer связывает метки с текстом, а prompt learning — это шаблоны.

- **Prompt learning** — Использование шаблонов prompt для управления поведением модели
- **Meta-learning** — Обучение стратегии адаптации задач в небольшом количестве шагов
- **Embedding similarity** — Сравнение эмбедингов примеров и входного запроса
- **Label verbalizer** — Привязка текстовых описаний классов к выходным токенам

🔮 Шаг 05:

Миссия 5

Дан список техник в few-/zero-shot, сопоставьте их с преимуществами:

Техники:

Ответ:

- Adapter tuning
- Prompt tuning
- Prefix tuning
- Linear probing

Преимущества:

1. Минимальное изменение параметров, лёгкость внедрения
2. Тонкая адаптация шаблонов prompt без полного fine-tuning
3. Добавляемые модули, не трогающие основную модель
4. Обучение лишь линейного слоя на замороженной основе

Подсказка: Подумайте: что остаётся замороженным при линейном probing, а что при adapter или prefix.

- **Adapter tuning** — Добавляемые модули, не трогающие основную модель
- **Prompt tuning** — Тонкая адаптация шаблонов prompt без полного fine-tuning
- **Prefix tuning** — Минимальное изменение параметров, лёгкость внедрения

- **Linear probing** — Обучение лишь линейного слоя на замороженной основе

🔮 Шаг 06:

Миссия 6

В командной переписке Иван рассказывает о loss functions, которые часто применяются в zero-shot / few-shot. Сопоставьте loss и ситуацию:

Loss:

Ответ:

- Contrastive loss
- Cross-entropy over verbalizer outputs
- Triplet loss
- Margin ranking loss

Ситуации:

1. Усиление расстояния между разными классами и сближение внутри класса
2. Прямая классификация через вербализатор
3. Оценка порядка близости между примерами и запросом
4. Обучение ранжированию похожести запрос-пример

Подсказка: Contrastive и triplet близки, ranking loss — про порядок.

- **Contrastive loss** — Усиление расстояния между разными классами и сближение внутри класса
- **Cross-entropy over verbalizer outputs** — Прямая классификация через вербализатор
- **Triplet loss** — Оценка порядка близости между примерами и запросом
- **Margin ranking loss** — Обучение ранжированию похожести запрос-пример

🔮 Шаг 07:

Миссия 7

Когда Лена тестирует zero-shot модель, она анализирует ошибки. Сопоставьте вид ошибки и возможную причину:

Виды ошибки:

Ответ:

- Класс неверно предсказан, но близок семантически
- Класс предсказан, хотя он вовсе не подходит
- Модель возвращает «неизвестно» для правильного класса
- Неправильная генерация текста не из списка меток

Причины:

1. Слабое различение между смежными метками
2. Вербализатор не охватывает весь смысл меток
3. Порог слишком консервативен
4. Ограничения decoding или шаблонов

Подсказка: Ошибки часто связаны либо с порогом, либо с покрытием verbalizer, либо с decoding.

- **Класс неверно предсказан, но близок семантически** — Слабое различение между смежными метками
- **Класс предсказан, хотя он вовсе не подходит** — Вербализатор не охватывает весь смысл меток

- **Модель возвращает «неизвестно» для правильного класса** — Порог слишком консервативен
- **Неправильная генерация текста не из списка меток** — Ограничения decoding или шаблонов

🔮 Шаг 08:

Миссия 8

Максим планирует pipeline для few-shot обучения. Расположите шаги по порядку:

1. Оценка качества на валидации
2. Выбор предобученной модели
3. Сбор и подготовка few-shot примеров
4. Настройка prompt / verbalizer / адаптеров
5. Обучение или адаптация под задачу
6. Тестирование на новых классах

Подсказка: Начинаете с модели, затем примеры, затем настройка и обучение, и в конце — проверка.

Ответ:

- Выбор предобученной модель
- Сбор и подготовка few-shot примеров
- Настройка prompt / verbalizer / адаптеров
- Обучение или адаптация под задачу
- Оценка качества на валидации
- Тестирование на новых классах

🔮 Шаг 09:

Миссия 9

Расположите архитектурные подходы по степени вмешательства в основную модель (от меньшего к большему):

1. Линейный probing
2. Prompt tuning
3. Prefix tuning
4. Adapter tuning
5. Полное fine-tuning

Подсказка: Чем больше слоёв вы модифицируете — тем выше вмешательство.

Ответ:

- Линейный probing
- Prompt tuning
- Prefix tuning
- Adapter tuning
- Полное fine-tuning

🔮 Шаг 10:

Миссия 10

При развертывании zero-shot системы на продакшене, упорядочите этапы:

1. Мониторинг ошибок в реальных запросах

2. Сбор новых данные для дообучения
3. Внедрение в продуктивную систему
4. Тестирование на контрпримерных запросах
5. Калибровка порогов / уверенности
6. Оценка на held-out задачах

Подсказка: После тестирования и оценки сначала калибруют, затем внедряют, потом мониторят и собирают данные.

Ответ:

- Мониторинг ошибок в реальных запросах
- Сбор новых данные для дообучения
- Внедрение в продуктивную систему
- Тестирование на контрпримерных запросах
- Калибровка порогов / уверенности
- Оценка на held-out задачах

Шаг 11:

Миссия 11

Команда планирует эволюцию системы от zero-shot к few-shot. Расположите стадии развития:

1. Zero-shot только с prompt
2. Zero-shot + вербализатор улучшенный
3. Few-shot с небольшим дополнением примеров
4. Few-shot с адаптерами
5. Полное fine-tuning при наличии большого числа примеров

Подсказка: Сначала минимальное вмешательство, затем добавление примеров, затем более гибкие методы, затем полный fine-tuning.

Ответ:

- Zero-shot только с prompt
- Zero-shot + вербализатор улучшенный
- Few-shot с небольшим дополнением примеров
- Few-shot с адаптерами
- Полное fine-tuning при наличии большого числа примеров

Шаг 12:

Миссия 12

При сравнении моделей Alice и Bob, упорядочите критерии, которые важны в продакшене few/zero shot:

1. Латентная память и объём модели
2. Скорость inference
3. Гибкость адаптации к новым классам
4. Устойчивость к переобучению на малых данных
5. Планируемые издержки поддержки

Подсказка: В продакшене часто важны latency, гибкость, устойчивость, и поддержка.

Ответ:

- Латентная память и объём модели

- Скорость inference
- Гибкость адаптации к новым классам
- Устойчивость к переобучению на малых данных
- Планируемые издержки поддержки

🔮 Шаг 13:

Миссия 13

В процессе экспериментов с few-shot моделью, упорядочите цикл итерации:

1. Анализ ошибок
2. Доработка verbalizer / prompt
3. Переобучение / адаптация модели
4. Оценка на валидации
5. Обновление продакшена

Подсказка: Анализ → корректировка → обучение → оценка → интеграция.

Ответ:

- Анализ ошибок
- Доработка verbalizer / prompt
- Переобучение / адаптация модели
- Оценка на валидации
- Обновление продакшена

🔮 Шаг 14:

Миссия 14

Оля хочет уменьшить «разрыв домена» между предварительным корпусом и её специализированными примерами. Какой метод может помочь?

Подсказка: Мы хотим адаптировать представления модели к новому домену без полного переобучения.

Ответ:

- **Domain adaptation через adapter или prompt-модули**
- Игнорировать домен и использовать общие примеры
- Разбить задачу на множество подзадач с отдельными моделями

🔮 Шаг 15:

Миссия 15

Вася экспериментирует: добавляет всего 1 пример нового класса. Какой риск наиболее вероятен?

Подсказка: Один пример — это крайний случай слабого количества данных.

Ответ:

- **Переобучение на этот один пример (overfitting)**
- Модель не обучится вовсе
- Она сразу станет превосходить zero-shot

🔮 Шаг 16:

Миссия 16

В продакшене выявлена категория, которой не было ни в тренировочных, ни в валидационных данных. Что делать?

Подсказка: Это ситуация open-set: нужно либо расширять verbalizer, либо отклонять запросы.

Ответ:

- **Добавить категорию через prompt / verbalizer и дообучить модель**
- Игнорировать ситуацию и всегда классифицировать в известные классы
- Выбросить все такие запросы без обработки

🔮 **Шаг 17:**

Миссия 17

Команда готовит статью: нужно выбрать лучший baseline для сравнения с их few-shot системой. Что разумно выбрать?

Подсказка: Baseline — это сильная стандартная модель с минимальной адаптацией.

Ответ:

- **Zero-shot prompt-набор без дообучения**
- Случайное угадывание
- Модель, обученная с нуля на few примерах

📁 **Модуль 15 - 🎯 Финальный проект твоя модель**

📖 **Урок 1 - 📋 Выбор задачи и постановка цели**

🔮 **Шаг 01:**

Миссия 1

Сергей хочет выбрать задачу для финального проекта: классификация, регрессия или генерация текста. Что полезно сделать в самом начале?

Подсказка: Прежде чем углубляться в архитектуры, важно понять цель и формат задачи.

Ответ:

- **Определить целевую метрику и формат выхода (label, число, текст)**
- Сразу выбирать архитектуру трансформера
- Сначала собирать огромный датасет

🔮 **Шаг 02:**

Миссия 2

Катя думает: «Может взять задачу перевода — ведь трансформеры уже хороши в этом». Какой аспект важно проверить прежде всего?

Подсказка: Убедись, что у тебя есть достаточные данные или хотя бы параллельные пары.

Ответ:

- **Наличие параллельных данных (исходный → целевой язык)**
- Наличие метрик BLEU сразу
- Использование GPU при первом прототипе

🔮 **Шаг 03:**

Миссия 3

Миша выбрал задачу прогнозирования цен. Он задумывается: использовать ли логарифм цены как выход. Это относится к:

Подсказка: Подумай: ты меняешь шкалу, но оставляешь задачу регрессии.

Ответ:

- **Преобразования целевой переменной перед обучением**
- Изменение архитектуры нейросети на классификационную
- Отказ от использования нормализации

🔮 Шаг 04:

Миссия 4

Алексей составляет список критериев выбора задачи. Сопоставьте критерий и его смысл:

Критерии:

Ответ:

- Четкость цели
- Измеримость
- Доступность данных
- Практическая полезность

Смыслы:

1. Можно объективно оценить результат
2. Цель сформулирована точно, без двусмысленностей
3. Есть данные для обучения и проверки модели
4. Проект решает реальную задачу или приносит ценность

Подсказка: Один критерий связан с тем, что ты сможешь измерить, другой — что данные реально собрать.

- **Четкость цели** — Цель сформулирована точно, без двусмысленностей
- **Измеримость** — Можно объективно оценить результат
- **Доступность данных** — Есть данные для обучения и проверки модели
- **Практическая полезность** — Проект решает реальную задачу или приносит ценность

🔮 Шаг 05:

Миссия 5

Инна сравнивает типы задач: классификация, регрессия, генерация. Сопоставьте тип задачи и подходящий пример проекта:

Типы задач:

Ответ:

- Классификация
- Регрессия
- Генерация
- Сегментация / структурный выход

Примеры проектов:

1. Предсказать цену дома по характеристикам
2. Определить эмоцию по тексту
3. Сгенерировать текст новости по заголовку
4. Выделить объект на фото (маска)

Подсказка: Генерация — это свободный текст, классификация — фиксированные классы, сегментация — пространственный выход.

- **Классификация** — Определить эмоцию по тексту
- **Регрессия** — Предсказать цену дома по характеристикам
- **Генерация** — Сгенерировать текст новости по заголовку
- **Сегментация / структурный выход** — Выделить объект на фото (маска)

🔮 Шаг 06:

Миссия 6

Жанна оценивает риски постановки задачи. Сопоставьте риск и его описание:

Риски:

Ответ:

- Неясная метрика
- Смещение данных
- Нехватка примеров крайних случаев
- Категории с перекрытием

Описание риска:

1. Метрика не отражает бизнес-цель или плохо информативна
2. Данные обучающей выборки отличаются от данных в продакшене
3. Редкие примеры плохо представлены и модель не учится на них
4. **Классы неразличимы в некоторых кейсах** — границы нечеткие

Подсказка: Некоторые риски связаны с распределениями, другие — с метрикой или «шумом» в данных.

- **Неясная метрика** — Метрика не отражает бизнес-цель или плохо информативна
- **Смещение данных** — Данные обучающей выборки отличаются от данных в продакшене
- **Нехватка примеров крайних случаев** — Редкие примеры плохо представлены и модель не учится на них
- **Категории с перекрытием** — Классы неразличимы в некоторых кейсах — границы нечеткие

🔮 Шаг 07:

Миссия 7

Дима думает о гранулярности задачи. Сопоставьте понятие и эффект:

Понятия:

Ответ:

- Узкая спецификация
- Широкий охват
- Модульная постановка
- Энд-ту-энд подход

Эффекты:

1. Модель решает сразу весь pipeline (от сырья до выхода)
2. Упрощение части задачи, легче итерации
3. Модель покрывает множество случаев, высокая общность
4. Уменьшение числа крайних случаев, проще верификация

Подсказка: Узкая спецификация — это точечная задача, широкий охват — обобщение.

- **Узкая спецификация** — Уменьшение числа крайних случаев, проще верификация
- **Широкий охват** — Модель покрывает множество случаев, высокая общность
- **Модульная постановка** — Упрощение части задачи, легче итерации
- **Энд-ту-энд подход** — Модель решает сразу весь pipeline (от сырья до выхода)

🔮 Шаг 08:

Миссия 8

Артем планирует шаги перед прототипом модели. Расположите по порядку:

1. Сбор минимального набора данных (микро-набор)
2. Создание baseline (простейшая модель)
3. Оценка baseline на валидации
4. Анализ ошибок baseline
5. Уточнение постановки задачи и ограничений

Подсказка: Начни с малого, потом benchmark, анализ ошибок и итерации постановки.

Ответ:

- Сбор минимального набора данных (микро-набор)
- Создание baseline (простейшая модель)
- Оценка baseline на валидации
- Анализ ошибок baseline
- Уточнение постановки задачи и ограничений

🔮 Шаг 09:

Миссия 9

Катя переходит от идеи к спецификации. Упорядочите следующие этапы:

1. Определение входных и выходных форматов
2. Описание ограничений (латенси, память и т.д.)
3. Выбор архитектурных подходов, которые впишутся в требования
4. Определение предварительной метрики качества
5. Подготовка примеров использования (use cases)

Подсказка: Сначала use cases, потом входы/выходы, ограничения, метрика, архитектура.

Ответ:

- Определение входных и выходных форматов
- Описание ограничений (латенси, память и т.д.)
- Выбор архитектурных подходов, которые впишутся в требования
- Определение предварительной метрики качества
- Подготовка примеров использования (use cases)

🔮 Шаг 10:

Миссия 10

Таня формирует дорожную карту проекта. Расположите фазы по порядку:

1. Исследование / бизнес-анализ
2. Прототип с baseline
3. Улучшения, эксперименты, инкременты

4. Валидация, тестирование, проверка на краевых случаях
5. Развертывание и мониторинг
6. Сбор обратной связи и доработка

Подсказка: Сначала понять задачу, затем базовый прототип, потом итерации, тестирование, запуск и фидбек.

Ответ:

- Исследование / бизнес-анализ
- Прототип с baseline
- Улучшения, эксперименты, инкременты
- Валидация, тестирование, проверка на краевых случаях
- Развертывание и мониторинг
- Сбор обратной связи и доработка

Шаг 11:

Миссия 11

Максим планирует итерации улучшения задачи. Упорядочите шаги:

1. Сбор новых данных после запуска
2. Анализ ошибок реального поведения
3. Уточнение метрики или постановки задачи
4. Формирование гипотез улучшений
5. Внедрение и эксперимент

Подсказка: Цикл — наблюдать, анализировать, гипотезы, тестировать, повторять.

Ответ:

- Сбор новых данных после запуска
- Анализ ошибок реального поведения
- Уточнение метрики или постановки задачи
- Формирование гипотез улучшений
- Внедрение и эксперимент

Шаг 12:

Миссия 12

Елена готовит чеклист готовности проекта к защите. Расположите шаги проверки:

1. Корректность метрик и их обоснование
2. Проверка требований (память, latency и др.)
3. Сценарии использования и их покрытия
4. Сравнение с baseline и абляции
5. Идентификация и обсуждение ограничений проекта

Подсказка: Защита начинается с метрик и требований, затем кейсы, сравнения и ограничения.

Ответ:

- Корректность метрик и их обоснование
- Проверка требований (память, latency и др.)
- Сценарии использования и их покрытия
- Сравнение с baseline и абляции
- Идентификация и обсуждение ограничений проекта

🔮 Шаг 13:

Миссия 13

После выбора задачи Илья думает о целевой аудитории. Почему это важно?

Подсказка: Модель может быть полезна или бесполезна в зависимости от контекста использования.

Ответ:

- **Чтобы настроить требования по качеству, latency и ограничениям**
- Чтобы выбрать цвет интерфейса
- Чтобы не использовать никакую метрику

🔮 Шаг 14:

Миссия 14

Ника хочет оценить сложность своей задачи заранее. Какой признак говорит о высокой сложности?

Подсказка: Сложность часто проявляется в размере пространства выходов, шуме и разнообразии данных.

Ответ:

- **Большое число классов, высокая вариативность или редкие случаи**
- Очень маленький датасет
- Простая линейная связь

🔮 Шаг 15:

Миссия 15

После выбора задачи и метрики Артем хочет собрать отзывы от пользователей о целях модели. Какую форму обратной связи стоит запросить?

Подсказка: Попроси не только «нравится/не нравится», но и случаи ошибок и желаемое поведение.

Ответ:

- **Конкретные примеры, где модель ошибается, и описания ожидаемого результата**
- Только общее одобрение или отказ
- Технический лог ошибок без контекста

🔮 Шаг 16:

Миссия 16

Перед тем как стартовать проект, команда обсуждает: «Когда мы признаем цель выполненной?» Что нужно зафиксировать заранее?

Подсказка: Постановка завершённости — это критерии успеха, пороги и условия.

Ответ:

- **Критерии успеха (метрики, пороговые значения, бизнес-условия)**
- Когда прошло определённое время
- Когда модель просто работает

Урок 2 - ⚙ Код, конфиги, метрики, деплой

Шаг 01:

Миссия 1

Вы — инженер ML, и вам нужно выбрать, как хранить гиперпараметры вашей модели на проде. Что вы выберете?

Подсказка: Нужно, чтобы конфигурации можно было изменять без перекомпиляции кода.

Ответ:

- **YAML/JSON конфиг-файл, загружаемый при старте**
- Хардкодить значения прямо в коде
- Сохранять гиперпараметры в комментариях к коду

Шаг 02:

Миссия 2

 Вы увидели, что в логах training падает метрика на валидации. Что ваш первый шаг?

Подсказка: Не сразу менять архитектуру — сначала проверить данные и логи.

Ответ:

- **Проверить распределения данных и потенциальные «утечки» между трейн и валидацией**
- Увеличить размер модели
- Увеличить learning rate

Шаг 03:

Миссия 3

 Ваша модель слишком большая, и inference на проде работает медленно. Что логичнее попробовать?

Подсказка: Сначала лёгкие методы, прежде чем кардинально переписывать модель.

Ответ:

- **Применить квантизацию или pruning**
- Сразу переписать модель в C++
- Добавить новые слои

Шаг 04:

Миссия 4

 Вы готовите записи метрик (loss, accuracy) в прод окружении. Что надо предусмотреть?

Подсказка: Логи должны быть структурированными и метрики — привязаны к версиям.

Ответ:

- **Логировать в JSON с указанием версии модели и шага**
- Выводить метрики в stdout без контекста
- Сохранять все логи в одном большом текстовом файле

Шаг 05:

Миссия 5

 Какой способ деплоя модели предпочтителен для совместимости и масштабируемости?

Подсказка: Контейнеры широко применяются в продакшене.

Ответ:

- **Docker-контейнер с API интерфейсом**
- Сборка в монолитное приложение на сервере
- Вставка модели в front-end-код

Шаг 06:

Миссия 6

Сопоставьте компоненты ML-системы и их роли:

Компоненты:

Ответ:

- Конфигурационный менеджер
- Метрики мониторинга
- Сервис inference
- Версионирование модели

Роли:

1. Хранит и управляет гиперпараметрами и настройками
2. Отслеживает производительность модели в продакшене
3. Отвечает на запросы пользователей, возвращая предсказания модели
4. Позволяет откатываться и сравнивать версии моделей

Подсказка: Подумайте, что делает каждая часть после запуска.

- **Конфигурационный менеджер** — Хранит и управляет гиперпараметрами и настройками
- **Метрики мониторинга** — Отслеживает производительность модели в продакшене
- **Сервис inference** — Отвечает на запросы пользователей, возвращая предсказания модели
- **Версионирование модели** — Позволяет откатываться и сравнивать версии моделей

Шаг 07:

Миссия 7

 Ассоциируйте уровень абстракции и формат кода/конфигурации:

Уровни:

Ответ:

- Код модели (архитектура)
- Гиперпараметры обучения
- Среда продакшена (ресурсы)
- Логгирование/метрики

Форматы:

1. YAML/JSON конфиг
2. Python класс / модуль
3. Docker / Kubernetes манифест
4. Серийные журналы, метрики

Подсказка: Denk, где лучше держать каждую часть — в коде или в конфиге?

- **Код модели (архитектура)** — Python класс / модуль
- **Гиперпараметры обучения** — YAML/JSON конфиг
- **Среда продакшена (ресурсы)** — Docker / Kubernetes манифест
- **Логгирование/метрики** — Серийные журналы, метрики

🔮 Шаг 08:

Миссия 8

Сопоставьте проблему и способ её диагностики:

Проблемы:

Ответ:

- Модель переобучилась
- Inference резко замедлился
- Метрики в проде расходятся с локальными
- Конфликты библиотек при деплое

Диагностика:

1. Сравнение сред и логи, проверить версии библиотек
2. Проверка распределения данных между трейн и прод
3. Профилирование времени inference, memory leak анализ
4. Регуляризация, ранняя остановка, data augmentation

Подсказка: Проблема — затем способ её обнаружить, подумайте логично.

- **Модель переобучилась** — Регуляризация, ранняя остановка, data augmentation
- **Inference резко замедлился** — Профилирование времени inference, memory leak анализ
- **Метрики в проде расходятся с локальными** — Проверка распределения данных между трейн и прод
- **Конфликты библиотек при деплое** — Сравнение сред и логи, проверить версии библиотек

🔮 Шаг 09:

Миссия 9

📄 Сопоставьте метрику и её свойство, важное на продакшене:

Метрики:

Ответ:

- latency (задержка ответа)
- throughput (пропускная способность)
- availability (доступность)
- error rate (процент ошибок)

Свойства:

1. Как часто система недоступна
2. Сколько запросов обрабатываются в секунду
3. Сколько времени занимает один запрос
4. Какой процент запросов возвращает ошибку

Подсказка: Думайте о смысле метрики в контексте live-сервиса.

- **latency (задержка ответа)** — Сколько времени занимает один запрос

- **throughput (пропускная способность)** — Сколько запросов обрабатываются в секунду
- **availability (доступность)** — Как часто система недоступна
- **error rate (процент ошибок)** — Какой процент запросов возвращает ошибку

🔧 Шаг 10:

Миссия 10

✂ Расположите этапы подготовки модели к продакшену в логическом порядке:

1. Мониторинг и алертинг в проде
2. Тестирование на edge cases и стресс-тесты
3. Подготовка конфигураций деплоя
4. Упаковка модели в контейнер
5. Интеграционное тестирование API
6. Запуск inference в режиме продакшена

Подсказка: Сначала готовим модель и окружение, затем тестируем, и лишь потом запускаем и следим.

Ответ:

- Мониторинг и алертинг в проде
- Тестирование на edge cases и стресс-тесты
- Подготовка конфигураций деплоя
- Упаковка модели в контейнер
- Интеграционное тестирование API
- Запуск inference в режиме продакшена

🔧 Шаг 11:

Миссия 11

↻ Упорядочите шаги rollback — если новая версия модели дала ошибку в продакшене:

1. Вернуть предыдущую стабильную версию модели
2. Проинформировать команду и зафиксировать событие
3. Отключить трафик на сломавшуюся версию
4. Проверить, что система работает как раньше

Подсказка: Что делать первым — отключить и вернуть стабильную, а потом сообщать и проверять.

Ответ:

- Вернуть предыдущую стабильную версию модели
- Проинформировать команду и зафиксировать событие
- Отключить трафик на сломавшуюся версию
- Проверить, что система работает как раньше

🔧 Шаг 12:

Миссия 12

📊 Расположите стадии CI/CD процесса при обновлении модели:

1. Сборка контейнера с новой моделью
2. Юнит-тесты кода и проверки конфигов
3. Деплой на staging среду

4. Интеграционное тестирование
5. Плавающий rollout или canary deployment
6. Мониторинг и проверка метрик

Подсказка: CI/CD — от сборки до мониторинга, разместите шаги как на конвейере.

Ответ:

- Сборка контейнера с новой моделью
- Юнит-тесты кода и проверки конфигов
- Деплой на staging среду
- Интеграционное тестирование
- Плавающий rollout или canary deployment
- Мониторинг и проверка метрик

Шаг 13:

Миссия 13

 Упорядочьте шаги отладки, если модель начала выдавать аномально высокие loss'ы:

1. Сравнить входные данные с тестовым набором
2. Проверить версии зависимостей и библиотек
3. Запустить профилирование вычислений и градиентов
4. Проверить конфигурацию hyperparams
5. Сравнить поведение в dev и prod окружении

Подсказка: Логика: сначала данные, потом окружение, затем внутренняя отладка.

Ответ:

- Сравнить входные данные с тестовым набором
- Проверить версии зависимостей и библиотек
- Запустить профилирование вычислений и градиентов
- Проверить конфигурацию hyperparams
- Сравнить поведение в dev и prod окружении

Шаг 14:

Миссия 14

 Расположите шаги A/B теста новой версии модели:

1. Разделить трафик между старой и новой версиями
2. Собирать метрики и сравнивать
3. Анализ статистической значимости результатов
4. Принять решение: полный релиз или откат
5. Запустить новую версию на весь трафик (если тест успешен)

Подсказка: A/B тест — это введение новой версии постепенно и решение по результатам.

Ответ:

- Разделить трафик между старой и новой версиями
- Собирать метрики и сравнивать
- Анализ статистической значимости результатов
- Принять решение: полный релиз или откат
- Запустить новую версию на весь трафик (если тест успешен)

Шаг 15:

Миссия 15

📋 Упорядочите этапы обработки запроса в API с моделью:

1. API принимает запрос и валидирует вход
2. Преобработка данных (нормализация, преобразование)
3. Передача данных в модель и получение предсказания
4. Постобработка результата (обратные трансформации)
5. Возврат ответа клиенту и логгирование

Подсказка: Думайте, как проходит путь данных от клиента до ответа.

Ответ:

- API принимает запрос и валидирует вход
- Преобработка данных (нормализация, преобразование)
- Передача данных в модель и получение предсказания
- Постобработка результата (обратные трансформации)
- Возврат ответа клиенту и логгирование

Шаг 16:

Миссия 16

🗍 Расположите фазы жизненного цикла модели в продакшене:

1. Обучение и валидация
2. Тестирование и вёрстка API
3. Деплой и запуск
4. Мониторинг и алертинг
5. Обновление / переобучение / rollback

Подсказка: Жизненный цикл — от тренировки до поддержки и обновления.

Ответ:

- Обучение и валидация
- Тестирование и вёрстка API
- Деплой и запуск
- Мониторинг и алертинг
- Обновление / переобучение / rollback

Шаг 17:

Миссия 17

⚙ Расположите в правильном порядке операции при выпуске новой версии модели в продакшен:

1. Сборка контейнера с новой моделью
2. Запуск на сапай / часть трафика
3. Мониторинг метрик новых версий и сравнение
4. Полный релиз, если показатели стабильны
5. Откат, если метрики ухудшаются

Подсказка: Вы не разворачиваете сразу на весь трафик — сначала тестируете небольшой процент.

Ответ:

- Сборка контейнера с новой моделью
- Запуск на сапугу / часть трафика
- Мониторинг метрик новых версий и сравнение
- Полный релиз, если показатели стабильны
- Откат, если метрики ухудшаются

Урок 3 - 🗣 Презентация результата + peer review

Шаг 01:

Миссия 1

Ты показал свою модель преподавателю: чему отдать приоритет в слайдах для защиты?

Подсказка: важно не перегрузить техническими деталями, но показать ключевые новшества.

Ответ:

- **Архитектура + примеры работы**
- Каждая строчка кода
- Только бизнес-причины

Шаг 02:

Миссия 2

 Коллега просит объяснить использование attention-масок в твоём решении. Что ты покажешь?

Подсказка: визуализация внимания часто помогает.

Ответ:

- **Тепловую карту весов внимания**
- Сырые числа весов
- Один числовой срез

Шаг 03:

Миссия 3

 Ты презентуешь метрики модели. Что стоит обязательно включить?

Подсказка: кроме точности, подумай о слабых и сильных сторонах.

Ответ:

- **Precision, recall, F1, ROC**
- Только ассурасу
- Скорость обучения

Шаг 04:

Миссия 4

 При защите проекта тебе задают вопрос: «Почему ты выбрал эту архитектуру?». Как отвечаешь?

Подсказка: демонстрируй компромиссы и обоснования.

Ответ:

- **Это лучший компромисс между метриками, скоростью и сложностью**
- Потому что все так делают

- Потому что она проще всех

🔮 Шаг 05:

Миссия 5

🎧 Сопоставьте часть презентации с её идеальным содержанием:

Части слайдов:

Ответ:

- Введение в задачу
- Описание архитектуры
- Результаты и метрики
- Выводы и дальнейшие шаги

Контент:

1. Обсуждение ограничений, планы улучшения
2. Цель проекта, мотивация, формулировка задачи
3. Диаграмма модели, слои, модули
4. Таблицы метрик, сравнение с baseline

Подсказка: логика презентации — рассказ от задачи к решению, к результатам, к выводам.

- **Введение в задачу** — Цель проекта, мотивация, формулировка задачи
- **Описание архитектуры** — Диаграмма модели, слои, модули
- **Результаты и метрики** — Таблицы метрик, сравнение с baseline
- **Выводы и дальнейшие шаги** — Обсуждение ограничений, планы улучшения

🔮 Шаг 06:

Миссия 6

🔧 Сопоставьте элементы модели с возможными проблемами на защите:

Элементы:

Ответ:

- Сложное регуляризаторное дополнение
- Гибридный Encoder-Decoder
- Придуманная потеря
- Многозадачное обучение

Проблемы:

1. Сложно объяснить в домашних условиях
2. Зрители могут назвать это «overfitting» на задачу
3. Непрозрачность мотивации
4. Слишком высокая взаимозависимость задач

Подсказка: подумай, какая часть может быть непонятной или спорной для аудитории.

- **Сложное регуляризаторное дополнение** — Сложно объяснить в домашних условиях
- **Гибридный Encoder-Decoder** — Непрозрачность мотивации
- **Придуманная потеря** — Зрители могут назвать это «overfitting» на задачу
- **Многозадачное обучение** — Слишком высокая взаимозависимость задач

🔧 Шаг 07:

Миссия 7

📋 Сопоставьте метрику с её сильной стороной:

Метрики:

Ответ:

- Accuracy
- Precision
- Recall
- F1-score

Сильные стороны:

1. Хорошо отражает баланс точности и полноты
2. Учитывает ложноположительные ошибки
3. Простая и понятная общая мера
4. Важно при низком проценте положительных классов

Подсказка: вспомни, в каких сценариях важнее precision или recall.

- **Accuracy** — Простая и понятная общая мера
- **Precision** — Учитывает ложноположительные ошибки
- **Recall** — Важно при низком проценте положительных классов
- **F1-score** — Хорошо отражает баланс точности и полноты

🔧 Шаг 08:

Миссия 8

🔗 Расположите этапы подготовки презентации перед защитой:

1. Репетиция выступления вслух
2. Проверка, что все слайды отображаются корректно
3. Сбор обратной связи от друга
4. Финальная правка по времени и структуре

Подсказка: сначала проверь технику, потом протестируй на аудитории, потом доработай.

Ответ:

- Репетиция выступления вслух
- Проверка, что все слайды отображаются корректно
- Сбор обратной связи от друга
- Финальная правка по времени и структуре

🔧 Шаг 09:

Миссия 9

📋 Расположите этапы peer review твоего проекта:

1. Получить рецензии от коллег
2. Отправить презентацию + модель на обзор
3. Обработать замечания и исправить
4. Поделиться финальной версией с рецензентами

Подсказка: обзор начинается с отправки, затем получение, правка, и обратная связь снова.

Ответ:

- Получить отзывы от коллег
- Отправить презентацию + модель на обзор
- Обработать замечания и исправить
- Поделиться финальной версией с рецензентами

 Шаг 10:

Миссия 10

Расположите шаги подготовки к QA и деплою после защиты:

1. Автотесты и smoke tests
2. Документация API и интерфейсов
3. Деплой на staging
4. Мониторинг и логирование в продакшене

Подсказка: сначала тесты, потом документация, потом деплой, затем мониторинг.

Ответ:

- Автотесты и smoke tests
- Документация API и интерфейсов
- Деплой на staging
- Мониторинг и логирование в продакшене

 Шаг 11:

Миссия 11

✂ Ты получил критику: «Не вижу доказательства устойчивости модели». Как реагируешь в ответ?

Подсказка: покажи, что ты думал об этом — визуализации, валидация, устойчивость к шуму.

Ответ:

- **Покажу robustness анализ: атаки, шум, разные входы**
- Скажу, что модель работает уже достаточно хорошо
- Скажу, что не было времени

 Шаг 12:

Миссия 12

🔗 Один рецензент пишет: «Ваш код не воспроизводим». Что ты добавишь в репозиторий прямо сейчас?

Подсказка: подумай о зависимостях, семенах, инструкции.

Ответ:

- **requirements.txt / environment.yml + seed и инструкция запуска**
- README только с описанием задачи
- Просто код без инструкций

 Шаг 13:

Миссия 13

🔍 Сопоставьте формат сдачи проекта с содержанием:

Формат:

Ответ:

- Код и репозиторий
- Презентация
- Отчет по экспериментам
- Инструкция по развёртыванию

Содержимое:

1. Как запустить модель, зависимости, API
2. Липкие места экспериментов, гиперпараметры, графики
3. Слайды: мотивация, архитектура, выводы
4. Исходный код, скрипты обучения, тесты

Подсказка: каждый формат файла дополняет общий рассказ о проекте.

- **Код и репозиторий** — Исходный код, скрипты обучения, тесты
- **Презентация** — Слайды: мотивация, архитектура, выводы
- **Отчет по экспериментам** — Липкие места экспериментов, гиперпараметры, графики
- **Инструкция по развёртыванию** — Как запустить модель, зависимости, API

 Шаг 14:

Миссия 14

 Сопоставьте тип доказательства с конкретным примером:

Тип доказательства:

Ответ:

- Анализ устойчивости
- Сравнение с baseline
- Ablation study
- Кросс-валидация

Примеры:

1. Убираем часть модуля и смотрим, что ухудшается
2. Тестирование на различных шумовых вариантах входа
3. Сравнение с более старой моделью на тех же данных
4. Разбиваем данные на фолды и усредняем метрики

Подсказка: вспомни стандартные методы валидации и анализа в исследованиях.

- **Анализ устойчивости** — Тестирование на различных шумовых вариантах входа
- **Сравнение с baseline** — Сравнение с более старой моделью на тех же данных
- **Ablation study** — Убираем часть модуля и смотрим, что ухудшается
- **Кросс-валидация** — Разбиваем данные на фолды и усредняем метрики

 Шаг 15:

Миссия 15

 Расставьте в правильном порядке шаги peer review как рецензент:

1. Ознакомление с кодом и инструкцией
2. Оценка результатов и метрик
3. Написание замечаний и рекомендаций
4. Повторная проверка после доработок

Подсказка: сначала ты читаешь, затем оцениваешь, потом пишешь, потом проверяешь доработки.

Ответ:

- Ознакомление с кодом и инструкцией
- Оценка результатов и метрик
- Написание замечаний и рекомендаций
- Повторная проверка после доработок

Шаг 16:

Миссия 16

✓ Расположите этапы улучшения проекта после peer review:

1. Внедрить правки и исправления
2. Повторный запуск экспериментов
3. Обновление документации и презентации
4. Заключительное тестирование и финализация

Подсказка: сначала исправляешь, затем проверяешь изменения, затем обновляешь документацию.

Ответ:

- Внедрить правки и исправления
- Повторный запуск экспериментов
- Обновление документации и презентации
- Заключительное тестирование и финализация

Шаг 17:

Миссия 17

🗒 Расположите фазы финальной презентации + обратной связи на курсе:

1. Студенты презентуют свои проекты
2. Жюри и сверстники задают вопросы и дают отзывы
3. Участники дорабатывают проекты после фидбэка
4. Объявление победителей и обсуждение лучших решений

Подсказка: презентация → отзывы → доработки → итоги.

Ответ:

- Студенты презентуют свои проекты
- Жюри и сверстники задают вопросы и дают отзывы
- Участники дорабатывают проекты после фидбэка
- Объявление победителей и обсуждение лучших решений

Урок 4 - 🏆 Получение статуса ML Architect Pro

Шаг 01:

Миссия 1

🕒 Ты только что завершил архитектуру модели — что первым добавишь в слайды презентации?

Подсказка: начни с мотивации и постановки задачи — это ваша отправная точка.

Ответ:

- **Постановка задачи и мотивация проекта**
- Все детали реализации сети
- Сразу листинг кода

Шаг 02:

Миссия 2

🗣 При защите проекта спросят: «Почему ты выбрал этот loss?» Как лучше ответить?

Подсказка: расскажи про свойства loss и как они связаны с задачей.

Ответ:

- **Потому что он напрямую оптимизирует нужную метрику и стабилен**
- Потому что он простой
- Потому что у всех так

Шаг 03:

Миссия 3

🗣 Коллега спрашивает: «Где узкое место в работе твоей модели?» Что покажешь?

Подсказка: профилируй, покажи временные узлы или узлы с максимальной нагрузкой.

Ответ:

- **Профиль времени по слоям и пиковые точки**
- Общий граф обучения
- Описание гиперпараметров

Шаг 04:

Миссия 4

🗣 Тебя спросят: «Как ты оцениваешь обобщающую способность?» Что покажешь?

Подсказка: не просто ассигасу — нужно показать, как ведёт себя модель на разных данных.

Ответ:

- **Кривые обучения, графики переобучения и validation**
- Только тестовую точность
- Только loss функции

Шаг 05:

Миссия 5

Сопоставьте этап презентации с её главной задачей:

Части:

Ответ:

- Мотивация и цель
- Архитектура
- Эксперименты и метрики
- Выводы и дальнейшие планы

Задачи:

1. Объяснить, зачем всё это делалось

2. Показать, как устроена модель
3. Подтвердить эффективность подхода
4. Показать путь развития и ограничения

Подсказка: следи за логической связью от мотивации к решению и к результатам.

- **Мотивация и цель** — Объяснить, зачем всё это делалось
- **Архитектура** — Показать, как устроена модель
- **Эксперименты и метрики** — Подтвердить эффективность подхода
- **Выводы и дальнейшие планы** — Показать путь развития и ограничения

Шаг 06:

Миссия 6

 Сопоставьте критику с тем, как ты её адресуешь:

Критики:

Ответ:

- «Малая стабильность»
- «Непонятен выбор гиперпараметров»
- «Недостаточное сравнение с конкурентами»
- «Слишком сложная модель без нужды»

Твои контрмеры:

1. Robustness тесты, шум, вариации данных
2. Ablation study и объяснение выбора
3. Сравнение с baseline и современными методами
4. Обоснование архитектурных решений и упрощения

Подсказка: подумай, как ты заранее включил доказательства в работу.

- **«Малая стабильность»** — Robustness тесты, шум, вариации данных
- **«Непонятен выбор гиперпараметров»** — Ablation study и объяснение выбора
- **«Недостаточное сравнение с конкурентами»** — Сравнение с baseline и современными методами
- **«Слишком сложная модель без нужды»** — Обоснование архитектурных решений и упрощения

Шаг 07:

Миссия 7

 Сопоставьте исследовательский метод с тем, что он демонстрирует:

Методы:

Ответ:

- Ablation study
- Кросс-валидация
- Robustness / стресс-тесты
- Сравнение с baseline

Что показывает:

1. Как удаление компонентов влияет на качество
2. Обобщение по разным фолдам

3. Устойчивость к шуму или искажениям
4. Преимущество над базовыми методами

Подсказка: вспомни стандартные методики научных работ в ML.

- **Ablation study** — Как удаление компонентов влияет на качество
- **Кросс-валидация** — Обобщение по разным фолдам
- **Robustness / стресс-тесты** — Устойчивость к шуму или искажениям
- **Сравнение с baseline** — Преимущество над базовыми методами

🔮 Шаг 08:

Миссия 8

📋 Расположите шаги подготовки peer review:

1. Отправка репозитория и презентации коллегам
2. Получение замечаний и вопросов
3. Внесение правок и улучшений
4. Отправка обновлённой версии для проверки

Подсказка: сначала отправляешь, затем получаешь отзывы, потом исправляешь и снова отправляешь.

Ответ:

- Отправка репозитория и презентации коллегам
- Получение замечаний и вопросов
- Внесение правок и улучшений
- Отправка обновлённой версии для проверки

🔮 Шаг 09:

Миссия 9

🔗 Расположите этапы финальной подготовки перед защитой:

1. Проверка работоспособности всех слайдов и кода
2. Репетиция выступления перед коллегами
3. Доработка по таймингу и логике рассказа
4. Уточнение возможных вопросов и ответов

Подсказка: сначала техника, затем репетиция, доработка и подготовка ответов.

Ответ:

- Проверка работоспособности всех слайдов и кода
- Репетиция выступления перед коллегами
- Доработка по таймингу и логике рассказа
- Уточнение возможных вопросов и ответов

🔮 Шаг 10:

Миссия 10

🔗 Расположите шаги после защиты до получения статуса ML Architect Pro:

1. Обработка комментариев от жюри и коллег
2. Внедрение улучшений и доработок
3. Финальное тестирование и проверка качества

4. Загрузка финальной версии проекта и отчёта в систему

Подсказка: свежие отзывы → доработка → проверка → сдача.

Ответ:

- Обработка комментариев от жюри и коллег
- Внедрение улучшений и доработок
- Финальное тестирование и проверка качества
- Загрузка финальной версии проекта и отчёта в систему

Шаг 11:

Миссия 11

 Тебе возвращают проект с просьбой улучшить устойчивость к шуму. Что сделаешь?

Подсказка: надирующие атаки или данные с шумом могут выявить слабые места.

Ответ:

- **Добавлю шум в тренировочные данные и проведу robustness тесты**
- Скажу, что модель уже устойчива
- Уберу слои, чтобы она была проще

Шаг 12:

Миссия 12

 Жюри спрашивает: «Как гарантируешь безопасность модели в продакшене?» Что ответишь?

Подсказка: защита от атак, мониторинг аномалий, контроль входов — всё важно.

Ответ:

- **Я внедряю валидацию входов, мониторинг аномалий и защиту от атак**
- Скажу, что доверяю пользователям
- Не буду говорить о безопасности

Шаг 13:

Миссия 13

 Сопоставьте проблемные сценарии с мерами, которые ты можешь принять:

Сценарии:

Ответ:

- Внезапный drift данных
- Злоумышленник подаёт ввод-подделку
- Модель деградирует со временем
- Сбой вычислительной среды

Меры:

1. Мониторинг производительности и переобучение
2. Защита от adversarial атак, проверка границ
3. Периодические переобучения и адаптация
4. Контроль ресурсов, fallback режимы

Подсказка: подумай, как система может реагировать на неожиданные события.

- **Внезапный drift данных** — Мониторинг производительности и переобучение
- **Злоумышленник подаёт ввод-подделку** — Защита от adversarial атак, проверка границ

- **Модель деградирует со временем** — Периодические переобучения и адаптация
- **Сбой вычислительной среды** — Контроль ресурсов, fallback режимы

🔮 Шаг 14:

Миссия 14

📖 Сопоставьте способ оценивания проекта с тем, что он показывает:

Методы оценки:

Ответ:

- Кросс-валидация
- Hold-out на новых данных
- Robustness тесты
- Ablation study

Что показывает:

1. Глобальное обобщение на разных фолдах
2. Поведение на неизведанных данных
3. Устойчивость к искажениям
4. Ценность каждого компонента модели

Подсказка: каждая методика даёт своё доказательство качества.

- **Кросс-валидация** — Глобальное обобщение на разных фолдах
- **Hold-out на новых данных** — Поведение на неизведанных данных
- **Robustness тесты** — Устойчивость к искажениям
- **Ablation study** — Ценность каждого компонента модели

🔮 Шаг 15:

Миссия 15

🔧 Расположите фазы безопасного развёртывания в продакшен:

1. Тестирование на staging / песочнице
2. Отладка логов и мониторинга в тестовом окружении
3. Пилотный выпуск на часть трафика (canary)
4. Полный запуск и мониторинг

Подсказка: сначала staging, потом тестирование, затем частичный выпуск и полный запуск.

Ответ:

- Тестирование на staging / песочнице
- Отладка логов и мониторинга в тестовом окружении
- Пилотный выпуск на часть трафика (canary)
- Полный запуск и мониторинг

🔮 Шаг 16:

Миссия 16

♥ Расположите шаги реакции на инцидент в продакшене:

1. Обнаружение аномалии и тревога
2. Изоляция компонента / fallback режим
3. Эвакуация нагрузки и откат версии

4. Анализ причины и улучшения

Подсказка: сначала замечают ошибку, затем ограничивают ущерб, откатывают и анализируют.

Ответ:

- Обнаружение аномалии и тревога
- Изоляция компонента / fallback режим
- Эвакуация нагрузки и откат версии
- Анализ причины и улучшения

Шаг 17:

Миссия 17

 Расположите шаги, ведущие к статусу «ML Architect Pro»:

1. Успешная защита проекта
2. Получение позитивного peer review и жюри
3. Доработка проекта с учётом фидбэка
4. Подтверждение качества, публикация и вручение статуса

Подсказка: защита → отзывы → доработка → признание.

Ответ:

- Успешная защита проекта
- Получение позитивного peer review и жюри
- Доработка проекта с учётом фидбэка
- Подтверждение качества, публикация и вручение статуса